PAA — Exercícios dos materiais complementares

Resolução logo abaixo de cada enunciado · IFMG, Campus Formiga · 2026/2
Professor: Mário Luiz Rodrigues Oliveira · Aluna: Luisa Caetano

O que é. As **43 questões** que vêm dos PDFs e slides da disciplina — sem as listas e sem a prova, que estão nos seus próprios documentos. Cada resolução vem **logo abaixo do enunciado**, para conferência rápida na tela.

Depois delas, os exemplos já resolvidos nos slides, reproduzidos do material original.

Loureiro — Análise de Complexidade (UFMG) · semana 1	6
Campello — Análise de Algoritmos II · notação assintótica	10
Uber / Barbosa — Crescimento de Funções (UEM) · notação assintótica	3
Diego Mello — Matemática Discreta (IFMG Formiga) · semana 4	8
Rezende — Relações de Recorrência (IC-UNICAMP) · semana 4	7
Walace Rodrigues — Somatórios (IFMG Formiga)	4
Princípio de Indução Finita (IME-USP)	4
Slides de MC458 (UNICAMP) · semana 5 · exercício deixado em aberto	1

Loureiro — Análise de Complexidade (UFMG) · semana 1

1. Loureiro, ex. 5

Uso do teorema: Exercício 5

$T(n) = 4T(n/2) + n$

[transcrição literal — Loureiro, Análise de Complexidade (UFMG), slide 168]

> Resolução
>
> **Resposta.** T(n) = Theta(n ^ 2) (Caso 1 do Teorema Mestre)
>
> TEOREMA MESTRE (CLRS, Teorema 4.1). Seja T(n) = a*T(n/b) + f(n), com a >= 1 e b > 1 constantes, e seja E = log_b(a). Entao: Caso 1: se f(n) = O(n ^ (E - eps)) para algum eps > 0, entao T(n) = Theta(n ^ E). Caso 2: se f(n) = Theta(n ^ E), entao T(n) = Theta(n ^ E * lg n). Caso 3: se f(n) = Omega(n ^ (E + eps)) para algum eps > 0 E vale a condicao de regularidade a*f(n/b) <= c*f(n) para alguma constante c < 1 e todo n suficientemente grande, entao T(n) = Theta(f(n)).
>
> Intuicao: n ^ E = n ^ (log_b a) e o custo total das FOLHAS da arvore de recursao e f(n) e o custo da RAIZ. O teorema compara os dois, mas exige folga POLINOMIAL: se f(n) fica abaixo de n ^ E por um fator n ^ eps, vale o Caso 1; se f(n) = Theta(n ^ E), o Caso 2 (aparece o fator lg n, que conta os niveis); se f(n) fica acima de n ^ E por um fator n ^ eps e vale a regularidade, o Caso 3. Entre os casos ha lacunas em que o teorema nao se aplica, por exemplo f(n) = n ^ E / lg n ou f(n) = n ^ E * lg n: nao basta 'as folhas dominarem'.
>
> **APLICACAO.** T(n) = 4T(n/2) + n, logo a = 4, b = 2, f(n) = n. E = log_2(4) = 2, ou seja n ^ E = n ^ 2.
>
> Comparacao: f(n) = n = n ^ 1 e n ^ E = n ^ 2, e 1 < 2. Tome eps = 1 > 0. Entao n ^ (E - eps) = n ^ (2-1) = n, e vale 0 <= n <= 1 * n para todo n >= 1, isto e, f(n) = O(n ^ (E - eps)) com testemunhas c = 1 e n0 = 1. (Qualquer eps em (0,1] serve; para 0 < eps < 1 basta n <= 1*n ^ (2-eps) para todo n >= 1, pois 2 - eps >= 1.)
>
> Estamos no CASO 1, e portanto T(n) = Theta(n ^ E) = Theta(n ^ 2).
>
> Leitura: as folhas dominam. A arvore tem log_2 n + 1 niveis (profundidades 0..log_2 n). Cada nivel interno i < log_2 n tem 4 ^ i subproblemas de tamanho n/2 ^ i com custo n/2 ^ i cada, ou seja 2 ^ i * n por nivel, e esses niveis somam n + 2n + ... + (n/2)*n = n ^ 2 - n. O nivel das folhas tem 4 ^ (log_2 n) = n ^ 2 nos de custo T(1) cada, ou seja n ^ 2 * T(1). A soma geometrica e dominada pelo fundo da arvore: o total e n ^ 2 - n + n ^ 2 * T(1) = Theta(n ^ 2), com T(1) > 0 constante (para n potencia de 2 e T(1) = 1, T(n) = 2n ^ 2 - n; fora das potencias de 2, com piso ou teto, a forma fechada muda, mas a classe Theta(n ^ 2) nao).

2. Loureiro, ex. 6

Uso do teorema: Exercício 6

$T(n) = 4T(n/2) + n^2$

[transcrição literal — Loureiro, Análise de Complexidade (UFMG), slide 169]

> Resolução
>
> **Resposta.** T(n) = Theta(n ^ 2 * lg n) (Caso 2 do Teorema Mestre)
>
> Mesmo Teorema Mestre (forma do CLRS, a do slide) enunciado na resolucao do Loureiro, ex. 5.
>
> **APLICACAO.** T(n) = 4T(n/2) + n ^ 2, logo a = 4, b = 2, f(n) = n ^ 2. E = log_2(4) = 2, ou seja n ^ E = n ^ 2.
>
> Comparacao: f(n) = n ^ 2 = n ^ E exatamente. Formalmente, f(n) = Theta(n ^ 2) com testemunhas c1 = 1, c2 = 1 e n0 = 1, pois 1 * n ^ 2 <= n ^ 2 <= 1 * n ^ 2 para todo n >= 1.

Estamos no CASO 2, e portanto T(n) = Theta(n ^ E * lg n) = Theta(n ^ 2 * lg n).

Leitura: aqui folhas e raiz custam a mesma ordem, e todos os niveis da arvore custam Theta(n ^ 2) cada um. Como ha Theta(lg n) niveis (o nivel i tem 4 ^ i subproblemas de tamanho n/2 ^ i, cada um custando (n/2 ^ i) ^ 2 = n ^ 2/4 ^ i, totalizando n ^ 2 por nivel), o custo total e n ^ 2 vezes o numero de niveis, isto e Theta(n ^ 2 * lg n). O fator lg n vem exatamente desse empate — ele nao aparece nos exercicios 5 e 7 do Loureiro.

---

3. Loureiro, ex. 7

Uso do teorema: Exercício 7

$T(n) = 4T(n/2) + n^3$

[transcrição literal — Loureiro, Análise de Complexidade (UFMG), slide 170]

Resolução

Resposta. T(n) = Theta(n ^ 3) (Caso 3 do Teorema Mestre, com a condicao de regularidade verificada)

Mesmo Teorema Mestre (forma do CLRS, a do slide) enunciado na resolucao do Loureiro, ex. 5.

APLICACAO. T(n) = 4T(n/2) + n ^ 3, logo a = 4, b = 2, f(n) = n ^ 3. E = log_2(4) = 2, ou seja n ^ E = n ^ 2.

Passo 1 — dominancia polinomial. Tome eps = 1 > 0. Entao n ^ (E + eps) = n ^ (2+1) = n ^ 3, e vale n ^ 3 >= 1 * n ^ 3 para todo n >= 1, isto e, f(n) = Omega(n ^ (E + eps)) com testemunhas c = 1 e n0 = 1. A raiz domina as folhas por um fator polinomial n ^ 1.

Passo 2 — condicao de regularidade (obrigatoria no Caso 3, nao basta citar). Precisamos de uma constante c < 1 com a*f(n/b) <= c*f(n) para todo n suficientemente grande: a * f(n/b) = 4 * (n/2) ^ 3 = 4 * n ^ 3/8 = n ^ 3/2 = 0,5 * n ^ 3. Logo a*f(n/b) <= c * f(n) = c * n ^ 3 vale com c = 1/2 < 1, para todo n >= 1 (na verdade vale com igualdade). Testemunhas: c = 1/2, n0 = 1. Condicao SATISFEITA.

Com os dois passos verificados, estamos no CASO 3, e portanto T(n) = Theta(f(n)) = Theta(n ^ 3).

Leitura: a raiz domina. O custo por nivel cai geometricamente por fator 1/2 (nivel 0 custa n ^ 3, nivel 1 custa 4*(n/2) ^ 3 = n ^ 3/2, nivel 2 custa 16*(n/4) ^ 3 = n ^ 3/4, ...), e a serie geometrica soma no maximo 2*n ^ 3 = Theta(n ^ 3). E exatamente isso que a condicao de regularidade formaliza.

---

4. Loureiro, ex. 8

Uso do teorema: Exercício 8

O tempo de execução de um algoritmo *A* é descrito pela recorrência

$T(n) = 7T(n/2) + n^2$

Um outro algoritmo A' tem um tempo de execução descrito pela recorrência

$T'(n) = aT'(n/4) + n^2$

Qual é o maior valor inteiro de a tal que A' é assintoticamente mais rápido que A?

[transcrição literal — Loureiro, Análise de Complexidade (UFMG), slide 171]

Resolução

Resposta. a = 48

Mesmo Teorema Mestre (forma do CLRS, a do slide) enunciado na resolucao do Loureiro, ex. 5.

PASSO 1 — resolver A: T(n) = 7T(n/2) + n ^ 2. Aqui a = 7, b = 2, f(n) = n ^ 2, e E_A = log_2(7) = 2,807354... (irracional, mas 2 < log_2(7) < 3 porque 4 < 7 < 8). Comparacao: f(n) = n ^ 2 e n ^ E_A = n ^ (log_2 7), com 2 < log_2(7). Tome eps = log_2(7) - 2 = 0,807354... > 0. Entao n ^ (E_A - eps) = n ^ 2 e vale 0 <= n ^ 2 <= 1 * n ^ 2 para todo n >= 1, ou seja f(n) = O(n ^ (E_A - eps)) com testemunhas c = 1 e n0 = 1. CASO 1, portanto: T(n) = Theta(n ^ (log_2 7)), com log_2(7) = 2,807354...

PASSO 2 — resolver A': T'(n) = a*T'(n/4) + n ^ 2. Aqui b = 4, f(n) = n ^ 2, e E' = log_4(a). O resultado depende de a: - se log_4(a) > 2, isto e a > 16: CASO 1 (tome eps = log_4(a) - 2 > 0; n ^ 2 <= 1*n ^ 2 para n >= 1) e T'(n) = Theta(n ^ (log_4 a)); - se a = 16: log_4(16) = 2, f(n) = Theta(n ^ 2), CASO 2 e T'(n) = Theta(n ^ 2 * lg n); - se a < 16: log_4(a) < 2, CASO 3 (eps = 2 - log_4(a) > 0; regularidade: a*(n/4) ^ 2 = (a/16)*n ^ 2 <= c*n ^ 2 com c = a/16 < 1) e T'(n) = Theta(n ^ 2). Como queremos o MAIOR a possivel, e todo a >= 16 ja da uma classe >= Theta(n ^ 2 lg n), o candidato relevante esta na faixa a > 16, onde T'(n) = Theta(n ^ (log_4 a)).

PASSO 3 — a condicao 'A' e assintoticamente mais rapido que A'. A' e assintoticamente mais rapido que A quando T'(n) = o(T(n)), isto e quando o expoente dominante de A' e ESTRITAMENTE MENOR que o de A: log_4(a) < log_2(7). Mudando de base (ln): log_4(a) = ln a / ln 4 = ln a / (2 ln 2) e log_2(7) = ln 7 / ln 2. Logo ln a / (2 ln 2) < ln 7 / ln 2 <=> ln a < 2 * ln 7 (multiplicando ambos os lados por 2 ln 2 > 0) <=> ln a < ln (7 ^ 2) <=> a < 49 (exponencial e estritamente crescente). Como a deve ser inteiro (numero de subproblemas), o maior valor e a = 48.

PASSO 4 — por que 49 NAO serve (o ponto delicado do exercicio). Para a = 49: log_4(49) = ln 49 / (2 ln 2) = 2 ln 7 / (2 ln 2) = ln 7 / ln 2 = log_2(7) EXATAMENTE. Note que 49 = 7 ^ 2 e 4 = 2 ^ 2, entao log_4(49) = log_(2 ^ 2)(7 ^ 2) = log_2(7). Nesse caso

f(n) = n^2 continua sendo O(n^(log_4(49) - eps)) com eps = log_2(7) - 2 > 0, logo ainda vale o CASO 1 e T'(n) = Theta(n^(log_4 49)) = Theta(n^(log_2 7)) = mesma classe de T(n). Os dois algoritmos ficam na MESMA classe assintotica — e um EMPATE, e empatar nao e 'ser assintoticamente mais rapido' (nao vale T'(n) = o(T(n)); a razao T'(n)/T(n) fica limitada entre duas constantes positivas, nao tende a 0). Por isso a resposta e 48, e nao 49.

CONFERENCIA DOS EXPOENTES (comparacao numerica): log_2(7) = 2,807355 log_4(48) = 2,792481 < 2,807355 -> A' mais rapido (VALE) log_4(49) = 2,807355 = 2,807355 -> empate (NAO vale) log_4(50) = 2,821928 > 2,807355 -> A' mais lento (NAO vale) E, de fato, para a = 48: T'(n) = Theta(n^(log_4 48)) e T(n) = Theta(n^(log_2 7)), com log_4 48 ~ 2,792481 e log_2 7 ~ 2,807355; a razao T'(n)/T(n) = Theta(n^(log_4 48 - log_2 7)), com expoente ~ -0,014874 < 0, logo -> 0, confirmando T'(n) = o(T(n)).

RESPOSTA: a = 48, e nesse caso T'(n) = Theta(n^(log_4 48)).

---

5. Loureiro, p. 87

Notação Θ: Exemplo

• Usando a definição formal de Θ prove que $6n^3 \neq \Theta(n^2)$.

Contexto do material (transcrição literal, necessária para resolver):
Loureiro, p. 84 — Notação Θ

• A notação Θ limita a função por fatores constantes.

• Escreve-se $f(n) = \Theta(g(n))$, se existirem constantes positivas c_1, c_2 e n_0 tais que para $n \geq n_0$, o valor de $f(n)$ está sempre entre $c_1 g(n)$ e $c_2 g(n)$ inclusive.
➜ Pode-se dizer que $g(n)$ é um limite assintótico firme (em inglês, *asymptotically tight bound*) para $f(n)$.

$f(n) = \Theta(g(n))$,
$\exists\, c_1 > 0, c_2 > 0, n_0, \mid 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n), \forall n \geq n_0$
Observe que a notação Θ define um conjunto de funções:

$\Theta(g(n)) =$
$\{f : \mathbb{N} \to \mathbb{R}^+ \mid \exists\, c_1 > 0, c_2 > 0, n_0, 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n), \forall n \geq n_0\}$.

[transcrição literal — Loureiro, p. 87]

Resolução

Resposta. 6n³ ∉ Θ(n²). Por contradição: se existissem c1 > 0, c2 > 0 e no com 0 ≤ c1n² ≤ 6n³ ≤ c2n² para todo n ≥ no, então, tomando n* = max(⌈no⌉, ⌊c2/6⌋ + 1), teríamos n* ≥ no e n* > c2/6, o que dá 6(n*)³ > c2(n*)², violando a cota superior. Logo não há constantes que satisfaçam a definição. Quem falha é só a cota superior; a inferior vale (por exemplo c1 = 6, no = 1).

INTERPRETAÇÃO DO ENUNCIADO:

A escrita "6n³ ≠ Θ(n²)" usa o abuso de notação do material (f(n) = Θ(g(n)) significa f pertence ao conjunto Θ(g(n))); portanto "≠" quer dizer "não pertence": 6n³ ∉ Θ(n²). Leituras possíveis: (i) não pertencimento ao conjunto Θ(n²) definido no slide da p. 84; (ii) desigualdade literal entre uma função e um conjunto, que seria trivial e sem conteúdo. Resolvo (i), que é a única que pede "a definição formal de Θ". As funções são f(n) = 6n³ e g(n) = n², com n natural (o conjunto do slide é de funções f : ℕ → ℝ⁺).

PASSO 1 — A DEFINIÇÃO E SUA NEGAÇÃO:

Pelo slide, f(n) = Θ(g(n)) se e só se ∃ c1 > 0, ∃ c2 > 0, ∃ no tais que ∀ n ≥ no: 0 ≤ c1g(n) ≤ f(n) ≤ c2g(n). A negação é: para todos c1 > 0, c2 > 0 e no, existe n ≥ no em que alguma das desigualdades 0 ≤ c1g(n), c1g(n) ≤ f(n), f(n) ≤ c2g(n) falha. Basta mostrar que, quaisquer que sejam c2 > 0 e no, existe n ≥ no (natural) com f(n) > c2g(n); isso já nega a definição para todo c1 também, pois c1 não interfere na desigualdade que falha.

PASSO 2 — SUPOSIÇÃO (PARA CONTRADIÇÃO):

Suponha que existam constantes c1 > 0, c2 > 0 e no tais que, para todo natural n ≥ no, 0 ≤ c1n² ≤ 6n³ ≤ c2n². (1) Em particular, para todo natural n ≥ no vale a cota superior 6n³ ≤ c2n².

PASSO 3 — O n QUE VIOLA A COTA SUPERIOR (EM FUNÇÃO DE c2 E no):

```
Defina
    n* = max(⌈n₀⌉, ⌊c₂/6⌋ + 1).
Propriedades de n*:
     (a) n* é natural e n* ≥ ⌈n₀⌉ ≥ n₀, logo (1) deveria valer em n = n*. (Se o material toma n₀ natural, ⌈n₀⌉ = n₀.)
    (b) n* ≥ ⌊c₂/6⌋ + 1 > c₂/6, porque ⌊x⌋ + 1 > x para todo real x. Como c₂ > 0, ⌊c₂/6⌋ ≥ 0, então n* ≥ 1 e, portanto, (n*)² > 0.
Aplicando a cota superior de (1) em n = n*:
    6(n*)³ ≤ c₂(n*)².
Dividindo os dois lados por (n*)² > 0 (o sentido da desigualdade se mantém):
    6n* ≤ c₂, isto é, n* ≤ c₂/6,
o que contradiz (b), n* > c₂/6. Diretamente: 6(n*)³ = 6n*·(n*)² > 6·(c₂/6)·(n*)² = c₂(n*)², a cota superior é violada em n*.
```

PASSO 4 — CONCLUSÃO:

A suposição (1) é falsa para quaisquer $c_1 > 0$, $c_2 > 0$ e n_0: sempre há o natural $n^* = \max(\lceil n_0 \rceil, \lfloor c_2/6 \rfloor + 1) \geq n_0$ com $6(n^*)^3 > c_2(n^*)^2$. Logo não existem constantes como exige a definição, e $6n^3 \notin \Theta(n^2)$, ou, na notação do slide, $6n^3 \neq \Theta(n^2)$. ∎

OBSERVAÇÃO — SÓ A COTA SUPERIOR FALHA:

A parte inferior de (1) é satisfazível: com $c_1 = 6$ e $n_0 = 1$, para todo $n \geq 1$ vale $0 \leq 6n^2 \leq 6n^3$, pois $n^3 \geq n^2$ quando $n \geq 1$; em $n = n_0 = 1$ exatamente: $0 \leq 6 \leq 6$, verdadeiro. Ou seja, $6n^3 = \Omega(n^2)$, e é apenas o $O(n^2)$ que não vale. Por isso a contradição foi construída sobre a cota superior, e o n^* depende só de c_2 e n_0, não de c_1.

CONFERÊNCIA DE FRONTEIRA (n EXATAMENTE NO LIMIAR):

O "+1" em $\lfloor c_2/6 \rfloor + 1$ é necessário: com $c_2 = 6$ e $n = c_2/6 = 1$, vale $6 \cdot 1^3 = 6 \leq 6 \cdot 1^2 = 6$, sem violação; já $n^* = \lfloor 6/6 \rfloor + 1 = 2$ dá $6 \cdot 8 = 48 > 6 \cdot 4 = 24$, violação. Com $c_2 = 12$: $n^* = 3$ (para $n_0 \leq 3$), $6 \cdot 27 = 162 > 12 \cdot 9 = 108$.

Conferência numérica (Python; não faz parte da prova): confere a fórmula $n^* = \max(\lceil n_0 \rceil, \lfloor c_2/6 \rfloor + 1)$ nos três casos testados (com n_0 pequeno): $c_2 = 1$ dá $n^* = \lfloor 1/6 \rfloor + 1 = 1$, e $6 \cdot 1 = 6 > 1 \cdot 1 = 1$; $c_2 = 10$ dá $n^* = \lfloor 10/6 \rfloor + 1 = 2$, e $6 \cdot 8 = 48 > 10 \cdot 4 = 40$; $c_2 = 1000$ dá $n^* = \lfloor 1000/6 \rfloor + 1 = 167$, e $6 \cdot 167 = 1002 > 1000$, isto é, $6 \cdot 167^3 > 1000 \cdot 167^2$. Nos três valores de n a cota superior é violada. As contas numéricas apenas ilustram; a prova é o Passo 3, que vale para todo $c_2 > 0$ e todo n_0.

---

6. Loureiro, p. 185

Modelagem usando equação de recorrência
Linhas no plano

• Problema:
– Qual é o número máximo de regiões L_n determinado por n retas no plano?

➜ Lembre-se que um plano sem nenhuma reta tem uma região, com uma reta tem duas regiões e com duas retas têm quatro regiões.

[Figura: três desenhos lado a lado. (i) plano vazio com a região rotulada 1, legenda $L_0 = 1$; (ii) uma reta, regiões rotuladas 1 e 2, legenda $L_1 = 2$; (iii) duas retas concorrentes, regiões rotuladas 1, 2, 3, 4, legenda $L_2 = 4$.]

> **Contexto do material** (transcrição literal, necessária para resolver):
> *Loureiro, p. 184 — Modelagem usando equação de recorrência / Estratégia para resolução da equação*
>
> A recorrência da Torre de Hanoi aparece em várias aplicações de todos os tipos. Normalmente, existem três etapas para achar uma forma fechada para o valor de $T(n)$:
>
> 1. Analisar pequenos casos. Com isto podemos ter um entendimento melhor do problema e, ao mesmo tempo, ajudar nos dois passos seguintes.
> 2. Achar e provar uma recorrência para o valor de $T(n)$.
> 3. Achar e provar uma forma fechada para a recorrência.
>
> *(pp. 179–183: Torre de Hanoi como exemplo trabalhado — $T(0) = 0$, $T(n) = 2T(n-1) + 1$ para $n > 0$; tabela de pequenos casos; forma fechada $T(n) = 2^n - 1$ provada por indução.)*

[transcrição literal — Loureiro, p. 185]

Resolução

Resposta. O número máximo de regiões determinado por n retas no plano satisfaz a recorrência $L_0 = 1$ e $L_n = L_{n-1} + n$ para $n \geq 1$, e portanto tem a forma fechada $L_n = n(n+1)/2 + 1$ para todo $n \geq 0$ ($L_0..L_5 = 1, 2, 4, 7, 11, 16$). A recorrência vale porque a n-ésima reta acrescenta no máximo n regiões a qualquer configuração de $n - 1$ retas, e acrescenta exatamente n quando as retas estão em posição geral (nenhum par paralelo, nenhum trio passando por um mesmo ponto), configuração que existe para todo n. Em particular $L_n \in \Theta(n^2)$, com $c_1 = 1/2$, $c_2 = 1$, $n_0 = 2$.

CHECAGEM DE INTERPRETAÇÃO: Leituras possíveis de "número máximo de regiões L_n determinado por n retas": (i) o máximo, sobre todas as configurações de n retas distintas no plano, do número de regiões (componentes conexas do plano menos a união das retas); (ii) o número de regiões de uma configuração específica. A palavra "máximo" e os valores do enunciado ($L_2 = 4$ exige duas retas concorrentes; duas paralelas dariam 3) fixam a leitura (i), que é a resolvida aqui. Seguindo a estratégia do slide 184, há três etapas: pequenos casos; achar e provar a recorrência; achar e provar a forma fechada. Se se admitissem retas repetidas, uma reta repetida não muda a união e acrescenta 0 regiões, o que não afeta o máximo; daqui em diante as retas são distintas.

ETAPA 1 — PEQUENOS CASOS: Sem retas, o plano é uma região: $L_0 = 1$. Uma reta divide o plano em dois semiplanos: $L_1 = 2$. Duas retas: se paralelas, 3 regiões; se concorrentes, 4; logo $L_2 = 4$. Três retas: a terceira reta, cortando as duas primeiras (concorrentes) em dois pontos distintos, fica dividida em 3 pedaços, e cada pedaço divide uma região em duas: $4 + 3 = 7$ regiões (o triângulo do meio, 3 regiões encostadas nos lados e 3 encostadas nos vértices). Se as três passam por um mesmo ponto, só 6. A sequência 1, 2, 4, 7 sugere que a n-ésima reta acrescenta n regiões: $L_n = L_{n-1} + n$.

ETAPA 2 — A RECORRÊNCIA. PRELIMINAR: AS REGIÕES SÃO CONVEXAS. Seja um conjunto finito de retas $r_1, \ldots, r_m$. Cada r_i separa o plano em dois semiplanos abertos. Para cada ponto p fora de todas as retas, o "vetor de lados" de p diz em qual semiplano aberto de cada r_i o ponto está. O conjunto C dos pontos com um dado vetor de lados é a interseção de m semiplanos abertos, logo é aberto e convexo (em particular conexo). Esses conjuntos (os não vazios) particionam o plano menos a união das retas, são abertos e disjuntos, e cada um é conexo; logo são exatamente as componentes conexas, isto é, as regiões. Conclusão: toda região é um aberto convexo, e dois pontos fora das retas estão na mesma região se e somente se estão do mesmo lado de cada reta. Com $m = 0$ há um único vetor (vazio) e uma única região, o plano.

ETAPA 2 — LEMA (QUANTO UMA RETA NOVA ACRESCENTA): Seja uma configuração de n − 1 retas distintas (n ≥ 1) e ℓ uma reta distinta de todas elas. Seja k o número de pontos distintos em que ℓ encontra as n − 1 retas. Então a configuração com as n retas tem exatamente (regiões antigas) + (k + 1) regiões.

Prova. Cada reta antiga, sendo distinta de ℓ, encontra ℓ em no máximo um ponto; logo os pontos de ℓ que estão em alguma reta antiga são exatamente k pontos distintos, e k ≤ n − 1. Retirar k pontos distintos de uma reta deixa k + 1 pedaços (intervalos abertos: duas semirretas e k − 1 segmentos se k ≥ 1; a reta inteira se k = 0). Cada pedaço é convexo e não encontra reta antiga, então todos os seus pontos estão do mesmo lado de cada reta antiga: o pedaço inteiro está numa única região antiga. Seja R uma região antiga com ℓ ∩ R ≠ ∅. O conjunto ℓ ∩ R é convexo, está contido em ℓ e não encontra reta antiga, então está contido num único pedaço P; como P está inteiro numa região, que encontra R, essa região é R, e P ⊂ ℓ ∩ R. Logo ℓ ∩ R = P. Assim a correspondência pedaço ↦ região que o contém é uma bijeção entre os k + 1 pedaços e as regiões antigas que ℓ encontra: são exatamente k + 1 regiões antigas cortadas por ℓ. Pelo preliminar, as regiões novas são as regiões antigas subdivididas pelo lado de ℓ. Uma região antiga R que não encontra ℓ é convexa e não toca ℓ, então está toda de um lado de ℓ e continua sendo uma única região. Uma região antiga R que encontra ℓ, num ponto q, é aberta, então contém um pequeno disco em torno de q, que tem pontos dos dois lados de ℓ; logo R se divide em exatamente dois conjuntos não vazios, R ∩ (semiplano aberto de um lado) e R ∩ (do outro lado). Contando: regiões novas = (antigas não cortadas) + 2·(antigas cortadas) = (antigas) + (antigas cortadas) = (antigas) + (k + 1). ∎

ETAPA 2 — COTA SUPERIOR: A n-ÉSIMA RETA ACRESCENTA NO MÁXIMO n REGIÕES. Pelo lema, como k ≤ n − 1, a n-ésima reta acrescenta k + 1 ≤ n regiões a qualquer configuração das n − 1 retas anteriores. Defina U_0 = 1 e U_n = U_{n−1} + n. Afirmação Q(n): toda configuração de n retas distintas tem no máximo U_n regiões. Base n = 0: 1 região = U_0. Passo (n ≥ 1): dada uma configuração de n retas, retire uma delas, ℓ; as n − 1 restantes têm, por Q(n − 1), no máximo U_{n−1} regiões; recolocando ℓ, o lema dá no máximo U_{n−1} + n = U_n regiões. Logo L_n ≤ U_n para todo n ≥ 0.

ETAPA 2 — n É ATINGÍVEL: POSIÇÃO GERAL. Tome as retas r_i: y = i·x − i², para i = 1, 2, 3, … Os coeficientes angulares são distintos, então quaisquer duas são concorrentes (nenhum par paralelo e todas distintas). A interseção de r_i e r_j (i ≠ j) resolve i·x − i² = j·x − j², isto é, (i − j)x = i² − j², x = i + j; o ponto é (i + j, i·j). Ao acrescentar r_n às retas r_1, …, r_{n−1}, os pontos de encontro têm abscissas n + 1, n + 2, …, n + (n − 1), todas distintas: são exatamente k = n − 1 pontos distintos. (Isso diz também que três retas da família nunca passam por um mesmo ponto.) Pelo lema, r_n acrescenta exatamente n regiões. Seja A_n o número de regiões de r_1, …, r_n; então A_0 = 1 e A_n = A_{n−1} + n, a mesma recorrência de U, de onde A_n = U_n para todo n ≥ 0 (indução imediata: A_0 = U_0, e A_{n−1} = U_{n−1} implica A_n = U_{n−1} + n = U_n).

ETAPA 2 — CONCLUSÃO: U_n = A_n ≤ L_n (L_n é o máximo e A_n é atingido) e L_n ≤ U_n (cota superior). Logo L_n = U_n, isto é, L_0 = 1 e L_n = L_{n−1} + n para n ≥ 1. Note que a cota "no máximo n" vale para toda configuração e a igualdade só para configurações em que a nova reta corta as anteriores em n − 1 pontos distintos; se a reta nova é paralela a alguma anterior, ou passa por um ponto onde duas anteriores já se cruzam, ela acrescenta menos que n (ex.: três retas por um ponto dão 6, não 7); e uma configuração com algum par paralelo ou algum trio concorrente tem menos que L_n regiões.

ETAPA 3 — FORMA FECHADA, POR INDUÇÃO: Desenrolando, L_n = 1 + (1 + 2 + … + n) = 1 + n(n+1)/2, que se prova por indução. P(n): L_n = n(n+1)/2 + 1. Base n = 0: L_0 = 1 e 0·1/2 + 1 = 1. Confere. (Também n = 1: 1·2/2 + 1 = 2; n = 2: 2·3/2 + 1 = 4, como no enunciado.) Hipótese de indução: para um n ≥ 1, vale P(n − 1): L_{n−1} = (n − 1)n/2 + 1. Passo: L_n = L_{n−1} + n = (n − 1)n/2 + 1 + n = (n² − n + 2n)/2 + 1 = (n² + n)/2 + 1 = n(n+1)/2 + 1. Logo P(n). Por indução, P(n) vale para todo n ≥ 0.

COMPORTAMENTO ASSINTÓTICO (complemento): L_n = (n² + n)/2 + 1 ∈ Θ(n²): existem $c_1 = 1/2$, $c_2 = 1$, $n_0 = 2$ tais que, para todo n ≥ 2, (1/2)n² ≤ L_n ≤ n². A desigualdade da esquerda vale para todo n ≥ 0, pois L_n − n²/2 = n/2 + 1 > 0. A da direita: n² − L_n = (n² − n − 2)/2 = (n − 2)(n + 1)/2 ≥ 0 para n ≥ 2. Em n = n_0 = 2: (1/2)·4 = 2 ≤ L_2 = 4 ≤ 4 = 2². Confere (com igualdade à direita).

Conferência numérica (Python; não faz parte da prova): conferem os valores da recorrência L_0..L_10 = 1, 2, 4, 7, 11, 16, 22, 29, 37, 46, 56, que a forma fechada n(n+1)/2 + 1 bate com eles para n = 0..39, que retas aleatórias (em posição geral, com probabilidade 1) produzem exatamente esses números de regiões para n = 0..10, e que com concorrência tripla o número cai (3 retas por um ponto dão 6), em acordo com o lema (k = 1 < 2, a terceira reta acrescenta só 2). Nada disso é prova; a prova é a das etapas 2 e 3.

Campello — Análise de Algoritmos II · notação assintótica

7. Campello, ex. 1

Exercício: f(n) = 5n é o(n) ? Justifique
[transcrição literal — Campello, Análise de Algoritmos II 27]

Resolução

Resposta. NÃO. 5n não é o(n).

Definição adotada no material: f(n) é o(g(n)) se, para QUALQUER constante real c > 0, existe n0 tal que f(n) < c*g(n) para todo n >= n0. A ordem dos quantificadores é: para todo c > 0, existe n0, para todo n >= n0. Ou seja, a desigualdade tem de valer para TODA constante c, por menor que seja — e não apenas para alguma.

Para negar, basta exibir UMA constante c > 0 para a qual nenhum n0 funciona.

Testemunha da falha: tome c = 1. Seria preciso ter 5n < 1*n para todo n >= n0, isto é, 5n < n, ou seja, 4n < 0, ou seja, n < 0. Isso é falso para todo n >= 1. Logo não existe n0 algum que sirva para c = 1, e a definição de little-o já está violada.

O caso geral deixa claro o que acontece. Para n > 0 podemos dividir por n: 5n < c*n <=> 5 < c. A desigualdade não depende de n: ela é verdadeira para todo n se c > 5, e falsa para todo n se c <= 5. Portanto, para qualquer c <= 5 (por exemplo c = 5, c = 4, c = 1, c = 0,5) NÃO existe nenhum n a partir do qual 5n < c*n. Como a definição exige que a propriedade valha para toda constante positiva c, e ela falha em toda a faixa 0 < c <= 5, concluímos que 5n não é o(n).

Interpretação (o coração do item): little-o exige crescimento ESTRITAMENTE menor, isto é, que a razão f(n)/g(n) tenda a zero. Aqui 5n/n = 5 para todo n, e 5 não tende a 0. De fato 5n = Theta(n): a função e a referência têm exatamente a mesma taxa de crescimento, e uma constante multiplicativa não muda ordem assintótica. Por isso 5n é O(n) (com c = 5 e n0 = 1) mas nunca o(n).

Regra geral que vale guardar: nenhuma função é little-o de si mesma, nem de qualquer função da mesma ordem. o(g) é sempre um subconjunto ESTRITO de O(g), e a diferença entre os dois é exatamente o quantificador: O(g) pede 'existe c', o(g) pede 'para todo c'.

Conferência: para c = 0,5; 1; 4; 5 não existe nenhum n no intervalo testado (n de 1 até 200000) satisfazendo 5n < c*n; a desigualdade só passa a valer quando c > 5 (já para c = 5,0001 vale para todo n >= 1). Confirmado.

---

8. Campello, ex. 2

Exercício: f(n) = $5n^2$ é ω(n^2) ? Justifique
[transcrição literal — Campello, Análise de Algoritmos II 28]

Resolução

Resposta. NÃO. 5n ^ 2 não é omega(n ^ 2).

Definição adotada no material: f(n) é omega(g(n)) se, para QUALQUER constante real c > 0, existe n0 tal que f(n) > c*g(n) para todo n >= n0. De novo o quantificador é universal em c: a desigualdade tem de valer para TODA constante positiva, inclusive as grandes.

Para negar, basta exibir uma constante c > 0 para a qual nenhum n0 funciona.

Testemunha da falha: tome c = 5. Seria preciso 5n ^ 2 > 5*n ^ 2 para todo n >= n0, isto é, 0 > 0. Falso para todo n. Logo não existe n0 para c = 5, e a definição de little-omega está violada.

Caso geral: para n > 0, dividindo por n ^ 2, 5n ^ 2 > c*n ^ 2 <=> 5 > c. Outra vez a desigualdade não depende de n: vale para todo n se c < 5 e falha para todo n se c >= 5. Como existe (infinitas) constantes c >= 5 para as quais nenhum n serve, 5n ^ 2 não é omega(n ^ 2).

Interpretação: little-omega é o espelho de little-o — exige crescimento ESTRITAMENTE maior, isto é, f(n)/g(n) -> infinito. Aqui 5n ^ 2/n ^ 2 = 5, que é constante e não tende a infinito. Como 5n ^ 2 = Theta(n ^ 2), as duas funções têm a MESMA taxa de crescimento; uma função nunca pode ser little-omega de outra da mesma ordem. Ela é, sim, Omega(n ^ 2) (Omega maiúsculo, com c = 5 e n0 = 1, ou qualquer c <= 5), mas isso é outro operador: Omega pede 'existe c', omega pede 'para todo c'.

Conferência: a desigualdade 5n ^ 2 > c*n ^ 2 foi testada para n de 1 até 200000; vale para todo n quando c = 1 ou c = 4,999 e falha para todo n quando c = 5 ou c = 6. Confirmado.

---

9. Campello, ex. 3

Exercícios Adicionais

Seja f(n) que assume o valor $3n^2$ se n for par ou 7n se n for ímpar. Mostre que essa função é O(n^2) e Ω(n)
[transcrição literal — Campello, Análise de Algoritmos II 32]

Resolução

Resposta. f(n) é O(n ^ 2) com c = 7 e n0 = 1; e é Omega(n) com c = 3 e n0 = 1. (Observação: f não é Theta(n ^ 2) nem Theta(n).)

Seja f(n) = 3n ^ 2 se n é par, e f(n) = 7n se n é ímpar. Note que f(n) > 0 para todo n >= 1, então a exigência 0 <= f(n) das definições está automaticamente satisfeita.

PARTE 1 — f(n) é O(n ^ 2). Precisamos exibir c > 0 e n0 > 0 tais que 0 <= f(n) <= c*n ^ 2 para todo n >= n0. Testemunhas: c = 7 e n0 = 1. Prova por casos, para n >= 1: - n par: f(n) = 3n ^ 2 <= 7n ^ 2, pois 3 <= 7. Vale. - n ímpar: f(n) = 7n <= 7n ^ 2, pois 7n ^ 2 - 7n = 7n(n - 1) >= 0 sempre que n >= 1. Vale. Como os dois casos cobrem todo n >= 1, temos 0 <= f(n) <= 7n ^ 2 para todo n >= 1. Logo f(n) = O(n ^ 2). (Qualquer c >= 7 também serve.)

PARTE 2 — f(n) é Omega(n). Precisamos exibir c > 0 e n0 > 0 tais que f(n) >= c*n >= 0 para todo n >= n0. Testemunhas: c = 3 e n0 = 1. Prova por casos, para n >= 1: - n par: f(n) = 3n ^ 2 >= 3n, pois 3n ^ 2 - 3n = 3n(n - 1) >= 0 para n >= 1. Vale. - n ímpar: f(n) = 7n >= 3n, pois 7 >= 3. Vale. Logo f(n) >= 3n para todo n >= 1, e f(n) = Omega(n). (Qualquer 0 < c <= 3 também serve.)

Observe o que foi feito: como a função é definida por casos, a técnica é limitar CADA ramo separadamente e depois tomar a constante que serve para os dois (o maior c no teto, o menor c no piso). O n0 escolhido tem de valer para os dois ramos simultaneamente.

OBSERVAÇÃO IMPORTANTE — f não é Theta(n ^ 2) nem Theta(n). Esse é o ponto interessante do exercício: f oscila entre duas ordens diferentes, então o teto e o piso justos não coincidem.

(a) f NÃO é Theta(n ^ 2). Para isso faltaria o piso: seria preciso existir c1 > 0 e n0 com f(n) >= c1*n ^ 2 para TODO n >= n0. Mas nos índices ímpares f(n) = 7n, e 7n >= c1*n ^ 2 equivale a n <= 7/c1. Como existem ímpares arbitrariamente grandes, a desigualdade falha em infinitos valores de n, qualquer que seja c1 > 0 e qualquer que seja n0. Exemplo concreto com c1 = 1: em n = 9 temos f(9) = 63 < 81 = 9 ^ 2; o mesmo ocorre em n = 11, 13, 15, 17, ...

(b) f NÃO é Theta(n). Aqui faltaria o teto: seria preciso existir c2 > 0 e n0 com f(n) <= c2*n para todo n >= n0. Mas nos índices pares f(n) = 3n ^ 2, e 3n ^ 2 <= c2*n equivale a n <= c2/3, que falha em todos os pares grandes. Exemplo concreto com c2 = 100: já em n = 34 temos f(34) = 3468 > 3400.

Conclusão: f(n) fica presa entre n e n ^ 2 — é O(n ^ 2) (teto) e Omega(n) (piso) —, mas não admite descrição Theta por nenhuma das duas, porque sua taxa de crescimento não é única: ela alterna infinitas vezes entre ordem linear e ordem quadrática.

Conferência: as desigualdades 0 <= f(n) <= 7n ^ 2 e f(n) >= 3n foram verificadas para todo n de 1 até 200000 — ambas válidas em todo o intervalo. As falhas apontadas em (a) e (b) foram localizadas exatamente em n = 9, 11, 13, ... (para c1 = 1) e n = 34, 36, 38, ... (para c2 = 100). Confirmado.

---

10. Campello, ex. 6

Exercícios Adicionais

Diz-se que um algoritmo que executa um no. constante de operações primitivas (que independe do tamanho n da entrada) “executa em tempo O(1)” ou simplesmente “é O(1)”. Mostre, usando a definição formal do operador BIG-O, que qualquer função f(n) = d, onde d é uma constante positiva qualquer, é O(1)
[transcrição literal — Campello, Análise de Algoritmos II 33]

Resolução

Resposta. Sim. Para f(n) = d com d > 0 constante, tome c = d e n0 = 1: vale 0 <= f(n) <= c*1 para todo n >= 1. Logo f(n) = O(1).

DEFINIÇÃO. f(n) = O(g(n)) se existem constantes c > 0 e n0 > 0 tais que 0 <= f(n) <= c*g(n) para todo n >= n0. Aqui g(n) = 1, a função constante igual a 1; portanto c*g(n) = c*1 = c para qualquer n.

DEMONSTRAÇÃO. Seja f(n) = d, com d uma constante real positiva qualquer (o valor de d não depende de n — é exatamente isso que significa 'número constante de operações primitivas, independente do tamanho da entrada').

Escolha das testemunhas: c = d e n0 = 1. Repare que ambas são constantes fixadas ANTES de qualquer n, respeitando a ordem 'existe c, existe n0, para todo n >= n0'; e c = d > 0 por hipótese, e n0 = 1 > 0.

Verificação, para todo n >= n0 = 1: - lado esquerdo: 0 <= f(n), pois f(n) = d > 0. OK. - lado direito: f(n) = d <= d*1 = c*g(n). Vale com IGUALDADE, o que é permitido, já que a definição usa <= e não <. Como a cadeia 0 <= d <= c*1 vale para todo n >= 1 (na verdade para todo n, pois nada na desigualdade depende de n), a definição está satisfeita. Portanto f(n) = d é O(1).

OBSERVAÇÕES. 1. Qualquer c >= d serve igualmente bem (por exemplo c = d + 1, com folga estrita), e qualquer n0 > 0 serve, porque a desigualdade não depende de n. A liberdade de escolha é grande justamente porque o problema é trivial — mas a definição exige que a escolha seja EXIBIDA, e é isso que a prova faz. 2. Vale também a cota inferior: f(n) = d >= d*1 com c = d e n0 = 1, logo f(n) = Omega(1) e, portanto, f(n) = Theta(1). Ou seja, um algoritmo com número constante de operações não só 'não passa de' tempo constante como de fato gasta tempo constante. 3. Por que se escreve O(1) e não, digamos, O(5) ou O(d): em notação assintótica a constante multiplicativa é absorvida por c. Como d <= d*1, toda função constante positiva pertence à mesma classe O(1); escrever O(d) seria redundante, pois O(d) = O(1) para qualquer d > 0 constante.

Conferência: a desigualdade 0 <= d <= c*1 com c = d é uma identidade que não envolve n; a verificação é imediata e não admite contraexemplo. Confirmado.

---

11. Campello, ex. 8

Exercícios Adicionais

Ordene as seguintes funções de acordo com as suas taxas de crescimento assintótico (justifique!): 4*n*log(n) + 2n, 2^{10}, $2^{\log(n)}$, 3n + 100log(n), 4n, 2^n, 3^{3n}, n^2 + 10n, n^3, n*log(n)
[transcrição literal — Campello, Análise de Algoritmos II 33]

Resolução

Resposta. Ordem crescente de taxa de crescimento: 2 ^ 10 < 2 ^ (lg n) = 3n + 100 log n = 4n < n log n = 4n log n + 2n < n ^ 2 + 10n < n ^ 3 < 2 ^ n < 3 ^ (3n). (Os sinais '=' indicam funções Theta uma da outra, isto é, empatadas; log na base 2, como no material. Em base b > 2, como 10 ou e, 2 ^ (log_b n) = n ^ (log_b 2) sai do empate com 4n e fica estritamente entre 2 ^ 10 e 4n.)

PASSO 1 — simplificar cada função à sua classe Theta. Em notação assintótica, constantes multiplicativas somem e apenas o termo dominante da soma sobrevive.

```
2^10 = 1024                -> Theta(1).      É um número, não depende de n.
  2^(lg n) = n               -> Theta(n).      Pois 2^(log_2 n) = n, por definição de logaritmo na base 2.
                                               (Se o material usasse log na base 10, seria 2^(log_10 n) = n^(log_10
2) = n^0,301,
                                                que ficaria ENTRE a constante e a linear. Adoto a convenção do cur-
so, lg = base 2.)
  3n + 100 log n              -> Theta(n).      log n = o(n), então 3n domina: 3n <= 3n + 100 log n <= 103n para n
>= 1.
  4n                         -> Theta(n).      Constante 4 é absorvida.
  n log n                    -> Theta(n log n).
  4n log n + 2n               -> Theta(n log n). Pois n = o(n log n): para n >= 2, 4n log n <= 4n log n + 2n <= 6n
log n.
  n^2 + 10n                  -> Theta(n^2).    Para n >= 10: n^2 <= n^2 + 10n <= 2n^2.
  n^3                        -> Theta(n^3).
  2^n                        -> Theta(2^n).
  3^(3n) = (3^3)^n = 27^n    -> Theta(27^n).   Aqui está a pegadinha do item: o expoente 3n vira base 27.
```

PASSO 2 — encadear as classes. A hierarquia usada é a padrão, e cada elo é justificado pelo limite da razão (razão -> 0 significa crescimento estritamente menor):

1 << n << n log n << n ^ 2 << n ^ 3 << 2 ^ n << 27 ^ n

- 1 << n: 1/n -> 0. - n << n log n: n/(n log n) = 1/log n -> 0 (para n >= 2 já vale n <= n log n). - n log n << n ^ 2: (n log n)/n ^ 2 = (log n)/n -> 0, pois log n cresce mais devagar que qualquer potência positiva de n. - n ^ 2 << n ^ 3: n ^ 2/n ^ 3 = 1/n -> 0. - n ^ 3 << 2 ^ n: toda exponencial de base > 1 supera qualquer polinômio; n ^ 3/2 ^ n -> 0 (por L'Hôpital três vezes, ou notando que 2 ^ n = e ^ (n ln 2) e n ^ 3 = e ^ (3 ln n), com n ln 2 >> 3 ln n). - 2 ^ n << 27 ^ n: 27 ^ n/2 ^ n = (27/2) ^ n = 13,5 ^ n -> infinito. Ou seja, 3 ^ (3n) não apenas supera 2 ^ n como o faz por um fator que ele próprio é exponencial. É de longe a função que mais cresce da lista.

PASSO 3 — ordenação final, com os empates agrupados.

1º) 2 ^ 10 -> Theta(1) (constante) 2º) 2 ^ (lg n), 3n + 100 log n, 4n -> Theta(n) (as três EMPATADAS: uma é Theta da outra) 3º) n log n, 4n log n + 2n -> Theta(n log n) (as duas EMPATADAS) 4º) n ^ 2 + 10n -> Theta(n ^ 2) 5º) n ^ 3 -> Theta(n ^ 3) 6º) 2 ^ n -> Theta(2 ^ n) 7º) 3 ^ (3n) = 27 ^ n -> Theta(27 ^ n)

Em linha: 2 ^ 10 < 2 ^ (lg n) = 3n + 100 log n = 4n < n log n = 4n log n + 2n < n ^ 2 + 10n < n ^ 3 < 2 ^ n < 3 ^ (3n)

JUSTIFICATIVA DOS EMPATES (por que '=' e não '<'): - 2 ^ (lg n) = n e 4n: n/(4n) = 1/4, constante; logo n = Theta(4n) (testemunhas: 1/4 * 4n <= n <= 1 * 4n, para todo n >= 1). Constante multiplicativa não altera ordem. - 3n + 100 log n e 4n: para n >= 1 temos 3n <= 3n + 100 log n <= 103n, e 103n = Theta(4n). Um termo de ordem menor somado não altera a classe. - n log n e 4n log n + 2n: para n >= 2, 1*(n log n) <= 4n log n + 2n <= 6*(n log n), pois 2n <= 2n log n quando log n >= 1. Testemunhas c1 = 1, c2 = 6, n0 = 2.

DOIS PONTOS DE ATENÇÃO DO EXERCÍCIO: (i) 2 ^ (lg n) parece exponencial mas NÃO é: a exponenciação e o logaritmo na mesma base se cancelam, e a função é exatamente linear. Quando o expoente é c·lg n, a função é polinomial (2 ^ (c lg n) = n ^ c); para crescer como uma exponencial b ^ n, o expoente precisa crescer pelo menos linearmente em n. (ii) 3 ^ (3n) parece 'só mais uma exponencial' mas é a maior de todas: reescrevendo como 27 ^ n fica claro que a base é 27, não 3. Comparar exponenciais exige colocá-las na forma b ^ n e comparar as bases.

Conferência: as razões entre funções consecutivas foram avaliadas em n crescentes e confirmam a ordem assintótica (por exemplo, (n log n)/n ^ 2 tende a 0, decrescente só a partir de n = 3; 27 ^ n/2 ^ n = 13,5 ^ n explodindo). Para n pequeno a ordem pode se inverter: n ^ 3 > 2 ^ n para n = 2..9. As desigualdades dos empates (1 <= (4n log n + 2n)/(n log n) <= 6 para n >= 2; 3 <= (3n + 100 log n)/n <= 103 para n >= 1) foram checadas numericamente. Confirmado.

---

12. Campello, ex. 7

Exercícios Adicionais

Explique porque, no **melhor caso** (com relação às possíveis composições da entrada) o tempo de execução do algoritmo de busca binária iterativo visto em aula é O(1)

Pode-se então concluir que esse algoritmo é Ω(1) ?
[transcrição literal — Campello, Análise de Algoritmos II 33]

Resolução

Resposta. Melhor caso O(1): para todo n ≥ 1 há uma entrada (x na primeira posição consultada) que custa uma constante, 6 operações, com c = 6 e n0 = 1. Quanto à pergunta: SIM, o algoritmo é Ω(1), para toda entrada (c = 1, n0 = 1) — mas NÃO "por ser O(1)": O é cota superior e não implica cota inferior (1/n é O(1) e não é Ω(1)). O Ω(1) do algoritmo vem de o custo do melhor caso ser também ≥ uma constante e ser o MÍNIMO sobre todas as entradas, de modo que toda execução custa pelo menos isso. E o algoritmo NÃO é O(1): o pior caso é Θ(lg n).

MODELO DE CUSTO. Busca binária iterativa sobre A[1..n] ordenado, procurando x, cada linha executada custando 1 (um teste e a atribuição que ele dispara contam separados): 1 esq ← 1 2 dir ← n 3 enquanto esq ≤ dir 4 meio ← ⌊(esq+dir)/2⌋ 5 se A[meio] = x devolva meio 6 senão se x < A[meio] então dir ← meio − 1 7 senão esq ← meio + 1 8 devolva "não encontrado" Custos: inicialização 2; iteração que devolve (teste do laço, linha 4, teste da linha 5, retorno) 4; iteração que não devolve (teste do laço, linha 4, teste 5, teste 6, uma atribuição) 5; saída por "não encontrado" (teste falho do laço, retorno) 2. Para uma entrada I = (A, x) de ta-

manho n, T(I) é o custo da execução; T_melhor(n) = mín{T(I) : |I| = n} e T_pior(n) = máx{T(I) : |I| = n}. Por definição, T_melhor(n) ≤ T(I) ≤ T_pior(n) para toda I com |I| = n.

PARTE 1 — POR QUE O MELHOR CASO É O(1). Seja n ≥ 1. Como esq = 1 ≤ n = dir, toda execução entra no laço ao menos uma vez. Se a primeira iteração devolve, o custo é 2 + 4 = 6; se não devolve, é pelo menos 2 + 5 + 2 = 9 > 6. E o custo 6 é atingido para todo n ≥ 1: basta qualquer A ordenado com x = A[⌊(1+n)/2⌋], que é o primeiro meio. "Melhor caso" é a melhor composição da entrada com n fixo — não "n pequeno". Logo T_melhor(n) = 6 para todo n ≥ 1, valor que não depende de n. O(1): c = 6, n0 = 1, pois 0 ≤ 6 ≤ 6·1 para todo n ≥ 1; em n = n0 = 1: 6 ≤ 6. Também Ω(1): c = 6, n0 = 1 (0 ≤ 6·1 ≤ 6). Portanto T_melhor ∈ Θ(1), com c1 = c2 = 6, n0 = 1. Vale para: as entradas de melhor caso (x na posição central). Contando só comparações com elementos do vetor, o melhor caso faz 1 comparação, e as testemunhas passam a c = 1, n0 = 1; o modelo muda a constante, não a conclusão.

CONTRAPONTO — O PIOR CASO (o que NÃO se transfere). Seja m = dir − esq + 1 o tamanho do intervalo; então meio = esq + ⌊(m−1)/2⌋. Indo para a direita, o novo tamanho é dir − meio = m − 1 − ⌊(m−1)/2⌋ = ⌊m/2⌋; indo para a esquerda, é meio − esq = ⌊(m−1)/2⌋ ≤ ⌊m/2⌋. Após j iterações que não devolvem, o tamanho é no máximo ⌊n/2^j⌋ (pois ⌊⌊n/2^j⌋/2⌋ = ⌊n/2^(j+1)⌋), e uma nova iteração exige tamanho ≥ 1: há no máximo ⌊lg n⌋ + 1 iterações. A entrada com x > A[n] atinge exatamente esse número (vai sempre para a direita, com tamanhos ⌊n/2^j⌋ exatos), com custo 2 + 5(⌊lg n⌋ + 1) + 2 = 5⌊lg n⌋ + 9; nenhuma execução custa mais, porque uma iteração final que devolve custa 4 < 5 + 2. Logo T_pior(n) = 5⌊lg n⌋ + 9. Cota de baixo: ⌊lg n⌋ > lg n − 1 dá T_pior(n) > 5 lg n + 4 ≥ 5 lg n para n ≥ 1. Cota de cima: T_pior(n) ≤ 5 lg n + 9 ≤ 14 lg n quando 9 ≤ 9 lg n, isto é, n ≥ 2. Portanto T_pior ∈ Θ(lg n), com c1 = 5, c2 = 14, n0 = 2; em n = 2: 5 ≤ 14 ≤ 14. T_pior ∉ O(1): se T_pior(n) ≤ c para todo n ≥ n0, tome n ≥ máx(n0, 2^⌈c⌉); então T_pior(n) ≥ 5 lg n ≥ 5c > c — contradição.

PARTE 2 — "PODE-SE ENTÃO CONCLUIR QUE ESSE ALGORITMO É Ω(1)?" "O algoritmo é Ω(g)" significa: existem c > 0 e n0 > 0 tais que, para todo n ≥ n0 e TODA entrada I de tamanho n, T(I) ≥ c·g(n). Pela cadeia T_melhor(n) ≤ T(I) ≤ T_pior(n), isso equivale a T_melhor ∈ Ω(g); do mesmo modo, "o algoritmo é O(g)" equivale a T_pior ∈ O(g). Ou seja: uma cota INFERIOR do melhor caso vale para toda entrada; uma cota SUPERIOR do melhor caso, em geral, NÃO se transfere para as outras entradas (só quando ela também limita o pior caso).

A pergunta admite três leituras, e as três têm resposta:

(A) A inferência "o melhor caso é O(1), LOGO o algoritmo é Ω(1)" é válida? NÃO. O é cota superior e não fornece cota inferior. Contraexemplo dentro das definições do curso: f(n) = 1/n está em O(1) (c = 1, n0 = 1, pois 0 ≤ 1/n ≤ 1), mas não em Ω(1): para qualquer c > 0, todo n > 1/c dá 1/n < c. A conclusão é verdadeira por um fato ADICIONAL: T_melhor(n) = 6 > 0 — toda execução faz ao menos as linhas 1 a 4. Esse fato é a parte Ω do Θ(1) do melhor caso, não a parte O.

(B) O algoritmo (em toda entrada) é Ω(1)? SIM — é a leitura pretendida. Para todo n ≥ 1 e toda I com |I| = n, T(I) ≥ T_melhor(n) = 6 ≥ 1·1. Testemunhas c = 1 (ou c = 6), n0 = 1; em n = 1: 1 ≤ 6. Vale para toda entrada. A justificativa é que o melhor caso é limite inferior de todas as execuções: o que se transfere é a cota inferior do melhor caso, não o "O(1)".

(C) O pior caso é Ω(1)? Sim, mas é uma afirmação fraca: T_pior(n) ≥ 6 ≥ 1 (c = 1, n0 = 1); a cota justa é Ω(lg n), com c = 5, n0 = 1.

O que a pergunta testa: saber QUAL cota do melhor caso "sobe" para o algoritmo inteiro. A resposta é "sim, é Ω(1), porque nenhuma entrada custa menos que o melhor caso" — com dois complementos: NÃO se pode concluir que o algoritmo é O(1) (o pior caso é Θ(lg n)), e, pelo mesmo motivo, também NÃO se pode dizer que ele é Ω(lg n) (a entrada de melhor caso custa 6).

RESUMO T_melhor ∈ Θ(1) c1 = c2 = 6, n0 = 1 entradas de melhor caso algoritmo ∈ Ω(1) c = 1, n0 = 1 toda entrada algoritmo ∈ O(1) FALSO falha no pior caso T_pior ∈ Θ(lg n) c1 = 5, c2 = 14, n0 = 2 pior caso (ex.: x > A[n]) algoritmo ∈ O(lg n) c = 14, n0 = 2 toda entrada

Conferência: o modelo acima foi executado por força bruta para n = 1..299 com todos os x possíveis (presentes e ausentes, entre e fora dos elementos): o mínimo foi 6 e o máximo 5⌊lg n⌋ + 9 em todos os n, e x > A[n] sempre atinge o máximo. O caso n = 0 fica fora por n0 = 1 (nele o custo seria 4, o que não altera nada).

---

13. Campello, ex. 4

Exercícios Adicionais

Qual é o maior tamanho de entrada n de um problema P tal que P pode ser resolvido em tempo t se o algoritmo para resolver P leva f(n) micro-segundos? Responda em uma tabela com todas as combinações de t = 1s, 1hora, 1mês, 1 século e f(n) = log(n), n, n*log(n), n^2, n^3, 2^n. Relacione os resultados obtidos com as discussões em aula
[transcrição literal — Campello, Análise de Algoritmos II 32]

Resolução

Resposta. Tabela abaixo. Destaque: para f(n) = log n o limite não é o tempo (nenhuma entrada com n elementos que se possa de fato armazenar chega perto do maior n, que é 2^t); para f(n) = 2^n, passar de 1 segundo para 1 século SOMA apenas 32 ao maior n resolvível (19 -> 51).

Modelo: o algoritmo gasta f(n) micro-segundos, e dispomos de t de tempo. O maior n resolvível é o maior inteiro n tal que f(n) <= t, com t convertido para micro-segundos.

Conversões (1 s = 10^6 us; mês = 30 dias; século = 100 anos de 365 dias): 1 segundo = 10^6 us 1 hora = 3600 * 10^6 = 3,6 * 10^9 us 1 mês = 30 * 24 * 3600 * 10^6 = 2,592 * 10^12 us 1 século = 100 * 365 * 24 * 3600 * 10^6 = 3,1536 * 10^15 us

Logaritmos na base 2 (lg).

f(n) | 1 segundo | 1 hora | 1 mês | 1 século ---------- + ------------- + -------------- + ------------------ + ---------------------- log n | 2 ^ (10 ^ 6) | 2 ^ (3,6*10 ^ 9) | 2 ^ (2,592*10 ^ 12) | 2 ^ (3,1536*10 ^ 15) n | 1.000.000 | 3.600.000.000 | 2.592.000.000.000 | 3.153.600.000.000.000.000 n log n | 62.746 | 133.378.058 | 71.870.856.404 | 68.610.956.750.570 n ^ 2 | 1.000 | 60.000 | 1.609.968 | 56.156.922 n ^ 3 | 100 | 1.532 | 13.736 | 146.645 2 ^ n | 19 | 31 | 41 | 51

(*) log n: resolver lg n <= 10 ^ 6 dá n <= 2 ^ (10 ^ 6), um número com cerca de 301.030 algarismos decimais — muito maior que o número de átomos do universo observável. Ou seja: o maior n existe e vale exatamente 2 ^ t, com t medido em microssegundos — mas é um número com da ordem de 3·10 ^ 5 a 9,5·10 ^ 14 algarismos decimais. Escrever o número n ainda é possível (2 ^ (10 ^ 6) ocupa cerca de 125 KB); o que é fisicamente impossível é uma ENTRADA com n elementos. Para f(n) = lg n, portanto, nenhuma entrada que se possa de fato armazenar chega perto do limite: o gargalo passa a ser armazenar ou ler a entrada, não o tempo de f(n).

Como cada célula foi obtida: - f(n) = n: n <= t direto. Ex.: 1 s -> n = 10 ^ 6. - f(n) = n ^ 2: n <= raiz(t). Ex.: 1 h -> raiz(3,6*10 ^ 9) = 60.000 exato. - f(n) = n ^ 3: n <= t ^ (1/3). Ex.: 1 s -> 100 exato; 1 h -> (3,6*10 ^ 9) ^ (1/3) ~ 1532,6 -> 1532. - f(n) = n lg n: não tem forma fechada elementar; resolve-se numericamente a equação n lg n = t. Ex.: lg(62.746) ~ 15,937236, logo 62.746 * lg(62.746) ~ 999.997,8 <= 10 ^ 6; ja 62.747 * lg(62.747) = 1.000.015,2 > 10 ^ 6, o que confirma 62.746 como o maior. (Cuidado ao arredondar: com lg ~ 15,94 o produto daria 1.000.171, acima do limite — o arredondamento para cima inverteria a conclusao.) - f(n) = 2 ^ n: n <= lg t. Ex.: 1 s -> lg(10 ^ 6) ~ 19,93 -> n = 19 (2 ^ 19 = 524.288 <= 10 ^ 6, mas 2 ^ 20 = 1.048.576 > 10 ^ 6).

LEITURA DA TABELA — o argumento sobre intratabilidade. De 1 segundo para 1 século o orçamento de tempo é multiplicado por 3,1536 * 10 ^ 9 (mais de três bilhões de vezes). Veja o efeito em cada linha: - linear: o n resolvível é MULTIPLICADO pelo mesmo fator ~3*10 ^ 9 (de 10 ^ 6 para 3,15*10 ^ 15); - quadrático: multiplicado por raiz(3,1536*10 ^ 9) ~ 56.157 (de 1.000 para 56 milhões); - cúbico: multiplicado por (3,1536*10 ^ 9) ^ (1/3) ~ 1.466 (de 100 para 146.645); - exponencial: SOMADO de 32 unidades apenas (de 19 para 51), porque n <= lg t e o logaritmo transforma o fator 3*10 ^ 9 em lg(3,1536*10 ^ 9) ~ 31,55.

Esse contraste é a essência da noção de intratabilidade: num algoritmo polinomial, ganhar hardware (ou tempo) mais rápido por um fator k aumenta o tamanho tratável por um fator de cerca de k ^ (1/grau) (exato, a menos de pisos, para f(n) = n ^ grau; para um polinômio com termos de ordem menor, só para t grande) — um ganho MULTIPLICATIVO. Num algoritmo exponencial b ^ n, o mesmo fator k só acrescenta cerca de log_b k ao tamanho tratável (lg k para 2 ^ n) — um ganho ADITIVO e minúsculo. Por isso um problema exponencial não é resolvido esperando o computador melhorar: mesmo um século de computação só leva de 19 para 51 elementos de entrada.

Conferência: a linha log n é analítica (n = 2 ^ t); as demais células foram obtidas por busca binária sobre n verificando f(n) <= t, e os valores de fronteira foram checados nos dois lados (o n listado cabe, o n+1 estoura). Confirmado.

---

14. Campello, ex. 5

Exercícios Adicionais

Qual a complexidade assintótica de **pior caso** para o tempo de execução do algoritmo de busca binária iterativo visto em aula? Justifique formalmente usando a definição do operador BIG-O
[transcrição literal — Campello, Análise de Algoritmos II 32]

Resolução

Resposta. O pior caso da busca binária iterativa é Theta(lg n); em particular, T_pior(n) = O(lg n), com testemunhas c = 2a + b e n0 = 2 (a e b são as constantes de custo por iteração e de custo fixo).

ALGORITMO. Busca binária iterativa em um vetor ORDENADO A[1..n] procurando x: mantém-se um intervalo [esq, dir], inicialmente [1, n]; enquanto esq <= dir, calcula-se meio = piso((esq+dir)/2), compara-se x com A[meio] e ou se devolve meio, ou se descarta metade do intervalo (esq = meio+1, ou dir = meio-1). Se o laço terminar, devolve-se 'não encontrado'.

PASSO 1 — quantas iterações o laço executa no pior caso. Seja n_k o número de elementos do intervalo de busca no INÍCIO da iteração k+1 (portanto, no início, o tamanho é n). Numa iteração, o elemento do meio é descartado junto com uma das metades, então n_{k+1} <= piso(n_k / 2) <= n_k / 2. Por indução simples em k, n_k <= n / 2 ^ k para todo k >= 0. (Base: com k = 0, o tamanho é n = n/2 ^ 0. Passo: se n_k <= n/2 ^ k, então n_{k+1} <= n_k/2 <= n/2 ^ (k+1).)

O laço só executa mais uma iteração enquanto o intervalo é não vazio, isto é, enquanto n_k >= 1. Se k > lg n, então 2 ^ k > n e portanto n_k <= n/2 ^ k < 1, ou seja, n_k = 0 e o laço já parou. Logo o número de iterações é no máximo piso(lg n) + 1. Essa cota é ATINGIDA (a menos de 1) quando x não está no vetor: nenhuma comparação acerta, e o algoritmo é obrigado a reduzir o intervalo até esvaziá-lo. Por isso o pior caso realmente cresce como lg n, e não menos.

PASSO 2 — custo total. Cada iteração faz um número constante de operações primitivas (uma soma, uma divisão inteira, uma ou duas comparações, uma atribuição): custo <= a, para alguma constante a > 0. Fora do laço há um custo fixo de inicialização e retorno, <= b, com b > 0. Portanto, para todo n >= 1, T_pior(n) <= a * (piso(lg n) + 1) + b <= a*lg n + a + b.

PASSO 3 — testemunhas formais de O(lg n). Definição: T(n) = O(lg n) se existem c > 0 e n0 > 0 tais que 0 <= T(n) <= c*lg n para todo n >= n0. Tome n0 = 2 e c = 2a + b (a cota inferior, no Passo 4, usa outra letra: c1). Justificativa: para n >= 2 vale lg n >= 1, logo a constante a + b pode ser absorvida, pois a + b <= (a + b)*lg n. Então, para todo n >= 2, T_pior(n) <= a*lg n + (a

+ b) <= a*lg n + (a + b)*lg n = (2a + b)*lg n = c*lg n. E T_pior(n) >= 0 trivialmente. Logo T_pior(n) = O(lg n), com c = 2a + b e n0 = 2, ambos constantes que não dependem de n — respeitando a ordem 'existe c, existe n0, para todo n >= n0'.

PASSO 4 — a cota inferior (por que é Theta e não só O). O enunciado pede O, mas vale registrar que a cota é justa. Para a cota inferior do PIOR caso basta exibir UMA entrada cara: tome x > A[n] (x maior que todos os elementos). Entao toda comparacao manda a busca para a direita, e o novo intervalo tem tamanho piso(n_k/2); como piso(piso(n/2^k)/2) = piso(n/2^(k+1)), apos k iteracoes o tamanho e exatamente piso(n/2^k), que e >= 1 se e somente se 2^k <= n. Logo essa entrada faz exatamente piso(lg n) + 1 iteracoes (conferido para n = 1..10^5). Pelo ramo esquerdo, de tamanho piso((n_k - 1)/2), o intervalo pode esvaziar antes (com n = 4, em 2 iteracoes); por isso a cota usa a entrada x > A[n], e nao 'qualquer x ausente'. Como cada iteracao custa pelo menos a' > 0, temos T_pior(n) >= a'*(piso(lg n) + 1) >= a'*lg n para todo n >= 1 (pois piso(lg n) + 1 > lg n). Testemunhas: c1 = a' e n0 = 1 (em n = 1: a'*1 >= a'*0), logo T_pior(n) = Omega(lg n). Combinando com o Passo 3, T_pior(n) = Theta(lg n).

OBSERVAÇÃO sobre a base do logaritmo: ela é irrelevante em notação assintótica, pois log_B n = lg n / lg B, e 1/lg B é constante positiva para toda base B > 1 — trocar de base só muda a constante c.

Conferência: a recorrência do pior caso, T(1) = 1 e T(n) = T(piso(n/2)) + 1, foi tabulada e comparada com piso(lg n) + 1; os valores coincidem para n de 1 até 2^13, inclusive nos casos de fronteira n = 1, 2, 3. Confirmado.

---

15. Campello, p. 11

Exercício

Mostre que $f(n) = \log_y n$ é $O(\log_x n)$ para x, y > 1, ou seja, a base logarítmica não afeta a complexidade assintótica da função / algoritmo

Contexto do material (transcrição literal, necessária para resolver):
Campello, p. 7 — Notação O (Big-O)

◆ Dadas as funções f(n) e g(n), dizemos que "f(n) é O(g(n))" se existirem as *constantes positivas* c (real) e n_0 (inteira) tais que:

- $f(n) \leq cg(n)$ para $n \geq n_0$

◆ Exemplo: 2n + 10 é O(n)

- $2n + 10 \leq cn$
- $cn - 2n \geq 10$
- $(c - 2)\, n \geq 10$
- $n \geq 10/(c - 2)$
- Pegue $c = 3$ e $n_0 = 10$

[Gráfico em escalas logarítmicas com 3n, 2n+10 e n, para n de 1 a 1,000.]

Campello, p. 9 (trecho que usa log): "$3\log n + 5$ é $O(\log n)$ — É preciso $c > 0$ e $n_0 \geq 1$ tais que $3\log n + 5 \leq c \log n$ para todo $n \geq n_0$ — Note que $3\log n + 5 \leq (3 + 5) \log n$ se $n > 1$ (pois $\log 1 = 0$) — Logo, basta tomar, por exemplo, $c = 8$ e $n_0 = 2$".

[transcrição literal — Campello, p. 11]

Resolução

Resposta. Para x, y > 1 fixos, a mudança de base dá log_y n = (log_y x) · log_x n para todo n ≥ 1. Logo f(n) = log_y n é O(log_x n) com as testemunhas c = log_y x = 1/log_x y (real, positiva porque x, y > 1; depende só de x e y, nunca de n) e n0 = 1. Em n = n0 = 1 os dois lados valem 0 e 0 ≤ 0. A recíproca também vale, com c' = log_x y: log_x n é O(log_y n). Por isso log_y n ∈ Θ(log_x n), com c1 = c2 = log_y x e n0 = 1, e a base do logaritmo não muda a classe assintótica. A prova usa x, y > 1 (que o enunciado fixa) para ter c > 0 e log_x n ≥ 0; a hipótese não pode ser simplesmente apagada: com base 1 o logaritmo não está definido, e com x = 1/2, y = 2 tem-se log_2 n NÃO é O(log_(1/2) n) (PASSO 5).

INTERPRETAÇÃO. Há duas leituras possíveis do enunciado. (A) A afirmação principal: para quaisquer bases fixas x, y > 1, log_y n ∈ O(log_x n), pela definição da p. 7 do Campello: existem c real positiva e n0 inteiro positivo tais que f(n) ≤ c·g(n) para todo n ≥ n0. (B) A paráfrase "ou seja, a base logarítmica não afeta a complexidade assintótica", que só fica justificada se valer também a recíproca, ou seja, Θ. Resolvo (A) com testemunhas explícitas e depois acrescento (B). Em (A), x e y são constantes fixadas antes. A ordem é: para cada par fixo (x, y), ∃c ∃n0 ∀n ≥ n0. Por isso c pode depender de x e de y, mas não de n. O domínio é n inteiro, n ≥ 1, onde log n está definido.

```
PASSO 1 — MUDANÇA DE BASE, A PARTIR DA DEFINIÇÃO DE LOGARITMO:
Para uma base b > 0 com b ≠ 1 e um real z > 0, log_b z é o único real s tal que b^s = z. Esse s é único porque s ↦
b^s é estritamente monótona: crescente se b > 1, decrescente se 0 < b < 1.
Fixe n ≥ 1 e escreva t = log_x n, isto é, n = x^t. Escreva também s = log_y x, isto é, x = y^s. Então
    n = x^t = (y^s)^t = y^(s·t),
e pela unicidade log_y n = s·t. Ou seja,
    log_y n = (log_y x) · log_x n    para todo n ≥ 1.          (1)
Fazendo n = y em (1): 1 = log_y y = (log_y x)(log_x y). Logo log_y x = 1/log_x y, e as duas formas de escrever c
coincidem.
```

PASSO 2 — O SINAL DA CONSTANTE: Como y > 1, a função s ↦ y^s é estritamente crescente e y^0 = 1. Como x > 1 = y^0, o expoente s = log_y x satisfaz s > 0. Então c = log_y x é um real positivo, como a definição exige. Pela mesma razão (x > 1 e y > 1), log_x y > 0.

```
PASSO 3 — AS TESTEMUNHAS PARA O:
Tome
    c = log_y x = 1/log_x y = (ln x)/(ln y)   e   n0 = 1.
Por (1), para todo n ≥ n0 = 1,
    f(n) = log_y n = c · log_x n ≤ c · log_x n.
A desigualdade é na verdade uma igualdade. Logo log_y n ∈ O(log_x n).
Conferência em n = n0 = 1: o lado esquerdo é log_y 1 = 0, o lado direito é c·log_x 1 = c·0 = 0, e 0 ≤ 0 vale. Ao
contrário do exemplo da p. 9 (3 log n + 5), aqui não há termo aditivo, então não é preciso fugir de n = 1: n0 = 1
basta. Qualquer c maior e qualquer n0 ≥ 1 também servem, porque log_x n ≥ 0 para n ≥ 1.
Exemplo numérico, com x = 2 e y = 10: c = log_10 2 ≈ 0,30103 e n0 = 1. Em n = 1: 0 ≤ 0. Em n = 8: log_10 8 ≈
0,90309 e c·log_2 8 = 0,30103·3 ≈ 0,90309, igualdade como (1) prevê. Com x = 10 e y = 2, c = log_2 10 ≈ 3,32.
```

```
PASSO 4 — A RECÍPROCA E O θ:
O mesmo argumento com os papéis de x e y trocados também vale, já que a hipótese é simétrica (x, y > 1). Ele dá
log_x n = (log_x y) · log_y n, com c' = log_x y > 0 e n0 = 1. Logo log_x n ∈ O(log_y n): a base nova também não
"cresce mais" que a antiga.
Juntando as duas direções na definição de θ usada na disciplina (CLRS, 3ª ed.: existem c1, c2 > 0 e n0 tais que 0
≤ c1·g(n) ≤ f(n) ≤ c2·g(n) para todo n ≥ n0), com f(n) = log_y n e g(n) = log_x n, tome
    c1 = c2 = log_y x   e   n0 = 1.
Para n ≥ 1 vale log_x n ≥ 0, então 0 ≤ c1·log_x n, e por (1) c1·log_x n = log_y n = c2·log_x n. Em n = 1, tudo
vale 0. Logo log_y n ∈ θ(log_x n). É isso que justifica a frase "a base logarítmica não afeta a complexidade as-
sintótica": funções não negativas (para n ≥ n0) que diferem por um fator constante positivo estão na mesma classe
θ. Por isso se escreve O(log n) sem base. Só a direção O, sozinha, não bastaria para essa frase: n ∈ O(n²), e mes-
mo assim n e n² têm complexidades diferentes.
```

PASSO 5 — ONDE A HIPÓTESE x, y > 1 ENTRA: O enunciado fixa x, y > 1, e a prova usa isso em dois pontos: c = log_y x > 0 (PASSO 2), que a definição exige, e log_x n ≥ 0 para n ≥ 1 (PASSO 4), que dá o 0 ≤ c1·g(n) do Θ. Não analiso aqui todas as bases fora de (1, ∞), porque o veredito delas depende de qual definição se usa (a do Campello não exige 0 ≤ f; a do CLRS exige) e porque logaritmo negativo não serve como custo de algoritmo. Dois fatos bastam para ver que a hipótese não pode ser simplesmente apagada: (i) Base 1: 1 ^ s = 1 para todo s, então log_1 n não está definido (para n ≥ 2 não existe s com 1 ^ s = n; para n = 1 todo s serve). (ii) Contraexemplo concreto, pela definição de O do material: y = 2 e x = 1/2. Então log_x n = −log_2 n. Afirmação: log_2 n NÃO é O(log_(1/2) n). Negação da definição: para toda c > 0 e todo n0 ≥ 1, existe n ≥ n0 com log_2 n > c·log_(1/2) n. Tome n = max(n0, 2). Então log_2 n ≥ 1 > 0 e c·log_(1/2) n = −c·log_2 n < 0. A desigualdade f(n) ≤ c·g(n) falha nesse n, qualquer que seja a escolha de c e n0.

Conferência numérica (Python; não faz parte da prova): para (x, y) = (2, 10), (10, 2), (2, 3), (3, 2) e (1,5; 100), que log_y n ≤ c·log_x n com c = 1/log_x y vale para todo n de 1 a 10^4, e que em n = 1 os dois lados valem 0. Os valores de c calculados batem com c = log_y x: 0,3010 = log_10 2, 3,3219 = log_2 10, 0,6309 = log_3 2, 1,5850 = log_2 3 e 0,0880 = log_100 1,5. Isso é coerente com a igualdade (1), que torna a desigualdade exata em todo n, e com a escolha n0 = 1.

---

16. Campello, p. 22

Exercício

Faça uma análise **detalhada** do no. de operações primitivas computadas por cada um dos algoritmos anteriores para cálculo da média de prefixo e mostre o número exato de operações executadas por cada um deles. Em seguida, prove usando a **definição** que Prefix1 é $O(n^2)$ e Prefix2 é O(n). Discuta se existem diferenças entre pior caso, melhor caso e caso médio desses algoritmos. Justifique

Contexto do material (transcrição literal, necessária para resolver):
Campello, p. 18 — Exemplo (Média de Prefixo)

◆ Vamos ilustrar a análise assintótica com dois algoritmos que fazem o cálculo da **média de prefixos**

◆ A i-**ésima média de prefixo** de um arranjo X é a média dos (i + 1) primeiros elementos de X:

$$A[i] = (X[0] + X[1] + \ldots + X[i]) / (i+1)$$

◆ O cálculo do vetor A de médias de prefixo de um outro vetor X possui aplicações em análise financeira

[Gráfico de barras: X e A para i = 0…6; eixo de 0 a 35.]

Campello, p. 19 — Exemplo (Média de Prefixo)

◆ O algoritmo abaixo calcula médias de prefixos em **tempo quadrático**, aplicando a definição:

```
Algoritmo Prefix1(X, n)
    Entrada: vetor X de n inteiros
    Saída: vetor A de médias de prefixos de X        #operações
    A ← novo vetor de n inteiros
    para i ← 0 até n − 1 faça                              ~ n
        s ← 0                                              ~ n
        para j ← 0 até i faça                    ~ 1 + 2 + …+ n
            s ← s + X[j]                         ~ 1 + 2 + …+ n
        A[i] ← s / (i + 1)                                 ~ n
    retorne A                                              1
```

Campello, p. 20 — Exemplo (Média de Prefixo)

◆ O tempo de execução de Prefix1 é O(1 + 2 + …+ n)

◆ A soma dos primeiros n inteiros é n(n + 1) / 2 (prog. aritmética):

- Pode-se provar graficamente
- Área em laranja ao lado é dada por $n^2/2 + n \cdot ½$

◆ Portanto, Prefix1 executa em tempo $O(n^2)$

[Figura: escada de 1 a 6 sobre grade 6 × 6, com a diagonal; colunas em amarelo/laranja abaixo, faixas azuis acima.]

Campello, p. 21 — Exemplo (Média de Prefixo)

◆ O algoritmo abaixo calcula a média de prefixos em **tempo linear**, varrendo e somando uma só vez:

```
Algoritmo Prefix2(X, n)
    Entrada: vetor X de n inteiros
    Saída: vetor A de médias de prefixos de X       #operações
    A ← novo vetor de n inteiros
    s ← 0                                             1
    para i ← 0 até n − 1 faça                        ~ n
        s ← s + X[i]                                 ~ n
        A[i] ← s / (i + 1)                           ~ n
    retorne A                                         1
```

Campello, p. 15 — Análise Assintótica de Algoritmos (única regra de contagem presente neste PDF)

◆ A análise assintótica de um algoritmo determina o seu tempo de execução em notação Big-O

◆ Para fazer a análise assintótica:

- Encontra-se o no. de operações primitivas executado pelo algoritmo no pior caso (em função do tamanho da entrada)
- Escreve-se essa função com notação Big-O

◆ Exemplo:

- Nós determinamos anteriormente que o algoritmo arrayMax executa no máximo 8n − 2 operações primitivas
- Então, dizemos que o algoritmo arrayMax "executa em tempo O(n)" ou "possui **complexidade assintótica linear**"

Campello, p. 7 — definição de Big-O: ver o contexto de "Campello, p. 11" ($f(n) \leq cg(n)$ para $n \geq n_0$, c real positiva, n_0 inteira positiva).

[transcrição literal — Campello, p. 22]

Resolução

Resposta. No modelo de custo M declarado abaixo (cada operação primitiva custa 1), Prefix1 executa exatamente $T1(n) = 3n^2 + 13n + 4$ operações e Prefix2 executa exatamente $T2(n) = 10n + 5$ operações, para todo $n \geq 0$ e para todo vetor X de tamanho n. Pela definição: $T1(n) \leq 20 \cdot n^2$ para todo $n \geq 1$ (c = 20, n0 = 1), logo Prefix1 é $O(n^2)$; $T2(n) \leq 15 \cdot n$ para todo $n \geq 1$ (c = 15, n0 = 1), logo Prefix2 é O(n). As cotas são justas: $T1 \in \Theta(n^2)$ (c1 = 3, c2 = 20, n0 = 1), $T2 \in \Theta(n)$ (c1 = 10, c2 = 15, n0 = 1), e T1 não é O(n). Não há diferença entre pior caso, melhor caso e caso médio: em cada algoritmo, os laços têm limites que dependem só de n e não há desvio condicional que dependa do conteúdo de X, de modo que todas as entradas de mesmo tamanho n executam exatamente a mesma sequência de operações; os três casos coincidem com T1(n) e com T2(n), respectivamente.

INTERPRETAÇÃO DO ENUNCIADO. "Os algoritmos anteriores para cálculo da média de prefixo" são Prefix1 (p. 19) e Prefix2 (p. 21). O enunciado pede quatro coisas: (i) uma análise detalhada, linha a linha, do número de operações primitivas; (ii) o número EXATO de operações de cada algoritmo; (iii) prova pela definição de que Prefix1 é $O(n^2)$ e Prefix2 é O(n); (iv) discussão de pior caso, melhor caso e caso médio, com justificativa. Leituras possíveis de (ii): (a) contar só quantas vezes cada linha executa, como faz a coluna "#operações" do slide; (b) contar operações primitivas com um modelo de custo explícito. Como o enunciado diz "número exato de operações", e a coluna do slide usa "∼" (aproximado), resolvo a leitura (b), mas parto da (a): primeiro conto quantas vezes cada linha executa (isso não depende de modelo), depois multiplico pelo custo de cada linha no modelo declarado. O material presente não traz a regra de contagem de operações primitivas (a p. 15 só diz "encontra-se o no. de operações primitivas ... no pior caso"), então o modelo precisa ser declarado; com outro modelo as constantes mudam, mas as contagens de execução por linha da seção seguinte não mudam.

```
MODELO DE CUSTO M (declarado).
Custa 1 cada: atribuição, operação aritmética (+, /), comparação, acesso indexado a vetor (X[j], A[i]), retorno, e
a alocação "A ← novo vetor de n inteiros" (a linha inteira custa 1). Um laço "para v ← a até b faça" custa: 1 pela
inicialização v ← a; 1 por teste v ≤ b (o limite b é tratado como valor já disponível, e o teste custa 1); e 2 por
incremento v ← v + 1 (uma soma e uma atribuição). Se o corpo executa k = b − a + 1 vezes (k ≥ 0), o laço faz k in-
crementos e k + 1 testes (o último teste é o que falha), logo o controle do laço custa 1 + (k + 1) + 2k = 3k + 2.
Custo de cada linha de corpo nesse modelo:
    s ← 0               : 1 (atribuição)
    s ← s + X[j]        : 3 (acesso X[j], soma, atribuição)
    s ← s + X[i]        : 3 (acesso X[i], soma, atribuição)
    A[i] ← s / (i + 1)  : 4 (soma i + 1, divisão, acesso indexado A[i], atribuição)
    retorne A           : 1
```

PASSO 1 — DE ONDE VEM CADA "~" DA TABELA DE PREFIX1 (quantas vezes cada linha executa).

```
    1: A ← novo vetor de n inteiros          executa 1 vez
    2: para i ← 0 até n − 1 faça             corpo executa n vezes; n + 1 testes; n incrementos
    3:     s ← 0                             executa n vezes (uma por valor de i)
    4:     para j ← 0 até i faça             para cada i, corpo executa i + 1 vezes; i + 2 testes
    5:         s ← s + X[j]                  executa Σ_{i=0}^{n−1} (i + 1) = 1 + 2 + … + n vezes
    6:     A[i] ← s / (i + 1)                executa n vezes
    7: retorne A                             executa 1 vez
```

O "~ n" da linha 2 é porque o teste do laço externo acontece n + 1 vezes, não n: o slide despreza o teste final. O "~ n" das linhas 3 e 6 é na verdade exato em número de execuções (n vezes cada); o "~" esconde só o custo de cada execução (1 e 4 operações em M). O "~ 1 + 2 + … + n" da linha 5 é exato em execuções; o da linha 4 é aproximado, porque o laço interno, para um i fixo, testa i + 2 vezes, e no total testa $\Sigma_{i=0}^{n-1} (i + 2) = (1 + 2 + \dots + n) + n$ vezes. Pela progressão aritmética (p. 20), $1 + 2 + \dots + n = n(n + 1)/2$; a "área em laranja" $n^2/2 + n \cdot \frac{1}{2}$ do slide é exatamente esse valor.

PASSO 2 — PREFIX1 EXATO NO MODELO M.

Linha 1: 1.
Linha 2 (controle do laço externo, k = n): 3n + 2.
Linha 3: n execuções × 1 = n.
Linha 4 (controle do laço interno, para cada i com k = i + 1): 3(i + 1) + 2 = 3i + 5; somando em i = 0, …, n − 1: $3 \cdot \Sigma_{i=0}^{n-1} i + 5n = 3 \cdot n(n - 1)/2 + 5n$.
Linha 5: n(n + 1)/2 execuções × 3 = 3·n(n + 1)/2.
Linha 6: n execuções × 4 = 4n.
Linha 7: 1.
Total:

$$T1(n) = 1 + (3n + 2) + n + [3n(n - 1)/2 + 5n] + 3n(n + 1)/2 + 4n + 1.$$

As duas frações somam $3n(n - 1)/2 + 3n(n + 1)/2 = (3/2) \cdot (n^2 - n + n^2 + n) = 3n^2$. Os termos lineares somam 3n + n + 5n + 4n = 13n. As constantes somam 1 + 2 + 1 = 4. Logo

$$T1(n) = 3n^2 + 13n + 4, \text{ para todo } n \geq 0.$$

Checagem em n = 0 (laço externo não executa): alocação 1 + inicialização i ← 0 1 + teste que falha 1 + retorno 1 = 4 = T1(0). Checagem em n = 1: linha 1: 1; linha 2: 3·1 + 2 = 5; linha 3: 1; linha 4 (i = 0): 5; linha 5: 3; linha 6: 4; linha 7: 1; soma 20 = 3 + 13 + 4 = T1(1).

PASSO 3 — DE ONDE VEM CADA "~" DA TABELA DE PREFIX2, E A CONTAGEM EXATA.

```
    1: A ← novo vetor de n inteiros          executa 1 vez                    custo 1
    2: s ← 0                                 executa 1 vez                    custo 1
    3: para i ← 0 até n − 1 faça             corpo n vezes; n + 1 testes      custo 3n + 2
    4:     s ← s + X[i]                      executa n vezes                  custo 3n
    5:     A[i] ← s / (i + 1)                executa n vezes                  custo 4n
    6: retorne A                             executa 1 vez                    custo 1
```

O "~ n" da linha 3 esconde de novo o teste final (n + 1 testes) e os incrementos; os "~ n" das linhas 4 e 5 são exatos em execuções e escondem o custo por execução (3 e 4). Total:

$$T2(n) = 1 + 1 + (3n + 2) + 3n + 4n + 1 = 10n + 5, \text{ para todo } n \geq 0.$$

Checagem: n = 0 dá 1 + 1 + 2 + 1 = 5 = T2(0); n = 1 dá 1 + 1 + 5 + 3 + 4 + 1 = 15 = T2(1).
A diferença essencial está na linha 5 de Prefix1: ela refaz, para cada i, a soma X[0] + … + X[i] desde o início, o que dá 1 + 2 + … + n execuções; Prefix2 mantém em s a soma do prefixo anterior e faz uma única soma por i.

PASSO 4 — PREFIX1 É $O(n^2)$, PELA DEFINIÇÃO.

Definição (Campello): f(n) é O(g(n)) se existem uma constante real c > 0 e um inteiro n0 ≥ 1 tais que $f(n) \leq c \cdot g(n)$ para todo n ≥ n0. Ordem dos quantificadores: ∃c ∃n0 ∀n ≥ n0.
Tome c = 20 e n0 = 1 (escolhidos antes de n, sem depender de n). Para todo inteiro n ≥ 1 vale $n \leq n^2$ e $1 \leq n^2$, logo $13n \leq 13n^2$ e $4 \leq 4n^2$. Somando a $3n^2 \leq 3n^2$:

$$T1(n) = 3n^2 + 13n + 4 \leq 3n^2 + 13n^2 + 4n^2 = 20n^2, \text{ para todo } n \geq 1.$$

Em n = n0 = 1 exatamente: T1(1) = 20 e $20 \cdot 1^2 = 20$; 20 ≤ 20 vale (com igualdade). Portanto Prefix1 é $O(n^2)$.

PASSO 5 — PREFIX2 É O(n), PELA DEFINIÇÃO.

Tome c = 15 e n0 = 1. Para todo inteiro n ≥ 1 vale 5 ≤ 5n, logo

$$T2(n) = 10n + 5 \leq 10n + 5n = 15n, \text{ para todo } n \geq 1.$$

Em n = n0 = 1 exatamente: T2(1) = 15 e 15·1 = 15; 15 ≤ 15 vale. Portanto Prefix2 é O(n).

PASSO 6 — AS COTAS SÃO JUSTAS (Θ) E PREFIX1 NÃO É O(n).

O enunciado só pede O, mas O é cota superior e não diz que Prefix1 é pior que Prefix2; para isso é preciso a cota inferior.
Prefix1 ∈ $\Theta(n^2)$: para todo n ≥ 1, 13n + 4 ≥ 0, logo $T1(n) \geq 3n^2$. Com c1 = 3, c2 = 20, n0 = 1: $3n^2 \leq T1(n) \leq 20n^2$ para todo n ≥ 1. Em n = 1: 3 ≤ 20 ≤ 20.
Prefix2 ∈ Θ(n): para todo n ≥ 1, $10n \leq 10n + 5 \leq 15n$. Com c1 = 10, c2 = 15, n0 = 1. Em n = 1: 10 ≤ 15 ≤ 15.
Prefix1 não é O(n). Negação da definição: para toda constante c > 0 e todo inteiro n0 ≥ 1, existe n ≥ n0 com T1(n) > c·n. Dados c e n0 quaisquer, tome $n = \max(n0, \lceil c/3 \rceil + 1)$. Então n ≥ n0 e n > c/3 (pois $n \geq \lceil c/3 \rceil + 1 > c/3$), logo 3n > c, e como n ≥ 1:

$$T1(n) = 3n^2 + 13n + 4 > 3n^2 = (3n) \cdot n > c \cdot n.$$

Exemplo: c = 100, n0 = 1 dá $n = \max(1, 34 + 1) = 35$, e $T1(35) = 3 \cdot 1225 + 455 + 4 = 4134 > 3500 = 100 \cdot 35$. Assim nenhuma escolha de c e n0 serve, e Prefix1 está em $\Theta(n^2)$ mas não em O(n): sua contagem cresce estritamente mais rápido que a de Prefix2.

```
PASSO 7 — PIOR CASO, MELHOR CASO E CASO MÉDIO.
Fixe n. Em Prefix1, o laço externo executa exatamente n vezes e, para cada i, o laço interno executa exatamente i
+ 1 vezes: esses números dependem só de n e de i, nunca dos valores X[0], …, X[n − 1]. Não há nenhum "se" nos al-
goritmos, e nenhuma linha é pulada ou repetida conforme o conteúdo de X. O mesmo vale para Prefix2, cujo único
laço executa exatamente n vezes. Logo, para cada n, todas as entradas de tamanho n produzem exatamente a mesma
sequência de operações, e no modelo M (custo 1 por operação, independente do valor dos operandos):
    pior caso de Prefix1 = melhor caso = caso médio = 3n² + 13n + 4;
    pior caso de Prefix2 = melhor caso = caso médio = 10n + 5.
O caso médio coincide com os outros dois para qualquer distribuição de probabilidade sobre as entradas, porque a
média de uma quantidade constante é essa constante. Em consequência, as afirmações dos PASSOS 4 a 6 valem para
toda entrada, não só para o pior caso: Prefix1 é θ(n²) e Prefix2 é θ(n) em pior, melhor e caso médio. Ressalva de
escopo: essa igualdade dos três casos é afirmada dentro do modelo de custo unitário M (o da disciplina); num mode-
lo que cobrasse aritmética pelo número de bits dos operandos, o custo de cada soma passaria a depender dos valores
de X, e a discussão teria de ser refeita.
```

Conferência numérica (Python; não faz parte da prova): uma implementação instrumentada no mesmo modelo M contou $T1(n) = 3n^2 + 13n + 4$ e $T2(n) = 10n + 5$ para n = 0, …, 300, com 3 vetores diferentes para cada n, e obteve a mesma contagem independentemente do conteúdo — o que bate com as fórmulas dos PASSOS 2 e 3 e com a igualdade dos três casos do PASSO 7. Confirmou também as testemunhas: $3n^2 + 13n + 4 \le 20n^2$ para $n \ge 1$, com igualdade $20 \le 20$ em n = 1, e $10n + 5 \le 15n$ para $n \ge 1$, com $15 \le 15$ em n = 1. A cota inferior (Θ) e a refutação de "Prefix1 é O(n)" não entram nessa contagem; estão provadas algebricamente no PASSO 6.

Uber / Barbosa — Crescimento de Funções (UEM) · notação assintótica

17. Uber (UEM), p. 8

Notação assintótica

Exemplos

Responda utilizando a definição na notação O, se cada expressão a seguir é verdadeira ou falsa.

1. $100n^2 = O(n^2)$
2. $\frac{1}{2}n^2 - 3n = O(n^2)$
3. $3n + 20 = O(n^2)$
4. $6n^3 = O(n^2)$
5. $720 = O(1)$

> **Contexto do material** (transcrição literal, necessária para resolver):
> [Definição — Uber (UEM), p. 6 do PDF, slide 4/33]
>
> **Notação assintótica**
>
> *Notação O*
>
> Para uma dada função g(n), denotamos por O(g(n)) o conjunto de funções
>
> $O(g(n)) = \{f(n)$: existem constantes positivas c e n_0 tais que $0 \le f(n) \le cg(n)$ para todo $n \ge n_0\}$
>
> A notação O descreve um limite assintótico superior.

[transcrição literal — Uber (UEM), p. 8]

> Resolução
>
> **Resposta.** (1) $100n^2 = O(n^2)$ é VERDADEIRA (c = 100, n0 = 1). (2) $\frac{1}{2}n^2 - 3n = O(n^2)$ é VERDADEIRA (c = ½, n0 = 6; nenhum n0 ≤ 5 serve, porque a definição exige $0 \le f(n)$ e $f(n) < 0$ para $1 \le n \le 5$ — entre os naturais, 6 é o menor n0). (3) $3n + 20 = O(n^2)$ é VERDADEIRA (c = 23, n0 = 1). (4) $6n^3 = O(n^2)$ é FALSA: para quaisquer c > 0 e n0 > 0, o natural $n = \text{máx}(\lceil n0 \rceil, \lfloor c/6 \rfloor + 1)$ satisfaz $n \ge n0$ e $6n^3 > c \cdot n^2$. (5) $720 = O(1)$ é VERDADEIRA (c = 720, n0 = 1).
>
> **INTERPRETAÇÃO.** Leituras possíveis de "f(n) = O(g(n))": (i) como abuso de notação para $f(n) \in O(g(n))$, o conjunto definido no slide 4 do material; (ii) como igualdade literal entre uma função e um conjunto, que não faz sentido. Resolvo a leitura (i), que é a convenção do próprio material ("denotamos por O(g(n)) o conjunto de funções"). O enunciado pede "utilizando a definição", então para cada item verdadeiro exibo constantes numéricas c e n0 e confiro $0 \le f(n) \le c \cdot g(n)$ para todo $n \ge n0$, inclusive em n = n0 exatamente; para o item falso nego a definição. Tomo n natural ($n \ge 1$), domínio usual de tamanho de entrada; os argumentos abaixo valem também para n real nos intervalos indicados.
>
> A definição, com a ordem dos quantificadores explícita: $f(n) \in O(g(n))$ se e somente se existem c > 0 e n0 > 0 tais que, para todo $n \ge n0$, $0 \le f(n) \le c \cdot g(n)$. Sua negação: para todo c > 0 e todo n0 > 0 existe $n \ge n0$ com $f(n) < 0$ ou $f(n) > c \cdot g(n)$. As constantes c e n0 escolhidas abaixo são números fixos, que não dependem de n.
>
> **ITEM 1.** $100n^2 = O(n^2)$: VERDADEIRA. Testemunhas: c = 100, n0 = 1. Para todo $n \ge 1$: $f(n) = 100n^2 \ge 0$, porque $n^2 > 0$; e $f(n) = 100n^2 = 100 \cdot n^2 = c \cdot g(n)$, logo $f(n) \le c \cdot g(n)$ (vale com igualdade). Conferência em n = n0 = 1: $0 \le 100 \cdot 1^2 = 100 \le 100 \cdot 1^2 = 100$. Verdadeiro.
>
> **ITEM 2.** $\frac{1}{2}n^2 - 3n = O(n^2)$: VERDADEIRA. Testemunhas: c = ½, n0 = 6. Desigualdade da esquerda: $f(n) = \frac{1}{2}n^2 - 3n = \frac{1}{2}n(n - 6)$. Para $n \ge 6$ os dois fatores n e $n - 6$ são ≥ 0, logo $f(n) \ge 0$. Desigualdade da direita: para $n \ge 6$ (na verdade para todo $n \ge 0$), $3n \ge 0$, logo $f(n) = \frac{1}{2}n^2 - 3n \le \frac{1}{2}n^2 = c \cdot g(n)$. Conferência em n = n0 = 6: $f(6) = \frac{1}{2} \cdot 36 - 18 = 18 - 18 = 0$, e $c \cdot g(6) = \frac{1}{2} \cdot 36 =$

18. Logo $0 \le 0 \le 18$. Verdadeiro. Por que nenhum $n0 \le 5$ serve (e 6 é o menor n0 natural): a definição do material exige $0 \le f(n)$. Para $1 \le n \le 5$, $f(n) = \tfrac{1}{2}n(n-6) < 0$ ($n > 0$ e $n - 6 < 0$); por exemplo $f(1) = -2{,}5$ e $f(5) = 12{,}5 - 15 = -2{,}5$. Assim nenhum $n0 \le 5$ serve, qualquer que seja c (inclusive com n0 não inteiro: qualquer $n0 \le 5$ deixa $n = 5 \ge n0$ com $f(5) < 0$). Com $n0 = 6$ a cota inferior vale. (Qualquer $c \ge \tfrac{1}{2}$ também serviria com $n0 = 6$, pois $\tfrac{1}{2}n^2 \le c \cdot n^2$. Escolhi o menor desses.)

ITEM 3. $3n + 20 = O(n^2)$: VERDADEIRA. Testemunhas: $c = 23$, $n0 = 1$. Para todo $n \ge 1$: $f(n) = 3n + 20 > 0$. Além disso, $n \ge 1$ implica $n \le n^2$ (multiplicando $n \ge 1$ por $n > 0$) e $1 \le n^2$. Então $3n + 20 \le 3n^2 + 20n^2 = 23n^2 = c \cdot g(n)$. Conferência em $n = n0 = 1$: $0 \le 3 \cdot 1 + 20 = 23 \le 23 \cdot 1^2 = 23$. Verdadeiro (com igualdade à direita).

```
ITEM 4. 6n³ = O(n²): FALSA.
Negação da definição: devo mostrar que, para todo c > 0 e todo n0 > 0, existe n ≥ n0 com 6n³ > c·n² (a outra meta-
de, f(n) ≥ 0, sempre vale, então a violação tem de ser na cota superior).
Sejam c > 0 e n0 > 0 arbitrários. Tome o natural
    n = máx(⌈n0⌉, ⌊c/6⌋ + 1).
Então n ≥ ⌈n0⌉ ≥ n0, e n ≥ ⌊c/6⌋ + 1 > c/6 (pois ⌊x⌋ + 1 > x para todo real x). De n > c/6 vem 6n > c, e multipli-
cando por n² > 0 (n ≥ 1): 6n³ = 6n·n² > c·n². Logo a desigualdade f(n) ≤ c·g(n) falha nesse n ≥ n0. Como c e n0
eram arbitrários, não existem constantes que satisfaçam a definição, e 6n³ ∉ O(n²).
Exemplo numérico do argumento: c = 1000, n0 = 1 dá n = máx(1, ⌊166,67⌋ + 1) = 167, e 6·167³ = 6·167·167² =
1002·167² > 1000·167².
```

ITEM 5. $720 = O(1)$: VERDADEIRA. Aqui $f(n) = 720$ e $g(n) = 1$ para todo n (funções constantes). Testemunhas: $c = 720$, $n0 = 1$. Para todo $n \ge 1$: $0 \le 720 \le 720 \cdot 1 = c \cdot g(n)$ (igualdade à direita). Conferência em $n = n0 = 1$: $0 \le 720 \le 720$. Verdadeiro. (Também vale com qualquer $c \ge 720$; o n0 é irrelevante porque nada depende de n, e $n0 = 1$ cumpre a exigência de ser positivo.)

OBSERVAÇÃO SOBRE O ALCANCE DAS AFIRMAÇÕES. As afirmações acima são sobre funções de n, não sobre algoritmos; não há pior ou melhor caso envolvido. Cada desigualdade vale para todo $n \ge n0$ indicado, e em nenhum item c ou n0 depende de n.

Conferência numérica (Python; não faz parte da prova): as testemunhas dos itens verdadeiros em todo n de n0 até 20000 — $100n^2 \le 100n^2$ (item 1), $0 \le \tfrac{1}{2}n^2 - 3n \le \tfrac{1}{2}n^2$ com falha exatamente em $n = 1, \ldots, 5$, o que justifica $n0 = 6$ entre os naturais (item 2), $3n + 20 \le 23n^2$ (item 3) e $720 \le 720$ (item 5) — e, no item 4, que $6n^3 \le c \cdot n^2$ exige $n \le c/6$, com $c = 1000$ falhando em $n = 167$, o mesmo n que a fórmula $\max(\lceil n0 \rceil, \lfloor c/6 \rfloor + 1)$ produz com $n0 = 1$.

---

18. Uber (UEM), p. 12

Notação assintótica

Exemplos

Responda utilizando a definição na notação Ω, se cada expressão a seguir é verdadeira ou falsa.

1. $100n^2 = \Omega(n^2)$
2. $\frac{1}{2}n^2 - 3n = \Omega(n^2)$
3. $3n^2 + 20 = \Omega(n)$
4. $6n = \Omega(n^2)$
5. $720 = \Omega(1)$

Contexto do material (transcrição literal, necessária para resolver):
[Definição — Uber (UEM), p. 10 do PDF, slide 7/33]

Notação assintótica

Notação Ω

Para uma dada função g(n), denotamos por Ω(g(n)) o conjunto de funções

$\Omega(g(n)) = \{f(n)$: existem constantes positivas c e n_0 tais que $0 \le cg(n) \le f(n)$ para todo $n \ge n_0\}$

A notação Ω descreve um limite assintótico inferior.

[transcrição literal — Uber (UEM), p. 12]

Resolução

Resposta. (1) $100n^2 = \Omega(n^2)$ é VERDADEIRA ($c = 100$, $n0 = 1$). (2) $(1/2)n^2 - 3n = \Omega(n^2)$ é VERDADEIRA ($c = 1/4$, $n0 = 12$). (3) $3n^2 + 20 = \Omega(n)$ é VERDADEIRA ($c = 3$, $n0 = 1$). (4) $6n = \Omega(n^2)$ é FALSA: para quaisquer $c > 0$ e $n0 > 0$, o natural $n = \lceil \max(n0, 6/c + 1) \rceil$ satisfaz $n \ge n0$ e $c \cdot n^2 > 6n$. (5) $720 = \Omega(1)$ é VERDADEIRA ($c = 720$, $n0 = 1$).

LEITURA DO ENUNCIADO. Leituras possíveis: (i) "$f(n) = \Omega(g(n))$" como abuso de notação para "$f(n) \in \Omega(g(n))$", conjunto definido no material (p. 10); (ii) "=" como igualdade de conjuntos, o que não faz sentido, pois o lado esquerdo é uma função. Adoto (i), que é a convenção do próprio material ("denotamos por Ω(g(n)) o conjunto de funções"). O enunciado pede para responder "utilizando a definição", então cada item usa exatamente a definição dada:

$f(n) \in \Omega(g(n)) \iff$ existem constantes positivas c e n0 tais que $0 \le c \cdot g(n) \le f(n)$ para todo $n \ge n0$.

A ordem dos quantificadores é $\exists c\, \exists n0\, \forall n \ge n0$: c e n0 são fixados antes e não podem depender de n. Para VERDADEIRO exibo c e n0 numéricos e provo a desigualdade para todo $n \ge n0$, conferindo em $n = n0$. Para FALSO nego a definição: para todo $c > 0$ e todo $n0 > 0$ existe $n \ge n0$ com $c \cdot g(n) > f(n)$ (a parte $0 \le c \cdot g(n)$ vale sempre aqui, já que $g(n) \ge 0$, logo a única desigualdade que

pode falhar é $c \cdot g(n) \leq f(n)$). Os itens tratam de funções de n (n natural, $n \geq 1$), não de algoritmos, de modo que não há distinção de pior ou melhor caso a fazer.

ITEM 1. $100n^2 = \Omega(n^2)$: VERDADEIRA. Testemunhas: $c = 100$, $n0 = 1$. Para todo $n \geq 1$: $c \cdot g(n) = 100n^2 \geq 0$ e $100n^2 \leq 100n^2$ (igualdade). Logo $0 \leq 100n^2 \leq 100n^2$ para todo $n \geq 1$. Conferência em $n = n0 = 1$: $0 \leq 100 \cdot 1 = 100 \leq 100$. Vale.

ITEM 2. $(1/2)n^2 - 3n = \Omega(n^2)$: VERDADEIRA. Testemunhas: $c = 1/4$, $n0 = 12$. Desigualdade da direita: $(1/4)n^2 \leq (1/2)n^2 - 3n \iff 3n \leq (1/2)n^2 - (1/4)n^2 = (1/4)n^2 \iff 12n \leq n^2 \iff 12 \leq n$ (dividindo por $n > 0$). Logo vale para todo $n \geq 12$. Desigualdade da esquerda: $0 \leq (1/4)n^2$ vale para todo n. Logo $0 \leq (1/4)n^2 \leq (1/2)n^2 - 3n$ para todo $n \geq 12$. Conferência em $n = n0 = 12$: $c \cdot g(12) = 144/4 = 36$; $f(12) = 144/2 - 36 = 72 - 36 = 36$. Então $0 \leq 36 \leq 36$. Vale (com igualdade). Observação: com $c = 1/4$ o $n0 = 12$ é o menor possível, pois para $1 \leq n \leq 11$ tem-se $n^2 < 12n$, isto é, $(1/4)n^2 > (1/2)n^2 - 3n$. Note também que f(n) é negativa para $1 \leq n \leq 5$ e nula em $n = 6$; isso não atrapalha, porque a definição só exige a desigualdade a partir de n0.

ITEM 3. $3n^2 + 20 = \Omega(n)$: VERDADEIRA. Testemunhas: $c = 3$, $n0 = 1$. Para todo $n \geq 1$ vale $n \leq n^2$ (multiplicando $1 \leq n$ por $n > 0$), logo $3n \leq 3n^2 < 3n^2 + 20$. E $0 \leq 3n$. Portanto $0 \leq 3n \leq 3n^2 + 20$ para todo $n \geq 1$. Conferência em $n = n0 = 1$: $0 \leq 3 \leq 3 + 20 = 23$. Vale.

```
ITEM 4. 6n = Ω(n²): FALSA.
Negação da definição: para todo c > 0 e todo n0 > 0 existe n ≥ n0 com c·n² > 6n.
Sejam c > 0 e n0 > 0 arbitrários. Tome n = ⌈máx(n0, 6/c + 1)⌉, que é natural. Então:
    n ≥ máx(n0, 6/c + 1) ≥ n0, logo n ≥ n0;
    n ≥ 6/c + 1 > 6/c, logo c·n > 6 (multiplicando por c > 0);
    como n > 0, multiplicando c·n > 6 por n: c·n² > 6n.
Assim a desigualdade c·n² ≤ 6n falha em um n ≥ n0. Como c e n0 eram arbitrários, não existem constantes positivas
c e n0 que satisfaçam a definição, e 6n ∉ Ω(n²).
Exemplo concreto (não é a prova, só ilustra): c = 0,01 e n0 = 1 dão n = ⌈máx(1, 601)⌉ = 601, e 0,01·601² = 3612,01
> 3606 = 6·601.
```

ITEM 5. $720 = \Omega(1)$: VERDADEIRA. Aqui $f(n) = 720$ e $g(n) = 1$ para todo n. Testemunhas: $c = 720$, $n0 = 1$. Para todo $n \geq 1$: $0 \leq 720 \cdot 1 = 720 \leq 720$. (Qualquer c com $0 < c \leq 720$ também serve, por exemplo $c = 1$.) Conferência em $n = n0 = 1$: $0 \leq 720 \leq 720$. Vale.

Conferência numérica (Python; não faz parte da prova): no item 2, $(1/4)n^2 \leq (1/2)n^2 - 3n$ vale de 12 a 20000 e falha exatamente em $n = 1, \ldots, 11$, o que bate com $n0 = 12$ ser o menor possível para $c = 1/4$ entre os naturais; no item 3, $3n \leq 3n^2 + 20$ vale de $n0 = 1$ a 20000 sem falhas; no item 4, para $c = 0{,}01$ a desigualdade $c \cdot n^2 \leq 6n$ falha em $n = 601$, exatamente o $n = \lceil 6/c + 1 \rceil = 601$ dado pela fórmula do item 4. Os itens 1 e 5 são as igualdades $100n^2 = 100n^2$ e $720 = 720$.

---

19. Uber (UEM), p. 17

Notação assintótica

Exemplos

Responda utilizando a definição na notação Θ, se cada expressão a seguir é verdadeira ou falsa.

1. $100n^2 = \Theta(n^2)$
2. $\frac{1}{2}n^2 - 3n = \Theta(n^2)$
3. $3n^2 + 20 = \Theta(n)$
4. $6n = \Theta(n^2)$
5. $720 = \Theta(1)$

Contexto do material (transcrição literal, necessária para resolver):
[Definição — Uber (UEM), p. 14 do PDF, slide 10/33]

Notação assintótica

Notação Θ

Para uma dada função g(n), denotamos por Θ(g(n)) o conjunto de funções

Θ(g(n)) = {f(n): existem constantes positivas c_1, c_2 e n_0 tais que $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ para todo $n \geq n_0$}

A notação Θ descreve um limite assintótico restrito.

[Também no material, p. 16, slide 11/33 — Teorema 3.1: Para duas funções quaisquer f(n) e g(n), temos que $f(n) = \Theta(g(n))$ se e somente se $f(n) = O(g(n))$ e $f(n) = \Omega(g(n))$.]

[transcrição literal — Uber (UEM), p. 17]

Resolução

Resposta. (1) $100n^2 = \Theta(n^2)$ é VERDADEIRA: $c_1 = c_2 = 100$, $n_0 = 1$. (2) $(1/2)n^2 - 3n = \Theta(n^2)$ é VERDADEIRA: $c_1 = 1/4$, $c_2 = 1/2$, $n_0 = 12$. (3) $3n^2 + 20 = \Theta(n)$ é FALSA: falha a cota superior $f(n) \leq c_2 \cdot n$; para quaisquer c_1, c_2, no positivos, $n = \max(\lceil n_0 \rceil, \lfloor c_2/3 \rfloor + 1)$ a viola. (4) $6n = \Theta(n^2)$ é FALSA: falha a cota inferior $c_1 \cdot n^2 \leq f(n)$; para quaisquer c_1, c_2, no positivos, $n = \max(\lceil n_0 \rceil, \lfloor 6/c_1 \rfloor + 1)$ a viola. (5) $720 = \Theta(1)$ é VERDADEIRA: $c_1 = c_2 = 720$, $n_0 = 1$.

INTERPRETAÇÃO. Leituras possíveis de "$f(n) = \Theta(g(n))$": (i) como igualdade de conjuntos, o que não faz sentido (à esquerda há uma função); (ii) como pertinência $f(n) \in \Theta(g(n))$, que é a convenção do material (CLRS, 3ª ed., seção 3.1, adotada no slide

10/33). Resolvo a leitura (ii). O enunciado pede "utilizando a definição na notação Θ", então cada item é decidido diretamente pela definição do slide 10/33, sem usar limites nem o Teorema 3.1 como atalho:

$\Theta(g(n)) = \{f(n)$: existem constantes positivas c_1, c_2 e n_0 tais que $0 \le c_1 g(n) \le f(n) \le c_2 g(n)$ para todo $n \ge n_0\}$.

A ordem dos quantificadores é $\exists c_1\ \exists c_2\ \exists n_0\ \forall n \ge n_0$. O domínio é n natural, $n \ge 1$. Não há algoritmo nem entradas: cada f é uma função fixa de n, então não cabe falar em pior ou melhor caso. A negação da definição, usada nos itens falsos, é: para TODAS as constantes positivas c_1, c_2, no existe $n \ge n_0$ tal que alguma das desigualdades $0 \le c_1 g(n)$, $c_1 g(n) \le f(n)$, $f(n) \le c_2 g(n)$ é falsa. Em cada item falso exibo esse n em função das constantes e digo qual desigualdade falha.

ITEM 1 — $100n^2 = \Theta(n^2)$: VERDADEIRA. Testemunhas: $c_1 = 100$, $c_2 = 100$, $n_0 = 1$. Para todo $n \ge 1$: $0 \le 100n^2$ ($n^2 > 0$), $100n^2 \le 100n^2$ e $100n^2 \le 100n^2$ (igualdades). Conferência em $n = n_0 = 1$: $0 \le 100 \cdot 1 = 100 \le 100 \le 100$. Vale.

ITEM 2 — $(1/2)n^2 - 3n = \Theta(n^2)$: VERDADEIRA. Seja $f(n) = n^2/2 - 3n$. Testemunhas: $c_1 = 1/4$, $c_2 = 1/2$, $n_0 = 12$. Cota inferior: para $n > 0$, $n^2/4 \le n^2/2 - 3n \iff 3n \le n^2/4 \iff 12 \le n$. Logo vale para todo $n \ge 12$ (e falha para $1 \le n \le 11$, por isso $n_0 = 12$ e não menor). Cota superior: $n^2/2 - 3n \le n^2/2 \iff 0 \le 3n$, verdadeiro para todo $n \ge 1$. Não negatividade: $c_1 n^2 = n^2/4 \ge 0$. Conferência em $n = n_0 = 12$: $c_1 g(12) = 144/4 = 36$; $f(12) = 72 - 36 = 36$; $c_2 g(12) = 72$. Então $0 \le 36 \le 36 \le 72$. Vale (com igualdade na cota inferior).

ITEM 3 — $3n^2 + 20 = \Theta(n)$: FALSA. A desigualdade que falha é a SUPERIOR, $f(n) \le c_2 \cdot n$. Sejam c_1, c_2, n_0 constantes positivas arbitrárias. Tome $n = \max(\lceil n_0 \rceil, \lfloor c_2/3 \rfloor + 1)$. Então $n \ge n_0$ e $n > c_2/3$ (porque $\lfloor x \rfloor + 1 > x$), isto é, $3n > c_2$. Multiplicando por $n > 0$: $3n^2 > c_2 n$, e portanto $f(n) = 3n^2 + 20 > 3n^2 > c_2 n$. Assim, para quaisquer c_1, c_2, n_0 há $n \ge n_0$ com $f(n) > c_2 g(n)$; a definição é negada e $3n^2 + 20 \notin \Theta(n)$. Exemplo numérico: $c_2 = 300$, $n_0 = 1$ dá $n = \max(1, 101) = 101$: $f(101) = 30623 > 300 \cdot 101 = 30300$. (A cota inferior, por outro lado, vale: com $c_1 = 3$, $3n \le 3n^2 + 20$ para todo $n \ge 1$; a função é $\Omega(n)$, mas não $O(n)$, e por isso não é $\Theta(n)$.)

ITEM 4 — $6n = \Theta(n^2)$: FALSA. A desigualdade que falha é a INFERIOR, $c_1 \cdot n^2 \le f(n)$. Sejam c_1, c_2, n_0 constantes positivas arbitrárias. Tome $n = \max(\lceil n_0 \rceil, \lfloor 6/c_1 \rfloor + 1)$. Então $n \ge n_0$ e $n > 6/c_1$, isto é, $c_1 n > 6$. Multiplicando por $n > 0$: $c_1 n^2 > 6n = f(n)$. Assim, para quaisquer c_1, c_2, n_0 há $n \ge n_0$ com $c_1 g(n) > f(n)$; a definição é negada e $6n \notin \Theta(n^2)$. Exemplo numérico: $c_1 = 0{,}01$, $n_0 = 1$ dá $n = \max(1, 601) = 601$: $c_1 n^2 = 0{,}01 \cdot 361201 = 3612{,}01 > 6 \cdot 601 = 3606$. (A cota superior, por outro lado, vale: $6n \le 6n^2$ para $n \ge 1$; a função é $O(n^2)$, mas não $\Omega(n^2)$.)

ITEM 5 — $720 = \Theta(1)$: VERDADEIRA. Aqui $f(n) = 720$ e $g(n) = 1$ são funções constantes de n. Testemunhas: $c_1 = 720$, $c_2 = 720$, $n_0 = 1$. Para todo $n \ge 1$: $0 \le 720 \cdot 1 \le 720 \le 720 \cdot 1$ (igualdades). Conferência em $n = n_0 = 1$: $0 \le 720 \le 720 \le 720$. Vale. (Qualquer $c_1 \le 720 \le c_2$ positivos também servem.)

Conferência numérica (Python; não faz parte da prova): item 1, $100n^2 \le 100n^2$ em todo $n \ge 1$; item 2, $n^2/4 \le n^2/2 - 3n \le n^2/2$ vale de 12 a 20000 e falha exatamente em $n = 1, \ldots, 11$, o que justifica $n_0 = 12$ entre os naturais com $c_1 = 1/4$; item 5, $720 \le 720$; item 3, $3n \le 3n^2 + 20$ em todo $n \ge 1$ (cota inferior) e, com $c_2 = 300$, a cota superior falha já em $n = 100$, onde $30020 > 30000$, e o $n = 101$ da fórmula também falha; item 4, com $c_1 = 0{,}01$ a cota inferior falha em $n = 601$, o mesmo n da fórmula.

Diego Mello — Matemática Discreta (IFMG Formiga) · semana 4

20. IFMG, ex. 3

Exercícios (Para Casa)

Projete um algoritmo recursivo, em pseudo-código, que calcule o valor F_n da sequência de Fibonacci para o valor de n informado.

[transcrição literal — Diego Mello, Matemática Discreta (IFMG), slide 24]

Resolução

Resposta. Algoritmo recursivo direto da definicao: FIBONACCI(n) devolve n se n <= 1 e devolve FIBONACCI(n-1) + FIBONACCI(n-2) caso contrario. Ele esta correto, mas custa tempo exponencial, Theta(phi ^ n) com phi = (1+raiz(5))/2 (aprox. 1,618).

DEFINICAO USADA. A sequencia de Fibonacci e definida por F_0 = 0, F_1 = 1 e F_n = F_{n-1} + F_{n-2} para n >= 2. O algoritmo abaixo e a transcricao literal dessa definicao.

PSEUDO-CODIGO.

```
FIBONACCI(n)
    1: if n = 0 then
    2:     return 0
    3: end if
    4: if n = 1 then
    5:     return 1
    6: end if
    7: return FIBONACCI(n-1) + FIBONACCI(n-2)
```

CORRETUDE (inducao forte sobre n). Seja P(n): FIBONACCI(n) devolve F_n. Base: n = 0 cai na linha 2 e devolve 0 = F_0; n = 1 cai na linha 5 e devolve 1 = F_1. Hipotese forte: suponha que P(k) vale para todo k com 0 <= k < n, para um n >= 2 fixo. Passo: para n >= 2 o algoritmo executa a linha 7. Como n-1 e n-2 estao ambos em [0, n-1], a hipotese forte garante que as chamadas devolvem F_{n-1} e F_{n-2}; a soma e F_{n-1} + F_{n-2} = F_n por definicao. Logo P(n) vale. Terminacao: cada chamada recursiva reduz o argumento em pelo menos 1, e argumentos 0 e 1 param; logo nao ha recursao infinita para n >= 0.

CUSTO. Seja C(n) o numero total de chamadas de FIBONACCI geradas por uma chamada FIBONACCI(n) (contando a propria). Entao

C(0) = C(1) = 1, C(n) = 1 + C(n-1) + C(n-2) para n >= 2.

A solucao em forma fechada e

C(n) = 2*F_{n+1} - 1.

Verificacao por inducao forte: para n = 0, 2*F_1 - 1 = 2*1 - 1 = 1 = C(0); para n = 1, 2*F_2 - 1 = 2*1 - 1 = 1 = C(1). Passo (n >= 2): C(n) = 1 + C(n-1) + C(n-2) = 1 + (2F_n - 1) + (2F_{n-1} - 1) = 2(F_n + F_{n-1}) - 1 = 2F_{n+1} - 1.

Como F_m = (phi^m - psi^m)/raiz(5), com phi = (1+raiz(5))/2 e |psi| = |(1-raiz(5))/2| < 1, tem-se F_m = Theta(phi^m), logo

C(n) = Theta(phi^n), com phi aprox. 1,618.

Testemunhas para a cota inferior exponencial. Usa-se o lema F_m >= 2^((m-2)/2), valido para todo m >= 1 (o lema com expoente (m-1)/2 seria FALSO em m = 2, pois F_2 = 1 < 2^(1/2) = 1,414). Prova do lema, por inducao com DUAS bases (o passo usa F_(m-1) e F_(m-2), entao uma base so nao basta): m = 1 da F_1 = 1 >= 2^(-1/2) = 0,707; m = 2 da F_2 = 1 >= 2^0 = 1. Passo, para m >= 3: F_m = F_(m-1) + F_(m-2) >= 2^((m-3)/2) + 2^((m-4)/2) = 2^((m-4)/2)*(raiz(2) + 1) >= 2^((m-4)/2)*2 = 2^((m-2)/2), pois raiz(2) + 1 = 2,414 > 2. Dele vem F_(n+1) >= 2^((n-1)/2), logo C(n) = 2*F_(n+1) - 1 >= 2*2^((n-1)/2) - 1 = 2^((n+1)/2) - 1. Como 2^((n+1)/2) - 1 >= 2^(n/2) equivale a 2^(n/2)*(raiz(2) - 1) >= 1, isto e n >= 2,54, a desigualdade vale a partir de n = 3. Portanto C(n) = Omega(2^(n/2)) com c = 1 e n0 = 3. Em palavras: o tempo cresce exponencialmente.

POR QUE E TAO CARO. A arvore de recursao recalcula os mesmos subproblemas muitas vezes: FIBONACCI(n-2) e resolvido tanto dentro de FIBONACCI(n-1) quanto diretamente por FIBONACCI(n), e isso se repete em cascata (ver o IFMG, ex. 4). Nao ha memoria entre os ramos.

COMO CORRIGIR (Theta(n)).

(a) Recursao com memorizacao (top-down): guarda-se cada F_k ja calculado num vetor M[0..n] inicializado com NIL.

```
FIB-MEMO(n, M)
    1: if n <= 1 then return n end if
    2: if M[n] <> NIL then return M[n] end if
    3: M[n] <- FIB-MEMO(n-1, M) + FIB-MEMO(n-2, M)
    4: return M[n]
```

Cada indice 0..n e calculado uma unica vez, com O(1) de trabalho proprio: tempo Theta(n), espaco Theta(n).

(b) Versao iterativa (bottom-up), com apenas duas variaveis:

```
FIB-ITER(n)
    1: if n <= 1 then return n end if
    2: a <- 0 ; b <- 1
    3: for i <- 2 to n do
    4:     t <- a + b ; a <- b ; b <- t
    5: end for
    6: return b
```

O laco executa n-1 iteracoes de custo O(1): tempo Theta(n) e espaco Theta(1) (contando operacoes aritmeticas a custo unitario).

CONFERENCIA. Os valores devolvidos coincidem com F_0..F_11 = 0,1,1,2,3,5,8,13,21,34,55,89. O numero de chamadas contado na execucao para n = 0..11 e 1, 1, 3, 5, 9, 15, 25, 41, 67, 109, 177, 287, exatamente igual a 2*F_{n+1} - 1 em todos esses casos.

CONFIANCA: confirmado.

---

21. IFMG, ex. 4

Exercícios (Para Casa)

Desenhe um diagrama de árvore que ilustre as chamadas recursivas do algoritmo acima para $n = 6$.

[transcrição literal — Diego Mello, Matemática Discreta (IFMG), slide 24]

Resolução

Resposta. Arvore de chamadas de FIBONACCI(6): 25 nos no total (= 2*F_7 - 1 = 2*13 - 1), altura 5 arestas (6 niveis de nos). F(4) aparece 2 vezes, F(3) 3 vezes, F(2) 5 vezes, F(1) 8 vezes e F(0) 5 vezes — as repeticoes seguem os proprios numeros de Fibonacci.

Cada no e uma chamada; os dois filhos de F(k) sao F(k-1) (esquerda) e F(k-2) (direita). As folhas sao F(1) e F(0), que devolvem 1 e 0 sem recursao. A arvore, em texto indentado (cada nivel de indentacao = um nivel de profundidade):

```
F(6)
|-- F(5)
|   |-- F(4)
|   |   |-- F(3)
|   |   |   |-- F(2)
|   |   |   |   |-- F(1)   -> 1
|   |   |   |   +-- F(0)   -> 0
|   |   |   +-- F(1)       -> 1
|   |   +-- F(2)
|   |       |-- F(1)       -> 1
|   |       +-- F(0)       -> 0
|   +-- F(3)
|       |-- F(2)
|       |   |-- F(1)       -> 1
|       |   +-- F(0)       -> 0
|       +-- F(1)           -> 1
+-- F(4)
    |-- F(3)
    |   |-- F(2)
    |   |   |-- F(1)       -> 1
    |   |   +-- F(0)       -> 0
    |   +-- F(1)           -> 1
    +-- F(2)
        |-- F(1)           -> 1
        +-- F(0)           -> 0
```

OS TRES PRIMEIROS NIVEIS, resumidos: Nivel 0: F(6) Nivel 1: F(5), F(4) Nivel 2: F(4), F(3) (filhos de F(5)) e F(3), F(2) (filhos de F(4))

SOMA DOS VALORES. Folhas com valor 1 sao as chamadas F(1): ha 8 delas, e 8 = F_6. As folhas F(0) valem 0 e nao contribuem. Como o valor de cada no e a soma dos filhos, o valor da raiz e o numero de folhas F(1), ou seja, F(6) = 8 — coerente com F_6 = 8.

REPETICAO DE SUBPROBLEMAS (o ponto do exercicio). Contando quantas vezes cada rotulo aparece na arvore inteira:

F(6): 1 vez F(5): 1 vez F(4): 2 vezes F(3): 3 vezes F(2): 5 vezes F(1): 8 vezes F(0): 5 vezes

Os numeros 1, 1, 2, 3, 5, 8 sao os proprios termos da sequencia de Fibonacci: em geral, na arvore de F(n), o rotulo F(k) aparece F_{n-k+1} vezes para 1 <= k <= n. Isso mostra por que o algoritmo e exponencial: o mesmo subproblema e resolvido do zero um numero de vezes que cresce como Fibonacci. Por exemplo, F(2) e recalculado 5 vezes so para n = 6.

TOTAL DE CHAMADAS. Somando: 1 + 1 + 2 + 3 + 5 + 8 + 5 = 25 nos. Isso confirma a formula C(n) = 2*F_{n+1} - 1 do IFMG, ex. 3: C(6) = 2*F_7 - 1 = 2*13 - 1 = 25. Contando ARESTAS (convencao padrao para altura), a altura da arvore e 5: o caminho mais longo, F(6) -> F(5) -> F(4) -> F(3) -> F(2) -> F(1), desce sempre pela esquerda e tem 6 nos, portanto 5 arestas. O caminho mais curto ate uma folha, F(6) -> F(4) -> F(2) -> F(0), desce sempre pela direita e tem 4 nos, portanto 3 arestas, o que mostra que a arvore e bastante desbalanceada — dai a base phi aprox. 1,618 em vez de 2.

CONFERENCIA. A arvore foi percorrida chamada a chamada: 25 nos no total, com as multiplicidades por rotulo exatamente 1, 1, 2, 3, 5, 8 e 5 para F(6) ate F(0), e 2*F_7 - 1 = 25.

CONFIANCA: confirmado.

---

22. IFMG, ex. 1

Exercícios (Para Casa)

Prove, para *n* inteiro positivo, que

$0^2 + 1^2 + 2^2 + \ldots n^2 = (2n + 1)(n + 1)(n) / 6$

[transcrição literal — Diego Mello, Matemática Discreta (IFMG), slide 14]

Resolução

Resposta. Verdadeiro: para todo inteiro n >= 1 vale 0^2 + 1^2 + ... + n^2 = n(n+1)(2n+1)/6. Provado por inducao em n.

ENUNCIADO. Seja P(n) a proposicao

P(n): soma de i^2 para i = 0 ate n = (2n+1)(n+1)(n)/6.

Observe que o termo i = 0 contribui 0^2 = 0, entao a soma que comeca em 0 e igual a soma que comeca em 1; isso nao muda nada, mas justifica a base valer ja em n = 1.

BASE (n = 1). Lado esquerdo: 0^2 + 1^2 = 0 + 1 = 1. Lado direito: (2*1+1)(1+1)(1)/6 = 3*2*1/6 = 6/6 = 1. Os dois lados valem 1, logo P(1) e verdadeira.

HIPOTESE DE INDUCAO (HI). Suponha que, para um inteiro fixo n >= 1, P(n) seja verdadeira, isto e,

0^2 + 1^2 + ... + n^2 = n(n+1)(2n+1)/6.

PASSO INDUTIVO. Queremos provar P(n+1), ou seja,

0^2 + 1^2 + ... + n^2 + (n+1)^2 = (n+1)(n+2)(2n+3)/6,

pois substituindo n por n+1 na formula obtem-se (2(n+1)+1)((n+1)+1)(n+1)/6 = (2n+3)(n+2)(n+1)/6.

Partindo do lado esquerdo e separando o ultimo termo:

[0^2 + 1^2 + ... + n^2] + (n+1)^2 = n(n+1)(2n+1)/6 + (n+1)^2 <-- AQUI se aplica a HI (troca do colchete pela forma fechada) = (n+1) * [n(2n+1) + 6(n+1)] / 6 (fator comum (n+1); o segundo termo virou 6(n+1)^2/6) = (n+1) * (2n^2 + n + 6n + 6) / 6 = (n+1) * (2n^2 + 7n + 6) / 6 = (n+1)(n+2)(2n+3) / 6,

onde na ultima linha usamos a fatoracao 2n^2 + 7n + 6 = (n+2)(2n+3), que se confere expandindo: (n+2)(2n+3) = 2n^2 + 3n + 4n + 6 = 2n^2 + 7n + 6.

O resultado obtido e exatamente o lado direito de P(n+1). Logo P(n) implica P(n+1).

CONCLUSAO. P(1) e verdadeira e P(n) => P(n+1) para todo n >= 1; pelo Principio da Inducao Matematica, P(n) vale para todo inteiro n >= 1. Em particular, a soma dos quadrados dos n primeiros inteiros e Theta(n^3), com as testemunhas c1 = 1/3, c2 = 1 e n0 = 1, que satisfazem (1/3)n^3 <= n(n+1)(2n+1)/6 <= n^3 para todo n >= 1.

CONFERENCIA. A igualdade foi conferida termo a termo para n = 1, 2, 3 (1 = 1; 1+4 = 5 = 2*3*5/6; 1+4+9 = 14 = 3*4*7/6) e vale sem excecao em toda a faixa de inteiros testada ate n = 2999.

CONFIANCA: confirmado.

---

23. IFMG, ex. 2

Exercícios (Para Casa)

Prove, para n inteiro positivo, que

$1 + 4 + 7 + \cdots + (3n - 2) = n(3n - 1) / 2$

[transcrição literal — Diego Mello, Matemática Discreta (IFMG), slide 14]

Resolução

Resposta. Verdadeiro: para todo inteiro n >= 1 vale 1 + 4 + 7 + ... + (3n-2) = n(3n-1)/2. Provado por inducao em n.

ENUNCIADO. O termo geral da soma e a_i = 3i - 2, com i = 1, 2, ..., n (de fato a_1 = 1, a_2 = 4, a_3 = 7, a_4 = 10, ...). Seja P(n) a proposicao

P(n): soma de (3i - 2) para i = 1 ate n = n(3n - 1)/2.

BASE (n = 1). Lado esquerdo: 3*1 - 2 = 1. Lado direito: 1*(3*1 - 1)/2 = 1*2/2 = 1. Os dois lados valem 1, logo P(1) e verdadeira.

HIPOTESE DE INDUCAO (HI). Suponha que, para um inteiro fixo n >= 1,

1 + 4 + 7 + ... + (3n - 2) = n(3n - 1)/2.

PASSO INDUTIVO. Queremos provar P(n+1). O proximo termo da soma e

a_{n+1} = 3(n+1) - 2 = 3n + 1,

e a formula avaliada em n+1 e

(n+1)(3(n+1) - 1)/2 = (n+1)(3n + 2)/2.

Partindo do lado esquerdo de P(n+1) e separando o ultimo termo:

[1 + 4 + ... + (3n - 2)] + (3n + 1) = n(3n - 1)/2 + (3n + 1) <-- AQUI se aplica a HI = (3n^2 - n)/2 + (6n + 2)/2 = (3n^2 - n + 6n + 2)/2 = (3n^2 + 5n + 2)/2 = (n + 1)(3n + 2)/2,

usando a fatoracao 3n^2 + 5n + 2 = (n+1)(3n+2), que se confere expandindo: (n+1)(3n+2) = 3n^2 + 2n + 3n + 2 = 3n^2 + 5n + 2.

O resultado e exatamente o lado direito de P(n+1). Logo P(n) implica P(n+1).

CONCLUSAO. P(1) vale e P(n) => P(n+1) para todo n >= 1; pelo Principio da Inducao Matematica, P(n) vale para todo inteiro n >= 1.

OBSERVACAO (por que a formula tinha de ser essa). Trata-se de uma progressao aritmetica de razao 3, primeiro termo 1 e ultimo termo 3n - 2. A soma de uma PA e (primeiro + ultimo) * numero de termos / 2 = (1 + 3n - 2) * n / 2 = n(3n - 1)/2, o que confirma a forma fechada por um segundo caminho independente. Assintoticamente a soma e Theta(n^2): testemunhas c1 = 1, c2 = 2 e n0 = 1 dao 1*n^2 <= n(3n-1)/2 <= 2*n^2 para todo n >= 1 (a esquerda: (3n^2 - n)/2 >= n^2 equivale a n^2 >= n, verdade para n >= 1; a direita: (3n^2 - n)/2 <= 2n^2 equivale a -n <= n^2, sempre verdade).

CONFERENCIA. Conferido termo a termo para n = 1, 2, 3 (1 = 1; 1+4 = 5 = 2*5/2; 1+4+7 = 12 = 3*8/2) e sem excecao em toda a faixa de inteiros testada ate n = 2999.

CONFIANCA: confirmado.

---

24. IFMG, ex. 5

Exercícios (Para Casa)

Dê a relação de recorrência para o algoritmo recursivo abaixo:

```
Algoritmo 9 ORDENACAO-POR-SELECAO(V: vetor, j: inteiro)
 1: if (j = 1) then
 2:     return V
 3: else
 4:     indice ← 1
 5:     for (i ← 1 to j) do
 6:         if (V[ i ] > V[ indice ]) then
 7:             indice ← i
 8:         end if
 9:     end for
10:     aux ← V[ j ]
11:     V[ j ] ← V[ indice ]
12:     V[ indice ] ← aux
13:     ORDENACAO-POR-SELECAO(V, j - 1)
14: end if
```

[transcrição literal — Diego Mello, Matemática Discreta (IFMG), slide 28]

Resolução

Resposta. T(1) = c (constante) e T(j) = T(j-1) + a*j + b para j >= 2, isto e, T(n) = T(n-1) + Theta(n) com T(1) = Theta(1). Resolvendo por expansao, T(n) = Theta(n ^ 2), no melhor e no pior caso (a ORDEM de crescimento nao depende do conteudo do vetor: o laco sempre faz j iteracoes e j comparacoes; so o numero de execucoes da linha 7 varia, e com ele a constante a, entre a1 e a2).

CONTAGEM DE CUSTO DE UMA CHAMADA. Para que a contagem linha a linha possa ser conferida, o pseudo-codigo do enunciado:

```
1: if (j = 1) then
   2:     return V
   3: else
   4:     indice <- 1
   5:     for (i <- 1 to j) do
   6:         if (V[i] > V[indice]) then
   7:             indice <- i
   8:         end if
   9:     end for
  10:     aux <- V[j]
  11:     V[j] <- V[indice]
  12:     V[indice] <- aux
  13:     ORDENACAO-POR-SELECAO(V, j-1)
  14: end if
```

Considere a chamada ORDENACAO-POR-SELECAO(V, j) e conte o trabalho feito fora da recursao:

- linha 1: um teste, O(1); - linha 4: uma atribuicao, O(1); - linhas 5-9: o laco 'for i <- 1 to j' executa exatamente j iteracoes, e cada iteracao faz uma comparacao (linha 6) e, no maximo, uma atribuicao (linha 7) — custo O(1) por iteracao. O numero de vezes que a linha 7 executa depende do vetor (para n = 5 ela roda 10 vezes num vetor crescente, 4 num decrescente e 0 num de elementos iguais), entao o custo do laco nao e uma expressao unica a*j: e a1*j <= custo <= a2*j, com 0 < a1 <= a2 constantes. O que vale para toda entrada e que o laco custa Theta(j); - linhas 10-12: a troca (tres atribuicoes), O(1); - linha 13: uma chamada recursiva sobre o subvetor de tamanho j-1, custo T(j-1).

Agrupando as parcelas constantes numa constante b > 0, o trabalho local fica entre a1*j + b e a2*j + b. Abaixo escrevo a*j + b, com a valendo a1 (para a cota inferior) ou a2 (para a cota superior): a conta e a mesma nos dois casos, e os dois resultados sao Theta(n ^ 2).

CASO BASE. Se j = 1, so se executa o teste da linha 1 e o return da linha 2: custo constante c > 0.

RELACAO DE RECORRENCIA.

T(1) = c T(j) = T(j-1) + a*j + b, para j >= 2

ou, em notacao assintotica,

T(1) = Theta(1) T(n) = T(n-1) + Theta(n).

Observe que nao ha distincao entre melhor e pior caso: o laco das linhas 5-9 sempre executa j iteracoes, qualquer que seja o conteudo de V. O teste da linha 6 pode alterar a linha 7, mas isso nao muda a contagem de iteracoes. Portanto, para TODA entrada, T(j) - T(j-1) fica entre a1*j + b e a2*j + b: o custo EXATO depende do vetor (pela linha 7), mas a forma T(n) = T(n-1) + Theta(n) vale para toda entrada, e o resultado assintotico e o mesmo em todos os casos.

RESOLUCAO POR EXPANSAO (iteracao).

```
T(n) = T(n-1) + a*n + b
       = T(n-2) + a*(n-1) + b + a*n + b
       = T(n-3) + a*(n-2) + a*(n-1) + a*n + 3b
       = ...
       = T(n-k) + a*[ n + (n-1) + ... + (n-k+1) ] + k*b.
```

Parando em n - k = 1, ou seja, k = n - 1:

T(n) = T(1) + a*[n + (n-1) + ... + 2] + (n-1)*b = c + a*(n(n+1)/2 - 1) + b*(n-1),

onde usamos a forma fechada 2 + 3 + ... + n = n(n+1)/2 - 1. Expandindo:

T(n) = (a/2)*n^2 + (a/2 + b)*n + (c - a - b).

E um polinomio de grau 2 com coeficiente lider a/2 > 0, logo T(n) = Theta(n^2).

TESTEMUNHAS EXPLICITAS. Tomando, para ilustrar, a = b = c = 1 (valores escolhidos para simplificar a conta, e NAO o custo de contar 1 por operacao elementar: com essa contagem, o caso base ja custa 2 — teste e return — e uma chamada custa cerca de 2j + 7), a expressao acima da

T(n) = (n^2 + 3n - 2)/2.

Afirmo que T(n) = Theta(n^2) com c1 = 1/2, c2 = 2 e n0 = 1: - cota inferior: (n^2 + 3n - 2)/2 >= (1/2)*n^2 <=> 3n - 2 >= 0 <=> n >= 2/3; como n e inteiro, isso equivale a n >= 1. Vale para todo n >= 1. - cota superior: (n^2 + 3n - 2)/2 <= 2*n^2 <=> n^2 + 3n - 2 <= 4n^2 <=> 3n^2 - 3n + 2 >= 0, cujo discriminante e 9 - 24 < 0, logo e sempre positivo. Vale para todo n >= 1. Assim, (1/2)*n^2 <= T(n) <= 2*n^2 para todo n >= n0 = 1, com c1 e c2 escolhidos antes de n e independentes de n.

No caso geral (a, b, c quaisquer positivos), a forma fechada e T(n) = (a/2)n^2 + (a/2 + b)n + (c - a - b). Tome c1 = a/4, c2 = a e construa n0 explicitamente: n0 = max(1, 4(a/2 + b)/a, 4|c - a - b|/a). Com esse n0 cada termo de grau menor fica limitado por (a/4)n^2 em modulo, de modo que (a/4)n^2 <= T(n) <= a*n^2 para todo n >= n0. O ponto e que n0 sai de a, b, c por uma formula, e nao e apenas declarado existir.

VERIFICACAO DA SOLUCAO POR INDUCAO (metodo da substituicao, com a constante intacta). Seja P(n): T(n) = (n^2 + 3n - 2)/2, com T(1) = 1 e T(n) = T(n-1) + n + 1. Base n = 1: lado esquerdo T(1) = 1; lado direito (1 + 3 - 2)/2 = 2/2 = 1. Confere. Hipotese: T(n-1) = ((n-1)^2 + 3(n-1) - 2)/2 = (n^2 + n - 4)/2, para um n >= 2 fixo. Passo: T(n) = T(n-1) + n + 1 = (n^2 + n - 4)/2 + (2n + 2)/2 <-- AQUI se aplica a hipotese = (n^2 + 3n - 2)/2, que e exatamente P(n). As constantes 1/2, 3/2 e -1 reaparecem identicas, sem crescer.

OBSERVACAO SOBRE O ALGORITMO. Ele e a ordenacao por selecao escrita recursivamente: em cada chamada localiza o indice do MAIOR elemento de V[1..j] (linhas 4-9), coloca-o na posicao j (linhas 10-12) e ordena recursivamente V[1..j-1]. Ao final, o vetor fica em ordem nao decrescente. O numero total de comparacoes e n + (n-1) + ... + 2 = n(n+1)/2 - 1, confirmando a ordem quadratica.

CONFERENCIA. Com a = b = c = 1, os valores ilustrativos adotados acima, a forma fechada T(n) = (n^2 + 3n - 2)/2 da T(1) = 1, T(2) = 4, T(3) = 8 e T(4) = 13. A razao T(n)/n^2 estabiliza (n = 50: 0,5296; n = 200: 0,5075; n = 800: 0,5019), convergindo para a constante a/2 = 1/2 prevista pela forma fechada — comportamento tipico de Theta(n^2).

CONFIANCA: confirmado.

---

25. IFMG, ex. 6

Exercícios (Para Casa)

Nos slides sobre algoritmos recursivos, conhecemos o jogo 'Torre de Hanoi'. Pelo que estudamos, o menor número de movimentações para transferir os discos da primeira torre para a última é dado pela relação de recorrência:

- $T(1) = 1$
- $T(n) = 2 \cdot T(n-1) + 1$

De posse desta informação, use o método '**expandir, conjecturar e verificar**' para encontrar uma solução em forma fechada que dê o número de movimentações necessárias para completar o jogo com n discos.

[transcrição literal — Diego Mello, Matemática Discreta (IFMG), slide 35]

Resolução

Resposta. T(n) = 2^n - 1 movimentos. Obtido pelo metodo expandir, conjecturar e verificar; a verificacao final e por inducao. Assintoticamente, T(n) = Theta(2^n).

RECORRENCIA DADA.

T(1) = 1 T(n) = 2*T(n-1) + 1, para n >= 2

(mover n discos = mover n-1 discos para o pino auxiliar, mover o disco maior, mover os n-1 de volta sobre ele).

PASSO 1 — EXPANDIR. Substituimos repetidamente a definicao de T:

```
T(n) = 2*T(n-1) + 1
        = 2*[2*T(n-2) + 1] + 1 = 4*T(n-2) + 2 + 1
        = 4*[2*T(n-3) + 1] + 2 + 1 = 8*T(n-3) + 4 + 2 + 1
        = 16*T(n-4) + 8 + 4 + 2 + 1
        = ...
```

Apos k substituicoes:

T(n) = 2 ^ k * T(n-k) + (2 ^ (k-1) + 2 ^ (k-2) + ... + 2 + 1).

O parenteses e uma progressao geometrica de razao 2 com k termos, cuja forma fechada e 2 ^ k - 1. Logo

T(n) = 2 ^ k * T(n-k) + 2 ^ k - 1, valido para 1 <= k <= n-1.

PASSO 2 — CONJECTURAR. A expansao so para quando o argumento chega no caso base n - k = 1, ou seja, k = n - 1. Substituindo:

```
T(n) = 2^(n-1) * T(1) + 2^(n-1) - 1
        = 2^(n-1) * 1 + 2^(n-1) - 1
        = 2 * 2^(n-1) - 1
        = 2^n - 1.
```

CONJECTURA: T(n) = 2 ^ n - 1 para todo n >= 1.

PASSO 3 — VERIFICAR (inducao sobre n). Este passo e obrigatorio: a expansao com reticencias e uma heuristica, nao uma prova. Seja

P(n): T(n) = 2 ^ n - 1.

BASE (n = 1). Pela recorrencia, T(1) = 1. Pela formula, 2 ^ 1 - 1 = 2 - 1 = 1. Os dois lados valem 1, logo P(1) e verdadeira.

HIPOTESE DE INDUCAO. Suponha que, para um inteiro fixo n >= 2, valha P(n-1), isto e, T(n-1) = 2 ^ (n-1) - 1.

PASSO INDUTIVO. Para n >= 2, a recorrencia da

```
T(n) = 2*T(n-1) + 1
        = 2*(2^(n-1) - 1) + 1        <-- AQUI se aplica a hipotese de inducao
        = 2^n - 2 + 1
        = 2^n - 1,
```

que e exatamente P(n). Note que em nenhum momento a tese foi usada para provar a tese: partiu-se da recorrencia e substituiu-se apenas o termo T(n-1) pela hipotese.

CONCLUSAO. P(1) vale e P(n-1) => P(n) para todo n >= 2; pelo Principio da Inducao Matematica, T(n) = 2 ^ n - 1 para todo inteiro n >= 1.

ORDEM ASSINTOTICA. T(n) = Theta(2 ^ n), com testemunhas c1 = 1/2, c2 = 1 e n0 = 1: - superior: 2 ^ n - 1 <= 1 * 2 ^ n para todo n (subtrair 1 so diminui); - inferior: 2 ^ n - 1 >= (1/2)*2 ^ n <=> (1/2)*2 ^ n >= 1 <=> 2 ^ (n-1) >= 1 <=> n >= 1. Logo (1/2)*2 ^ n <= T(n) <= 2 ^ n para todo n >= 1. Em termos praticos: cada disco a mais dobra (e mais um) o numero de movimentos; para os 64 discos da lenda seriam 2 ^ 64 - 1 = 18.446.744.073.709.551.615 movimentos.

CONFERENCIA. Os primeiros valores gerados pela recorrencia sao 1, 3, 7, 15, 31, 63, 127, 255 para n = 1..8, coincidindo com 2 ^ n - 1 (2-1, 4-1, 8-1, 16-1, 32-1, 64-1, 128-1, 256-1). A igualdade foi conferida sem excecao em toda a faixa testada, n = 1 ate 499.

CONFIANCA: confirmado.

---

26. IFMG, slide 23

Pergunta: como deve ser a primeira chamada do algoritmo recursivo acima?

Pergunta: mostre, passo a passo, a ordenação de V = [10, 2, 13, 22, 8, 3, 12, 9, 1, 50]

Contexto do material (transcrição literal, necessária para resolver):
(slide 23, mesma página das perguntas)

Seja o algoritmo abaixo, que ordena um vetor por meio de um procedimento recursivo:

```
Algoritmo 6 ORDENACAO-POR-SELECAO(V: vetor, j: inteiro)
 1: if (j = 1) then
 2:     return V
 3: else
 4:     indice ← 1
 5:     for (i ← 1 to j) do
 6:         if (V[ i ] > V[ indice ]) then
 7:             indice ← i
 8:         end if
 9:     end for
10:     aux ← V[ j ]
11:     V[ j ] ← V[ indice ]
```

```
12:     V[ indice ] ← aux
13:     ORDENACAO-POR-SELECAO(V, j − 1)
14: end if
```

[transcrição literal — IFMG, slide 23]

Resolução

Resposta. A primeira chamada deve ser ORDENACAO-POR-SELECAO(V, n), com V o vetor inteiro e j = n = número de elementos de V; para o vetor dado, ORDENACAO-POR-SELECAO(V, 10). O parâmetro j é o tamanho do prefixo V[1..j] que ainda não está ordenado, e no início nada está ordenado. Executando o algoritmo, há 10 chamadas (j = 10, 9, …, 1). Em cada chamada com j ≥ 2, o maior elemento de V[1..j] vai para a posição j. Os estados sucessivos são: [10, 2, 13, 22, 8, 3, 12, 9, 1, 50] → (j=10, sem mudança) → (j=9) [10, 2, 13, 1, 8, 3, 12, 9, 22, 50] → (j=8) [10, 2, 9, 1, 8, 3, 12, 13, 22, 50] → (j=7, sem mudança) → (j=6) [3, 2, 9, 1, 8, 10, 12, 13, 22, 50] → (j=5) [3, 2, 8, 1, 9, 10, 12, 13, 22, 50] → (j=4) [3, 2, 1, 8, 9, 10, 12, 13, 22, 50] → (j=3) [1, 2, 3, 8, 9, 10, 12, 13, 22, 50] → (j=2, sem mudança) → (j=1, caso base) devolve [1, 2, 3, 8, 9, 10, 12, 13, 22, 50].

INTERPRETAÇÃO. A primeira pergunta pode ser lida de duas formas: (i) quais argumentos passar na chamada inicial; (ii) o que acontece nessa chamada. Respondo (i), que é o que "como deve ser a primeira chamada" pede, e uso (ii) para justificar. A segunda pergunta pede a execução literal do Algoritmo 6 a partir dessa chamada, mostrando o vetor depois de cada troca (linhas 10–12). Convenções do próprio código: os índices começam em 1; a comparação da linha 6 é estrita (>), então, em caso de empate, fica o primeiro índice (no vetor dado não há valores repetidos). As trocas das linhas 10–12 e a chamada da linha 13 atuam sobre o mesmo V: o vetor é passado por referência e ordenado no lugar. É por isso que o ramo else não precisa de return: quando a recursão termina, quem chamou já vê o V ordenado.

PARTE 1 — O QUE O PARÂMETRO j SIGNIFICA E POR QUE A PRIMEIRA CHAMADA É (V, n). Na chamada com parâmetro j, o laço das linhas 5–9 percorre só V[1..j]. As linhas 10–12 trocam V[j] com V[indice], e a linha 13 chama de novo com j − 1. Então j é o fim do trecho que ainda falta ordenar. O trecho V[j+1..n] já está pronto e não é mais tocado.

Invariante da recursão. Na chamada com parâmetro j, valem: (a) V é uma permutação do vetor original; (b) V[j+1..n] está em ordem crescente; (c) todo elemento de V[1..j] é ≤ todo elemento de V[j+1..n].

Para que a primeira chamada satisfaça o invariante com qualquer vetor de entrada, o trecho "pronto" V[j+1..n] tem de ser vazio, e isso exige j = n. Com j = n, as condições (b) e (c) valem por vacuidade e (a) vale trivialmente.

```
Por que nenhum outro valor serve:
    j < n: as posições j+1..n nunca são examinadas (todas as chamadas usam índices ≤ j). Contraexemplo: V = [2, 1]
com j = 1 cai direto no caso base e devolve [2, 1], que não está ordenado. (Com o vetor deste slide, a chamada (V,
9) por acaso daria certo, porque V[10] = 50 já é o máximo. Isso é coincidência desta entrada e não justifica j =
9.)
    j > n: a linha 6 acessa V[n+1], fora do vetor.
     j = 0 (ou j < 1): o teste da linha 1 falha, o laço da linha 5 não executa e a linha 10 acessa V[0], fora do
vetor. Além disso, j − 1 nunca chega a 1, então a recursão não para.
```

Para ver que j = n de fato ordena, verifica-se o passo do invariante. Suponha (a), (b), (c) na chamada com j ≥ 2. Invariante do laço: depois da iteração i, V[indice] é o máximo de V[1..i]. Vale com i = 1, porque indice = 1. Se vale para i − 1, a linha 6 troca indice por i exatamente quando V[i] é maior que o máximo de V[1..i−1]. Ao fim do laço, V[indice] = máx V[1..j]. As linhas 10–12 trocam V[j] com esse máximo. Isso preserva (a); deixa V[j] ≥ todo elemento de V[1..j−1]; e, por (c), V[j] ≤ todo elemento de V[j+1..n]. Logo V[j..n] fica crescente e todo elemento de V[1..j−1] é ≤ todo elemento de V[j..n]. Esse é o invariante para a chamada com j − 1. No caso base j = 1, (b) diz que V[2..n] está crescente e (c) diz que V[1] ≤ V[2..n]. Logo V inteiro está ordenado. Partindo de j = n, isso vale para toda entrada com n ≥ 1.

PARTE 2 — EXECUÇÃO LITERAL COM V = [10, 2, 13, 22, 8, 3, 12, 9, 1, 50] E PRIMEIRA CHAMADA (V, 10). Em cada chamada mostro a varredura da linha 6 (quando indice muda), a troca das linhas 10–12 e o vetor depois da troca. Os elementos já na posição final aparecem depois da barra "|".

```
CHAMADA j = 10.
    Varredura de V[1..10] = 10, 2, 13, 22, 8, 3, 12, 9, 1, 50: indice = 1 (10); i = 3: 13 > 10, indice = 3; i = 4:
22 > 13, indice = 4; i = 10: 50 > 22, indice = 10. Os outros i não mudam indice.
    Troca V[10] ↔ V[10] (o elemento troca consigo mesmo).
    V = [10, 2, 13, 22, 8, 3, 12, 9, 1 | 50]
```

```
CHAMADA j = 9.
    Varredura de V[1..9] = 10, 2, 13, 22, 8, 3, 12, 9, 1: indice = 1 → 3 (13) → 4 (22). Máximo 22 em V[4].
    Troca V[9] = 1 ↔ V[4] = 22.
    V = [10, 2, 13, 1, 8, 3, 12, 9 | 22, 50]
```

```
CHAMADA j = 8.
    Varredura de V[1..8] = 10, 2, 13, 1, 8, 3, 12, 9: indice = 1 → 3 (13). Máximo 13 em V[3].
    Troca V[8] = 9 ↔ V[3] = 13.
    V = [10, 2, 9, 1, 8, 3, 12 | 13, 22, 50]
```

```
CHAMADA j = 7.
    Varredura de V[1..7] = 10, 2, 9, 1, 8, 3, 12: indice = 1 → 7 (12 > 10). Máximo 12 em V[7].
    Troca V[7] ↔ V[7] (o elemento troca consigo mesmo).
    V = [10, 2, 9, 1, 8, 3 | 12, 13, 22, 50]
```

```
CHAMADA j = 6.
    Varredura de V[1..6] = 10, 2, 9, 1, 8, 3: nenhum supera 10, indice = 1.
    Troca V[6] = 3 ↔ V[1] = 10.
    V = [3, 2, 9, 1, 8 | 10, 12, 13, 22, 50]
```

```
CHAMADA j = 5.
    Varredura de V[1..5] = 3, 2, 9, 1, 8: indice = 1 → 3 (9). Máximo 9 em V[3].
    Troca V[5] = 8 ↔ V[3] = 9.
    V = [3, 2, 8, 1 | 9, 10, 12, 13, 22, 50]
```

```
CHAMADA j = 4.
    Varredura de V[1..4] = 3, 2, 8, 1: indice = 1 → 3 (8). Máximo 8 em V[3].
    Troca V[4] = 1 ↔ V[3] = 8.
    V = [3, 2, 1 | 8, 9, 10, 12, 13, 22, 50]
```

```
CHAMADA j = 3.
    Varredura de V[1..3] = 3, 2, 1: nenhum supera 3, indice = 1.
    Troca V[3] = 1 ↔ V[1] = 3.
    V = [1, 2 | 3, 8, 9, 10, 12, 13, 22, 50]
```

```
CHAMADA j = 2.
    Varredura de V[1..2] = 1, 2: indice = 1 → 2 (2 > 1). Máximo 2 em V[2].
    Troca V[2] ↔ V[2] (o elemento troca consigo mesmo).
    V = [1 | 2, 3, 8, 9, 10, 12, 13, 22, 50]
```

CHAMADA j = 1. Caso base (linhas 1–2): devolve V = [1, 2, 3, 8, 9, 10, 12, 13, 22, 50], que está ordenado.

Resumo da execução: 10 chamadas. As chamadas com j = 10, 7 e 2 trocam um elemento consigo mesmo, então o vetor não muda nelas. As seis chamadas com j = 9, 8, 6, 5, 4, 3 alteram o vetor. A linha 6 executa j vezes na chamada com parâmetro j ≥ 2 (para i = 1, …, j) e nenhuma vez no caso base. No total, para n = 10, são 2 + 3 + … + 10 = 10·11/2 − 1 = 54 comparações entre elementos. Esse número vale para qualquer vetor de 10 elementos, porque o laço não para antes do fim.

Conferência numérica (Python; não faz parte da prova): a simulação do Algoritmo 6 com a primeira chamada (V, 10) confirma, chamada por chamada, os mesmos índices do máximo (10, 4, 3, 7, 1, 3, 3, 1, 2), as mesmas trocas, os mesmos vetores intermediários e o resultado final [1, 2, 3, 8, 9, 10, 12, 13, 22, 50] ordenado (True). A justificativa de j = n e o invariante não dependem da simulação.

---

27. IFMG, slide 26

Exercício: Esboce uma árvore com as chamadas recursivas do algoritmo acima para resolver o problema para $n = 4$ discos.

Contexto do material (transcrição literal, necessária para resolver):
(slide 26, mesma página do exercício; título 'Algoritmos Recursivos e Relações de Recorrência')

Como o problema pode ser resolvido para *n* discos?

[figura: quatro quadros com os pilares A, B, C — estado inicial com os discos em A; “H(4,A,B)”; “m(A,C)”; “H(4,B,C)”]

```
Algoritmo 7 TORRE-HANOI(n, origem, destino, auxiliar: inteiros)
 1: if (n = 1) then
 2:     Movimente n de origem para destino
 3: else
 4:     TORRE-HANOI(n − 1, origem, auxiliar, destino)
 5:     Movimente n de origem para destino
 6:     TORRE-HANOI(n − 1, auxiliar, destino, origem)
 7: end if
```

[transcrição literal — IFMG, slide 26]

Resolução

Resposta. Com a chamada inicial TORRE-HANOI(4, A, C, B) (origem = A, destino = C, auxiliar = B, como na figura do slide), a árvore de chamadas é binária completa com 4 níveis: 1 chamada com n = 4, 2 com n = 3, 4 com n = 2 e 8 com n = 1, num total de 15 nós, dos quais 7 internos (n ≥ 2) e 8 folhas (n = 1). Cada nó executa exatamente um movimento (linha 2 ou linha 5), logo são 15 movimentos; o disco 1 move-se 8 vezes, o disco 2 quatro, o disco 3 duas e o disco 4 uma. A ordem dos movimentos é a do percurso em ordem simétrica da árvore (subárvore esquerda, movimento do nó, subárvore direita). O número de chamadas (e o de movimentos) satisfaz C(1) = 1, C(n) = 2C(n − 1) + 1, cuja solução é $C(n) = 2^n - 1$, e C(4) = 15.

INTERPRETAÇÃO DO ENUNCIADO.

O enunciado pede "uma árvore com as chamadas recursivas do algoritmo acima" para n = 4. Leituras possíveis: (i) a árvore de chamadas do Algoritmo 7, com os parâmetros de cada chamada; (ii) uma árvore de recursão de custos (cada nó rotulado pelo custo). Resolvo a leitura (i), que é a pedida literalmente ("árvore com as chamadas recursivas"), e acrescento em cada nó o movimento que ele executa, de onde sai também a leitura (ii) (custo 1 movimento por nó). O enunciado não fixa os pilares da chamada inicial; adoto os da figura do slide, em que os discos começam em A e terminam em C, passando por B: TORRE-HANOI(4, A, C, B). Qualquer outra escolha de pilares produz a mesma forma de árvore, apenas com os rótulos dos pilares permutados.

Observação sobre a figura: ela rotula os quatro quadros "H(4,A,B)", "m(A,C)", "H(4,B,C)". Pelo Algoritmo 7, as duas chamadas que a chamada com n = 4 faz têm parâmetro n − 1 = 3 (linhas 4 e 6); então o primeiro e o último quadro correspondem, no algoritmo, às chamadas TORRE-HANOI(3, A, B, C) e TORRE-HANOI(3, B, C, A), e "m(A,C)" é a linha 5, que move o disco 4 de A para C. A árvore abaixo segue o algoritmo.

ÁRVORE DE CHAMADAS (n = 4).

Notação: H(n, origem, destino, auxiliar) é a chamada TORRE-HANOI(n, origem, destino, auxiliar). Ao lado de cada nó está o movimento que ele executa e, entre colchetes, a posição desse movimento na sequência de execução (1 a 15). Filhos aparecem recuados abaixo do pai, na ordem em que são chamados (linha 4 e depois linha 6).

```
H(4, A, C, B)  move disco 4: A → C  [8]
       H(3, A, B, C)  move disco 3: A → B  [4]
           H(2, A, C, B)  move disco 2: A → C  [2]
               H(1, A, B, C)  move disco 1: A → B  [1]   (folha)
               H(1, B, C, A)  move disco 1: B → C  [3]   (folha)
           H(2, C, B, A)  move disco 2: C → B  [6]
               H(1, C, A, B)  move disco 1: C → A  [5]   (folha)
               H(1, A, B, C)  move disco 1: A → B  [7]   (folha)
       H(3, B, C, A)  move disco 3: B → C  [12]
           H(2, B, A, C)  move disco 2: B → A  [10]
               H(1, B, C, A)  move disco 1: B → C  [9]   (folha)
               H(1, C, A, B)  move disco 1: C → A  [11]  (folha)
           H(2, A, C, B)  move disco 2: A → C  [14]
               H(1, A, B, C)  move disco 1: A → B  [13]  (folha)
               H(1, B, C, A)  move disco 1: B → C  [15]  (folha)
```

Como os filhos de cada nó foram obtidos: um nó H(n, o, d, x) com n ≥ 2 tem o teste da linha 1 falso e executa, nesta ordem, a linha 4, que chama H(n − 1, o, x, d) (filho esquerdo: origem e destino do pai, mas com destino e auxiliar trocados), a linha 5, que move o disco n de o para d, e a linha 6, que chama H(n − 1, x, d, o) (filho direito). Um nó H(1, o, d, x) tem o teste da linha 1 verdadeiro, move o disco 1 de o para d (linha 2) e não faz chamada: é folha. Por exemplo, a raiz H(4, A, C, B) tem filhos H(3, A, B, C) e H(3, B, C, A), e o nó H(3, B, C, A) tem filhos H(2, B, A, C) e H(2, A, C, B).

Atenção à ordem: a árvore está desenhada por pais e filhos, mas o movimento de um nó só acontece depois que toda a sua subárvore esquerda terminou (linha 5 vem depois da linha 4). Por isso a numeração [1] a [15] é a ordem simétrica da árvore, e não a ordem de cima para baixo do desenho.

SEQUÊNCIA DE MOVIMENTOS, NA ORDEM DE EXECUÇÃO.

```
 1: disco 1 A → B
     2: disco 2 A → C
     3: disco 1 B → C
     4: disco 3 A → B
     5: disco 1 C → A
     6: disco 2 C → B
     7: disco 1 A → B
     8: disco 4 A → C
     9: disco 1 B → C
    10: disco 2 B → A
    11: disco 1 C → A
    12: disco 3 B → C
    13: disco 1 A → B
    14: disco 2 A → C
    15: disco 1 B → C
```

Acompanhando os pilares (topo à direita), partindo de A = [4 3 2 1], B = [ ], C = [ ]: após o movimento 7 tem-se A = [4], B = [3 2 1], C = [ ] (os três discos menores foram para o auxiliar, que é exatamente o efeito de H(3, A, B, C)); o movimento 8 leva o disco 4 para C; após o movimento 15, A = [ ], B = [ ], C = [4 3 2 1]. Conferi passo a passo que nenhum dos 15 movimentos coloca um disco sobre um disco menor. (A correção do algoritmo para n geral não é pedida no exercício e não é provada aqui; a afirmação vale para esta execução com n = 4.)

CONTAGEM NA ÁRVORE DE n = 4.

Por níveis: nível 0 tem 1 nó (n = 4), nível 1 tem 2 nós (n = 3), nível 2 tem 4 nós (n = 2), nível 3 tem 8 nós (n = 1). Total de nós: 1 + 2 + 4 + 8 = 15 = $2^4 - 1$. Nós internos (n ≥ 2): 1 + 2 + 4 = 7 = $2^3 - 1$. Folhas (n = 1): 8 = 2^3. Movimentos: um por nó, 15. Por disco: o disco k é movido exatamente pelos nós com parâmetro n = k (linha 2 se k = 1, linha 5 se k ≥ 2), e há 2 ^ (4−k) desses nós; logo disco 4: 1, disco 3: 2, disco 2: 4, disco 1: 8, e 1 + 2 + 4 + 8 = 15, o que bate com a lista acima.

RELAÇÃO COM T(n) = 2T(n − 1) + 1.

Seja C(n) o número total de chamadas (nós da árvore, contando a própria chamada) e M(n) o número de movimentos executados por uma chamada TORRE-HANOI(n, o, d, x), para n ≥ 1 inteiro (o algoritmo não trata n = 0). Pela estrutura do algoritmo, C e M não dependem dos pilares o, d, x: dependem só de n. A leitura das linhas dá:

C(1) = 1, C(n) = 1 + 2C(n − 1) para n ≥ 2 (a própria chamada + as duas das linhas 4 e 6); M(1) = 1, M(n) = 2M(n − 1) + 1 para n ≥ 2 (os movimentos das duas subchamadas + o da linha 5).

São a mesma recorrência T(n) = 2T(n − 1) + 1 com T(1) = 1. Na árvore, o "+1" é o próprio nó e o "2T(n − 1)" são as duas subárvores, cada uma de altura uma unidade menor.

Afirmação: para todo inteiro $n \geq 1$, $C(n) = M(n) = 2^n - 1$, e o número de folhas é $F(n) = 2^{n-1}$.

Prova por indução em n. P(n): "para quaisquer pilares distintos o, d, x, a chamada TORRE-HANOI(n, o, d, x) gera exatamente $2^n - 1$ chamadas no total (contando ela própria), executa exatamente $2^n - 1$ movimentos e a sua árvore tem exatamente 2^{n-1} folhas". (P(n) é enunciada para quaisquer pilares porque as subchamadas usam pilares em outra ordem.)

Base, $n = 1$: o teste da linha 1 é verdadeiro; a chamada executa um movimento (linha 2) e nenhuma subchamada. Chamadas: $1 = 2^1 - 1$. Movimentos: $1 = 2^1 - 1$. Folhas: a própria chamada, $1 = 2^0$. P(1) vale.

Passo: seja $n \geq 2$ e suponha $P(n-1)$ (hipótese de indução, para quaisquer pilares). A chamada TORRE-HANOI(n, o, d, x) tem o teste da linha 1 falso ($n \neq 1$) e executa as linhas 4, 5 e 6: as subchamadas TORRE-HANOI($n-1$, o, x, d) e TORRE-HANOI($n-1$, x, d, o), com $n - 1 \geq 1$, e um movimento. Pela hipótese aplicada a cada uma (com os pilares (o, x, d) e (x, d, o), respectivamente), cada subchamada gera $2^{n-1} - 1$ chamadas, executa $2^{n-1} - 1$ movimentos e tem 2^{n-2} folhas. Logo: chamadas $= 1 + 2(2^{n-1} - 1) = 2^n - 1$; movimentos $= 2(2^{n-1} - 1) + 1 = 2^n - 1$; folhas $= 2 \cdot 2^{n-2} = 2^{n-1}$ (a chamada em si não é folha, pois faz subchamadas). Isso é P(n). Por indução, P(n) vale para todo $n \geq 1$.

Para $n = 4$: $C(4) = M(4) = 2^4 - 1 = 15$ e $F(4) = 2^3 = 8$, como contado na árvore. Desenrolando a recorrência: $T(4) = 2T(3) + 1 = 2(2T(2) + 1) + 1 = 4T(2) + 3 = 4(2T(1) + 1) + 3 = 8T(1) + 7 = 15$; os coeficientes 1, 2, 4, 8 são os números de nós por nível, e $7 = 1 + 2 + 4$ é o número de nós internos.

ORDEM DE CRESCIMENTO.

Tomando como custo o número de movimentos (ou de chamadas; cada chamada faz, fora das subchamadas, um teste e um movimento, isto é, trabalho limitado por uma constante), $T(n) = 2^n - 1 \in \Theta(2^n)$. Testemunhas: $c_1 = 1/2$, $c_2 = 1$, $n_0 = 1$, isto é, para todo $n \geq 1$, $(1/2)\cdot 2^n \leq 2^n - 1 \leq 1\cdot 2^n$. Lado direito: $2^n - 1 \leq 2^n$, pois $-1 \leq 0$. Lado esquerdo: $2^n - 1 \geq 2^{n-1}$ equivale a $2^{n-1} \geq 1$, verdadeiro para $n \geq 1$. Em $n = n_0 = 1$ exatamente: $(1/2)\cdot 2 = 1 \leq 2^1 - 1 = 1 \leq 2$. Vale (com igualdade à esquerda). Esse valor vale para toda entrada de tamanho n, não só no pior caso: o algoritmo não toma decisão que dependa de outra coisa além de n, então melhor caso, pior caso e qualquer caso coincidem.

Conferência numérica (Python; não faz parte da prova): a execução literal do Algoritmo 7 com TORRE-HANOI(4, A, C, B) lista exatamente as 15 chamadas da árvore acima com os mesmos parâmetros e a mesma relação pai–filho (indentação), e os 15 movimentos na mesma ordem da sequência numerada 1 a 15: 15 chamadas e 15 movimentos, o que coincide com $C(4) = M(4) = 2^4 - 1 = 15$.

Rezende — Relações de Recorrência (IC-UNICAMP) · semana 4

28. Rezende (UNICAMP), p. 9, recorrência (3)

(p. 9) Se uma recorrência é muito parecida com outra que já foi vista, é razoável supor que sua solução também seja parecida. Por exemplo, as recorrências (3) e (4) são ambas pertencentes a $\Theta(n \log n)$ (deixamos a prova de que ambas são pertencentes a $\Omega(n \log n)$ como exercício para o leitor).

Exercício (recorrência (3)): provar que a recorrência (3) pertence a $\Omega(n \log n)$.

> **Contexto do material** (transcrição literal, necessária para resolver):
> (p. 7) $T(n) = 2T(n/2) + n, \quad T(1) = ?. \qquad (3)$
>
> (p. 7, legenda da Figura 1) Árvore de Recorrência para a expressão $T(n) = 2T(n/2) + n$

[transcrição literal — Rezende (UNICAMP), p. 9, recorrência (3)]

> Resolução
>
> **Resposta.** Para n potência de 2 e caso base T(1) = d com $d \geq 0$ (por exemplo T(1) = 1), a recorrência $T(n) = 2T(n/2) + n$ satisfaz $T(n) \geq n \lg n$ para toda potência de 2 $n \geq 1$, provado pelo método da substituição (indução em k, $n = 2^k$) com a mesma constante $c = 1$ na hipótese e na conclusão. Logo $T(n) \in \Omega(n \lg n)$ com testemunhas $c = 1$ e $n_0 = 1$. Se o leitor adotar $T(1) = d < 0$, a conclusão continua valendo: prova-se por substituição a hipótese reforçada $T(n) \geq n \lg n + d\cdot n$, e daí $T(n) \geq (1/2)\cdot n \lg n$ para $n \geq n_0 = 2^{\lceil -2d \rceil}$ (testemunhas $c = 1/2$, $n_0 = 2^{\lceil -2d \rceil}$). Como log_b n = lg n / lg b, o resultado vale para qualquer base $b > 1$ do "log" do enunciado.
>
> **LEITURA DO ENUNCIADO (INTERPRETAÇÃO):** O exercício pede provar que a recorrência (3), $T(n) = 2T(n/2) + n$ com $T(1) = ?$, pertence a $\Omega(n \log n)$. Há três pontos em aberto no texto. (i) O caso base não é dado ("T(1) = ?"); a resposta assintótica não deve depender dele, então adoto T(1) = d e trato separadamente $d \geq 0$ (caso principal; inclui o natural T(1) = 1) e $d < 0$ (observação final). (ii) A recorrência escreve n/2 sem piso nem teto, e a Figura 1 (árvore de recorrência) divide n ao meio em todos os níveis; a única leitura em que T(n/2) é sempre um argumento inteiro definido é $n = 2^k$, $k = 0, 1, 2, \ldots$. Por isso o domínio desta prova é o conjunto das potências de 2, e a afirmação Ω vale sobre esse domínio; não afirmo nada sobre n que não sejam potências de 2 (estender exigiria saber que T é não decrescente, o que o enunciado não dá). (iii) "log" sem base: uso lg = $\log_2$ e no fim mostro que a base não importa. A pergunta é sobre uma função T definida pela recorrência, não sobre pior/melhor caso de um algoritmo: a afirmação vale para todo n do domínio.
>
> **DEFINIÇÃO USADA:** $T(n) \in \Omega(g(n))$ se existem constantes $c > 0$ e $n_0 > 0$ tais que, para todo $n \geq n_0$ (no domínio), $0 \leq c\cdot g(n) \leq T(n)$. Ordem dos quantificadores: $\exists c\ \exists n_0\ \forall n \geq n_0$; c e n_0 abaixo são números fixos, que não dependem de n.

```
PASSO 1 — A AFIRMAÇÃO A PROVAR POR INDUÇÃO (CASO d ≥ 0):
Seja d = T(1) ≥ 0. Para k ≥ 0, seja
    P(k): T(2^k) ≥ 2^k · k,
isto é, T(n) ≥ 1 · n lg n para n = 2^k (pois lg 2^k = k). A constante da substituição é c = 1.
```

PASSO 2 — BASE (k = 0, n = 1): T(1) = d ≥ 0 = 1 · 1 · lg 1, pois lg 1 = 0. Logo P(0) vale. (A base é exatamente n = 1; não é preciso nenhuma base extra, porque o passo abaixo só usa o valor em n/2 ≥ 1.)

```
PASSO 3 — PASSO INDUTIVO (k ≥ 1, n = 2^k ≥ 2):
Hipótese de indução, exatamente como enunciada: P(k − 1), isto é, T(2^(k−1)) ≥ 2^(k−1) · (k − 1), ou seja, T(n/2) ≥
(n/2)·lg(n/2) = (n/2)(lg n − 1).
Como n = 2^k com k ≥ 1, n/2 = 2^(k−1) é potência de 2 maior ou igual a 1, e a recorrência se aplica:
    T(n) = 2T(n/2) + n
         ≥ 2 · (n/2)(lg n − 1) + n        (hipótese de indução, multiplicada por 2 > 0)
         = n lg n − n + n
         = n lg n
         = 1 · 2^k · k.
Logo P(k) vale. A constante c = 1 da hipótese reaparece intacta na conclusão (não "cresceu" no meio), e nenhum
termo foi descartado: a desigualdade usa só a hipótese.
```

PASSO 4 — CONCLUSÃO Ω COM TESTEMUNHAS (d ≥ 0): Por indução, T(n) ≥ n lg n para toda potência de 2 n ≥ 1. Tome c = 1 e no = 1. Para todo n ≥ no potência de 2: 0 ≤ 1 · n lg n (pois n ≥ 1 implica lg n ≥ 0) e 1 · n lg n ≤ T(n) (Passo 3). Conferência em n = no = 1 exatamente: c · 1 · lg 1 = 0 e T(1) = d ≥ 0, então 0 ≤ 0 ≤ d. Em n = 2: 1 · 2 · lg 2 = 2 e T(2) = 2d + 2 ≥ 2. Portanto T(n) ∈ Ω(n lg n) sobre as potências de 2.

PASSO 5 — A BASE DO LOGARITMO NÃO IMPORTA: Para qualquer base b > 1, log_b n = lg n / lg b, então n lg n = (lg b) · n log_b n. Logo T(n) ≥ (lg b) · n log_b n para n ≥ 1 potência de 2, e T(n) ∈ Ω(n log_b n) com testemunhas c = lg b > 0 e no = 1. Em particular vale para o "log" do enunciado, qualquer que seja a base usada pelo material.

```
OBSERVAÇÃO — SE O CASO BASE ADOTADO FOR NEGATIVO (d < 0):
A hipótese P(k) acima falha já na base (T(1) = d < 0 = 1 lg 1), então é preciso reforçá-la com um termo de ordem
menor, como é usual no método da substituição. Seja
    Q(k): T(2^k) ≥ 2^k · k + d · 2^k, isto é, T(n) ≥ n lg n + d·n para n = 2^k.
Base (k = 0): T(1) = d = 1 · 0 + d · 1, vale com igualdade.
Passo (k ≥ 1): pela hipótese Q(k − 1), T(n/2) ≥ (n/2)(lg n − 1) + d·(n/2); então
    T(n) = 2T(n/2) + n ≥ n(lg n − 1) + d·n + n = n lg n + d·n,
que é Q(k), com as mesmas constantes (1 e d) da hipótese.
Agora, para n potência de 2 com lg n ≥ −2d: d·n ≥ −(1/2)·n lg n (pois −d ≤ (1/2) lg n e n > 0), logo T(n) ≥ n lg n
− (1/2) n lg n = (1/2) n lg n. Tome c = 1/2 e n₀ = 2^⌈−2d⌉ (um número fixo que depende só da constante d, não de
n). Para n ≥ n₀ potência de 2, lg n ≥ ⌈−2d⌉ ≥ −2d, e como d < 0 temos ⌈−2d⌉ ≥ 1, logo lg n ≥ 1 > 0 e 0 ≤ (1/2) n
lg n ≤ T(n). Conferência em n = n₀ exatamente: lg n₀ = ⌈−2d⌉ ≥ −2d, que é a condição usada. Assim T(n) ∈ Ω(n lg n)
também neste caso. (Este argumento também cobre d ≥ 0, mas lá a prova mais simples dos Passos 1–4 já basta.)
```

Conferência numérica (Python; não faz parte da prova): para a recorrência (3) com T(1) = 1 e n = 2^k, a forma fechada T(2^k) = n lg n + n, que é exatamente o caso d = 1 da igualdade T(n) = n lg n + d·n obtida na observação (a desigualdade Q(k) vale ali com igualdade); em particular T(n) ≥ n lg n, coerente com as testemunhas c = 1, no = 1.

---

29. Rezende (UNICAMP), p. 9, recorrência (4)

(p. 9) Se uma recorrência é muito parecida com outra que já foi vista, é razoável supor que sua solução também seja parecida. Por exemplo, as recorrências (3) e (4) são ambas pertencentes a $\Theta(n \log n)$ (deixamos a prova de que ambas são pertencentes a $\Omega(n \log n)$ como exercício para o leitor).

Exercício (recorrência (4)): provar que a recorrência (4) pertence a $\Omega(n \log n)$.

Contexto do material (transcrição literal, necessária para resolver):
(p. 8) Vamos, então, provar por indução que a recorrência:

$T(n) = 2T(n/2) + n, \quad T(1) = ? \qquad (4)$

pertence a $O(n \log n)$, ou seja, vamos mostrar que existem constantes $c \in \mathbb{N}$ e $n_0 \in \mathbb{N}$ tais que $T(n) \leq cn \log n$, para todo $n \geq n_0$.

A única diferença da expressão (4) e da expressão (3) é o fato de estarmos usando o piso de $n/2$, ao invés da fração pura.[2]

Para realizar a prova, usaremos como hipótese o caso $\lfloor n/2 \rfloor$. [...] A partir da hipótese, podemos afirmar que

$T(n) = 2T(\lfloor n/2 \rfloor) + n \leq 2c\lfloor n/2 \rfloor \log \lfloor n/2 \rfloor + n$

(p. 9) [...] Para completar nossa prova, falta apenas encontrarmos a base da indução. A intuição comum é considerar $T(1) = 1$. [...] Se usarmos $T(2)$ e $T(3)$ como bases, a recorrência não dependerá mais do valor de $T(1)$ diretamente, para $n > 3$. A partir da expressão (4), calculamos que $T(2) = 4$ e $T(3) = 5$.

[transcrição literal — Rezende (UNICAMP), p. 9, recorrência (4)]

Resolução

Resposta. Com a recorrência (4) lida como T(n) = 2T(⌊n/2⌋) + n para n ≥ 2 e T(1) = 1 (o que dá T(2) = 4 e T(3) = 5, os valores que o texto calcula), vale T(n) ≥ (1/2)·n lg n para todo n ≥ 1. Logo T(n) ∈ Ω(n lg n), com testemunhas c = 1/2 e n0 = 1. A prova é por indução forte (método da substituição): os casos base são n = 1 e n = 2, e o passo vale para todo n ≥ 3. A constante c = 1 não serve: T(7) = 17 < 7 lg 7 ≈ 19,65. A mesma prova vale para qualquer T(1) ≥ 0.

INTERPRETAÇÃO DO ENUNCIADO: Há três leituras possíveis. (a) A equação (4) com piso, T(n) = 2T(⌊n/2⌋) + n. O texto diz que "a única diferença da expressão (4) e da expressão (3) é o fato de estarmos usando o piso de n/2", e a dedução da p. 8 usa ⌊n/2⌋. Na transcrição, a equação (4) aparece impressa como "T(n) = 2T(n/2) + n, T(1) = ?", sem o símbolo de piso. (b) A fração pura n/2. Essa leitura é a recorrência (3), que só fica definida em potências de 2, e não é o exercício. (c) T(1) indeterminado ("T(1) = ?"). Resolvo a leitura (a), com T(1) = 1. É a "intuição comum" citada na p. 9 e o único valor compatível com T(2) = 4 e T(3) = 5: T(2) = 2T(1) + 2 = 4 e T(3) = 2T(⌊3/2⌋) + 3 = 2T(1) + 3 = 5. No fim, uma observação mostra que o resultado vale para qualquer T(1) ≥ 0, o que também cobre a leitura (c) com T(1) ≥ 0. Uso lg = $\log_2$ (na recorrência (3), T(2^k) = n lg n + n). A classe Ω(n log n) não depende da base do logaritmo. Só a constante c depende dela.

DEFINIÇÃO USADA: T(n) ∈ Ω(g(n)) se existem constantes reais c > 0 e n0 tais que, para todo n ≥ n0, 0 ≤ c·g(n) ≤ T(n). A ordem dos quantificadores é ∃c ∃n0 ∀n ≥ n0. O texto, para a cota O, pede c ∈ ℕ. Para a cota inferior isso não funciona em base 2, porque c = 1 falha (ver abaixo), e por isso uso c real positivo, como no CLRS. Se o leitor exigir c natural, a mesma prova dá c = 1 com logaritmo na base 4: (1/2)·n lg n = n $\log_4$ n.

AFIRMAÇÃO P(n): P(n): T(n) ≥ (1/2)·n lg n. Provo P(n) para todo n ≥ 1 por indução forte em n.

CASOS BASE (n = 1 e n = 2): n = 1: T(1) = 1 e (1/2)·1·lg 1 = 0. Como 1 ≥ 0, P(1) vale. n = 2: T(2) = 4 e (1/2)·2·lg 2 = 1. Como 4 ≥ 1, P(2) vale. O caso n = 2 precisa ser base porque o passo abaixo exige n ≥ 3. Com n = 2, a cota (n−1)/2 = 1/2 fica menor que 1 e o logaritmo dela é negativo.

HIPÓTESE DE INDUÇÃO: Seja n ≥ 3 e suponha P(k) para todo k com 1 ≤ k < n, ou seja, T(k) ≥ (1/2)·k lg k. Uso essa hipótese exatamente uma vez, em k = ⌊n/2⌋. Esse k está no intervalo, porque para n ≥ 3 vale 1 ≤ ⌊n/2⌋ ≤ n/2 < n.

PASSO INDUTIVO (n ≥ 3): Passo 1 (definição e hipótese). T(n) = 2T(⌊n/2⌋) + n ≥ 2·(1/2)·⌊n/2⌋·lg⌊n/2⌋ + n = ⌊n/2⌋·lg⌊n/2⌋ + n.

Passo 2 (tirar o piso). Para todo n vale ⌊n/2⌋ ≥ (n−1)/2: a igualdade vale com n ímpar, e com n par ⌊n/2⌋ = n/2 > (n−1)/2. Com n ≥ 3, temos (n−1)/2 ≥ 1. A função x ↦ x lg x é crescente para x ≥ 1, porque os dois fatores, x e lg x, são não negativos e não decrescentes nesse intervalo. Então ⌊n/2⌋·lg⌊n/2⌋ ≥ ((n−1)/2)·lg((n−1)/2) = ((n−1)/2)·(lg(n−1) − 1). Portanto T(n) ≥ ((n−1)/2)·(lg(n−1) − 1) + n.

```
Passo 3 (comparar com o alvo). Quero T(n) - (1/2)·n lg n ≥ 0. Pelo passo 2,
    T(n) - (1/2)·n lg n ≥ n - (n-1)/2 - (1/2)·[n lg n - (n-1) lg(n-1)].
Seja m = n - 1 ≥ 2. Então
    n lg n - (n-1) lg(n-1) = lg n + m·(lg n - lg m) = lg n + lg((1 + 1/m)^m).
Como ln(1 + x) < x para x > 0, vale m·ln(1 + 1/m) < 1, ou seja, (1 + 1/m)^m < e < 4. Logo lg((1 + 1/m)^m) < 2, e
    n lg n - (n-1) lg(n-1) < lg n + 2.
Substituindo:
    T(n) - (1/2)·n lg n > n - (n-1)/2 - (1/2)·(lg n + 2) = (n - lg n - 1)/2.
```

Passo 4 (n − lg n − 1 ≥ 0 para todo n ≥ 1). Isso equivale a 2^(n−1) ≥ n, que se prova por indução simples. Em n = 1: 2^0 = 1 ≥ 1. Se 2^(n−1) ≥ n, então 2^n = 2·2^(n−1) ≥ 2n ≥ n + 1. Logo T(n) − (1/2)·n lg n > 0, e P(n) vale.

Com P(1), P(2) e o passo para todo n ≥ 3, P(n) vale para todo n ≥ 1.

TESTEMUNHAS E CONFERÊNCIA EM n = n0: c = 1/2 e n0 = 1. Em n = n0 = 1: c·g(1) = (1/2)·1·lg 1 = 0 e T(1) = 1, logo 0 ≤ 0 ≤ 1. Para todo n ≥ 1, 0 ≤ (1/2)·n lg n ≤ T(n). A cota vale para toda entrada n ≥ 1, porque T é uma função de n e não há pior ou melhor caso a distinguir. Portanto T(n) ∈ Ω(n lg n). Com a prova da cota O(n lg n) feita no texto, T(n) ∈ Θ(n lg n).

POR QUE c = 1 NÃO FECHA: Com c = 1, a mesma conta do passo 3 dá a cota inferior n − (n−1) − (lg n + 2) = −1 − lg n, que é negativa. Isso não é só falha da técnica: a afirmação T(n) ≥ n lg n é falsa. T(7) = 2T(3) + 7 = 17, mas 7 lg 7 ≈ 19,65. O piso ⌊n/2⌋ < n/2 com n ímpar faz T ficar abaixo de n lg n em vários n (7, 11, 13, 14, 15, ...). Por isso c precisa ficar abaixo de 1. O valor 1/2 deixa folga suficiente para absorver tanto o piso quanto o termo lg n + 2.

OBSERVAÇÃO (T(1) arbitrário não negativo): O passo indutivo não usa o valor de T(1). Os casos base valem para qualquer T(1) ≥ 0: P(1) é 0 ≤ T(1), e P(2) é T(2) = 2T(1) + 2 ≥ 2 ≥ 1. Logo T(n) ≥ (1/2)·n lg n para todo n ≥ 1 sempre que T(1) ≥ 0, com as mesmas testemunhas c = 1/2 e n0 = 1. Não afirmo nada para T(1) < 0.

Conferência numérica (Python; não faz parte da prova): T(2) = 4 e T(3) = 5, os valores que o texto calcula; e T(n) ≥ 0,5·n lg n para n = 2, …, 2·10⁵, o que é coerente com a prova (que também cobre n = 1, onde a desigualdade é 0 ≤ 1). Confirma ainda que T(n) ≥ n lg n é falsa, com n = 7 como primeiro contraexemplo, que é o motivo de c = 1 falhar. A razão mínima T(n)/(n lg n) ≈ 0,8361 nesse intervalo mostra que a folga entre 1/2 e a melhor constante empírica é grande. Isso é conferência numérica, não faz parte da prova.

---

30. Rezende (UNICAMP), p. 10

Semelhantemente, alterar o termo pelo qual n é dividido a cada passo também modifica a solução da recorrência. Uma árvore apresentando a expansão da relação de recorrência

$T(n) = 2T(\lfloor n/3 \rfloor) + n \qquad (8)$

é apresentada na figura 3. Convidamos o leitor a provar que a solução dessa relação de recorrência pertence a $O(n)$.

Contexto do material (transcrição literal, necessária para resolver):
(p. 10) Para uma recorrência com a forma $T(n) = aT(\lfloor n/b \rfloor) + n$, se a for igual a b, ocorrerá que no i-ésimo passo da expansão da recorrência (com $i \geq 0$), teremos a soma de a^i termos iguais a n/b^i. Como $a = b$, a expressão resultante do i-ésimo passo será $a^i n/a^i = n$. A árvo-

re resultante de uma expansão como essa tem altura $\log_b n$ e, conseqüentemente, a solução da recorrência pertence a $O(n \log n)$.

[transcrição literal — Rezende (UNICAMP), p. 10]

Resolução

Resposta. T(n) = 2T(⌊n/3⌋) + n pertence a O(n). Como o texto não dá o caso base, adotamos: a recorrência vale para todo n ≥ 1 e T(0) = d, com d ≥ 0 uma constante real (custo de tempo não é negativo; no caso usual, d = 0, que dá T(1) = 1, T(2) = 2). Testemunhas: c = máx(3, 2d + 1) e n0 = 1; com d = 0, c = 3 e n0 = 1. Prova por indução forte em n (método da substituição): T(n) ≤ c·n para todo n ≥ 1. No passo, para n ≥ 3, a hipótese aplicada a ⌊n/3⌋ (que é ≥ 1 e < n) dá T(n) ≤ 2c⌊n/3⌋ + n ≤ (2c/3)n + n ≤ c·n, e a última desigualdade vale exatamente porque c ≥ 3: a constante sai do passo igual à que entrou.

INTERPRETAÇÃO. O enunciado ("Convidamos o leitor a provar que a solução dessa relação de recorrência pertence a O(n)") admite duas leituras: (i) provar a cota superior O(n) formalmente, com testemunhas; (ii) apenas justificar pela árvore da figura 3 que a soma dos níveis é limitada. A leitura (i) é a que a disciplina exige (testemunhas numéricas), e é a que resolvemos; a árvore aparece só como motivação para a escolha de c. O texto também não fixa o caso base nem a partir de que n a recorrência vale; por isso o declaramos explicitamente e mostramos, no fim, que outra escolha razoável só muda o valor numérico de c, não a conclusão. O pedido é só O(n) (cota superior); não é pedido Ω nem Θ.

```
CASO BASE ADOTADO.
T está definida nos naturais n ≥ 0, com T(0) = d (constante d ≥ 0, não depende de n) e T(n) = 2T(⌊n/3⌋) + n para
todo n ≥ 1. Note que ⌊n/3⌋ < n para todo n ≥ 1, então a recorrência define T(n) para todo n a partir de T(0). Em
particular, como ⌊1/3⌋ = ⌊2/3⌋ = 0:
    T(1) = 2d + 1,
    T(2) = 2d + 2.
A afirmação "T(n) ∈ O(n)" é sobre n ≥ n0 = 1; o ponto n = 0 fica de fora (se d > 0, T(0) = d > c·0, e não há nada
a provar ali, pois a definição só exige n ≥ n0).
```

DE ONDE VEM O 3 (MOTIVAÇÃO, NÃO É PROVA). Pela árvore, o nível i tem 2^i nós, cada um com custo no máximo n/3^i, logo o nível custa no máximo (2/3)^i·n. Somando a série geométrica, n·Σ_{i≥0} (2/3)^i = n·1/(1 − 2/3) = 3n. Isso sugere c = 3. Essa conta ignora os pisos e o caso base; a prova abaixo é a que vale.

AFIRMAÇÃO A PROVAR. Seja c = máx(3, 2d + 1). Para todo n ≥ 1, P(n): T(n) ≤ c·n. Ordem dos quantificadores: existe c (= máx(3, 2d + 1), número fixo, que depende só da constante d do caso base e não de n), existe n0 = 1, tal que para todo n ≥ n0 vale T(n) ≤ c·n. Junto com 0 ≤ T(n), que vale para todo n porque d ≥ 0 (indução: T(n) = 2T(⌊n/3⌋) + n com T(⌊n/3⌋) ≥ 0), isso é exatamente a definição de T(n) ∈ O(n). Com d = 0: c = 3, n0 = 1.

```
BASE (n = 1 e n = 2).
São os valores de n ≥ 1 para os quais ⌊n/3⌋ = 0, isto é, em que a recorrência cai em T(0), onde a hipótese P não
está disponível (P só fala de n ≥ 1). Por isso são tratados à parte.
    n = 1 (que é n0 exatamente): T(1) = 2d + 1 ≤ c = c·1, porque c ≥ 2d + 1. Vale.
    n = 2: T(2) = 2d + 2. Como c ≥ 2d + 1, temos 2c ≥ 4d + 2. Precisamos 2d + 2 ≤ 2c. Como d ≥ 0: 2d + 2 ≤ 4d + 2
≤ 2c. Vale.
Com d = 0: T(1) = 1 ≤ 3 e T(2) = 2 ≤ 6.
```

HIPÓTESE DE INDUÇÃO (FORTE). Seja n ≥ 3 e suponha P(k) verdadeira para todo k com 1 ≤ k < n, isto é, T(k) ≤ c·k para esses k, com o MESMO c.

```
PASSO (n ≥ 3).
Como n ≥ 3, k = ⌊n/3⌋ satisfaz k ≥ 1; e k ≤ n/3 < n. Logo a hipótese se aplica a k: T(⌊n/3⌋) ≤ c·⌊n/3⌋. Então
    T(n) = 2T(⌊n/3⌋) + n
         ≤ 2c·⌊n/3⌋ + n          (hipótese em k = ⌊n/3⌋)
         ≤ 2c·(n/3) + n          (⌊n/3⌋ ≤ n/3 e c > 0)
         = c·n − (c/3)·n + n
         = c·n − (c/3 − 1)·n
         ≤ c·n,
pois c ≥ 3 dá c/3 − 1 ≥ 0 e n > 0. Isso é P(n).
A constante não cresce: a hipótese foi usada com c e a conclusão saiu com o mesmo c, sem nenhuma sobra positiva do
tipo "c·n + n". O termo extra n foi absorvido pela folga (c/3)·n, que é ≥ n justamente porque c ≥ 3. Com c < 3 o
passo não fecharia por esse argumento (a folga (c/3)·n seria menor que n).
Fencepost do passo, n = 3 exatamente: k = ⌊3/3⌋ = 1, e T(3) = 2T(1) + 3 = 2(2d + 1) + 3 ≤ 2c + 3 ≤ 2c + c = 3c,
usando T(1) ≤ c (base) e 3 ≤ c. Com d = 0: T(3) = 5 ≤ 9.
```

CONCLUSÃO. Por indução forte, T(n) ≤ c·n para todo n ≥ 1, com c = máx(3, 2d + 1). Portanto T(n) ∈ O(n), com testemunhas c = máx(3, 2d + 1) e n0 = 1 (c = 3, n0 = 1 no caso T(0) = 0). Note que T(n) não é custo de "pior caso" ou "melhor caso" de nada: é uma função de n bem definida pela recorrência e pelo caso base, e a cota vale para todo n ≥ 1.

SE O CASO BASE FOR OUTRO. Se, em vez disso, a recorrência valer só para n ≥ 2, com T(0) = d0 ≥ 0 e T(1) = d1 ≥ 0 dados, o mesmo argumento funciona com c = máx(3, d1, d0 + 1) e n0 = 1: a base é n = 1 (T(1) = d1 ≤ c) e n = 2 (T(2) = 2d0 + 2 ≤ 2c porque c ≥ d0 + 1); o passo para n ≥ 3 é idêntico, palavra por palavra. Por exemplo, T(0) = 1, T(1) = 1 dá c = máx(3, 1, 2) = 3. Em qualquer escolha de constantes iniciais não negativas, a conclusão O(n) não muda; muda só o número c. (Com d < 0 a cota T(n) ≤ c·n ainda sai da mesma conta, mas T fica negativo para n pequeno — com d = −10, em 118 valores de n, o último n = 129 — e então n0 = 1 não satisfaz o 0 ≤ T(n) da definição; custo negativo não modela tempo, por isso o excluímos.)

OBSERVAÇÃO (NÃO PEDIDA). Como d ≥ 0, T(n) ≥ 0 para todo n (indução: T(n) = 2T(⌊n/3⌋) + n com T(⌊n/3⌋) ≥ 0), logo T(n) ≥ n para n ≥ 1, e portanto T(n) ∈ Θ(n) com c1 = 1, c2 = c, n0 = 1. O contraste com o texto do material: lá a = b dá n por nível e Θ(n lg n); aqui a = 2 < b = 3, os níveis decrescem geometricamente e o total é linear.

Conferência numérica (Python; não faz parte da prova): calculei T(n)/n para n = 1 até 2·10^5 com T(0) = 0, T(1) = 1 (o nosso caso d = 0) e obteve máximo 2,9769 < 3, e T(n) ≤ 3n em todo o intervalo, de acordo com c = 3; com T(0) = 1, T(1) = 1 (o caso "SE O CASO BASE FOR OUTRO", c = 3) obteve máximo 2,9827 < 3, também de acordo. Isso só confere a prova num intervalo finito; a prova é a indução acima.

---

31. Rezende (UNICAMP), p. 17

Exercício: O algoritmo acima descrito tem complexidade linear, quadrática ou exponencial? Justifique sua resposta.

Contexto do material (transcrição literal, necessária para resolver):
(p. 15) 4.2 Subgrafo Induzido Maximal

Este problema consiste em, dados um grafo não-orientado $G = (V, E)$ e um inteiro k, encontrar um subgrafo induzido $H = (U, F)$ de G de tamanho máximo tal que todos os vértices de H tenham grau $\geq k$ (em H), ou concluir que nenhum tal subgrafo induzido existe.

Para resolver este problema, pode ser usado o seguinte algoritmo indutivo baseado no parâmetro $n = |V|$, o número de vértices de G:

(p. 16) **Base**: para $n = 1$, o grafo G tem um subgrafo induzido que satisfaz o enunciado (o próprio G) somente no caso em que $k = 0$.

Hipótese: Dados um grafo não-orientado $G = (V, E)$ e um inteiro k, sabemos encontrar um subgrafo induzido $H = (U, F)$ de G de tamanho máximo tal que todos os vértices de H tenham grau $\geq k$ (em H), ou concluir que nenhum tal subgrafo induzido existe, desde que o número de vértices de G seja $< n$.

Passo: Sejam $G = (V, E)$ um grafo não-orientado com $|V| = n$ e k um inteiro. Se $|V| < k + 1$, nenhum vértice pode ter grau ≥ k, conseqüentemente, não há nenhum subgrafo induzido com a propriedade desejada.

Assumamos, então, que $n \geq k + 1$. Se todos os vértices de G têm grau $\geq k$, então, basta tomarmos $H = G$. Caso contrário, selecionamos um vértice v cujo grau seja $< k$ e o removemos de G, junto com todas as suas arestas incidentes. O grafo resultante tem $n - 1$ vértices e, por hipótese de indução, sabemos resolver esse problema. CQD

Complexidade: Para analisar a complexidade deste algoritmo, é necessário formalizar um pouco mais a representação do grafo. Vamos levar em consideração apenas as duas representações mais usuais: lista de adjacências e matriz de adjacências.

[Figuras 4 e 5: exemplo de grafo com vértices 1–5 e arestas {1,4}, {2,3}, {2,4}, {2,5}, {4,5}, e sua lista de adjacências: 1→4; 2→3→4→5; 3→2; 4→1→2→5; 5→2→4.]

Na representação por lista de adjacências, é usado um vetor onde cada posição corresponde a um vértice do grafo G e, para cada vértice v de G, (p. 17) temos uma lista encadeada onde cada nó da lista corresponde a um vértice de G adjacente a v. [...]

É razoável supor que, durante a construção de qualquer das duas representações, podemos armazenar em cada vértice, seu grau. Assim, a questão de se determinar se um dado vértice tem grau menor que k se torna uma operação de tempo constante.

Usando a representação por lista de adjacências, a cada passo é necessário encontrar um vértice v cujo grau seja menor que k (tempo n, no pior caso). Uma vez que v seja encontrado, este deve ser removido e devem ser removidas as entradas correspondentes a ele nas listas de adjacências de todos os vértices adjacentes a v. Essa segunda operação depende diretamente do grau de cada vértice da lista de adjacências de v e requer um total de $\sum_{i=1}^{k-1} g(v_i)$ onde, no pior caso, $g(v_i) = n-1$. A recorrência que expressa a complexidade do algoritmo para a representação por lista de adjacências é, portanto, a seguinte:

$T(n) = T(n - 1) + n + (k - 1)(n - 1)$.

Como estamos considerando que k é uma entrada do problema, a solução desta recorrência pertence a $O(kn^2)$.

[transcrição literal — Rezende (UNICAMP), p. 17]

Resolução

Resposta. Na representação por lista de adjacências, a recorrência do texto, T(n) = T(n − 1) + n + (k − 1)(n − 1), tem forma fechada T(n) = T(0) + n(n + 1)/2 + (k − 1)·n(n − 1)/2 = T(0) + (k·n² − (k − 2)·n)/2. Para todo k ≥ 1 e todo n ≥ 2 vale (1/4)·k·n² ≤ T(n) ≤ ((3 + T(0))/4)·k·n², com constantes que não dependem nem de n nem de k; logo T(n) ∈ Θ(k·n²), o que confirma o O(kn²) do texto. Com k constante (k ≥ 1), o algoritmo é QUADRÁTICO em n: T(n) ∈ Θ(n²) (c_1 = k/4, c_2 = (3 + T(0))·k/4, n_0 = 2), não é linear (T ∉ O(n)) e não é exponencial (T ∉ Ω(2^n)). Se k puder crescer com n, a classificação só vale com ressalva: k útil vai no máximo até n − 1 (com k ≥ n o passo para de imediato), e com k = ⌊n/2⌋ o custo da recorrência é Θ(n³) — nem linear, nem quadrático. Em nenhum caso é exponencial: para k arbitrário o custo do modelo T' do Passo 4 (que para quando restam m ≤ k vértices) é O(n³). Todas as afirmações são sobre o custo de pior caso que a recorrência do texto modela (é uma cota superior para o tempo do algoritmo).

INTERPRETATION CHECK.

Leitura 1, "o algoritmo acima": o algoritmo indutivo do Subgrafo Induzido Maximal (retirar um vértice de grau < k e recorrer em n − 1 vértices), analisado na representação por lista de adjacências — a única cuja análise está no trecho, com a recorrência T(n) = T(n − 1) + n + (k − 1)(n − 1). No material, o Exercício vem logo depois da análise por lista de adjacências; a análise por matriz de adjacências vem depois dele (T(n) = T(n − 1) + n + n, isto é, T(n) = T(0) + n(n + 1), também Θ(n²) e quadrática), então a resposta não depende da representação. Esta é a leitura que resolvo.

Leitura 2, "linear, quadrática ou exponencial" em função de quê: do parâmetro da indução, n = |V|. O texto escreve O(kn²) e diz que "k é uma entrada do problema"; então há duas sub-leituras: (a) k fixo (constante independente de n) — a pergunta de múltipla escolha só tem resposta exata aqui, e a resposta é "quadrática"; (b) k livre, podendo crescer com n — trato também, porque muda a resposta (passa a cúbica no pior caso de k), e mostro que nem assim fica exponencial.

Leitura 3, "complexidade": custo de pior caso, como o próprio texto modela (busca do vértice em tempo n no pior caso; graus $g(v_i) = n - 1$ no pior caso). A recorrência é uma cota superior do tempo do algoritmo; os Θ abaixo são do valor $T(n)$ da recorrência. Não afirmo que toda entrada gaste isso: o algoritmo pode parar antes (por exemplo, se todos os vértices já têm grau $\geq k$ na primeira verificação).

Hipótese sobre k: tomo $k \geq 1$. Com $k \leq 0$ todo vértice tem grau $\geq k$, o algoritmo devolve $H = G$ na primeira verificação, e a recorrência nem se aplica (o termo $(k - 1)(n - 1)$ seria negativo, um "custo" sem sentido).

PASSO 1 — FORMA FECHADA DA RECORRÊNCIA LITERAL:

A recorrência vale para $n \geq 1$, com $T(0) = t_0 \geq 0$ uma constante. Desenrolando (telescópio), $T(n) - T(0) = \Sigma_{j=1}^{n} [T(j) - T(j - 1)] = \Sigma_{j=1}^{n} [j + (k - 1)(j - 1)]$.

Fechando as duas somas: $\Sigma_{j=1}^{n} j = n(n + 1)/2$ e $\Sigma_{j=1}^{n} (j - 1) = \Sigma_{i=0}^{n-1} i = n(n - 1)/2$. Logo

$T(n) = t_0 + n(n + 1)/2 + (k - 1)\cdot n(n - 1)/2 = t_0 + [n^2 + n + (k - 1)n^2 - (k - 1)n]/2 = t_0 + (k\cdot n^2 - (k - 2)\cdot n)/2$.

Conferência à mão nos primeiros valores. $n = 1$: pela recorrência, $T(1) = t_0 + 1 + (k - 1)\cdot 0 = t_0 + 1$; pela fórmula, $t_0 + (k - k + 2)/2 = t_0 + 1$. $n = 2$: pela recorrência, $T(2) = T(1) + 2 + (k - 1)\cdot 1 = t_0 + k + 2$; pela fórmula, $t_0 + (4k - 2k + 4)/2 = t_0 + k + 2$. Batem.

PASSO 2 — $T(n) \in \Theta(k\cdot n^2)$ COM CONSTANTES UNIFORMES EM k:

Afirmação: para todo $k \geq 1$ e todo $n \geq 2$, $(1/4)\cdot k\cdot n^2 \leq T(n) \leq ((3 + t_0)/4)\cdot k\cdot n^2$.

Cota inferior. Como $t_0 \geq 0$ e $-(k - 2) \geq -k$, $T(n) \geq (k\cdot n^2 - k\cdot n)/2 = k\cdot n(n - 1)/2$. Para $n \geq 2$, $n - 1 \geq n/2$, logo $T(n) \geq k\cdot n\cdot(n/2)/2 = k\cdot n^2/4$. Em $n = n_0 = 2$ exatamente: $T(2) = t_0 + k + 2 \geq k = (1/4)\cdot k\cdot 4$. Vale.

Cota superior. $-(k - 2)\cdot n/2 = -k\cdot n/2 + n \leq n/2$ quando $k \geq 1$ (pois $-k\cdot n/2 + n \leq -n/2 + n = n/2$). Então $T(n) \leq t_0 + k\cdot n^2/2 + n/2$. Para $n \geq 2$: $n/2 \leq n^2/4 \leq k\cdot n^2/4$ (pois $k \geq 1$); e $t_0 \leq t_0\cdot k\cdot n^2/4$, porque $k\cdot n^2/4 \geq 1\cdot 4/4 = 1$. Somando, $T(n) \leq k\cdot n^2/2 + k\cdot n^2/4 + t_0\cdot k\cdot n^2/4 = ((3 + t_0)/4)\cdot k\cdot n^2$. Em $n = n_0 = 2$ com $k = 1$ (o caso mais apertado): $T(2) = t_0 + 3$ e $((3 + t_0)/4)\cdot 1\cdot 4 = 3 + t_0$ — igualdade, a desigualdade vale.

As constantes $c_1 = 1/4$, $c_2 = (3 + t_0)/4$ e $n_0 = 2$ são escolhidas antes de n e não dependem de n nem de k. Isso confirma o $O(kn^2)$ do texto e mostra que ele é justo (também $\Omega(kn^2)$) para a recorrência.

PASSO 3 — k CONSTANTE: QUADRÁTICA.

Fixado $k \geq 1$ (constante), o Passo 2 dá, para todo $n \geq n_0 = 2$, $c_1\cdot n^2 \leq T(n) \leq c_2\cdot n^2$ com $c_1 = k/4 > 0$ e $c_2 = (3 + t_0)\cdot k/4$. Logo $T(n) \in \Theta(n^2)$: quadrática. Em $n = 2$: $c_1\cdot 4 = k \leq t_0 + k + 2 = T(2) \leq (3 + t_0)\cdot k = c_2\cdot 4$, pois $t_0 + k + 2 \leq (3 + t_0)k$ equivale a $t_0(k - 1) + 2(k - 1) \geq 0$, verdadeiro para $k \geq 1$.

Não é linear: $T \notin O(n)$. Negando a definição: para todo $c > 0$ e todo $n_0 \geq 1$, tome $n = \max(n_0, 2, \lfloor 4c/k \rfloor + 1)$. Então $n \geq n_0$, $n \geq 2$ e $n > 4c/k$, logo $T(n) \geq k\cdot n^2/4 = (k\cdot n/4)\cdot n > c\cdot n$. Isso viola $T(n) \leq c\cdot n$ nesse $n \geq n_0$.

Não é exponencial: $T \notin \Omega(2^n)$. Primeiro, $m^2 \leq 2^m$ para todo inteiro $m \geq 4$ (indução: P(m): $m^2 \leq 2^m$. Base $m = 4$: $16 \leq 16$. Passo: se $m \geq 4$ e $m^2 \leq 2^m$, então $(m + 1)^2 = m^2 + 2m + 1 \leq 2m^2$ porque $m^2 - 2m - 1 = (m - 1)^2 - 2 \geq 9 - 2 > 0$; logo $(m + 1)^2 \leq 2\cdot m^2 \leq 2\cdot 2^m = 2^{(m+1)}$). Para $n = 2m$ par com $m \geq 4$: $n^2 = 4m^2 \leq 4\cdot 2^m = 4\cdot 2^{(n/2)}$. Agora a negação: para todo $c > 0$ e todo n_0, tome m inteiro com $m \geq \max(4, n_0)$ e $2^m > 4c_2/c$ (existe, pois 2^m cresce sem limite), e $n = 2m \geq n_0$. Então $T(n) \leq c_2\cdot n^2 \leq 4c_2\cdot 2^m < c\cdot 2^m\cdot 2^m = c\cdot 2^n$. Isso viola $T(n) \geq c\cdot 2^n$ nesse $n \geq n_0$. Como $a^n \geq 2^n$ para $a \geq 2$, também $T \notin \Omega(a^n)$ para toda base $a \geq 2$.

PASSO 4 — k CRESCENDO COM n: ATÉ ONDE, E O QUE MUDA.

Até onde k pode ir. O passo do algoritmo diz: se $|V| < k + 1$, não há subgrafo com a propriedade, e para. Então com $k \geq n$ o algoritmo termina na primeira verificação, em tempo constante; o único k que gera trabalho é $1 \leq k \leq n - 1$. Além disso, como k é fixo durante toda a recursão e o número de vértices cai de um em um, a recursão só continua enquanto o grafo corrente tem $m \geq k + 1$ vértices. Um modelo mais fiel ao passo é, portanto,

$T'(m) = c_0$ (constante, $c_0 \geq 0$) para $m \leq k$, $T'(m) = T'(m - 1) + m + (k - 1)(m - 1)$ para $m \geq k + 1$,

cuja forma fechada, para $n \geq k$, é $T'(n) = c_0 + \Sigma_{m=k+1}^{n} [m + (k - 1)(m - 1)] = c_0 + [n(n + 1) - k(k + 1)]/2 + (k - 1)\cdot[n(n - 1) - k(k - 1)]/2$. Conferência em $n = k + 1$: a soma tem um termo, $(k + 1) + (k - 1)k$; a fórmula dá $[(k + 1)(k + 2) - k(k + 1)]/2 + (k - 1)[(k + 1)k - k(k - 1)]/2 = (k + 1) + (k - 1)k$. Bate.

Cota superior em função só de n. As parcelas somadas em $T'(n) - c_0$ são parte das parcelas (todas ≥ 0) de $T(n) - t_0$ do Passo 1, logo $T'(n) - c_0 \leq (k\cdot n^2 - (k - 2)\cdot n)/2 \leq (3/4)\cdot k\cdot n^2$ para $n \geq 2$ (Passo 2 com $t_0 = 0$). Com $k \leq n - 1 < n$: $T'(n) \leq c_0 + (3/4)\cdot n^3$. Então, qualquer que seja k, o custo modelado é $O(n^3)$: para $n \geq 1$, $T'(n) \leq (3/4 + c_0)\cdot n^3$ (pois $c_0 \leq c_0\cdot n^3$). Em particular nunca é exponencial (ver nota abaixo).

Nota sobre "nunca exponencial" com cota n^3: para $m \geq 10$, $m^3 \leq 2^m$ (P(m): $m^3 \leq 2^m$; base $m = 10$: $1000 \leq 1024$; passo: $(m + 1)^3 \leq 2m^3$ porque $((m + 1)/m)^3 \leq (1{,}1)^3 = 1{,}331 \leq 2$ para $m \geq 10$). Para $n = 3m$ com $m \geq 10$: $n^3 = 27m^3 \leq 27\cdot 2^m = 27\cdot 2^{(n/3)}$. Dado $c > 0$ e n_0, tome $m \geq \max(10, n_0)$ com $2^{(2m)} > 27\cdot(3/4 + c_0)/c$ e $n = 3m$: $T'(n) \leq (3/4 + c_0)\cdot 27\cdot 2^m < c\cdot 2^{(2m)}\cdot 2^m = c\cdot 2^n$. Logo $T' \notin \Omega(2^n)$.

Com $k = \lfloor n/2 \rfloor$ o custo é cúbico, não quadrático. Tome $k = \lfloor n/2 \rfloor$ e $n \geq 6$ (então $k \geq 3 \geq 1$ e $k \leq n/2$). Pelo fechamento de T', $T'(n) - c_0 \geq (k - 1)\cdot[n(n - 1) - k(k - 1)]/2$, já que a outra parcela, $[n(n + 1) - k(k + 1)]/2$, é ≥ 0 ($n \geq k$). Com $k \leq n/2$: $k(k - 1) \leq (n/2)(n/2 - 1) = n^2/4 - n/2$, logo $n(n - 1) - k(k - 1) \geq n^2 - n - n^2/4 + n/2 = 3n^2/4 - n/2 \geq n^2/2$ para $n \geq 2$. E $k -$

$1 \geq (n-1)/2 - 1 = (n-3)/2 \geq n/4$ para $n \geq 6$. Portanto $T'(n) \geq (n/4)\cdot(n^2/2)/2 = n^3/16$ para $n \geq 6$. Em $n = n_0 = 6$ exatamente: $k = 3$, $T'(6) - c_0 = \Sigma_{m=4}^{6} [m + 2(m-1)] = 10 + 13 + 16 = 39 \geq 216/16 = 13{,}5$. Vale. Do lado de cima, $T'(n) \leq c_0 + (3/4)\cdot k\cdot n^2 \leq c_0 + (3/8)\cdot n^3$. Logo, com $k = \lfloor n/2 \rfloor$, $T'(n) \in \Theta(n^3)$ ($c_1 = 1/16$, $c_2 = 3/8 + c_0$, $n_0 = 6$; em $n = 6$: $13{,}5 \leq 39 + c_0 \leq (3/8 + c_0)\cdot 216 = 81 + 216c_0$).

Não é quadrático nesse regime: para todo $c > 0$ e n_0, tome $n = \max(n_0, 6, \lfloor 16c \rfloor + 1)$; então $T'(n) \geq n^3/16 = (n/16)\cdot n^2 > c\cdot n^2$, violando $T'(n) \leq c\cdot n^2$. O mesmo vale para a recorrência literal do texto: com $k = \lfloor n/2 \rfloor \geq n/4$ ($n \geq 2$), o Passo 2 dá $T(n) \geq k\cdot n^2/4 \geq n^3/16$.

Resumo do Passo 4: a resposta "quadrática" pressupõe k constante (ou, mais geralmente, k limitado por uma constante). Se k é uma entrada que pode crescer junto com n, a recorrência literal dá $\Theta(k\cdot n^2)$; o modelo T' dá $O(k\cdot n^2) \subseteq O(n^3)$, que vai de quadrático (k constante) a cúbico ($k = \lfloor \alpha n \rfloor$ com $0 < \alpha < 1$ fixo, por exemplo $k = \lfloor n/2 \rfloor$), e o teto é $O(n^3)$ porque k útil é no máximo $n - 1$. Com k perto de n ($k = n - d$, $d \geq 1$ constante) restam só $O(1)$ passos de custo $\Theta(k\cdot n)$ e T' volta a $\Theta(n^2)$: com $k = n - 1$, $T'(n) = c_0 + n + (n-2)(n-1)$. Em nenhum regime é exponencial nem, para $k \geq 1$ com a recorrência dada, linear.

CONCLUSÃO.

Pela recorrência do texto (lista de adjacências, pior caso), o algoritmo tem custo $\Theta(k\cdot n^2)$. Como a pergunta é em função de n e oferece "linear, quadrática ou exponencial", a resposta é: quadrática, desde que k seja tratado como constante. Ela não é linear ($T \notin O(n)$) nem exponencial ($T \notin \Omega(2^n)$); o algoritmo é polinomial. Se k crescer com n, a dependência em k aparece: com k proporcional a n e longe de n ($k = \lfloor \alpha n \rfloor$, $0 < \alpha < 1$; por exemplo $k = \lfloor n/2 \rfloor$) o custo modelado chega a $\Theta(n^3)$, e nunca passa de $O(n^3)$, porque com $k \geq n$ o algoritmo para na primeira verificação.

Conferência numérica (Python; não faz parte da prova): a forma fechada do Passo 1, $T(n) - T(0) = n(n+1)/2 + (k-1)\cdot n(n-1)/2$, confere. A conferência numérica não cobriu as testemunhas $c_1 = 1/4$, $c_2 = (3 + t_0)/4$, $n_0 = 2$, a forma fechada de T' nem o regime $k = \lfloor n/2 \rfloor$; esses foram conferidos à mão nos valores de fronteira ($n = 1, 2$ para T; $n = k + 1$ e $n = 6$ com $k = 3$ para T').

---

32. Rezende (UNICAMP), p. 18

Exercício: Com o uso da estrutura auxiliar de *heap* como descrito acima, qual a complexidade final do algoritmo?

Contexto do material (transcrição literal, necessária para resolver):
(p. 15–17) [Mesmo problema e algoritmo do exercício da p. 17: Subgrafo Induzido Maximal — ver contexto daquele item; recorrência para lista de adjacências $T(n) = T(n-1) + n + (k-1)(n-1)$, solução em $O(kn^2)$.]

(p. 17) Se usarmos a representação por matriz de adjacências, a tarefa do algoritmo se torna um pouco mais simples. Neste caso, quando encontramos um vértice v (p. 18) de grau $< k$, precisamos percorrer apenas uma linha (correspondente ao vértice v) e uma coluna (correspondente aos vértices aos quais v é adjacente) da matriz. A complexidade para esta representação é descrita pela seguinte relação de recorrência:

$T(n) = T(n-1) + n + n$.

A solução para esta recorrência pertence a $O(n^2)$, ou seja, a representação da informação aumentou a velocidade do algoritmo.

É possível reduzir ainda mais o tempo necessário para se encontrar os vértices de grau menor que k através do uso de uma estrutura auxiliar como, por exemplo, um *heap* [2] com a chave de acesso sendo o grau de cada vértice. O tempo necessário para construir o *heap* inicial é $O(n \log n)$ e a remoção do i-ésimo elemento do *heap* toma tempo $O(\log(n - i + 1))$.

(p. 17) Na representação por matriz de adjacências, é construída uma matriz $n \times n$, M, onde as posições $M_{i,j}$ ($1 \leq i, j \leq n$) são preenchidas de modo que $M_{i,j} = 0$ se não há aresta entre os vértices i e j, $M_{i,j} = 1$ se há tal aresta. [...]

[transcrição literal — Rezende (UNICAMP), p. 18]

Resolução

Resposta. A complexidade final depende da representação do grafo e de k; tudo abaixo vale no pior caso sobre grafos com n vértices (e m arestas), com os graus já armazenados durante a construção da representação, como o texto supõe. Com matriz de adjacências, o algoritmo fica em $O(n^2 + k\cdot n \log n)$ para todo $k \geq 0$. Para k constante, $k \geq 1$, isso é $\Theta(n^2)$, a mesma classe $O(n^2)$ da matriz sem heap: cada vértice retirado ainda custa a leitura de uma linha e uma coluna ($2n$ posições), e o grafo sem arestas força $n - k$ retiradas. Com $k = 0$ nada é retirado e resta a construção do heap, $O(n \log n)$; com $k \geq n$ o algoritmo para no primeiro teste ($|V| < k + 1$), em $O(1)$. O heap acelera só a busca do vértice de grau $< k$, não a classe. Com listas de adjacências, sob as hipóteses H1–H3 do Passo 3, o algoritmo fica em $O(m + (k+1)\cdot n \log n)$, que para k constante é $O(m + n \log n)$; para $k \geq 1$, essa cota é de classe menor que o $O(k\, n^2)$ do texto sem heap sempre que $m \in o(n^2)$. Em qualquer das duas representações, a parcela do heap descrita no texto (construção $O(n \log n)$ mais as remoções, $\Sigma \lg(n - i + 1)$) soma $O(n \log n)$, e para k constante, $k \geq 1$, essa soma de cotas é de ordem exatamente $n \log n$ na pior entrada ($\Sigma_{j=k+1}^{n} \lg j = \lg n! - \lg k!$). Ela mede só o custo de encontrar e retirar os vértices, não o do algoritmo completo.

INTERPRETAÇÃO.
O enunciado pergunta pela "complexidade final do algoritmo" com o heap "como descrito acima". O algoritmo é o do Subgrafo Induzido Maximal, no passo literal do texto, repetido sobre o grafo atual G com |V| vértices:
    (i) se |V| < k + 1, para: "nenhum vértice pode ter grau ≥ k", e não há subgrafo induzido com a propriedade;
    (ii) senão, se todos os vértices de G têm grau ≥ k, devolve H = G;
    (iii) senão, seleciona um vértice v de grau < k, remove-o de G junto com suas arestas e repete.
O texto supõe que os graus já estão disponíveis: "É razoável supor que, durante a construção de qualquer das duas representações, podemos armazenar em cada vértice, seu grau. Assim, a questão de se determinar se um dado vértice tem grau menor que k se torna uma operação de tempo constante." Adoto essa premissa: os graus iniciais não são calculados pelo algoritmo. O heap tem como chave o grau de cada vértice; o teste (i) usa o contador |V|, o teste (ii) olha a chave mínima do heap, e o passo (iii) retira o mínimo do heap. A resposta tem de incluir tudo o que o algoritmo faz: os testes, a construção do heap, as retiradas, a enumeração dos vizinhos de v e a atualização dos graus deles. O custo das operações do heap (Passos 1 e 2) é uma parcela desse total; os Passos 4 e 5 dão o total em cada representação. Considero k inteiro, k ≥ 0.

QUANTAS RETIRADAS. Seja r o número de vértices retirados. Antes da i-ésima retirada o grafo atual tem n − i + 1 vértices, e o teste (i) exige n − i + 1 ≥ k + 1, isto é, i ≤ n − k. Logo:
    k = 0: o teste (ii) já passa (todo grau é ≥ 0), r = 0;
    k ≥ n: o teste (i) para de saída (n < k + 1), r = 0;
    1 ≤ k ≤ n − 1: r ≤ n − k ≤ n − 1.
Em todos os casos r ≤ n − 1 quando n ≥ 2, e o número de execuções do teste (i), que é r + 1, é ≤ n para todo n ≥ 1.

MODELO DE CUSTO (constantes do texto).
Os custos do texto vêm em notação O sem constantes, então as testemunhas abaixo são dadas em função das constantes dessas cotas, não como números fixos. Sejam:
    b > 0: a construção do heap com n chaves custa no máximo b · n lg n, para n ≥ 2;
    a > 0: a i-ésima remoção do heap custa no máximo a · (1 + lg(n − i + 1));
    g > 0: cada repetição dos testes (i) e (ii), mais a marcação de v como retirado, custa no máximo g.
O "1 +" é necessário: se o argumento for 1 (heap com um só elemento), lg 1 = 0, mas a remoção custa pelo menos uma operação; a leitura literal "custo ≤ a · lg 1 = 0" seria impossível. Com o "1 +", a cota vale para todo i de 1 a n. Uso lg = $\log_2$; a base não altera a classe.

PASSO 1 — PARCELA DO HEAP: CONSTRUÇÃO MAIS REMOÇÕES (COTA SUPERIOR).
Há r ≤ n remoções. Como cada parcela a(1 + lg(n − i + 1)) é ≥ 0, a soma sobre i = 1..r é no máximo a soma sobre i = 1..n:
    Σ_{i=1}^{n} a(1 + lg(n − i + 1)) = a(n + Σ_{j=1}^{n} lg j) = a(n + lg n!),
trocando j = n − i + 1 (quando i vai de 1 a n, j vai de n a 1). Como cada fator de n! = 1 · 2 · … · n é ≤ n, n! ≤ n^n e lg n! ≤ n lg n. Para n ≥ 2, lg n ≥ 1, logo n ≤ n lg n. Assim, para todo n ≥ 2,
    custo do heap ≤ b n lg n + a(n + n lg n) ≤ b n lg n + 2a n lg n = (b + 2a) n lg n.
Testemunhas: c = b + 2a, n_0 = 2. Em n = n_0 = 2: n lg n = 2, n = 2 ≤ 2 e lg 2! = 1 ≤ 2; a desigualdade vale. Logo a parcela do heap está em O(n log n) no pior caso sobre grafos com n vértices, para todo k ≥ 0 (a cota não depende de quantos vértices são de fato retirados; com k ≥ n o heap nem chega a ser construído, e a cota continua valendo como cota superior).

PASSO 2 — A SOMA DAS COTAS DO HEAP NÃO PODE SER MELHORADA (k CONSTANTE, k ≥ 1).
Primeiro, lg n! ∈ θ(n lg n): a cota superior lg n! ≤ n lg n vale para todo n ≥ 1 (Passo 1). Para a inferior, lg n! ≥ Σ_{i=⌈n/2⌉}^{n} lg i, pois as parcelas omitidas lg 1, …, lg(⌈n/2⌉ − 1) são ≥ 0. A soma restante tem n − ⌈n/2⌉ + 1 ≥ n/2 parcelas (n par: n/2 + 1; n ímpar: (n + 1)/2), cada uma ≥ lg⌈n/2⌉ ≥ lg(n/2) ≥ 0 para n ≥ 2. Então lg n! ≥ (n/2) lg(n/2) = (n/2)(lg n − 1), e (n/2)(lg n − 1) ≥ (n/4) lg n equivale a lg n ≥ 2, isto é, n ≥ 4. Em n = n_0 = 4: lg 4! = lg 24 ≈ 4,585 ≥ (4/4) · 2 = 2. Logo lg n! ∈ θ(n lg n), com c_1 = 1/4, c_2 = 1, n_0 = 4.
Agora a soma das cotas das remoções que de fato ocorrem. Fixe k ≥ 1 e tome o grafo sem arestas com n ≥ k + 1 vértices: todo grau é 0 < k, então o algoritmo retira vértices enquanto |V| ≥ k + 1, ou seja, r = n − k, e para no teste (i) com |V| = k. A i-ésima remoção ocorre com o heap de tamanho n − i + 1, e a soma das cotas é
    Σ_{i=1}^{n−k} a(1 + lg(n − i + 1)) = a((n − k) + Σ_{j=k+1}^{n} lg j) = a((n − k) + lg n! − lg k!).
Para n ≥ 2k + 2 vale n/2 ≥ k + 1, logo ⌈n/2⌉ ≥ k + 1, e a soma Σ_{j=k+1}^{n} lg j contém todas as parcelas Σ_{j=⌈n/2⌉}^{n} lg j usadas acima; então, para n ≥ max(4, 2k + 2), Σ_{j=k+1}^{n} lg j ≥ (n/4) lg n. Em n = n_0 = max(4, 2k + 2) com k = 1 (n_0 = 4): lg 2 + lg 3 + lg 4 = lg 24 ≈ 4,585 ≥ 2. Junto com a cota do Passo 1, a soma das cotas das remoções está em θ(n log n) nessa família, para cada k ≥ 1 fixo. Com k = 0 ou k ≥ n não há remoção, e a parcela do heap é só a construção (k = 0) ou nada (k ≥ n).
Isso mostra que somar as cotas do texto dá ordem n log n, não menos; não afirma um limite inferior para o tempo real das remoções, porque o texto só fornece cotas superiores. Observação: a cota O(n log n) da construção é a que o texto dá; um heap binário pode ser construído em O(n), mas isso não muda a ordem da parcela do heap quando k ≥ 1 é constante, porque a soma das cotas das remoções já é de ordem n log n na família acima.

PASSO 3 — HIPÓTESES QUE O TEXTO DEIXA IMPLÍCITAS. H1 (atualização das chaves). Retirar v remove as arestas de v; cada vizinho u de v ainda presente perde 1 de grau, e a chave de u no heap fica desatualizada. Se não for corrigida, o heap pode deixar de devolver um vértice de grau < k que existe. É preciso uma operação de diminuição de chave, que num heap binário custa O(log tamanho) ≤ O(log n), desde que se guarde, para cada vértice, sua posição no vetor do heap (um índice atualizado a cada troca, O(1) extra por troca). Seja d > 0 tal que cada diminuição de chave custa no máximo d(1 + lg n). H2 (quantas atualizações). No momento da remoção, v tem grau atual < k, ou seja, no máximo k − 1 vizinhos ainda presentes. Logo há no máximo k − 1 diminuições de chave por remoção; com r ≤ n − k remoções (1 ≤ k ≤ n − 1), são no máximo (k − 1)(n − k) ≤ k·n ao todo. Com k = 0 ou k ≥ n não há remoção nem diminuição. Em todos os casos o total D satisfaz D ≤ k·n, e também D ≤ m, pois cada aresta provoca no máximo uma diminuição (quando o primeiro dos seus extremos sai). Este é o mesmo fator (k − 1) que aparece na recorrência da lista de adjacências do texto, T(n) = T(n − 1) + n + (k − 1)(n − 1). H3 (representação: como achar os vizinhos de v). Matriz: ao retirar v, percorre-se a linha e a coluna de v, como diz o texto, 2n leituras, ignorando os vizinhos já retirados (marca por vértice, O(1)). Listas: percorre-se a lista original de v ignorando os marcados como retirados, sem apagar v das listas dos vizinhos. Cada vértice é retirado no máximo uma vez, então cada lista é percorrida no máximo uma vez: no máximo Σ_v grau(v) = 2m entradas lidas. (Se,

em vez de marcar, se apagasse v da lista de cada vizinho percorrendo-a, como na recorrência do texto, então, para $k \geq 2$, cada remoção pagaria até $(k-1)(n-1)$ e a cota voltaria a $O(k\,n^2)$, sem ganho na cota com o heap. Por exemplo, a estrela $K_{1,n-1}$ com $k = 2$ e $n \geq 3$: as $n-1$ folhas têm grau $1 < 2$ e o centro tem grau ≥ 2 até restarem 2 vértices, quando o teste (i) para; saem $n-2$ folhas. No pior caso sobre a ordem das listas, cada folha retirada está no fim da lista do centro, e apagá-la custa $(n-1) + (n-2) + \ldots + 2 = n(n-1)/2 - 1$ passos, $\Theta(n^2)$; com a ordem favorável custaria só $n-2$. Com $k = 1$ o vértice retirado tem grau 0, a parcela de apagar é nula e o ganho permanece.) Se os graus não estivessem armazenados, calculá-los custaria n^2 leituras com matriz e $O(n+m)$ com listas; as cotas superiores dos Passos 4 e 5 não mudariam (n^2 já é parcela da cota da matriz, e $n + m \leq m + n \lg n$ para $n \geq 2$), mas o caso $k = 0$ da matriz passaria a custar n^2 leituras em vez de só a construção do heap.

PASSO 4 — ALGORITMO COMPLETO, MATRIZ DE ADJACÊNCIAS.
Seja $e > 0$ tal que ler uma posição da matriz e testar a marca custa no máximo e. Com r retiradas e D diminuições, para $n \geq 2$,
$T(n) \leq g\,n$ (testes, $r + 1 \leq n$) $+ b\,n \lg n$ (construção) $+ a\,\Sigma_{i=1}^{r}(1 + \lg(n-i+1))$ (remoções) $+ e \cdot 2n \cdot r$ (linha e coluna dos retirados) $+ d(1 + \lg n) \cdot D$ (H1, H2).
Em forma de recorrência, cada remoção custa $O(\log n)$ no heap mais $2n$ na matriz mais as diminuições de chave: $T(n) = T(n-1) + O(\log n) + 2n + $ (diminuições), sobre a construção do heap. Pelo Passo 1, as duas parcelas do heap somam $\leq (b + 2a)\,n \lg n$. Com $r \leq n$, a parcela da matriz é $\leq 2e\,n^2$. Com $D \leq k\,n$ e $1 + \lg n \leq 2 \lg n$ ($n \geq 2$), a última é $\leq 2d\,k\,n \lg n$. Como $n \leq n \lg n \leq n^2$ para $n \geq 2$ ($\lg n \geq 1$ e $\lg n \leq n$),
$T(n) \leq (g + b + 2a + 2e)\,n^2 + 2d \cdot k\,n \lg n \leq C(n^2 + k\,n \lg n)$, com $C = g + b + 2a + 2e + 2d$ e $n_0 = 2$.
Em $n = n_0 = 2$: $n = 2 = n \lg n \leq 4 = n^2$ e $1 + \lg 2 = 2 = 2 \lg 2$; vale. Logo $T \in O(n^2 + k\,n \log n)$ no pior caso sobre grafos de n vértices, para todo $k \geq 0$.
k constante, $k \geq 1$ (k fixo, independente de n). Cota superior: $k\,n \lg n \leq k\,n^2$ (pois $\lg n \leq n$), e $T(n) \leq C(1 + k)\,n^2$ para $n \geq 2$: $O(n^2)$, testemunhas $c_2 = C(1 + k)$, $n_0 = 2$. Cota inferior no pior caso: no grafo sem arestas com $n \geq k + 1$, saem $r = n - k$ vértices (Passo 2), cada um com $2n$ leituras, num total de $2n(n-k)$ leituras, cada uma custando pelo menos uma operação. Para $n \geq 2k$ vale $n - k \geq n/2$ e $2n(n-k) \geq n^2$; em $n = n_0 = 2k$: $2 \cdot 2k \cdot k = 4k^2 = n^2$, e $2k \geq k + 1$ porque $k \geq 1$. Logo $T \in \theta(n^2)$ no pior caso, com $c_1 = 1$, $c_2 = C(1 + k)$, $n_0 = 2k$ (≥ 2). Por exemplo, com $k = 1$ o grafo sem arestas perde $n - 1$ vértices e custa $2n(n-1)$ leituras. É a mesma classe da matriz sem heap ($T(n) = T(n-1) + n + n$ do texto, $O(n^2)$): com matriz, percorrer linha e coluna de v já custa $2n$ por remoção, e o heap não reduz isso. $\theta(n^2)$ é afirmação de pior caso: há entradas bem mais baratas (por exemplo, o grafo completo K_n com $k \leq n - 1$ não perde vértice nenhum).
$k = 0$. Não há remoção, leitura da matriz nem diminuição; o custo é o teste (i), a construção do heap e o teste (ii): $T(n) \leq 2g + b\,n \lg n \leq (2g + b)\,n \lg n$ para $n \geq 2$ (pois $n \lg n \geq 1$; em $n = n_0 = 2$, $n \lg n = 2$ e $2g \leq 4g$), $O(n \log n)$ com $c = 2g + b$, $n_0 = 2$, em toda entrada. (Com uma construção em $O(n)$, seria $O(n)$.)
$k \geq n$. O teste (i) para de saída: $T(n) \leq g$, $O(1)$, em toda entrada, supondo $n = |V|$ disponível (é o tamanho do vetor ou da matriz). Por exemplo, K_n com $k = n$ cai aqui: nenhum vértice tem grau $\geq n$, e o algoritmo conclui que não há subgrafo sem retirar nada.
k variável. Pela H2, $D \leq (k-1)(n-k) \leq (n-1)^2/4$ quando $1 \leq k \leq n - 1$ (o produto de dois números de soma $n - 1$ é no máximo $((n-1)/2)^2$), e $D = 0$ fora desse intervalo. Logo $D \leq n^2$ e a cota fica $O(n^2 + n^2 \log n) = O(n^2 \log n)$. Essa cota é maior que o $O(n^2)$ sem heap; não afirmo que o custo real chegue a $n^2 \log n$, porque o texto só dá cotas superiores para as operações do heap.

PASSO 5 — ALGORITMO COMPLETO, LISTAS DE ADJACÊNCIAS.
Seja $e > 0$ a constante por entrada de lista lida (com o teste da marca). Para $n \geq 2$,
$T(n) \leq g\,n$ (testes) $+ e \cdot 2m$ (listas dos retirados, H3) $+ (b + 2a)\,n \lg n$ (heap, Passo 1) $+ 2d\,k\,n \lg n$ (H1, H2, com $1 + \lg n \leq 2 \lg n$).
Como $n \leq n \lg n$ para $n \geq 2$, $T(n) \leq 2e\,m + (g + b + 2a)\,n \lg n + 2d\,k\,n \lg n \leq C(m + (k+1)\,n \lg n)$, com $C = 2e + g + b + 2a + 2d$ e $n_0 = 2$. Em $n = n_0 = 2$: $n = 2 = n \lg n$; vale. Logo $T \in O(m + (k+1)\,n \log n)$ no pior caso sobre grafos com n vértices e m arestas, para todo $k \geq 0$. O "+ 1" cobre $k = 0$: mesmo sem nenhuma remoção, a construção do heap custa a parcela $n \log n$. Para k constante (k fixo), $k + 1$ é constante e $T \in O(m + n \log n)$, com $c = C(k+1)$, $n_0 = 2$. Para $k \geq 1$, a cota é melhor que o $O(k\,n^2)$ da lista sem heap sempre que $m \in o(n^2)$: $m + (k+1)\,n \lg n \leq m + 2k\,n \lg n$, e dividindo por $k\,n^2$ fica $\leq m/n^2 + 2 \lg n / n$, que tende a 0; logo a cota está em $o(k\,n^2)$. Com $k = 0$ nada é retirado e a comparação não se aplica; para grafos densos, $m \in \theta(n^2)$ e, com k constante, o termo m domina e a cota volta a $O(n^2)$. As cotas deste passo são superiores; não afirmo que sejam atingidas.

PASSO 6 — RESUMO. No pior caso, com os graus armazenados como o texto supõe, o algoritmo completo com o heap custa $O(n^2 + k\,n \log n)$ com matriz, que é $\Theta(n^2)$ para k constante, $k \geq 1$, $O(n \log n)$ para $k = 0$ e $O(1)$ para $k \geq n$; com listas, $O(m + (k+1)\,n \log n)$ (Passos 4 e 5). A frase do texto, "construção $O(n \log n)$ e remoção do i-ésimo elemento $O(\log(n - i + 1))$", soma $O(n \log n)$ (Passo 1), e para k constante, $k \geq 1$, essa soma de cotas é de ordem exatamente $n \log n$ na pior entrada (Passo 2). Essa soma é a parcela do heap, ou seja, o custo de encontrar e retirar os vértices de grau $< k$. Só seria o custo do algoritmo inteiro se a enumeração dos vizinhos (H3) e as atualizações de grau (H1) fossem desconsideradas; com matriz, a enumeração sozinha já custa $2n$ por vértice retirado, e o heap não melhora a classe.

Conferência numérica (Python; não faz parte da prova): implementei o algoritmo literal (teste $|V| < k + 1$, depois o teste do grau mínimo, depois a retirada) com matriz de adjacências, graus pré-armazenados e heap binário com diminuição de chave e vetor de posições. Rodei em todos os 33.867 grafos rotulados com 1 a 6 vértices e em 6.000 grafos aleatórios com 7 e 8 vértices, para cada k de 0 a $n + 1$. Em todos: o resultado coincide com a força bruta (subconjunto de tamanho máximo com grau induzido $\geq k$, ou "nenhum"); o número de leituras da matriz é exatamente $2n \cdot r$; $r \leq \max(0, n - k)$, e $r = 0$ para $k = 0$ e para $k \geq n$ (com $k \geq n$ o heap nem é construído); a i-ésima remoção ocorre com o heap de tamanho $n - i + 1$; o número de diminuições de chave é $\leq \min((k-1) \cdot r, m)$. Para cada $n \leq 6$ e $1 \leq k \leq n - 1$, o máximo de leituras sobre todos os grafos é exatamente $2n(n-k)$, atingido pelo grafo sem arestas (conferido também no grafo sem arestas para n até 39, com $2n(n-k) \geq n^2$ quando $n \geq 2k$), e o máximo de diminuições observado foi $(k-1)(n-k)$. No K_n com $k = n$ (n de 1 a 11) o algoritmo para sem retirar nem ler nada. Com listas de adjacências (20.000 grafos aleatórios com até 9 vértices, listas em ordem aleatória): entradas lidas $\leq 2m$ e diminuições $\leq \min(m, (k-1) \cdot r)$. Estrela $K_{1,n-1}$ com $k = 2$, variante que apaga das listas, n de 3 a 59: saem $n - 2$ folhas, o centro nunca sai, e o custo de apagar é $n(n-1)/2 - 1$ na pior ordem da lista do centro e $n - 2$ na melhor. Também: $(n/4) \lg n \leq \lg n! \leq n \lg n$ para n de 4 a

2999 (lg n!/(n lg n) ≈ 0,9276 em n = 10⁶); Σ_{j=k+1}^{n} lg j ≥ (n/4) lg n para k de 1 a 29 e n de max(4, 2k + 2) a 799; 1 + lg n ≤ 2 lg n, n ≤ n lg n e n lg n ≤ n² para n de 2 a 9999.

---

33. Rezende (UNICAMP), p. 19

Exercício: Na representação decimal, quantos dígitos tem o número $2^{2^{2^{2}}}$ (que é o *menor* inteiro N para o qual $\log_2^{*} N$ é maior que 4)?

Contexto do material (transcrição literal, necessária para resolver):
(p. 18) **4.3.1 A Função Logaritmo Iterado**

Inicialmente, definimos a função[5] $\log^{(i)} n$ recursivamente para inteiros não-negativos por:

$$\log^{(i)} n = \begin{cases} n, & \text{se } i = 0 \\ \log(\log^{(i-1)} n), & \text{se } i > 0 \text{ e } \log^{(i-1)} n > 0 \\ \text{indefinida}, & \text{se } i > 0 \text{ e } \log^{(i-1)} n \le 0 \text{ ou } \log^{(i-1)} \text{ é indefinida} \end{cases}$$

O leitor deve tomar o cuidado de não confundir as notações $\log^{(i)} n$ (i-ésima iterada da função logaritmo) e $\log^{i} n$ (i-ésima potência do logaritmo de n); esta última é o mesmo que $(\log n)^i$.

Definimos a *função logaritmo iterado*, denotada por $\log^{*} n$, lê-se "log estrela de n"como:
$\log^{*} n = \min \{i \ge 0 : \log^{(i)} n \le 1\}$

Vejamos como é lento o crescimento da função logaritmo iterado $\log_2^{*} n$:

[5] Omitimos aqui a base do logaritmo, mas, quando for conveniente, esta pode ser explicitada.

(p. 19)

$$\begin{aligned} \log_2^{*} 2 &= 1 \\ \log_2^{*} 2^{2} &= \log_2^{*} 4 = 2 \\ \log_2^{*} 2^{2^{2}} &= \log_2^{*} 2^{4} = \log_2^{*} 16 = 3 \\ \log_2^{*} 2^{2^{2^{2}}} &= \log_2^{*} 2^{16} = \log_2^{*} 65536 = 4 \\ \log_2^{*} 2^{2^{2^{2}}} &= \log_2^{*} 2^{65536} = 5 \end{aligned}$$

[transcrição literal — Rezende (UNICAMP), p. 19]

Resolução

Resposta. O número do enunciado é a torre 2^(2^(2^(2^2))) = 2^65536, e ele tem 19729 dígitos decimais: 10^19728 < 2^65536 < 10^19729, ou seja, ⌊65536·log₁₀ 2⌋ + 1 = 19728 + 1. O parêntese do enunciado, porém, não é verdadeiro pela definição de log* do próprio material: o menor inteiro N com log*₂ N > 4 é 65537 (log*₂ 65536 = 4 e log*₂ 65537 = 5). O que vale para 2^65536 é: ele é o MAIOR inteiro N com log*₂ N = 5 (equivalentemente, o menor N com log*₂ N > 5 é 2^65536 + 1). O enunciado não é alterado; a observação fica registrada aqui.

INTERPRETAÇÃO. Leitura 1 (torre): "2^(2222)" é a notação do material para a torre de cinco 2's. A própria tabela da p. 19 escreve 2^(22) = 2^4 = 16, 2^(222) = 2^16 = 65536 e 2^(2222) = 2^65536; logo cada "2" a mais na sequência é mais um nível da torre. Leitura 2 (literal): 2 elevado ao número 2222. Ela contradiz a linha 2^(2222) = 2^(65536) do material e o próprio parêntese (2^2222 nada tem de especial para log*), por isso resolvo a Leitura 1. Por completude, a Leitura 2 dá 669 dígitos (ver ITEM 4, com as mesmas cotas).

PASSO 1 — POR QUE ⌊log₁₀ x⌋ + 1 CONTA DÍGITOS: Um inteiro x ≥ 1 tem d dígitos decimais se e somente se 10^(d−1) ≤ x < 10^d (o menor número de d dígitos é 10^(d−1), o maior é 10^d − 1). Como log₁₀ é estritamente crescente, isso equivale a d − 1 ≤ log₁₀ x < d, isto é, d − 1 = ⌊log₁₀ x⌋, d = ⌊log₁₀ x⌋ + 1. A fórmula vale para todo inteiro x ≥ 1, inclusive potências de 10 (10^k tem k + 1 dígitos e log₁₀ 10^k = k).

Para x = 2^65536: log₁₀ x = 65536·log₁₀ 2. Esse número não é inteiro: se fosse igual a um inteiro k, teríamos 2^65536 = 10^k; k = 0 é impossível (2^65536 ≠ 1) e, para k ≥ 1, 10^k = 2^k·5^k é divisível por 5, enquanto 2^65536 não é (fatoração única). Portanto 2^65536 não é potência de 10, a primeira desigualdade da caracterização é estrita, e basta provar 19728 < 65536·log₁₀ 2 < 19729.

PASSO 2 — COTAS RACIONAIS DE log₁₀ 2: Vou provar 205/681 < log₁₀ 2 < 87/289, o que equivale a 2^681 > 10^205 e 2^289 < 10^87.

Ferramenta: 2^10 = 1024 = 1,024·10^3, logo 2^(10k) = 1,024^k·10^(3k). As potências de 1,024 por quadrados sucessivos são exatas em 1,024^(2^j) = 2^(10·2^j)/10^(3·2^j); arredondo cada uma para baixo (cota inferior) ou para cima (cota superior), e uso que, para reais positivos, a ≥ a' implica a² ≥ a'² e a·b ≥ a'·b' quando também b ≥ b' (idem com ≤).

```
1,024^2 = 1,048576 (exato)
    1,024^4 = 1,048576² = 1,099511627776 (exato) ∈ [1,0995116 ; 1,0995117]
    1,024^8 ≥ 1,0995116² = 1,20892575854... ≥ 1,2089257
    1,024^8 ≤ 1,0995117² = 1,20892597843... ≤ 1,208926
    1,024^16 ≥ 1,2089257² = 1,46150134812... ≥ 1,4615013
    1,024^16 ≤ 1,208926² = 1,4615020735... ≤ 1,4615021
    1,024^32 ≥ 1,4615013² = 2,13598604... ≥ 2,135986
    1,024^64 ≥ 2,135986² = 4,56243619... ≥ 4,562436
```

Cota inferior. 2^681 = 2·(2^10)^68 = 2·1,024^68·10^204, e 1,024^68 = 1,024^64·1,024^4 ≥ 4,562436·1,0995116 = 5,01645... > 5. Logo 2^681 > 2·5·10^204 = 10^205, e tomando log₁₀: 681·log₁₀ 2 > 205, isto é, log₁₀ 2 > 205/681.

```
Cota superior. 2^289 = 2^9·(2^10)^28 = 512·1,024^28·10^84, e
    1,024^28 = 1,024^16·1,024^8·1,024^4 ≤ 1,4615021·1,208926·1,0995117
             ≤ 1,7668479·1,0995117 = 1,94266... < 1,953125 = 1000/512
(1,4615021·1,208926 = 1,76684788...). Logo 2^289 < 512·(1000/512)·10^84 = 10^87, e log₁₀ 2 < 87/289.
```

(Valores: 205/681 = 0,3010279..., 87/289 = 0,3010380...; log₁₀ 2 = 0,3010299... está entre eles.)

```
PASSO 3 — O PISO:
Multiplicando por 65536 > 0:
    65536·205/681 = 13434880/681 e 19728 = 13434768/681, com 13434880 > 13434768, logo 65536·log₁₀ 2 > 19728;
    65536·87/289 = 5701632/289 e 19729 = 5701681/289, com 5701632 < 5701681, logo 65536·log₁₀ 2 < 19729.
Então 19728 < log₁₀ 2^65536 < 19729, ⌊log₁₀ 2^65536⌋ = 19728 e, pelo Passo 1, 2^65536 tem 19728 + 1 = 19729 dígi-
tos. Equivalentemente, 10^19728 < 2^65536 < 10^19729.
```

PASSO 4 — O PARÊNTESE: QUEM É O MENOR N COM log*₂ N > 4: Uso a definição do material: log^(0) n = n; log^(i) n = lg(log^(i−1) n) se log^(i−1) n está definida e é > 0 (indefinida caso contrário); log* n = min{i ≥ 0 : log^(i) n ≤ 1}. A recursão é aplicada a valores reais (lg 65537 não é inteiro), então leio a definição para reais n ≥ 0.

Torres: T(0) = 1 e T(k) = 2^T(k−1), ou seja T(1) = 2, T(2) = 4, T(3) = 16, T(4) = 65536, T(5) = 2^65536.

Lema A (deslocamento). Se n > 1, então para todo i ≥ 1, log^(i) n = log^(i−1)(lg n), um lado definido se e somente se o outro está. Prova por indução em i. Base i = 1: log^(1) n = lg n (definida, pois n > 0) = log^(0)(lg n). Passo i ≥ 2: log^(i) n está definida e vale lg(log^(i−1) n) exatamente quando log^(i−1) n está definida e é > 0; pela hipótese log^(i−1) n = log^(i−2)(lg n), e essa é exatamente a condição para log^(i−1)(lg n) estar definida e valer lg(log^(i−2)(lg n)) = lg(log^(i−1) n).

Lema B. P(k): para todo real n ≥ 0, existe i com 0 ≤ i ≤ k, log^(i) n definida e ≤ 1, se e somente se n ≤ T(k). Em particular log* n ≤ k ⇔ n ≤ T(k). Prova por indução em k ≥ 0. Base k = 0: o único índice é i = 0, e log^(0) n = n ≤ 1 = T(0). Passo k ≥ 1, supondo P(k−1) para todo real ≥ 0. Se n ≤ 1, o índice i = 0 serve e n ≤ 1 ≤ T(k): os dois lados são verdadeiros. Se n > 1, o índice 0 não serve (n > 1); pelo Lema A, existe i ∈ {1, …, k} com log^(i) n ≤ 1 se e somente se existe j = i − 1 ∈ {0, …, k−1} com log^(j)(lg n) ≤ 1, o que, por P(k−1) aplicado a lg n > 0, equivale a lg n ≤ T(k−1), isto é, n ≤ 2^T(k−1) = T(k). Como "log* n ≤ k" significa exatamente que algum índice i ≤ k pertence ao conjunto do mínimo, segue log* n ≤ k ⇔ n ≤ T(k). (Como T(k) cresce sem limite, o conjunto nunca é vazio e log* n está sempre definida.)

```
Consequências para inteiros N ≥ 1:
    log*₂ N > 4 ⇔ N > T(4) = 65536, logo o menor inteiro com log*₂ N > 4 é 65537, e não 2^65536.
    Conferência direta: log^(1) 65536 = 16, log^(2) = 4, log^(3) = 2, log^(4) = 1 ≤ 1 e nenhum índice anterior é ≤
1, então log*₂ 65536 = 4; e 65537 > T(4), 65537 ≤ T(5), então log*₂ 65537 = 5.
    log*₂ N = 5 ⇔ 65536 < N ≤ 2^65536: 2^65536 é o MAIOR inteiro com log*₂ N = 5 (de acordo com a última linha da
tabela, log*₂ 2^65536 = 5), e o menor com log*₂ N > 5 é 2^65536 + 1.
Portanto o parêntese está errado como escrito. Uma redação correta seria "(que é o maior inteiro N para o qual
log*₂ N é no máximo 5)" ou, mantendo "menor" e "maior que", "o menor inteiro N para o qual log*₂ N é maior que 5 é
2^65536 + 1". Isso não muda a pergunta principal, que é sobre os dígitos de 2^65536.
```

ITEM 4 — LEITURA LITERAL 2^2222 (rejeitada, só por completude): Com as mesmas cotas, 2222·205/681 = 455510/681 > 668 (pois 681·668 = 454908) e 2222·87/289 = 193314/289 < 669 (pois 289·669 = 193341). Logo 668 < 2222·log₁₀ 2 < 669 e 2^2222 teria 669 dígitos.

Conferência numérica (Python; não faz parte da prova): conferem a torre [1, 2, 4, 16, 65536] com T(5) = 2^65536, que log*₂ 65536 = 4 e log*₂ 65537 = 5 (logo o menor N com log*₂ N > 4 é 65537, como provado no Passo 4, e não 2^65536), e que 2^65536 tem 19729 dígitos = ⌊65536·log₁₀ 2⌋ + 1, igual ao provado nos Passos 2 e 3 com as cotas racionais 205/681 < log₁₀ 2 < 87/289.

---

34. Rezende (UNICAMP), p. 21, Exercícios Suplementares, item 1

5 Exercícios Suplementares

1. Encontre um limite superior para a solução da relação de recorrência $T(n) = 168T(\lfloor n/672 \rfloor + 135) + 168T(\lfloor n/672 \rfloor + 135) + 168T(\lfloor n/672 \rfloor + 189) + 168T(\lfloor n/672 \rfloor + 189) + 456n + 1$.

Contexto do material (transcrição literal, necessária para resolver):
Nenhum contexto adicional no texto: o item é autônomo; não há caso base dado. As técnicas do texto que se aplicam são as das p. 9–10 (termos constantes somados ao argumento, ex.: (p. 9) "$T(n) = 2T(\lfloor n/2 \rfloor + 37) + n$. (5) Intuitivamente, isso é verdade porque 37 é uma constante e se fizermos n suficientemente grande, $\lfloor n/2 \rfloor + 37$ se torna muito próximo de $\lfloor n/2 \rfloor$.") e o "Teorema Master" (Teorema 1, p. 13, não transcrito).

[transcrição literal — Rezende (UNICAMP), p. 21, Exercícios Suplementares, item 1]

Resolução

Resposta. T(n) ∈ O(n lg n). Juntando os termos iguais, a recorrência é T(n) = 336·T(⌊n/672⌋ + 135) + 336·T(⌊n/672⌋ + 189) + 456n + 1. Com o caso base declarado abaixo (T(n) = 1 para 1 ≤ n ≤ 189, recorrência para n ≥ 190), vale T(n) ≤ c·n·lg n para todo n ≥ n0, com c = 2·10¹⁰ e n0 = 2. O deslocamento (+135, +189) não muda a classe: para n ≥ 127008 o argumento ⌊n/672⌋ + 189 é no máximo n/336, e o que ele custa a mais é uma parcela da ordem de c·lg n, que é o(n) e cabe na folga c·n·lg 336 que a

divisão produz. (Observação, não provada aqui: com a = 672 = b^1 e f(n) = 456n + 1, o mesmo raciocínio do Teorema Master indica que o limite é justo, Θ(n lg n); o enunciado só pede o limite superior.)

INTERPRETAÇÃO DO ENUNCIADO. Leituras possíveis de "encontre um limite superior": (i) qualquer função g com T ∈ O(g); (ii) o limite superior assintótico natural, aquele que a técnica do texto (p. 9–10, constantes somadas ao argumento) produz. A leitura (i) é trivialmente satisfeita por funções enormes e não é o que o exercício treina; resolvo a (ii), com prova por substituição e testemunhas numéricas. O argumento é inteiro: n percorre os naturais n ≥ 1, e ⌊·⌋ é o piso. Afirmações sobre T valem para todo n do domínio (a recorrência é determinística; não há pior ou melhor caso aqui).

```
SIMPLIFICAÇÃO.
Os dois primeiros termos são iguais, assim como os dois últimos, logo
    T(n) = 336·T(⌊n/672⌋ + 135) + 336·T(⌊n/672⌋ + 189) + 456n + 1.
O total de chamadas é a = 336 + 336 = 672, e os argumentos são ≈ n/672.
```

```
CASO BASE (O TEXTO NÃO DÁ; DECLARO).
Para 1 ≤ n ≤ 189 temos ⌊n/672⌋ = 0, logo o segundo argumento vale 189 ≥ n: a recorrência não reduz o argumento (em
n = 189 ela chamaria T(189) para definir T(189)). Por isso declaro o caso base em toda a faixa 1 ≤ n ≤ 189 — é uma
escolha de conveniência (não a única possível), que garante que, para n ≥ 190, a recorrência só chame argumentos
menores que n. Declaro:
    T(n) = 1 para 1 ≤ n ≤ 189;  a recorrência vale para n ≥ 190.
Ela está bem definida para n ≥ 190: os argumentos m₁ = ⌊n/672⌋ + 135 e m₂ = ⌊n/672⌋ + 189 satisfazem 135 ≤ m₁ < m₂
≤ n/672 + 189 < n, porque n/672 + 189 < n ⇔ 189 < 671n/672 ⇔ n > 189,28…, o que vale para n ≥ 190 (em n = 190 exa-
tamente: m₂ = 0 + 189 = 189 < 190). Nunca se chega a argumento 0. Outro caso base positivo muda apenas a constante
c, não a classe (ver a observação no fim do PASSO 3).
```

PALPITE. a = 672 chamadas sobre argumentos ≈ n/b com b = 672 e custo não recursivo linear: a = b^1, o caso "a = b^k" do Teorema Master, que dá n lg n. Palpite: T(n) ≤ c·n·lg n.

AFIRMAÇÃO A PROVAR. P(n): T(n) ≤ c·n·lg n, com c = $2·10^{10}$. Provo P(n) para todo n ≥ n0 = 2, por indução forte em n. (n0 = 2 porque lg 1 = 0 e T(1) = 1 > 0; a afirmação é falsa em n = 1 e não é necessária lá: os argumentos recursivos são sempre ≥ 135.) A ordem dos quantificadores é a da definição: existem c = $2·10^{10}$ e n0 = 2, fixos, tais que para todo n ≥ 2 vale 0 ≤ T(n) ≤ c·n·lg n. (T(n) ≥ 0 é imediato: a base é positiva e a recorrência soma termos não negativos.)

```
PASSO 1 — BASE DA INDUÇÃO: 2 ≤ n ≤ 524287.
Cotas explícitas para T nessa faixa, todas por cálculo direto da recorrência:
(a) 2 ≤ n ≤ 189: T(n) = 1.
(b) n = 190: ⌊190/672⌋ = 0, argumentos 135 e 189 (base), então T(190) = 336·1 + 336·1 + 456·190 + 1 = 672 + 86640
+ 1 = 87313.
(c) 190 ≤ n ≤ 969: ⌊n/672⌋ ≤ 1, logo os argumentos são ≤ 1 + 189 = 190, e por (a) e (b) cada T(argumento) ≤ 87313.
Assim T(n) ≤ 672·87313 + 456·969 + 1 = 58674336 + 441864 + 1 = 59116201.
(d) 190 ≤ n ≤ 524287: ⌊n/672⌋ ≤ ⌊524287/672⌋ = 780 (pois 780·672 = 524160 ≤ 524287 < 524832 = 781·672), logo os
argumentos são ≤ 780 + 189 = 969, e por (a)–(c) cada T(argumento) ≤ 59116201. Assim
    T(n) ≤ 672·59116201 + 456·524287 + 1 = 39726087072 + 239074872 + 1 = 39965161945 < $4·10^{10}$.
(Contas: 59116201·672 = 59116201·700 − 59116201·28 = 41381340700 − 1655253628; 456·524287 = 456·$2^{19}$ − 456 =
239075328 − 456.)
Logo T(n) < $4·10^{10}$ para todo 2 ≤ n ≤ 524287. Como n·lg n ≥ 2·lg 2 = 2 nessa faixa (n lg n é crescente para n ≥ 1),
c·n·lg n ≥ $2·10^{10}$·2 = $4·10^{10}$ > T(n). P(n) vale para 2 ≤ n ≤ 524287; em n = n0 = 2 exatamente: T(2) = 1 ≤
$2·10^{10}$·2·1 = $4·10^{10}$.
```

```
PASSO 2 — PASSO INDUTIVO: n ≥ 524288 = $2^{19}$.
Hipótese de indução (forte): P(k) vale para todo k com 2 ≤ k ≤ n − 1.
Os argumentos m₁, m₂ satisfazem 2 ≤ 135 ≤ m₁ ≤ m₂ < n (visto no CASO BASE), então a hipótese se aplica a ambos:
    T(n) ≤ 336·c·m₁·lg m₁ + 336·c·m₂·lg m₂ + 456n + 1.
Seja m = n/672 + 189 (real). Tem-se m₁ ≤ m₂ ≤ m (o piso só diminui), e x·lg x é crescente para x ≥ 1, logo
    T(n) ≤ 672·c·m·lg m + 456n + 1 = c·(n + 127008)·lg m + 456n + 1,
pois 672·m = n + 672·189 = n + 127008.
Tratamento do deslocamento: como n ≥ 524288 ≥ 127008 = 672·189, vale 189 ≤ n/672, logo m ≤ n/672 + n/672 = n/336 e
lg m ≤ lg n − lg 336 < lg n − 8 (porque 336 > 256 = $2^8$). O fator lg n − 8 é positivo (lg n ≥ 19), e n + 127008 >
0, então
    T(n) ≤ c·(n + 127008)·(lg n − 8) + 456n + 1
         = c·n·lg n − [ 8cn − 127008·c·(lg n − 8) − 456n − 1 ].
Basta mostrar que o colchete é ≥ 0. Separo 8cn = 4cn + 4cn:
(i) 4cn − 456n − 1 ≥ 0: como c ≥ 115, 4cn ≥ 460n = 456n + 4n ≥ 456n + 1 para n ≥ 1.
(ii) 4cn − 127008·c·(lg n − 8) ≥ 0 ⇔ n ≥ 31752·(lg n − 8) (dividindo por 4c > 0; 127008/4 = 31752). Seja h(n) = n
− 31752·(lg n − 8). Em n = $2^{19}$ exatamente: h = 524288 − 31752·11 = 524288 − 349272 = 175016 > 0. E h é crescente
para n ≥ $2^{19}$: h(n+1) − h(n) = 1 − 31752·lg(1 + 1/n), e lg(1 + 1/n) ≤ (1/n)/ln 2 < 2/n, logo h(n+1) − h(n) > 1 −
63504/n ≥ 1 − 63504/524288 > 0. Portanto h(n) > 0 para todo n ≥ $2^{19}$.
Somando (i) e (ii), o colchete é ≥ 0 e T(n) ≤ c·n·lg n, isto é, P(n). Pelo princípio da indução forte (base no
PASSO 1 cobrindo 2..524287, passo para n ≥ 524288), P(n) vale para todo n ≥ 2.
```

PASSO 3 — CONCLUSÃO E TESTEMUNHAS. Com c = $2·10^{10}$ e n0 = 2: para todo n ≥ 2, 0 ≤ T(n) ≤ c·n·lg n. Logo T(n) ∈ O(n lg n). Em n = n0 = 2: T(2) = 1 ≤ $4·10^{10}$ = c·2·lg 2. A constante é enorme porque absorve os valores de T na faixa onde o passo indutivo ainda não funciona (n < 2^{19}); o passo em si só exige c ≥ 115. Isso não afeta a classe. Observação sobre o caso base: se, em vez de T(n) = 1, a base for T(n) = d_n ≥ 0 com d_n ≤ M para 1 ≤ n ≤ 189 (M ≥ 1), as cotas (b)–(d) do PASSO 1 crescem no máximo por um fator M (cada uma é ≤ M vezes a anterior, pois 456n + 1 ≤ M·(456n + 1)), e c = $2·10^{10}$·M serve; o PASSO 2 não usa a base. A classe O(n lg n) não depende da base.

POR QUE O DESLOCAMENTO NÃO MUDA A CLASSE. É a intuição da p. 9 do texto ("se fizermos n suficientemente grande, ⌊n/2⌋ + 37 se torna muito próximo de ⌊n/2⌋"), aqui tornada quantitativa: 1: O piso só diminui o argumento, e x lg x é crescente, então ele só ajuda numa cota superior. 2: A constante +189 (a maior das duas) é absorvida assim que n/672 ≥ 189: o argumento fica ≤ n/336, que ainda é uma fração fixa de n; em vez de ganhar lg 672 por nível, ganha-se lg 336 > 8, o que ainda produz a folga c·n·(lg 336) > 8cn. 3: O preço do deslocamento é a parcela c·127008·lg m ≤ c·127008·(lg n − 8), que é o(n); ela cabe em metade da folga (4cn) para n ≥ 2^{19}. 4: Só sobra a faixa finita n < 2^{19}, onde T é limitado por uma constante — e um número finito de valores é sempre absorvido aumentando c. Nada disso depende de 135 e 189 serem esses números: qualquer deslocamento constante só muda o limiar (127008, 2^{19}) e a constante c.

Conferência numérica (Python; não faz parte da prova): com outro caso base, T(n) = 1 para n ≤ 1000, a recorrência dá T(n)/(n lg n) entre 27,5 e 48,3 nas potências de 10 de 10^4 a 10^9 (numa amostra densa do mesmo intervalo, entre cerca de 23,9 e 52,2) — razão limitada, compatível com T ∈ O(n lg n), e bem abaixo da testemunha c = $2 \cdot 10^{10}$, que é folgada de propósito. A oscilação da razão (sem tendência de crescimento) também é compatível com o palpite Θ(n lg n) do Teorema Master. Confere ainda o domínio: o argumento ⌊n/672⌋ + 189 só fica menor que n para n > ~190, que é exatamente o limite n ≥ 190 usado acima. Os números do PASSO 1 e do PASSO 2 estão escritos por extenso para conferência.

Walace Rodrigues — Somatórios (IFMG Formiga)

35. Walace (IFMG), ex. 1

3.2. Perturbação

Exercícios:

Examine o processo de perturbação da soma dos quadrados

$\sum_{i=1}^{n+1} i^2 = ?$

[transcrição literal — Walace Rodrigues, Somatórios (IFMG), slide 16]

Resolução

Resposta. soma de i^2, para i = 1 ate n+1 = (n+1)(n+2)(2n+3)/6 = (2n^3 + 9n^2 + 13n + 6)/6. Observacao central do exercicio: a perturbacao aplicada diretamente a soma dos QUADRADOS nao produz a formula dos quadrados — ela cancela S_n e devolve a soma dos PRIMEIROS POTENCIAS, soma de i = n(n+1)/2. Para obter a soma dos quadrados e preciso perturbar a soma dos CUBOS.

O METODO DA PERTURBACAO

Seja S_n = soma de a_i, para i = 1 ate n. A ideia e escrever a soma estendida S_{n+1} de DUAS maneiras diferentes:

Forma A (destacando o ULTIMO termo): S_{n+1} = S_n + a_{n+1}. Forma B (destacando o PRIMEIRO termo e reindexando i -> i+1): S_{n+1} = a_1 + soma de a_{i+1}, para i = 1 ate n.

Como as duas expressoes representam o mesmo numero, igualam-se. O termo S_n aparece nos dois lados; quando a_{i+1} pode ser escrito como a_i mais termos de ordem menor, o S_n CANCELA e sobra uma equacao envolvendo apenas somatorios mais simples. O objetivo geral do metodo e fazer S_n reaparecer do lado direito e isola-lo: e assim que se resolve a soma geometrica: com S_n = soma de x^i para i = 0 ate n, a perturbacao da S_n + x^(n+1) = 1 + x*S_n, e dai S_n = (x^(n+1) - 1)/(x - 1) para x diferente de 1. Quando, como aqui, a_{i+1} = a_i + termos de ordem menor, o S_n CANCELA: o metodo entao nao fornece S_n, mas uma relacao entre somas de grau inferior.

EXAME DO PROCESSO PARA OS QUADRADOS

Seja S_n = soma de i^2, para i = 1 ate n.

Forma A: S_{n+1} = S_n + (n+1)^2.

Forma B: S_{n+1} = 1^2 + soma de (i+1)^2, para i = 1 ate n = 1 + soma de (i^2 + 2i + 1), para i = 1 ate n = 1 + S_n + 2*(soma de i, i = 1..n) + n.

Igualando A e B: S_n + (n+1)^2 = 1 + S_n + 2*(soma de i) + n.

O S_n CANCELA dos dois lados. Sobra: (n+1)^2 = 1 + n + 2*(soma de i) => soma de i, para i = 1 ate n = ((n+1)^2 - n - 1)/2 = (n^2 + n)/2 = n(n+1)/2.

Este e exatamente o ponto que o enunciado manda examinar: perturbar a soma dos quadrados NAO fornece a soma dos quadrados; fornece a formula da soma dos primeiros n inteiros. Em geral, perturbar a soma das potencias de expoente k devolve uma equacao para a soma de expoente k-1, que se resolve usando as somas de expoente menor ja conhecidas. Logo, para achar a soma dos quadrados, deve-se perturbar a soma dos CUBOS.

PERTURBACAO DOS CUBOS (para obter os quadrados)

Seja C_n = soma de i^3, para i = 1 ate n, e mantenha S_n = soma de i^2 e T_n = soma de i = n(n+1)/2 (ja obtida acima).

Forma A: C_{n+1} = C_n + (n+1)^3.

Forma B: C_{n+1} = 1^3 + soma de (i+1)^3, para i = 1 ate n = 1 + soma de (i^3 + 3i^2 + 3i + 1), para i = 1 ate n = 1 + C_n + 3*S_n + 3*T_n + n.

Igualando e cancelando C_n: (n+1)^3 = 1 + 3*S_n + 3*T_n + n.

```
Substituindo T_n = n(n+1)/2 e (n+1)^3 = n^3 + 3n^2 + 3n + 1:
        n^3 + 3n^2 + 3n + 1 = 1 + n + 3*S_n + 3n(n+1)/2
  =>    3*S_n = n^3 + 3n^2 + 2n - (3n^2 + 3n)/2
              = (2n^3 + 6n^2 + 4n - 3n^2 - 3n)/2
              = (2n^3 + 3n^2 + n)/2
              = n(n+1)(2n+1)/2
  =>    S_n = n(n+1)(2n+1)/6.
```

RESPOSTA DO ENUNCIADO (soma ate n+1)

Basta avaliar a forma fechada em m = n+1: soma de i^2, para i = 1 ate n+1 = S_{n+1} = (n+1)((n+1)+1)(2(n+1)+1)/6 = (n+1)(n+2)(2n+3)/6.

Expandindo: (n+1)(n+2)(2n+3)/6 = (2n^3 + 9n^2 + 13n + 6)/6.

VERIFICACAO

n = 0: formula da (1)(2)(3)/6 = 1; soma de i^2 de 1 a 1 = 1. OK. n = 1: (2)(3)(5)/6 = 5; soma = 1 + 4 = 5. OK. n = 2: (3)(4)(7)/6 = 14; soma = 1 + 4 + 9 = 14. OK. n = 3: (4)(5)(9)/6 = 30; soma = 1 + 4 + 9 + 16 = 30. OK.

Consistencia com a Forma A: S_{n+1} - S_n = (n+1)(n+2)(2n+3)/6 - n(n+1)(2n+1)/6 = (n+1)[(n+2)(2n+3) - n(2n+1)]/6 = (n+1)[2n^2+7n+6 - 2n^2 - n]/6 = (n+1)(6n+6)/6 = (n+1)^2, que e de fato o ultimo termo. OK.

COMPLEXIDADE ASSINTOTICA (leitura usual em PAA)

(n+1)(n+2)(2n+3)/6 = Theta(n^3). Testemunhas: para todo n >= 1 vale (n+1)(n+2)(2n+3)/6 >= (n)(n)(2n)/6 = n^3/3 e (n+1)(n+2)(2n+3)/6 <= (2n)(3n)(5n)/6 = 5n^3. Logo c1 = 1/3, c2 = 5, n0 = 1. Em n = n0 = 1: valor exato 5; c1*1 = 1/3 <= 5 e c2*1 = 5 >= 5. Desigualdades validas em n0.

---

36. Walace (IFMG), ex. 2

3.3. Operador diferença

Exercício:

Resolva usando o operador diferença

$\sum_{i=1}^{n} i * 2^{i} = ?$

[transcrição literal — Walace Rodrigues, Somatórios (IFMG), slide 19]

Resolução

Resposta. soma de i*2^i, para i = 1 ate n = (n-1)*2^(n+1) + 2.

O OPERADOR DIFERENCA E A SOMA TELESCOPICA

Define-se o operador diferenca (analogo discreto da derivada) por (Delta f)(i) = f(i+1) - f(i).

O analogo discreto do Teorema Fundamental do Calculo e a soma telescopica: somando (Delta f)(i) de i = a ate b, todos os termos intermediarios se cancelam, soma de (Delta f)(i), para i = a ate b = [f(a+1) - f(a)] + [f(a+2) - f(a+1)] + ... + [f(b+1) - f(b)] = f(b+1) - f(a).

Portanto, para resolver uma soma de termos g(i) basta encontrar uma "antidiferenca" f tal que (Delta f)(i) = g(i); a soma sai avaliada apenas nos extremos.

ESCOLHA DO CANDIDATO

Queremos f com (Delta f)(i) = i*2^i. Como o termo e o produto de um polinomio de grau 1 por 2^i, e a diferenca de (polinomio)*2^i devolve outro (polinomio de mesmo grau)*2^i, tenta-se o mesmo formato: f(i) = (a*i + b)*2^i, com a, b constantes a determinar.

```
Entao
        (Delta f)(i) = f(i+1) - f(i)
                     = (a(i+1) + b)*2^(i+1) - (a*i + b)*2^i
                     = 2^i * [ 2*(a*i + a + b) - (a*i + b) ]
                     = 2^i * (2a*i + 2a + 2b - a*i - b)
                     = 2^i * (a*i + 2a + b).
```

Para que isso seja identicamente igual a i*2^i, igualam-se os coeficientes: coeficiente de i: a = 1; termo independente: 2a + b = 0 => b = -2.

Logo a antidiferenca e f(i) = (i - 2)*2^i.

Conferencia direta: (Delta f)(i) = (i-1)*2^(i+1) - (i-2)*2^i = 2^i*[2(i-1) - (i-2)] = 2^i*(2i - 2 - i + 2) = i*2^i. Confere.

APLICACAO DA SOMA TELESCOPICA

```
soma de i*2^i, para i = 1 ate n = soma de (Delta f)(i), para i = 1 ate n = f(n+1) - f(1)
                                    = ((n+1) - 2)*2^(n+1) - (1 - 2)*2^1
                                    = (n - 1)*2^(n+1) - (-1)*2
                                    = (n - 1)*2^(n+1) + 2.
```

Forma fechada: soma de i*2^i, para i = 1 ate n = (n - 1)*2^(n+1) + 2.

VERIFICACAO

Caso base n = 1: (1-1)*2^2 + 2 = 0 + 2 = 2, e a soma e 1*2^1 = 2. OK. n = 2: (2-1)*2^3 + 2 = 8 + 2 = 10, e 2 + 8 = 10. OK. n = 3: (3-1)*2^4 + 2 = 32 + 2 = 34, e 2 + 8 + 24 = 34. OK. n = 4: (4-1)*2^5 + 2 = 96 + 2 = 98, e 34 + 64 = 98. OK. n = 5: 4*2^6 + 2 = 258, e 98 + 160 = 258. OK.

Verificacao geral por passo indutivo: F(n) = (n-1)*2^(n+1) + 2 satisfaz F(n) - F(n-1) = (n-1)*2^(n+1) - (n-2)*2^n = 2^n*[2(n-1) - (n-2)] = n*2^n, que e exatamente o n-esimo termo. Junto com o caso base n = 1, a formula esta provada por inducao.

COMPLEXIDADE ASSINTOTICA (leitura usual em PAA)

```
(n-1)*2^(n+1) + 2 = Theta(n*2^n). Testemunhas: para todo n >= 2 vale
        (n-1)*2^(n+1) + 2 >= (n-1)*2^(n+1) >= (n/2)*2^(n+1) = n*2^n,
pois n - 1 >= n/2 quando n >= 2; e
        (n-1)*2^(n+1) + 2 <= n*2^(n+1) + n*2^n = 3*n*2^n.
Logo c1 = 1, c2 = 3, n0 = 2. Em n = n0 = 2: valor exato 10; c1*n*2^n = 2*4 = 8 <= 10 e c2*n*2^n = 3*8 = 24 >= 10.
Desigualdades validas em n0.
```

---

37. Walace (IFMG), slide 16, item 2

Dica: perturbe $i!$

$\Sigma_{i=1}^{n} i * i! = ?$

Contexto do material (transcrição literal, necessária para resolver):
3.2. Perturbação (slide 15)

- Vantagem: método poderoso, resolve muitos somatórios
- Modo de usar: acrescente mais um termo e reescreva a soma
- Desvantagem: pode ser difícil saber o que perturbar

[quadro] $\Sigma_{i=1}^{n} i^2 = ?$ Dica: perturbe a soma dos cubos
$\Sigma_{i=1}^{\mathbf{n+1}} i^3 = \Sigma_{i=1}^{n} i^3 + (n+1)^3 = \Sigma_{i=0}^{n} (i+1)^3 = 1 + \Sigma_{i=1}^{n} (i+1)^3$
$= 1 + \Sigma_{i=1}^{n} (i^3 + 3i^2 + 3i + 1) = 1 + \Sigma_{i=1}^{n} i^3 + 3\Sigma_{i=1}^{n} i^2 + 3\Sigma_{i=1}^{n} i + \Sigma_{i=1}^{n} 1$
$= 1 + \Sigma_{i=1}^{n} i^3 + 3\Sigma_{i=1}^{n} i^2 + 3(n/2)(n+1) + n$
= agora continue daqui

3.2. Perturbação (slide 16) — **Exercícios:**

1. Examine o processo de perturbação da soma dos quadrados
   $\Sigma_{i=1}^{\mathbf{n+1}} i^2 = ?$

[transcrição literal — Walace (IFMG), slide 16, item 2]

Resolução

Resposta. Σ_(i=1)^(n) i·i! = (n + 1)! − 1, para todo inteiro n ≥ 0 (em n = 0 a soma é vazia e vale 0 = 1! − 1). Obtém-se pelo método da perturbação aplicado à soma auxiliar T(n) = Σ_(i=1)^(n) i! ("perturbe i!"): escrevendo T(n + 1) de duas formas, T(n) aparece dos dois lados e se cancela, sobrando S(n) = (n + 1)! − 1. A forma fechada é conferida por indução em n.

INTERPRETAÇÃO.

O símbolo "∗" do enunciado é a multiplicação e "i!" é o fatorial de i; a soma pedida é S(n) = Σ_(i=1)^(n) i·i! = 1·1! + 2·2! + … + n·n!, e "= ?" pede uma forma fechada em função de n. A dica "perturbe i!" pode ser lida de duas maneiras: (a) perturbar a própria soma S(n), acrescentando o termo (n + 1)·(n + 1)!; (b) perturbar a soma auxiliar dos fatoriais, T(n) = Σ_(i=1)^(n) i!, do mesmo modo que o slide 15 perturba a soma dos cubos para achar a soma dos quadrados. A leitura (b) é a que segue o modelo do slide 15 (perturba-se uma soma "vizinha", cuja expansão faz aparecer a soma procurada) e é a que funciona diretamente, porque (i + 1)! = (i + 1)·i! = i·i! + i! faz surgir exatamente o termo i·i!. É a que resolvo. (A leitura (a) também pode ser levada a termo, mas a expansão de (i + 1)·(i + 1)! não reproduz S(n) com coeficiente que se cancele de forma útil; não é o caminho da dica.)

PASSO 1 — A SOMA AUXILIAR E O TERMO A MAIS:

Seja T(n) = Σ_(i=1)^(n) i!, para n ≥ 0 (T(0) = 0, soma vazia). Acrescentando mais um termo:

```
1: T(n + 1) = Σ_(i=1)^(n+1) i! = Σ_(i=1)^(n) i! + (n + 1)! = T(n) + (n + 1)!.
```

PASSO 2 — A MESMA SOMA REESCRITA DE OUTRA FORMA:

Trocando o índice i por i + 1 (o novo índice vai de 0 a n) e separando o termo i = 0:

```
2: T(n + 1) = Σ_(i=0)^(n) (i + 1)! = 1! + Σ_(i=1)^(n) (i + 1)! = 1 + Σ_(i=1)^(n) (i + 1)!.
```

Pela definição de fatorial, (i + 1)! = (i + 1)·i! = i·i! + i! para todo i ≥ 0. Logo, pela linearidade da soma finita,

```
3: T(n + 1) = 1 + Σ_(i=1)^(n) (i·i! + i!) = 1 + Σ_(i=1)^(n) i·i! + Σ_(i=1)^(n) i! = 1 + S(n) + T(n).
```

Para n = 0 as linhas 2 e 3 continuam válidas: Σ_(i=0)^(0) (i + 1)! = 1! = 1, e as somas de i = 1 a 0 são vazias, valendo 0; fica T(1) = 1 = 1 + 0 + 0.

PASSO 3 — IGUALAR AS DUAS FORMAS:

As linhas 1 e 3 são duas expressões do mesmo número T(n + 1):

```
4: T(n) + (n + 1)! = 1 + S(n) + T(n).
```

T(n) é um número finito (soma de n parcelas finitas), então pode ser subtraído dos dois lados:

```
5: S(n) = (n + 1)! − 1.
```

Como no exemplo do slide 15 (onde Σ i³ aparece dos dois lados e se cancela), a soma perturbada T(n) se cancela por inteiro: não é preciso conhecer T(n), que serve só de veículo para chegar a S(n). A diferença está no que sobra: no slide 15 sobram as somas auxiliares Σ i e Σ 1, cujas formas fechadas é preciso conhecer, e a incógnita Σ i² aparece com coeficiente 3; aqui não sobra nenhuma soma além de S(n), que aparece com coeficiente 1.

PASSO 4 — CONFERÊNCIA DA FORMA FECHADA POR INDUÇÃO:

Seja P(n): Σ_(i=1)^(n) i·i! = (n + 1)! − 1. Provo P(n) para todo inteiro n ≥ 0.

Base, n = 0: o lado esquerdo é a soma vazia, igual a 0; o lado direito é 1! − 1 = 1 − 1 = 0. P(0) vale. (Se se preferir começar em n = 1, como o enunciado sugere: lado esquerdo 1·1! = 1, lado direito 2! − 1 = 1; P(1) vale.)

Passo: seja n ≥ 0 e suponha P(n), isto é, Σ_(i=1)^(n) i·i! = (n + 1)! − 1 (hipótese de indução). Então

```
6: Σ_(i=1)^(n+1) i·i! = Σ_(i=1)^(n) i·i! + (n + 1)·(n + 1)!
    7:                  = (n + 1)! − 1 + (n + 1)·(n + 1)!          (hipótese de indução)
    8:                  = (n + 1)!·(1 + (n + 1)) − 1
    9:                  = (n + 2)·(n + 1)! − 1
    10:                 = (n + 2)! − 1,
```

que é exatamente P(n + 1). Pelo princípio da indução, P(n) vale para todo n ≥ 0.

OBSERVAÇÃO — POR QUE FUNCIONA (TELESCOPAGEM).

A identidade da linha 3, i·i! = (i + 1)! − i!, mostra que cada parcela é uma diferença de fatoriais consecutivos; somando de i = 1 a n, os termos intermediários se cancelam: S(n) = (2! − 1!) + (3! − 2!) + … + ((n + 1)! − n!) = (n + 1)! − 1! = (n + 1)! − 1. É o mesmo resultado, obtido sem a soma auxiliar; a perturbação faz esse cancelamento de forma implícita.

Exemplos: n = 1: 1 = 2! − 1; n = 2: 1 + 4 = 5 = 3! − 1; n = 3: 1 + 4 + 18 = 23 = 4! − 1; n = 4: 23 + 96 = 119 = 5! − 1.

Conferência numérica (Python; não faz parte da prova): Σ i·i! = (n + 1)! − 1 para n = 0..14, o que coincide com a forma fechada obtida pela perturbação e provada por indução (inclusive em n = 0, soma vazia). A conferência numérica não substitui a prova; a prova é a do PASSO 4.

---

38. Walace (IFMG), slide 16, item 3

Se vire com esse

$\Sigma_{i=1}^{n} i * 2^{i} = ?$

Contexto do material (transcrição literal, necessária para resolver):
3.2. Perturbação (slide 15)

- Vantagem: método poderoso, resolve muitos somatórios
- Modo de usar: acrescente mais um termo e reescreva a soma
- Desvantagem: pode ser difícil saber o que perturbar

[quadro] $\Sigma_{i=1}^{n} i^2 = ?$ Dica: perturbe a soma dos cubos
$\Sigma_{i=1}^{\mathbf{n+1}} i^3 = \Sigma_{i=1}^{n} i^3 + (n+1)^3 = \Sigma_{i=0}^{n} (i+1)^3 = 1 + \Sigma_{i=1}^{n} (i+1)^3$
$= 1 + \Sigma_{i=1}^{n} (i^3 + 3i^2 + 3i + 1) = 1 + \Sigma_{i=1}^{n} i^3 + 3\,\Sigma_{i=1}^{n} i^2 + 3\,\Sigma_{i=1}^{n} i + \Sigma_{i=1}^{n} 1$
$= 1 + \Sigma_{i=1}^{n} i^3 + 3\,\Sigma_{i=1}^{n} i^2 + 3(n/2)(n+1) + n$
= agora continue daqui

3.2. Perturbação (slide 16) — **Exercícios:**

1. Examine o processo de perturbação da soma dos quadrados
   $\Sigma_{i=1}^{\mathbf{n+1}} i^2 = ?$

[transcrição literal — Walace (IFMG), slide 16, item 3]

Resolução

Resposta. Para todo inteiro n ≥ 0 (com a soma vazia valendo 0 quando n = 0), Σ_(i=1)^(n) i * 2^i = (n − 1)·2^(n+1) + 2. Obtém-se pelo método da perturbação (slide 15): escreve-se S_(n+1) de duas formas, isolando o último termo e deslocando o índice, e isola-se S_n. A forma fechada é depois confirmada por indução em n.

LEITURA DO ENUNCIADO: O símbolo "*" é a multiplicação, como no restante do material, e "2^(i)" é a potência 2 elevada a i. A pergunta "= ?" pede a forma fechada da soma S_n = Σ_(i=1)^(n) i·2^i em função de n. Outra leitura possível seria pedir só a ordem de crescimento (por exemplo Θ(n·2^n)). Fica a da forma fechada exata, porque o exercício está na seção "3.2. Perturbação",

cujo método produz uma expressão exata, e a ordem de crescimento sai dela de imediato. O limite inferior é i = 1. O termo i = 0 valeria 0·2^0 = 0, então começar em 0 ou em 1 dá o mesmo valor, e isso é usado abaixo.

NOTAÇÃO E FATO AUXILIAR: Seja S_n = Σ_(i=1)^(n) i·2^i, para n ≥ 0 (S_0 = 0, soma vazia). Usa-se também a soma geométrica G_n = Σ_(i=0)^(n) 2^i = 2^(n+1) − 1, para n ≥ 0. Ela também sai por perturbação. De um lado, G_(n+1) = G_n + 2^(n+1), separando o último termo. Do outro, G_(n+1) = 2^0 + Σ_(i=1)^(n+1) 2^i = 1 + Σ_(j=0)^(n) 2^(j+1) = 1 + 2·G_n (com i = j + 1). Igualando, G_n + 2^(n+1) = 1 + 2·G_n, logo G_n = 2^(n+1) − 1.

PASSO 1 — PRIMEIRA FORMA DE S_(n+1) (ACRESCENTAR UM TERMO): Separando o termo i = n + 1: S_(n+1) = Σ_(i=1)^(n) i·2^i + (n + 1)·2^(n+1) = S_n + (n + 1)·2^(n+1).

```
PASSO 2 — SEGUNDA FORMA DE S_(n+1) (REESCREVER A SOMA):
Com a troca de índice i = j + 1 (quando i vai de 1 a n + 1, j vai de 0 a n):
    S_(n+1) = Σ_(j=0)^(n) (j + 1)·2^(j+1)
            = Σ_(j=0)^(n) (2·j·2^j + 2·2^j)
            = 2·Σ_(j=0)^(n) j·2^j + 2·Σ_(j=0)^(n) 2^j.
Na primeira soma, o termo j = 0 vale 0·2^0 = 0, então Σ_(j=0)^(n) j·2^j = Σ_(j=1)^(n) j·2^j = S_n. A segunda soma
é G_n = 2^(n+1) − 1. Logo
    S_(n+1) = 2·S_n + 2·(2^(n+1) − 1) = 2·S_n + 2^(n+2) − 2.
```

```
PASSO 3 — ISOLAR S_n:
Igualando as duas expressões de S_(n+1):
    S_n + (n + 1)·2^(n+1) = 2·S_n + 2^(n+2) − 2,
    S_n = (n + 1)·2^(n+1) − 2^(n+2) + 2 = (n + 1)·2^(n+1) − 2·2^(n+1) + 2 = (n − 1)·2^(n+1) + 2.
Esta é a mesma estratégia do slide 15, que aplica ao somatório Σ i³ um termo a mais e reescreve a soma. Aqui a
soma perturbada já é a própria S_n: a parte que "sobra" do lado direito (2·S_n) traz S_n com coeficiente 2 ≠ 1,
então ela não se cancela e pode ser isolada. Não foi preciso perturbar uma soma de grau maior, como acontece com Σ
i² no slide.
```

```
PASSO 4 — CONFERÊNCIA POR INDUÇÃO EM n:
P(n): Σ_(i=1)^(n) i·2^i = (n − 1)·2^(n+1) + 2, para o inteiro n ≥ 0.
Base, n = 0: o lado esquerdo é a soma vazia, igual a 0. O lado direito é (0 − 1)·2^1 + 2 = −2 + 2 = 0. P(0) vale.
(Para quem preferir começar em n = 1: o lado esquerdo é 1·2^1 = 2, e o direito é (1 − 1)·2^2 + 2 = 2. P(1) também
vale.)
Passo: seja n ≥ 0 e suponha P(n), isto é, S_n = (n − 1)·2^(n+1) + 2 (hipótese de indução). Então
    S_(n+1) = S_n + (n + 1)·2^(n+1)                    (separando o último termo)
            = (n − 1)·2^(n+1) + 2 + (n + 1)·2^(n+1)    (hipótese de indução)
            = 2n·2^(n+1) + 2
            = n·2^(n+2) + 2
            = ((n + 1) − 1)·2^((n+1)+1) + 2,
que é exatamente P(n + 1). Pelo princípio da indução, P(n) vale para todo inteiro n ≥ 0.
```

OBSERVAÇÃO SOBRE A ORDEM DE CRESCIMENTO (consequência, não pedida): Para n ≥ 1, S_n = (n − 1)·2^(n+1) + 2 ≤ n·2^(n+1) = 2·n·2^n, porque (n − 1)·2^(n+1) + 2 ≤ n·2^(n+1) equivale a 2 ≤ 2^(n+1), que vale para n ≥ 0. Também S_n ≥ (n − 1)·2^(n+1) ≥ (n/2)·2^(n+1) = n·2^n quando n − 1 ≥ n/2, isto é, n ≥ 2. Então S_n ∈ Θ(n·2^n) com testemunhas c1 = 1, c2 = 2, no = 2. Em n = no = 2: S_2 = 1·2 + 2·4 = 10, c1·n·2^n = 8 ≤ 10 e c2·n·2^n = 16 ≥ 10.

Conferência numérica (Python; não faz parte da prova): Σ i·2^i = (n − 1)·2^(n+1) + 2 para n = 0..14, a mesma forma fechada obtida acima. Contas à mão em valores pequenos batem: n = 0 dá 0, n = 1 dá 2, n = 2 dá 2 + 8 = 10 = 1·8 + 2, n = 3 dá 10 + 24 = 34 = 2·16 + 2.

Princípio de Indução Finita (IME-USP)

39. USP, ex. 2.1

Exercício 2.1: (Trabalho p/ casa)

Mostre que

$1/(1 \cdot 2) + 1/(2 \cdot 3) + \cdots + 1/(n \cdot (n+1)) = n/(n+1)$,

para todos $n \geq 1$.

[transcrição literal — Princípio de Indução Finita (IME-USP) 9]

Resolução

Resposta. Vale a igualdade 1/(1*2) + 1/(2*3) + ... + 1/(n*(n+1)) = n/(n+1) para todo n >= 1, demonstrada por inducao sobre n.

ENUNCIADO DE P(n) Para cada inteiro n >= 1, seja P(n) a afirmacao:

P(n): soma_{k=1}^{n} 1/(k*(k+1)) = n/(n+1).

BASE (n = 1) Lado esquerdo: a soma tem um unico termo, k = 1, e vale 1/(1*2) = 1/2. Lado direito: n/(n+1) com n = 1 da 1/(1+1) = 1/2. Os dois lados valem 1/2, logo P(1) e verdadeira.

(Confirmacao adicional em n = 2, so para ilustrar: 1/2 + 1/(2*3) = 1/2 + 1/6 = 2/3 = 2/(2+1).)

HIPOTESE DE INDUCAO Seja n >= 1 um inteiro fixo e suponha que P(n) e verdadeira, isto e:

(HI) soma_{k=1}^{n} 1/(k*(k+1)) = n/(n+1).

PASSO INDUTIVO (P(n) => P(n+1)) Queremos mostrar que soma_{k=1}^{n+1} 1/(k*(k+1)) = (n+1)/(n+2). Separando o ultimo termo da soma (o termo k = n+1, que vale 1/((n+1)*(n+2))):

soma_{k=1}^{n+1} 1/(k*(k+1)) = [soma_{k=1}^{n} 1/(k*(k+1))] + 1/((n+1)*(n+2)) = n/(n+1) + 1/((n+1)*(n+2)) <-- AQUI se aplica a hipotese (HI)

Agora e algebra pura. Colocando tudo sobre o denominador comum (n+1)*(n+2):

```
n/(n+1) + 1/((n+1)*(n+2))
    = [ n*(n+2) ] / [ (n+1)*(n+2) ]  +  1 / [ (n+1)*(n+2) ]
    = [ n*(n+2) + 1 ] / [ (n+1)*(n+2) ]
    = [ n^2 + 2n + 1 ] / [ (n+1)*(n+2) ]
    = (n+1)^2 / [ (n+1)*(n+2) ]
    = (n+1)/(n+2).
```

Observe que (n+1) e (n+2) sao positivos para n >= 1, entao o cancelamento de um fator (n+1) e legitimo (nao ha divisao por zero).

O resultado (n+1)/(n+2) e exatamente a formula m/(m+1) avaliada em m = n+1. Logo P(n+1) e verdadeira.

CONCLUSAO P(1) vale e, para todo n >= 1, P(n) => P(n+1). Pelo Principio de Inducao Finita, P(n) vale para todo inteiro n >= 1, ou seja:

1/(1*2) + 1/(2*3) + ... + 1/(n*(n+1)) = n/(n+1) para todo n >= 1.

OBSERVACAO (por que a formula 'funciona') O motivo estrutural e que 1/(k*(k+1)) = 1/k - 1/(k+1), de modo que a soma telescopa e sobra 1 - 1/(n+1) = n/(n+1). Isso nao substitui a inducao pedida, mas explica por que o passo indutivo fecha tao limpo.

---

40. USP, ex. 2.2

Exercício 2.2: (Trabalho p/ casa)

Mostre que $2n^3 > 3n^2 + 3n + 1$ para todos $n \geq 3$.

[transcrição literal — Princípio de Indução Finita (IME-USP) 9]

Resolução

Resposta. Vale 2n^3 > 3n^2 + 3n + 1 para todo inteiro n >= 3. A base tem de ser n = 3 porque a desigualdade e FALSA em n = 1 e em n = 2.

ENUNCIADO DE P(n) Para cada inteiro n >= 3, seja P(n) a afirmacao:

P(n): 2n^3 > 3n^2 + 3n + 1.

POR QUE A BASE E n = 3 (e nao antes) A desigualdade nao vale para n < 3: n = 1: lado esquerdo 2*1 = 2; lado direito 3 + 3 + 1 = 7. Como 2 > 7 e falso, P(1) e falsa. n = 2: lado esquerdo 2*8 = 16; lado direito 3*4 + 3*2 + 1 = 12 + 6 + 1 = 19. Como 16 > 19 e falso, P(2) e falsa. Portanto o menor inteiro em que a afirmacao pode comecar a valer e n = 3, e e ali que a base deve ser feita. (Isso e importante: o passo indutivo abaixo so sera usado a partir de n >= 3.)

BASE (n = 3) Lado esquerdo: 2*3^3 = 2*27 = 54. Lado direito: 3*3^2 + 3*3 + 1 = 27 + 9 + 1 = 37. Como 54 > 37, P(3) e verdadeira.

HIPOTESE DE INDUCAO Seja n >= 3 um inteiro fixo e suponha que P(n) e verdadeira, isto e:

(HI) 2n^3 > 3n^2 + 3n + 1.

PASSO INDUTIVO (P(n) => P(n+1), para n >= 3) Queremos mostrar que 2(n+1)^3 > 3(n+1)^2 + 3(n+1) + 1.

1) Expandindo o lado esquerdo em n+1: 2(n+1)^3 = 2*(n^3 + 3n^2 + 3n + 1) = 2n^3 + 6n^2 + 6n + 2.

2) Aplicando a hipotese (HI) ao pedaco 2n^3 (a desigualdade e estrita e os demais termos sao somados dos dois lados, o que preserva a desigualdade estrita): 2n^3 + 6n^2 + 6n + 2 > (3n^2 + 3n + 1) + 6n^2 + 6n + 2 <-- AQUI se aplica (HI) = 9n^2 + 9n + 3.

```
3) Expandindo o lado direito em n+1:
  3(n+1)^2 + 3(n+1) + 1 = 3*(n^2 + 2n + 1) + 3n + 3 + 1
                        = 3n^2 + 6n + 3 + 3n + 4
                        = 3n^2 + 9n + 7.
```

4) Falta comparar o que obtivemos (9n^2 + 9n + 3) com o alvo (3n^2 + 9n + 7): 9n^2 + 9n + 3 >= 3n^2 + 9n + 7 <=> 9n^2 - 3n^2 >= 7 - 3 <=> 6n^2 >= 4. Como n >= 3, temos n^2 >= 9, logo 6n^2 >= 54 >= 4. (De fato 6n^2 >= 4 ja vale para todo n >= 1, pois 6*1 = 6 >= 4; para n >= 3 vale com folga enorme.) Portanto 9n^2 + 9n + 3 >= 3n^2 + 9n + 7.

5) Encadeando 1)-4): 2(n+1)^3 = 2n^3 + 6n^2 + 6n + 2 > 9n^2 + 9n + 3 >= 3n^2 + 9n + 7 = 3(n+1)^2 + 3(n+1) + 1.

A cadeia tem uma desigualdade estrita (>) seguida de uma nao-estrita (>=), o que da desigualdade estrita no total: 2(n+1)^3 > 3(n+1)^2 + 3(n+1) + 1, ou seja, P(n+1) e verdadeira.

CONCLUSAO P(3) vale e, para todo n >= 3, P(n) => P(n+1). Pelo Principio de Inducao Finita (com base deslocada para 3), P(n) vale para todo inteiro n >= 3:

2n^3 > 3n^2 + 3n + 1 para todo n >= 3.

OBSERVACAO SOBRE A BASE DESLOCADA A versao do PIF usada aqui e a padrao com inicio em n0: se P(n0) vale e P(n) => P(n+1) para todo n >= n0, entao P(n) vale para todo n >= n0. Aqui n0 = 3. Nada foi afirmado sobre n = 1 e n = 2 — e bom, porque nesses pontos a afirmacao e falsa, como calculado acima.

---

41. USP, EX. 2.3

Exercício 2.3: (Trabalho p/ casa)

Mostre que na sequencia [sic] de Fibonacci F_{5n} sempre é um multiplo [sic] de 5.

[transcrição literal — Princípio de Indução Finita (IME-USP) 9]

RESOLUÇÃO

Resposta. F_(5n) e multiplo de 5 para todo n >= 1 (e tambem para n = 0, pois F_0 = 0). A prova sai por inducao usando a identidade F_(5n+5) = 5*F_(5n+1) + 3*F_(5n).

CONVENCAO ADOTADA Uso a convencao padrao F_0 = 0, F_1 = 1, e F_k = F_(k-1) + F_(k-2) para todo k >= 2. Assim a sequencia comeca 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ... (Quem prefere F_1 = F_2 = 1 obtem exatamente os mesmos valores para indices >= 1, entao o enunciado nao muda.)

ENUNCIADO DE P(n) Para cada inteiro n >= 1, seja P(n) a afirmacao:

P(n): 5 divide F_(5n), isto e, existe um inteiro t tal que F_(5n) = 5t.

BASE (n = 1) Calculando os primeiros termos: F_0 = 0, F_1 = 1, F_2 = 1, F_3 = 2, F_4 = 3, F_5 = 5. Logo F_(5*1) = F_5 = 5 = 5*1, que e multiplo de 5. P(1) e verdadeira. (Conferindo os proximos, so para ilustrar: F_10 = 55 = 5*11, F_15 = 610 = 5*122, F_20 = 6765 = 5*1353.)

LEMA (identidade que liga F_(5n+5) a F_(5n)) Afirmo que, para todo inteiro m >= 0,

(L) F_(m+5) = 5*F_(m+1) + 3*F_m.

Demonstracao do lema: basta aplicar a recorrencia F_k = F_(k-1) + F_(k-2) cinco vezes, sempre reescrevendo tudo em funcao de F_(m+1) e F_m: F_(m+2) = F_(m+1) + F_m = 1*F_(m+1) + 1*F_m F_(m+3) = F_(m+2) + F_(m+1) = (F_(m+1) + F_m) + F_(m+1) = 2*F_(m+1) + 1*F_m F_(m+4) = F_(m+3) + F_(m+2) = (2F_(m+1) + F_m) + (F_(m+1) + F_m) = 3*F_(m+1) + 2*F_m F_(m+5) = F_(m+4) + F_(m+3) = (3F_(m+1) + 2F_m) + (2F_(m+1) + F_m) = 5*F_(m+1) + 3*F_m. Isso prova (L). (Nota: (L) e um caso particular da identidade de adicao de Fibonacci F_(m+k) = F_m*F_(k+1) + F_(m-1)*F_k, resultado padrao; com k = 5 ela da F_(m+5) = 8*F_m + 5*F_(m-1). Qualquer uma das duas serve — usarei (L), que acabei de provar do zero pela propria recorrencia.)

HIPOTESE DE INDUCAO Seja n >= 1 um inteiro fixo e suponha que P(n) e verdadeira, isto e:

(HI) F_(5n) = 5t para algum inteiro t.

PASSO INDUTIVO (P(n) => P(n+1)) Queremos mostrar que 5 divide F_(5(n+1)) = F_(5n+5). Aplicando o lema (L) com m = 5n (valido, pois 5n >= 0):

F_(5n+5) = 5*F_(5n+1) + 3*F_(5n).

Pela hipotese (HI), F_(5n) = 5t, logo <-- AQUI se aplica (HI)

F_(5n+5) = 5*F_(5n+1) + 3*(5t) = 5*(F_(5n+1) + 3t).

Como F_(5n+1) e t sao inteiros, F_(5n+1) + 3t e inteiro. Portanto F_(5n+5) e 5 vezes um inteiro, ou seja, 5 divide F_(5(n+1)). Logo P(n+1) e verdadeira.

CONCLUSAO P(1) vale e, para todo n >= 1, P(n) => P(n+1). Pelo Principio de Inducao Finita, P(n) vale para todo inteiro n >= 1:

F_(5n) e multiplo de 5 para todo n >= 1.

(Se quisermos incluir n = 0, tambem vale trivialmente: F_0 = 0 = 5*0.)

CAMINHO ALTERNATIVO (equivalente) Usando a identidade de adicao com m = 5n e k = 5: F_(5n+5) = F_(5n)*F_6 + F_(5n-1)*F_5 = 8*F_(5n) + 5*F_(5n-1). Pela hipotese 5 | F_(5n), entao 5 | 8*F_(5n); e 5 | 5*F_(5n-1) obviamente. Como a soma de dois multiplos de 5 e multiplo de 5, segue 5 | F_(5n+5). O unico cuidado extra desse caminho e que ele exige n >= 1 (para que o indice 5n-1 seja >= 0) e o uso de um resultado que nao foi provado aqui — por isso preferi o lema (L), autocontido.

---

42. USP, EX. 2.4

Exercício 2.4: (Trabalho p/ casa)

Mostre que (*PIF*) ⇒ (*PBO*), ou seja por indução, mostre que todo conjunto não-vazio dos inteiros positivos contém elemento minimal.

[transcrição literal — Princípio de Indução Finita (IME-USP) 9]

Resolução

Resposta. (PIF) => (PBO): todo subconjunto nao-vazio S dos inteiros positivos possui elemento minimo. A prova e por absurdo: supondo que S nao tenha minimo, uma inducao mostra que nenhum inteiro positivo esta em S, logo S seria vazio.

O QUE SE QUER PROVAR Principio da Boa Ordenacao (PBO): se S e subconjunto dos inteiros positivos Z+ = {1, 2, 3, ...} e S e nao-vazio, entao existe m em S tal que m <= s para todo s em S (m e o elemento minimo de S). Vamos deduzir isso admitindo apenas o Principio de Inducao Finita (PIF).

ESTRATEGIA Prova por contradicao. Suponha, por absurdo, que exista S subconjunto de Z+ com S nao-vazio e SEM elemento minimo. A partir dessa suposicao, mostraremos por inducao que S nao contem nenhum inteiro positivo — o que forca S = vazio e contradiz S nao-vazio.

ENUNCIADO DE P(n) Fixada a suposicao de absurdo acima, para cada inteiro n >= 1 seja P(n) a afirmacao:

P(n): nenhum dos inteiros 1, 2, ..., n pertence a S. (equivalentemente: S nao contem elemento algum <= n)

ATENCAO A FORMA DE P(n): ela nao fala de um unico valor, e sim de TODOS os inteiros de 1 ate n. Isso e proposital — e o modo padrao de simular inducao forte dentro do PIF comum. A hipotese de inducao, portanto, me dara de uma vez a informacao sobre 1, 2, ..., n, e nao apenas sobre n. Nao ha troca indevida de hipotese: e exatamente a afirmacao P(n) enunciada acima que sera suposta e usada.

BASE (n = 1) Afirmo que 1 nao pertence a S. Suponha que 1 pertencesse a S. Como S esta contido em Z+, todo s em S satisfaz s >= 1; logo 1 <= s para todo s em S, e 1 pertence a S. Ou seja, 1 seria o elemento minimo de S — contradizendo a suposicao de que S nao tem minimo. Portanto 1 nao pertence a S, e P(1) e verdadeira.

HIPOTESE DE INDUCAO Seja n >= 1 fixo e suponha P(n), isto e:

(HI) nenhum dos inteiros 1, 2, ..., n pertence a S.

PASSO INDUTIVO (P(n) => P(n+1)) Basta mostrar que n+1 tambem nao pertence a S, pois entao nenhum dos inteiros 1, ..., n, n+1 estara em S, que e exatamente P(n+1). Suponha, por absurdo dentro do passo, que n+1 pertenca a S. Tome s em S qualquer. Como s e inteiro positivo, ou s <= n ou s >= n+1. O caso s <= n esta excluido pela hipotese (HI), que diz que nenhum inteiro de 1 a n esta em S. <-- AQUI se aplica (HI) Logo s >= n+1 para todo s em S. Como alem disso n+1 pertence a S, concluimos que n+1 e o elemento minimo de S — contradizendo novamente a suposicao de que S nao tem minimo. Portanto n+1 nao pertence a S, e P(n+1) e verdadeira.

APLICACAO DO PIF P(1) vale e P(n) => P(n+1) para todo n >= 1. Pelo PIF, P(n) e verdadeira para todo inteiro n >= 1.

FECHAMENTO DA CONTRADICAO Seja k um inteiro positivo qualquer. Tomando n = k, a afirmacao P(k) diz em particular que k nao pertence a S. Como k era arbitrario, nenhum inteiro positivo pertence a S. Mas S esta contido em Z+, logo S = vazio. Isso contradiz a hipotese de que S e nao-vazio.

CONCLUSAO A suposicao de absurdo (existir S subconjunto de Z+, nao-vazio, sem elemento minimo) e insustentavel. Logo todo subconjunto nao-vazio de Z+ possui elemento minimo, isto e, (PIF) => (PBO). Como se queria demonstrar.

COMENTARIO CONCEITUAL (por que a prova tem esse formato) O PBO fala de existencia de um objeto (o minimo) dentro de um conjunto sobre o qual nao sabemos nada alem de 'nao-vazio'. Uma forma natural de prova-lo, a adotada aqui, e inverter: em vez de procurar o minimo, usa-se a inducao para varrer 1, 2, 3, ... e mostrar que a ausencia de minimo obriga cada um desses inteiros a ficar de fora de S. (Ha tambem uma prova direta, sem absurdo: com Q(n) = 'todo S contido em Z+ que tem algum elemento em {1, ..., n} possui minimo', a base vale porque, se 1 esta em S, 1 e o minimo; no passo, se S tem elemento em {1, ..., n} aplica-se a hipotese, e senao n+1 esta em S e e o minimo. Como S e nao-vazio, contem algum k, e Q(k) da o minimo. Nas duas provas, 'minimo' (menor que todos) implica 'minimal' (nenhum menor), que e o que o enunciado pede; em Z+, ordem total, os dois coincidem.) A varredura esgota Z+, e um subconjunto de Z+ que exclui todos os inteiros positivos so pode ser o vazio. O 'nao-vazio' do enunciado e justamente o que colide com esse esvaziamento. Vale notar ainda que a reciproca tambem e verdadeira ((PBO) => (PIF)), de modo que os dois principios sao equivalentes sobre os inteiros positivos; aqui foi pedida apenas uma das direcoes.

Slides de MC458 (UNICAMP) · semana 5 · exercício deixado em aberto

43. Slides UNICAMP MC458, Exemplo 3

Contexto — slides de MC458 (UNICAMP), "Indução matemática no projeto de algoritmos", Exemplo 3

Dada uma árvore binária *A* com *n* nós, calcular o fator de balanceamento (f.b.) de cada nó — a altura da subárvore esquerda menos a da direita. Convenção: árvore vazia tem altura *−1* e f.b. zero. A hipótese de indução **fortalecida** ("sabemos calcular f.b. *e alturas* de árvores com menos que *n* nós") leva ao algoritmo:

No slide ("Exemplo 3 - Algoritmo"), transcrição literal:

FatorAltura (A)
▷ Entrada: Uma árvore binária A com n ≥ 0 nós
▷ Saída: A árvore A os f.b.s nos seus nós e a altura de A
1. se n = 0 então h ← −1
2. senão
3. seja r a raiz de A
3. h_e ← FatorAltura(A_e)

4. $h_d \leftarrow$ FatorAltura(A_d)
5. ▷ armazena o f.b. na raiz em r.f
6. r.f $\leftarrow h_e - h_d$
7. h $\leftarrow \max(h_e, h_d) + 1$
8. devolva h

O slide numera duas linhas como "3." e termina em 8. **Nesta resolução as linhas são citadas pela numeração corrida abaixo** (1 a 9), que é a mesma do slide até a primeira linha 3 e soma 1 dali em diante:

FatorAltura(A) — numeração corrida usada nesta resolução
1. se n = 0 então h ← −1
2. senão
3. seja r a raiz de A
4. he ← FatorAltura(Ae)
5. hd ← FatorAltura(Ad)
6. ▷ armazena o f.b. na raiz em r.f
7. r.f ← he − hd
8. h ← max(he, hd) + 1
9. devolva h

Os dois slides seguintes ("Exemplo 3 - Complexidade"), transcrição literal:

"Seja T(n) o número de operações executadas pelo algoritmo para calcular os f.b.s e a altura de uma árvore A de n nós. Então $T(n) = \Theta(1)$, se $n = 0$; $T(n) = T(n_e) + T(n_d) + \Theta(1)$, se $n > 0$, onde n_e, n_d são os números de nós das subárvores esquerda e direita."

"O pior caso da recorrência parece ser quando, ao longo da recursão, um de n_e, n_d é sempre igual a zero (e o outro é sempre igual a $n - 1$). Chega-se então a: $T(n) = T(1) + \Sigma_{i=2}^{n} \Theta(1) = (n + 1)\Theta(1) + n\Theta(1) \in \Theta(n)$."

E então o slide deixa em aberto:

Exercício: "Há algum ganho em complexidade quando ambos n_e e n_d são aproximadamente $n/2$ ao longo da recursão?"

[transcrição literal do algoritmo, dos dois slides de complexidade e do exercício — MC458 (UNICAMP), p. 2–3; o contexto em prosa e a numeração corrida são nossos]

Resolução

1. Modelo de custo

Duas constantes positivas, fixas e **uniformes sobre todas as chamadas** — é o que os $\Theta(1)$ do slide escondem, e explicitá-las é necessário para poder somar um número *variável* delas:

$a > 0$: custo de uma chamada com A vazia (linha 1 e a devolução)

$b > 0$: custo da parte não recursiva de uma chamada com A não vazia (linha 1 e linhas 3, 7, 8, 9)

A recorrência estrutural exata é $T(A) = a$ se A é vazia, e $T(A) = b + T(A_e) + T(A_d)$ caso contrário.

2. O caso balanceado pelo Teorema Mestre

Com $n_e \approx n_d \approx n/2$ a recorrência vira $T(n) = 2 \cdot T(n/2) + \Theta(1)$. Para não colidir com as constantes de custo a e b, escrevo os parâmetros do teorema como α (número de chamadas recursivas), β (fator de redução) e k, na forma $T(n) = \alpha \cdot T(n/\beta) + c \cdot n^k$, com o critério $\alpha \gtrless \beta^k$.

parâmetro	valor	de onde vem
α	2	linhas 4 e 5: a esquerda *e* a direita são visitadas
β	2	cada subárvore tem ≈ n/2 nós
k	0	o trabalho fora da recursão é constante: $c \cdot n^0$

$\beta^k = 2^0 = 1$ e $\alpha = 2$, logo $\alpha > \beta^k$ — o **terceiro** dos três casos, na ordem em que o formulário da prova os imprime. Portanto

$$\mathbf{T(n) = \Theta(n^{\log_\beta \alpha}) = \Theta(n^{\log_2 2}) = \Theta(n)}$$

> Detalhe de rigor: "$T(n) = 2 \cdot T(n/2) + \Theta(1)$" é uma aproximação — as subárvores somam $n - 1$ nós, não n, e o formulário pede termo motor *literalmente* $c \cdot n^k$. A forma exata cabe no formulário. Numa árvore perfeitamente balanceada ($n = 2^{h+1} - 1$), cada filho tem $(n-1)/2$ nós e $T(n) = 2 \cdot T((n-1)/2) + b$, com $T(0) = a$. Trocando $U(m) = T(m - 1)$, fica $U(m) = 2 \cdot U(m/2) + b$ com $U(1) = a$ e $m = n + 1$ potência exata de 2: $\alpha = 2$, $\beta = 2$, $k = 0$, termo motor $b \cdot m^0$ com $b > 0$, e U positiva e crescente. Terceiro caso: $U(m) = \Theta(m)$, isto é, $T(n) = \Theta(n)$. Para formas só aproximadamente balanceadas, quem decide é o teorema da seção 3, que vale para toda árvore.

3. A razão estrutural: a forma da árvore é irrelevante para o custo

As duas contas acima cobrem apenas os dois extremos. O que segue vale para **toda** árvore binária, e é a resposta de verdade.

Teorema. Para toda árvore binária A com $n \geq 0$ nós, $\mathbf{T(A) = (n+1) \cdot a + n \cdot b}$, qualquer que seja a forma de A.

> **Prova, por indução forte em n.**
>
> **Base (n = 0).** A é vazia; por definição $T(A) = a$. A fórmula dá $(0+1)a + 0 \cdot b = a$. ✓

Passo (n ≥ 1). Suponha a fórmula válida para toda árvore com estritamente menos que n nós — a hipótese *forte* é necessária, porque não se sabe como os $n-1$ nós se repartem. Seja A com n nós, subárvores A_e com n_e nós e A_d com n_d. Como a raiz não está em nenhuma das duas, $n_e + n_d = n - 1$; e como $n \geq 1$, vale $n_e, n_d \leq n-1 < n$, logo a hipótese se aplica às duas. Então

$$\begin{aligned} T(A) &= b + [(n_e+1)a + n_e b] + [(n_d+1)a + n_d b] \\ &= b + (n_e + n_d + 2)\cdot a + (n_e + n_d)\cdot b \\ &= b + (n+1)\cdot a + (n-1)\cdot b \quad \leftarrow \text{usando } n_e + n_d = n - 1 \\ &= (n+1)\cdot a + n\cdot b. \ \blacksquare \end{aligned}$$

Corolário. Tomando $a = b = 1$, o número total de invocações de FatorAltura numa árvore de n nós é **exatamente $2n + 1$**: n chamadas com árvore não vazia (uma por nó) e $n + 1$ com árvore vazia — que são exatamente os $n+1$ ponteiros nulos de uma árvore binária de n nós (contando, em $n = 0$, o ponteiro para a raiz). Casos pequenos: $n = 0 \to 1$; $n = 1 \to 3$ (a raiz e os dois filhos vazios).

Consequência. Escrever "$T(n)$" para um algoritmo cujo custo dependesse da *forma* da entrada só faria sentido como pior caso. Aqui o custo depende **só de n**: $T(n)$ é função genuína de n, e pior caso = melhor caso = caso médio. Não há forma de árvore boa nem ruim.

Testemunhas. Com $T(n) = (a+b)n + a$: $\mathbf{c_1 = a + b}$, $\mathbf{c_2 = 2a + b}$, $\mathbf{n_0 = 1}$. Em $n = n_0 = 1$ exatamente: $T(1) = 2a + b$; a cota inferior dá $a + b \leq 2a + b \iff 0 \leq a$ ✓ (estrito, pois $a > 0$), e a superior $2a + b \geq 2a + b$ ✓ com igualdade. No modelo unitário ($a = b = 1$): $T(n) = 2n + 1$, $c_1 = 2$, $c_2 = 3$, $n_0 = 1$; em $n = 1$: $2 \leq 3 \leq 3$; em $n = 5$: $10 \leq 11 \leq 15$.

4. Por que tinha de ser assim — e o contraste que torna isso não trivial

O algoritmo faz, **por nó**, uma quantidade fixa de trabalho: uma subtração, um max, uma soma. Nenhum nó é visitado duas vezes e nenhum é pulado — é um percurso em pós-ordem disfarçado de recorrência. A forma da árvore *redistribui* as n unidades de trabalho entre os níveis, mas não muda quantas são.

O contraste: se o passo de combinação custasse $\Theta(\text{tamanho da subárvore})$ em vez de $\Theta(1)$ — como no MergeSort —, a forma importaria muito: cadeia daria $\Theta(n^2)$ e balanceada $\Theta(n \lg n)$. O que anula o efeito da forma aqui é especificamente o custo de combinação ser *constante*.

Cota inferior do problema. Qualquer algoritmo correto precisa escrever um f.b. em cada um dos n nós, logo executa $\Omega(n)$ operações (testemunhas: ao menos 1 escrita por nó, isto é $c = 1$, $n_0 = 1$). Como FatorAltura é $O(n)$, ele é **assintoticamente ótimo**. Nenhum balanceamento baixa a classe: o gargalo é o tamanho da saída, não a estratégia.

5. O que muda e o que não muda

Aspecto	Cadeia	Balanceada	Muda?
Tempo assintótico	$\Theta(n)$	$\Theta(n)$	não
Número exato de chamadas	$2n + 1$	$2n + 1$	não — *idênticos*
Custo exato	$(n+1)a + n\cdot b$	$(n+1)a + n\cdot b$	não
Profundidade máxima da recursão	$n + 1 = \Theta(n)$	$\lfloor \lg n \rfloor + 2$ no split perfeito; $\Theta(\lg n)$ em qualquer split de fração constante	**sim**
Risco de estouro de pilha	alto para n grande	praticamente nulo	**sim**

Os quadros simultaneamente ativos formam um caminho da raiz até uma chamada vazia. Numa cadeia de n nós são $n + 1$ quadros; numa árvore balanceada de altura $\lfloor \lg n \rfloor$ são $\lfloor \lg n \rfloor + 2$. Conferindo em $n = 1$: cadeia dá $1+1 = 2$, balanceada dá $\lfloor \lg 1 \rfloor + 2 = 2$ — coincidem, como devem, pois com um nó só as duas formas são a mesma árvore.

Para $n = 10^6$ é a diferença entre $10^6 + 1$ quadros (estouro certo na maioria dos ambientes) e $\lfloor \lg 10^6 \rfloor + 2 = 21$ quadros. Não é ganho de complexidade de *tempo*; é ganho de complexidade de *espaço auxiliar*, de $\Theta(n)$ para $\Theta(\lg n)$. Responder "Θ(n) nos dois casos, logo tanto faz" seria errado por omissão.

> **Por que não é como a busca binária.** É tentador esperar ganho logarítmico "porque a árvore está balanceada". A busca binária tem $T(n) = T(n/2) + \Theta(1)$, ou seja $\alpha = 1$, $\beta = 2$, $k = 0$, $\beta^k = 1$; aqui $\alpha = \beta^k$, segundo caso, dando $\Theta(n^k \lg n) = \Theta(\lg n)$. A queda vem de $\alpha = 1$: a busca binária **descarta** uma das metades. FatorAltura tem $\alpha = 2$ — desce nas duas e não descarta nada, porque precisa do f.b. de todo mundo. Com combinação de custo constante e as duas partes visitadas ($\alpha = 2$), o equilíbrio não muda o tempo. O que dá à busca binária o ganho logarítmico são *duas coisas juntas*: descartar uma das partes ($\alpha = 1$) *e* ficar, a cada passo, com no máximo uma fração constante $q < 1$ de n — então $T(n) \leq T(q\cdot n) + \Theta(1)$ e $T(n) = O(\lg n)$. A metade da busca binária é só um caso: ficar com $9/10$ também dá $O(\lg n)$. Descartando um só elemento por vez, $T(n) = T(n-1) + \Theta(1) = \Theta(n)$. E com combinação de custo proporcional ao tamanho, como no contraste da seção 4, a divisão equilibrada ganha mesmo sem descartar nada: $\Theta(n \lg n)$ contra $\Theta(n^2)$.

6. Uma nota sobre a contagem escrita no slide

O **resultado final do slide está exatamente certo**: a expressão "$(n+1)\Theta(1) + n\Theta(1)$" corresponde termo a termo ao que foi provado — $n + 1$ chamadas com árvore vazia e n com árvore não vazia. Não é sequer aproximação: é o custo exato, e vale para qualquer forma, não só para a cadeia.

E o *passo intermediário* é abreviado, não errado — vale desfazer a objeção fácil. O slide escreve $T(n) = T(1) + \sum_{i=2}^{n} \Theta(1)$, e isso de fato liga os extremos: cada $\Theta(1)$ da soma é o incremento de um passo do desenrolar, que já embute o $T(0)$ do filho vazio, de modo que

$$T(1) + (n-1)\cdot(T(0) + b) = (2a + b) + (n-1)(a + b) = (n+1)a + n\cdot b$$

que é exatamente o lado direito que o slide escreve. Objetar que "a soma tem $n-1$ parcelas e o total tem $2n+1$" seria contar parcelas — e $\Theta(1)$ não se compromete com constante nenhuma. O que vale registrar é outra coisa: somar um número *variável* de termos $\Theta(1)$ só é legítimo porque existe uma constante uniforme limitando todos eles — $a + b$, o valor de cada incremento $T(0) + b$ —, e essa uniformidade não está na notação, está no algoritmo. E o caso base declarado é $n = 0$: parar em $T(1)$ é atalho legítimo, desde que se saiba que $T(1) = 2a + b$, e não a.

E o slide diz "o pior caso da recorrência *parece ser* quando um de n_e, n_d é sempre zero". O teorema da seção 3 mostra que, no modelo de contagem de operações, **não há pior caso de forma**: toda árvore com n nós custa exatamente o mesmo. A cadeia é o pior caso para outra grandeza — a profundidade da pilha. O "parece ser" do slide é prudente, e o exercício é justamente o convite a descobrir que a suspeita não se confirma no tempo.

Exemplos resolvidos dos slides

reproduzidos do material original — enunciado e prova juntos

Loureiro — Análise de Complexidade (UFMG) · semana 1

E1.Exemplo de modelagem: — p. 32

Exemplo de modelagem: Malha rodoviária (1)

Suponha a malha rodoviária entre as seis cidades A, B, C, D, E, e F.

Problema: Achar um subconjunto da malha rodoviária representada pela tabela abaixo que ligue todas as cidades e tenha um comprimento total mínimo.

	B	C	D	E	F
A	5	–	10	–	–
B		5	10	20	–
C			20	–	30
D				20	–
E					10

E2.Função de complexidade — p. 57

Função de complexidade

- Para medir o custo de execução de um algoritmo é comum definir uma função de custo ou **função de complexidade** f.
- $f(n)$ é a medida do tempo necessário para executar um algoritmo para um problema de tamanho n.
- Função de **complexidade de tempo**: $f(n)$ mede o tempo necessário para executar um algoritmo para um problema de tamanho n.
- Função de **complexidade de espaço**: $f(n)$ mede a memória necessária para executar um algoritmo para um problema de tamanho n.
- Utilizaremos f para denotar uma função de complexidade de tempo daqui para a frente.
- Na realidade, a complexidade de tempo não representa tempo diretamente, mas o número de vezes que determinada operação considerada relevante é executada.

Exemplo: Registros de um arquivo

- Considere o problema de acessar os **registros** de um arquivo.
- Cada registro contém uma **chave** única que é utilizada para recuperar registros do arquivo.
- O problema: dada uma chave qualquer, localize o registro que contenha esta chave.
- O algoritmo de pesquisa mais simples é o que faz a **pesquisa seqüencial**.
- Seja f uma função de complexidade tal que $f(n)$ é o número de registros consultados no arquivo (número de vezes que a chave de consulta é comparada com a chave de cada registro).
  - Melhor caso: $f(n) = 1$ (registro procurado é o primeiro consultado);
  - Pior caso: $f(n) = n$ (registro procurado é o último consultado ou não está presente no arquivo);
  - Caso médio: $f(n) = \frac{n+1}{2}$.

Exemplo: Registros de um arquivo

- No estudo do caso médio, vamos considerar que toda pesquisa recupera um registro.
- Se p_i for a probabilidade de que o i-ésimo registro seja procurado, e considerando que para recuperar o i-ésimo registro são necessárias i comparações, então
  $f(n) = 1 \times p_1 + 2 \times p_2 + 3 \times p_3 + \cdots + n \times p_n$.
- Para calcular $f(n)$ basta conhecer a distribuição de probabilidades p_i.
- Se cada registro tiver a mesma probabilidade de ser acessado que todos os outros, então $p_i = 1/n, 1 \leq i \leq n$.
- Neste caso $f(n) = \frac{1}{n}(1 + 2 + 3 + \cdots + n) = \frac{1}{n}\left(\frac{n(n+1)}{2}\right) = \frac{n+1}{2}$.
- A análise do caso esperado revela que uma pesquisa com sucesso examina aproximadamente metade dos registros.

Exemplo: Maior e menor elementos (1)

- Considere o problema de encontrar o maior e o menor elemento de um vetor de inteiros $A[1..n], n \geq 1$.
- Um algoritmo simples pode ser derivado do algoritmo apresentado no programa para achar o maior elemento.

```
procedure MaxMin1 (var A: Vetor; var Max, Min: integer);
var i: integer;
begin
  Max := A[1];  Min := A[1];
  for i := 2 to n do
    begin
      if A[i] > Max then Max := A[i]; {Testa se A[i] contém o maior elemento}
      if A[i] < Min then Min := A[i]; {Testa se A[i] contém o menor elemento}
    end;
end;
```

- Seja $f(n)$ o número de comparações entre os elementos de A, se A tiver n elementos.
- Logo $f(n) = 2(n-1)$, para $n > 0$, para o melhor caso, pior caso e caso médio.

Exemplo: Maior e menor elementos (2)

- MaxMin1 pode ser facilmente melhorado:
  - a comparação $A[i] <$ Min só é necessária quando o resultado da comparação $A[i] >$ Max for falso.

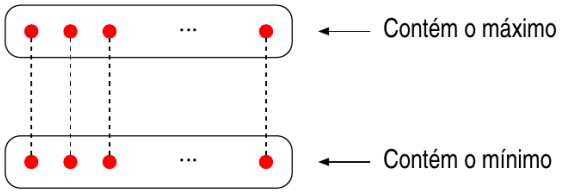
```
procedure MaxMin2 (var A: Vetor; var Max, Min: integer);
var i: integer;
begin
  Max := A[1];  Min := A[1];
  for i := 2 to n do
    if A[i] > Max
    then Max := A[i]
    else if A[i] < Min then Min := A[i];
end;
```


Exemplo: Maior e menor elementos (2)

- Para a nova implementação temos:
  - Melhor caso: $f(n) = n - 1$ (quando os elementos estão em ordem crescente);
  - Pior caso: $f(n) = 2(n - 1)$ (quando os elementos estão em ordem decrescente);
  - Caso médio: $f(n) = \frac{3n}{2} - \frac{3}{2}$.
- Caso médio:
  - $A[i]$ é maior do que Max a metade das vezes.
  - Logo, $f(n) = n - 1 + \frac{n-1}{2} = \frac{3n}{2} - \frac{3}{2}$, para $n > 0$.

Exemplo: Maior e menor elementos (3)

- Considerando o número de comparações realizadas, existe a possibilidade de obter um algoritmo mais eficiente:
  1. Compare os elementos de A aos pares, separando-os em dois subconjuntos (maiores em um e menores em outro), a um custo de $\lceil n/2 \rceil$ comparações.
  2. O máximo é obtido do subconjunto que contém os maiores elementos, a um custo de $\lceil n/2 \rceil - 1$ comparações.
  3. O mínimo é obtido do subconjunto que contém os menores elementos, a um custo de $\lceil n/2 \rceil - 1$ comparações.



Exemplo: Maior e menor elementos (3)

```
procedure MaxMin3(var A: Vetor;
                  var Max, Min: integer);
var i,
    FimDoAnel: integer;
begin
  {Garante uma qte par de elementos no vetor para evitar caso de exceção}
  if (n mod 2) > 0
  then begin
         A[n+1]    := A[n];
         FimDoAnel := n;
       end
  else FimDoAnel := n-1;

  {Determina maior e menor elementos iniciais}
  if A[1] > A[2]
  then begin
         Max := A[1]; Min := A[2];
       end
  else begin
         Max := A[2]; Min := A[1];
       end;
```

Exemplo: Maior e menor elementos (3)

```
 i:= 3;
 while i <= FimDoAnel do
   begin
   {Compara os elementos aos pares}
   if A[i] > A[i+1]
   then begin
          if A[i] > Max then   Max := A[i];
          if A[i+1] < Min then Min := A[i+1];
        end
   else begin
          if A[i] < Min then   Min := A[i];
          if A[i+1] > Max then Max := A[i+1];
        end;
   i:= i + 2;
   end;
end;
```


Exemplo: Maior e menor elementos (3)

- Os elementos de A são comparados dois a dois e os elementos maiores são comparados com `Max` e os elementos menores são comparados com `Min`.
- Quando n é ímpar, o elemento que está na posição $A[n]$ é duplicado na posição $A[n+1]$ para evitar um tratamento de exceção.
- Para esta implementação, $f(n) = \frac{n}{2} + \frac{n-2}{2} + \frac{n-2}{2} = \frac{3n}{2} - 2$, para $n > 0$, para o melhor caso, pior caso e caso médio.

E7.Como medir o custo de execução de um algoritmo? — p. 80

Como medir o custo de execução de um algoritmo?

- **Função de Custo** ou **Função de Complexidade**
  - $f(n)$ = medida de custo necessário para executar um algoritmo para um problema de tamanho n.
  - Se $f(n)$ é uma medida da quantidade de tempo necessário para executar um algoritmo para um problema de tamanho n, então f é chamada *função de complexidade de tempo de algoritmo*.
  - Se $f(n)$ é uma medida da quantidade de memória necessária para executar um algoritmo para um problema de tamanho n, então f é chamada *função de complexidade de espaço de algoritmo*.

- Observação: tempo não é tempo!
  - É importante ressaltar que a complexidade de tempo na realidade não representa tempo diretamente, mas o número de vezes que determinada operação considerada relevante é executada

E8.Exemplo de algoritmo exponencial — p. 128

Exemplo de algoritmo exponencial

- Um algoritmo simples seria verificar todas as rotas e escolher a menor delas.
- Há $(n-1)!$ rotas possíveis e a distância total percorrida em cada rota envolve n adições, logo o número total de adições é $n!$.
- No exemplo anterior teríamos 24 adições.
- Suponha agora 50 cidades: o número de adições seria $50! \approx 10^{64}$.
- Em um computador que executa 10^9 adições por segundo, o tempo total para resolver o problema com 50 cidades seria maior do que 10^{45} séculos só para executar as adições.
- O problema do caixeiro viajante aparece com freqüência em problemas relacionados com transporte, mas também aplicações importantes relacionadas com otimização de caminho percorrido.

Razões para limitar a recursão

- Memória necessária para acomodar variáveis a cada chamada
- O estado corrente da computação tem que ser armazenado para permitir a volta da chamada recursiva.

Exemplo:

```
function F(i : integer) : integer;
begin
  if i > 0
  then F := i * F(i-1)
  else F := 1;
end;
```

F(4) →	1	4∗F(3)
	2	3∗F(2)
	3	2∗F(1)
	4	1∗F(0)
	1	

Quando não usar recursividade

- Algoritmos recursivos são apropriados quando o problema é definido em termos recursivos
- Entretanto, uma definição recursiva não implica necessariamente que a implementação recursiva é a melhor solução!
- Casos onde evitar recursividade:
  - P ≡ if condição then $(S_i; P)$
    Exemplo: P ≡ **if** $i < n$ **then** $(\overbrace{i := i + 1; F := i * F}; P)$

Exemplo de resolução de equação de recorrência

Substitui-se os termos $T(k)$, $k < n$, até que todos os termos $T(k)$, $k > 1$, tenham sido substituídos por fórmulas contendo apenas $T(1)$.

$$\begin{aligned} T(n) &= n + T(n/3) \\ T(n/3) &= n/3 + T(n/3/3) \\ T(n/3/3) &= n/3/3 + T(n/3/3/3) \\ &\vdots \\ T(n/3/3\cdots/3) &= n/3/3\cdots/3 + T(n/3\cdots/3) \end{aligned}$$

Adicionando lado a lado, temos

$$T(n) = n + n\cdot(1/3) + n\cdot(1/3^2) + n\cdot(1/3^3) + \cdots + (n/3/3\cdots/3)$$

que representa a soma de uma série geométrica de razão $1/3$, multiplicada por n, e adicionada de $T(n/3/3\cdots/3)$, que é menor ou igual a 1.

Exemplo de resolução de equação de recorrência

$$T(n) = n + n\cdot(1/3) + n\cdot(1/3^2) + n\cdot(1/3^3) + \cdots + + (n/3/3\cdots/3)$$

Se desprezarmos o termo $T(n/3/3\cdots/3)$, quando n tende para infinito, então

$$T(n) = n\sum_{i=0}^{\infty}(1/3)^i = n\left(\frac{1}{1-\frac{1}{3}}\right) = \frac{3n}{2}.$$

Se considerarmos o termo $T(n/3/3/3\cdots/3)$ e denominarmos x o número de subdivisões por 3 do tamanho do problema, então $n/3^x = 1$, e $n = 3^x$. Logo $x = \log_3 n$.

Exemplo de resolução de equação de recorrência

Lembrando que $T(1) = 1$ temos

$$\begin{aligned} T(n) &= \sum_{i=0}^{x-1} \frac{n}{3^i} + T(\frac{n}{3^x}) \\ &= n \sum_{i=0}^{x-1} (1/3)^i + 1 \\ &= \frac{n(1 - (\frac{1}{3})^x)}{(1 - \frac{1}{3})} + 1 \\ &= \frac{3n}{2} - \frac{1}{2}. \end{aligned}$$

Logo, o programa do exemplo é $O(n)$.

E12.Notação assintótica em funções — p. 172

Notação assintótica em funções

Normalmente, a notação assintótica é usada em fórmulas matemáticas. Por exemplo, usando a notação O pode-se escrever que $n = O(n^2)$. Também pode-se escrever que

$$2n^2 + 3n + 1 = 2n^2 + \Theta(n)$$

Como se interpreta uma fórmula como esta?

Modelagem usando equação de recorrência
Torre de Hanoi

- Objetivo:
  - Transferir toda a torre para uma das outras varetas, movendo um disco de cada vez, mas nunca movendo um disco maior sobre um menor
- Soluções particulares:
  - Seja $T(n)$ o número mínimo de movimentos para transferir n discos de uma vareta para outra de acordo com as regras definidas no enunciado do problema.
  - Não é difícil observar que:

$$
\begin{aligned}
T(0) &= 0 \quad \text{[nenhum movimento é necessário]} \\
T(1) &= 1 \quad \text{[apenas um movimento]} \\
T(2) &= 3 \quad \text{[três movimentos usando as duas varetas]}
\end{aligned}
$$

Exemplo: Incrementar um contador binário

- Exemplo didático (*toy example*).
- Estrutura de dados:
  - Arranjo de k bits, usado para armazenar/contar números de 0 a $2^k - 1$ e depois novamente a 0.
- Operação:
  - Incrementar.
- Custo:
  - Número de bits trocados.
  - Exemplo: arranjo de 5 bits.

Exemplo: Incrementar um contador binário

Bits	Custo
00000	
00001	1
00010	2
00011	1
00100	3

➜ Custo varia!

➜ Custo máximo: $O(\log n)$.

E15.Exemplo 1 — p. 21

Notação O

Exemplo 1

Provar que $n^2 \in O(2^n)$.

prova: Suponha $n \geq 4$.

Se $n = 4$, então $4^2 = 16 \leq 2^4$

Hipótese de indução: Seja k um inteiro tal que $k \geq 4$. Suponha que $k^2 \leq 2^k$.

passo: $(k+1)^2 = k^2 + 2k + 1 \leq 2^k + 2k + 1$, pela hipótese de indução.

$(k+1)^2 \leq 2^k + 2k + 1 \leq 2^k + k^2$, pois $k \geq 4$.

Então $(k+1)^2 \leq 2^k + k^2 \leq 2^k + 2^k = 2(2^k) = 2^{k+1}$. □

E16.Exemplo 1 — p. 51

Exemplos de prova

Exemplo 1

Prove que $n^2 + 2n + 1 \in O(n^2)$.

prova: Observe que quando $n \geq 1$, tem-se $2n \leq 2n^2$.

Então $n^2 + 2n + 1 \leq n^2 + 2n^2 + n^2 = 4n^2$.

Então existe uma constante $c = 4$ tal que $n^2 + 2n + 1 \leq c \cdot n^2 = 4n^2$, sempre que $n \geq 1$. □

E17.Exemplo 2 — p. 52

Exemplos de prova

Exemplo 2

Prove que $7n^2 \in O(n^3)$.

prova: Observe que quando $n \geq 7$, tem-se $7n^2 \leq n^3$.

Então existe uma constante $c = 1$ tal que $7n^2 \leq c \cdot n^3 = n^3$, sempre que $n \geq 7$. □

E18.Exemplo 3 — p. 53

Exemplos de prova

Exemplo 3

Prove que n^2 não pertence à $O(n)$.

prova: Suponha, por contradição que $n^2 \in O(n)$.

Então existe uma constante positiva c tal que $n^2 \leq c \cdot n$, para todo $n \geq k \geq 0$.

Quando $n = 0$, a desigualdade é satisfeita, porém, para qualquer valor de $n > 0$, tem-se: $\frac{n^2}{n} \leq c$.

Então, quando $n > 0$, existe uma constante c tal que $n < c$, para todo $n \geq k$, um absurdo, já que c é constante e n não é. □

Exemplos de prova

Exemplo 4

É verdade que $x^3 \in O(7x^2)$?

prova: Se $x^3 \in O(7x^2)$, então existem constantes c e k tal que $x^3 \leq c \cdot 7x^2$ para todo valor de $x \geq k$.

Suponha $x > 0$. Então, $\frac{x^3}{x^2} \leq 7c$, para todo $x \geq k$.

Então, $x \leq 7c$, para todo $x \geq k$. Um absurdo, já que $7c$ é constante e x não é. $\square$

Exemplos de prova

Exemplo 5

Prove que $n! \in O(n^n)$.

prova: $n! = 1 \cdot 2 \cdot 3 \ldots n \leq n^n$, para todo $n \geq 1$.

Então existe uma constante positiva $c = 1$ tal que $n! \leq c \cdot n^n$, para todo $n \geq 1$. □

Exemplos de prova

Observe que $\log n! \leq \log n^n = n \log n$.

Então $\log n! \in O(n \log n)$.

E21.Exemplo 6 — p. 57

Exemplos de prova

Exemplo 6

Mostre que $n \in O(2^n)$.

prova: Basta provar que $n \leq 2^n$ para todo $n \geq 0$.
A prova é por indução. Quando $n = 0$, $0 \leq 1$, satisfazendo a inequação.

Suponha, por hipótese de indução que existe um inteiro k tal que $k \leq 2^k$.

Então, pela hipótese, $k + 1 \leq 2^k + 1 \leq 2^k + 2^k$, para todo $k \geq 0$.

Logo, $k + 1 \leq 2^k + 2^k = 2 \cdot 2^k = 2^{k+1}$.

Então existe uma constante $c = 1$ tal que $n \leq 2^n$ para todo $n \geq 0$. □

E22.Exemplo 7 — p. 58

Exemplos de prova

Exemplo 7

Mostre que $\log n \in O(n)$.

prova: Pela prova anterior, $n \leq 2^n$, para todo $n \geq 0$.

Fazendo o log de ambos os lados, a desigualdade é preservada:

$\log n \leq \log 2^n$

Então, $\log n \leq n \log 2$, para $n \geq 1$.

Não importa qual a base do logaritmo, $\log 2$ é uma constante positiva.

Portanto, existe uma constante positiva $c = \log 2$ tal que $\log n \leq c \cdot n$, para todo $n \geq 1$. □

Exemplos de prova

Exemplo 8

Mostre que $3n\log(n!) + (n^2+3)\log n \in O(n^2 \log n)$.

prova: Como vimos, $\log(n!) \in O(n \log n)$.

Então, existe uma constante positiva c_1 tal que $\log(n!) \leq c_1 \cdot n \log n$, para todo $n \geq n_1$.

Note que $3n \in O(n)$, pois $3n \leq c_2 \cdot n$, para todo $n \geq 0$, quando $c_2 = 3$.

Então, $3n\log(n!) \leq (c_1 n \log n)(3n) = 3c_1 n^2 \log n$.

Então, existe uma constante positiva $c = 3c_1$ tal que $3n\log(n!) \leq cn^2 \log n$.

Portanto, $3n\log(n!) \in O(n^2 \log n)$.

Exemplos de prova

Observe que $n^2 + 3 \leq n^2 + 3n^2 = 4n^2$, para todo $n \geq 0$. Então $n^2 + 3 \in O(n^2)$.

Por fim, $\log n \in O(\log n)$, para todo $n \geq 1$, pois $\log n = c_3 \log n$ quando $c_3 = 1$.

Lembre-se que se $f(n) \in O(p(n))$ e $g(n) \in O(q(n))$, então $f(n) \cdot g(n) \in O(p(n) \cdot q(n))$.

Portanto, $(n^2+3)\log n \in O(n^2 \log n)$.

Exemplos de prova

Lembre-se que se $f(n) \in O(p(n))$ e $g(n) \in O(q(n))$, então $f(n) + g(n) \in O(p(n) + q(n))$.

Portanto, $3n\log(n!) + (n^2 + 3)\log n \in O(n^2 \log n + n^2 \log n)$.

Então, $3n\log(n!) + (n^2 + 3)\log n \leq c \cdot 2(n^2 \log n)$.

Como $2c$ é uma constante, $3n\log(n!) + (n^2 + 3)\log n \in O(n^2 \log n)$.□

E24.Exemplo 9 — p. 62

Exemplos de prova

Exemplo 9

Mostre que $8n^3 + 5n^2 + 7 \in \Omega(n^3)$.

prova: Observe que $8n^3 + 5n^2 + 7 \geq 8n^3$, para todo $n \geq 0$.

Então, existe uma constante positiva, $c = 8$ tal que $8n^3 + 5n^2 + 7 \geq c \cdot n^3$, para todo $n \geq 0$.

Portanto, pela definição da notação Ω, $8n^3 + 5n^2 + 7 \in \Omega(n^3)$. □

Exemplos de prova

Exemplo 10

Mostre que $8n^3 + 5n^2 + 7 \in \Theta(n^3)$.

Para provar que $f(n) \in \Theta(g(n))$, precisamos provar que $f(n) \in \Omega(g(n))$ e $f(n) \in O(g(n))$.

Na prova anterior, mostramos que $8n^3 + 5n^2 + 7 \in \Omega(n^3)$.

Então, falta mostrar que $8n^3 + 5n^2 + 7 \in O(n^3)$.

Exemplos de prova

Então, falta mostrar que $8n^3 + 5n^2 + 7 \in O(n^3)$.

Observe que $5n^2 \leq n^3$, para todo $n \geq 5$.

Observe que $7 \leq n^3$, para todo $n \geq 7$.

Então, $8n^3 + 5n^2 + 7 \leq 8n^3 + n^3 + n^3$, para todo $n \geq 7$.

Portanto, $8n^3 + 5n^2 + 7 \in O(n^3)$.

Como $8n^3 + 5n^2 + 7 \in \Omega(n^3)$ e $8n^3 + 5n^2 + 7 \in O(n^3)$, pode-se concluir que $8n^3 + 5n^2 + 7 \in \Theta(n^3)$. $\square$

E26.Exemplo 11 — p. 65

Exemplos de prova

Exemplo 11

Mostre que $n^2 \in o(n^3)$.

prova: Pela definição da notação o, é preciso mostrar que

$$\lim_{n\to\infty} \frac{n^2}{n^3} = 0.$$

$$\lim_{n\to\infty} \frac{n^2}{n^3} = \lim_{n\to\infty} \frac{1}{n} = 0$$

E27.Exemplo 12 — p. 66

Exemplos de prova

Exemplo 12

Mostre que $n^2 \in \omega(n \log n)$.

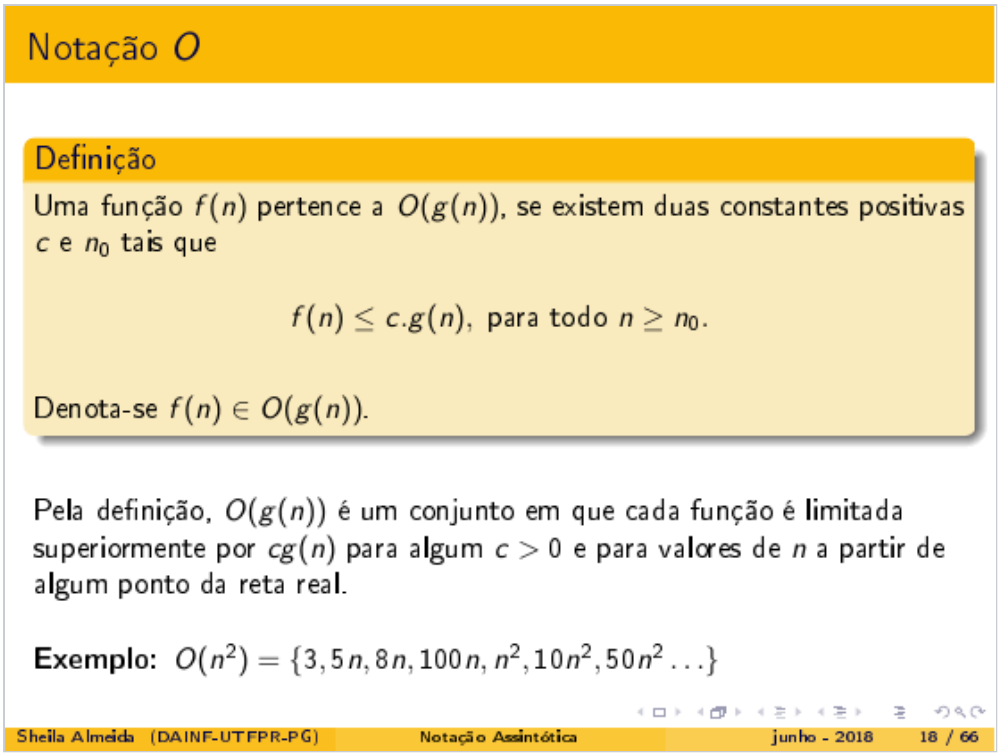
prova: Pela definição da notação ω, é preciso mostrar que

$$\lim_{n\to\infty} \frac{n \log n}{n^2} = 0.$$

$\lim_{n\to\infty} \frac{n \log n}{n^2} = \lim_{n\to\infty} \frac{\log n}{n} = \frac{\frac{d}{dn} \log n}{\frac{d}{dn} n} = \frac{\frac{d}{dn} \frac{\ln n}{\ln 10}}{1} = \frac{1}{(\ln 10)x} = 0$ □

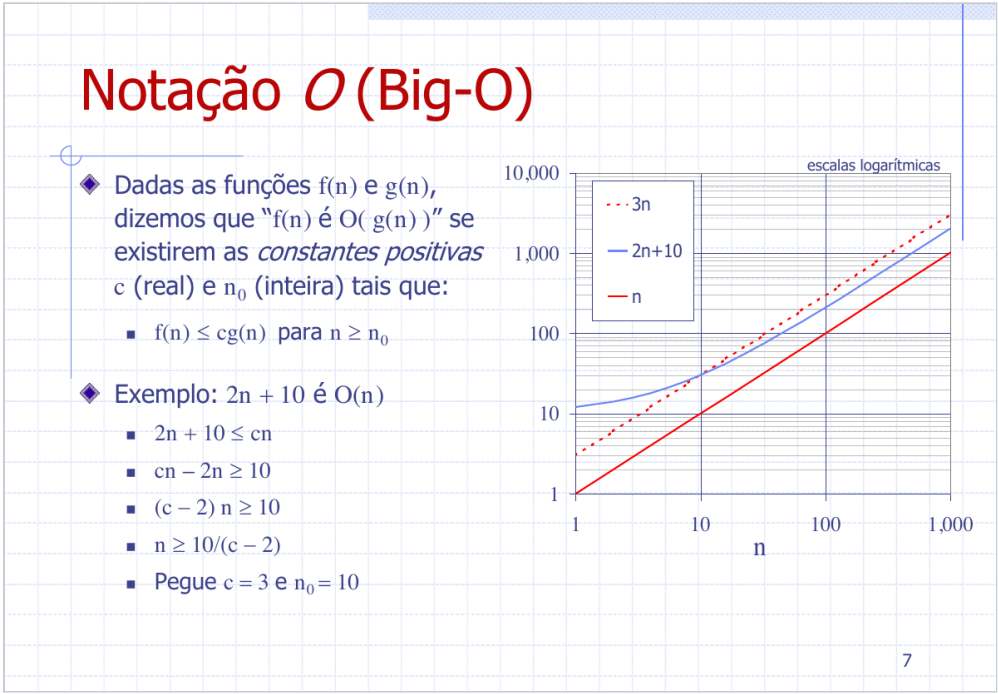
Sheila Almeida — Notação Assintótica (UTFPR) · exemplo extra

E28.Notação O — p. 18



Campello — Análise de Algoritmos II · exemplos

E29.Notação O (Big-O) — p. 7



E30.Exercício — p. 11

Exercício

Mostre que $f(n) = \log_y n$ é $O(\log_x n)$ para $x, y > 1$, ou seja, a base logarítmica não afeta a complexidade assintótica da função / algoritmo

11

E31.Análise Assintótica de Algoritmos — p. 15

Análise Assintótica de Algoritmos

- A análise assintótica de um algoritmo determina o seu tempo de execução em notação Big-O
- Para fazer a análise assintótica:
  - Encontra-se o no. de operações primitivas executado pelo algoritmo no pior caso (em função do tamanho da entrada)
  - Escreve-se essa função com notação Big-O
- Exemplo:
  - Nós determinamos anteriormente que o algoritmo `arrayMax` executa no máximo $8n - 2$ operações primitivas
  - Então, dizemos que o algoritmo `arrayMax` "executa em tempo $O(n)$" ou "possui **complexidade assintótica linear**"

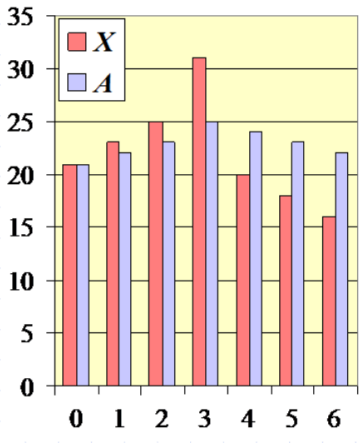
15

Exemplo (Média de Prefixo)

- Vamos ilustrar a análise assintótica com dois algoritmos que fazem o cálculo da **média de prefixos**
- A i-**ésima média de prefixo** de um arranjo X é a média dos (i + 1) primeiros elementos de X:

$$A[i] = (X[0] + X[1] + \ldots + X[i]) / (i+1)$$

- O cálculo do vetor A de médias de prefixo de um outro vetor X possui aplicações em análise financeira



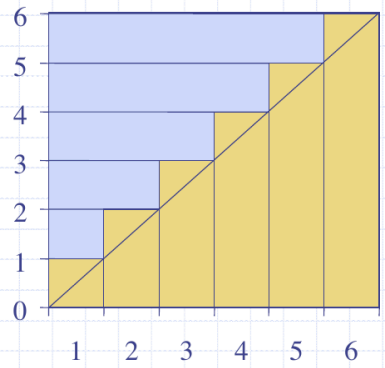
Exemplo (Média de Prefixo)

- O algoritmo abaixo calcula médias de prefixos em **tempo quadrático**, aplicando a definição:

```
Algoritmo Prefix1(X, n)
    Entrada: vetor X de n inteiros
    Saída: vetor A de médias de prefixos de X      #operações
    A ← novo vetor de n inteiros
    para i ← 0 até n − 1 faça                        ~ n
        s ← 0                                        ~ n
        para j ← 0 até i faça                        ~ 1 + 2 + …+ n
            s ← s + X[j]                             ~ 1 + 2 + …+ n
        A[i] ← s / (i + 1)                           ~ n
    retorne A                                        1
```

Exemplo (Média de Prefixo)

- O tempo de execução de Prefix1 é $O(1 + 2 + \ldots + n)$
- A soma dos primeiros n inteiros é $n(n + 1) / 2$ (prog. aritmética):
  - Pode-se provar graficamente
  - Área em laranja ao lado é dada por $n^2/2 + n \cdot ½$
- Portanto, Prefix1 executa em tempo $O(n^2)$



Exemplo (Média de Prefixo)

- O algoritmo abaixo calcula a média de prefixos em **tempo linear**, varrendo e somando uma só vez:

```
Algoritmo Prefix2(X, n)
    Entrada: vetor X de n inteiros
    Saída: vetor A de médias de prefixos de X        #operações
    A ← novo vetor de n inteiros
    s ← 0                                              1
    para i ← 0 até n – 1 faça                          ~ n
        s ← s + X[i]                                   ~ n
        A[i] ← s / (i + 1)                             ~ n
    retorne A                                          1
```

E33.Exercício — p. 22

Exercício

Faça uma análise **detalhada** do no. de operações primitivas computadas por cada um dos algoritmos anteriores para cálculo da média de prefixo e mostre o número exato de operações executadas por cada um deles. Em seguida, prove usando a **definição** que Prefix1 é $O(n^2)$ e Prefix2 é $O(n)$. Discuta se existem diferenças entre pior caso, melhor caso e caso médio desses algoritmos. Justifique

E34.Parentes do Big-O — p. 25

Parentes do Big-O



- **Exemplo**:
  - Seja uma função definida como $f(n) = 10n^2 + 5n$ para valores pares de n e como $f(n) = 3n + 3$ para valores ímpares de n
  - É possível mostrar que $f(n)$ é $\Omega(n)$ e $O(n^2)$
- **Exemplo**:
  - Seja um algoritmo que execute $T(n) = 8n + 3$ operações primárias no **pior caso** (correspondente a uma dada composição da sua entrada, de tamanho n) e $T(n) = 15$ operações no **melhor caso**
  - A complexidade assintótica desse algoritmo é $\Omega(1)$ e $O(n)$
    - **constante** no *melhor caso* e **linear** no *pior caso*

Forma Assintótica usando Limites

Extraído do material auxiliar da Profa. M. Carolina Monard

Forma Assintótica	Relação	Definição
$f(n) \in \Theta(g(n))$	$f(n) \equiv g(n)$	$0 < \lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty$
$f(n) \in O(g(n))$	$f(n) \preceq g(n)$	$0 \leq \lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty$
$f(n) \in \Omega(g(n))$	$f(n) \succeq g(n)$	$0 < \lim_{n\to\infty} \frac{f(n)}{g(n)}$
$f(n) \in o(g(n))$	$f(n) \prec g(n)$	$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$
$f(n) \in \omega(g(n))$	$f(n) \succ g(n)$	$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$

Exemplo 1: f(n) = $2n^2+n$ e g(n) = n^2

- $\lim_{n\to\infty}$ ($2n^2+n$ / n^2) = $\lim_{n\to\infty}$ (2 + 1/n) = 2

Exemplo 2: f(n) = n^2 e g(n) = $2n^2+n$

- $\lim_{n\to\infty}$ (n^2 / $2n^2+n$) = [L'Hôpital 2x] $\lim_{n\to\infty}$ (2 / 4) = 1/2

E36.Exemplo 1 — p. 17–21

Princípio da indução matemática (fraca)
Exemplo 1

Prove que

$$P(n) : 1 + 2 + \ldots + n = \frac{n(n+1)}{2}$$

para todos inteiros $n \geq 1$.

Prova (por indução matemática):

1. Passo base: $P(n_0) = P(1)$: Para $n_0 = 1$, $1 = \frac{1(1+1)}{2} = 1$ e a fórmula é verdadeira para $n_0 = 1$.
2. Passo indutivo: se a fórmula é verdadeira para $n = k$ então deve ser verdadeira para $n = k + 1$, i.e., $P(k) \rightarrow P(k+1)$.
   - Suponha que a fórmula é verdadeira para $n = k$, i.e.,

     $$P(k) : 1 + 2 + \ldots + k = \frac{k(k+1)}{2}$$

     para algum inteiro $k \geq 1$. [hipótese indutiva]

Princípio da indução matemática (fraca)
Exemplo 1

Deve-se mostrar que

$$P(k+1) : 1 + 2 + \ldots + (k+1) = \frac{(k+1)(k+2)}{2}$$

Sabe-se que

$$\begin{aligned} 1 + 2 + \ldots + k + (k+1) &= \frac{k(k+1)}{2} + (k+1) \\ &= \frac{k(k+1)}{2} + \frac{2(k+1)}{2} \\ &= \frac{k^2 + 3k + 2}{2} \\ &= \frac{(k+1)(k+2)}{2} \end{aligned}$$

[Isto era o que devia ser provado.]

Princípio da indução matemática (fraca)
Exemplo 1: Comentários

Observe que na prova por indução matemática devemos usar obrigatoriamente o predicado $P(k)$. Esse é um dos grandes desafios neste tipo de prova, como veremos em outros exemplos.

A soma

$$1 + 2 + \ldots + k + (k + 1),$$

que aparece no predicado $P(k + 1)$, possui os termos 1 a k, cuja soma $(1 + 2 + \ldots + k)$ vale $\frac{k(k+1)}{2}$ pela hipótese indutiva. Como estamos supondo que ela é verdadeira, podemos definir uma igualdade onde esses termos do lado esquerdo são substituídos por essa fração do lado direito:

$$1 + 2 + \ldots + k + (k + 1) = \frac{k(k + 1)}{2} + (k + 1).$$

Princípio da indução matemática (fraca)
Exemplo 1: Comentários

Nessa demonstração pode parecer que estamos usando o fato de $P(k)$ ser V para deduzir que $P(k + 1)$ é V, para em seguida deduzir que $P(k)$ é V. Parece circular! O que está ocorrendo?

Nao é isso que está acontecendo. Dado um k e o predicado associado, temos duas possibilidades:
(a) $P(k)$ é V
(b) $P(k)$ é F

A hipótese indutiva **não afirma** que $P(k)$ seja verdadeiro. O que afirma é que **caso** $P(k)$ seja V então $P(k+1)$ também será V. Isto é, se k faz com que $P(k)$ seja verdadeiro e, assim, esteja na categoria (a) acima, então $k + 1$ também fará com que $P(k + 1)$ seja V e, assim, também esteja em (a).

Princípio da indução matemática (fraca)
Exemplo 1: Comentários

Por exemplo, seja

$$P(n) : 1 + 2 + \ldots + n = \frac{n(n+1)}{2} + 1.$$

Isto nao é correto! Neste exemplo, o predicado $P(k)$ é falso para **todo** k.

Em geral, devemos tentar mostrar que **caso** $P(k)$ seja V, então $P(k+1)$ tambem é V.

Isso ficará evidente no próximo exemplo, quando vamos supor que $P(k)$ seja V e vamos chegar a uma contradição para $P(k+1)$. Ou seja, $P(k)$ é F.

Princípio da indução matemática (fraca)
Exemplo 2

Prove que

$$P(n) : 0 + 1 + 2 + \ldots + n = \frac{n(n+2)}{2}$$ ERRADO!

para todos inteiros $n \geq 0$.

Prova (por indução matemática):

1. Passo base: $P(n_0) = P(0)$: Para $n_0 = 0$, $0 = \frac{0(0+2)}{2} = 0$ e a fórmula é verdadeira para $n_0 = 0$.
2. Passo indutivo: se a fórmula é verdadeira para $n = k$ então deve ser verdadeira para $n = k + 1$, i.e., $P(k) \rightarrow P(k+1)$.
   - Suponha que a fórmula é verdadeira para $n = k$, i.e.,

     $$P(k) : 0 + 1 + 2 + \ldots + k = \frac{k(k+2)}{2} = \frac{k^2 + 2k}{2}$$

     para algum inteiro $k \geq 0$. [hipótese indutiva]

Princípio da indução matemática (fraca)
Exemplo 2

Deve-se mostrar que

$$P(k+1) : 0 + 1 + 2 + \ldots + (k+1) = \frac{(k+1)(k+3)}{2} = \frac{k^2 + 4k + 3}{2}$$

Sabe-se que

$$\begin{aligned} 0 + 1 + 2 + \ldots + k + (k+1) &= \frac{k^2 + 2k}{2} + (k+1) \\ &= \frac{k^2 + 2k + 2(k+1)}{2} \\ &= \frac{k^2 + 4k + 2}{2} \end{aligned}$$

[Assim, não foi possível derivar a conclusão a partir da hipótese. Isto significa que o predicado original é falso.]

Princípio da indução matemática (fraca)
Exemplo 3

Prove que

$$P(n) : \sum_{i=0}^{n} r^i = \frac{r^{n+1} - 1}{r - 1}$$

para todos inteiros $n \geq 0$ e para todos números reais r, $r \neq 1$.

Prova (por indução matemática):

1. Passo base: $P(n_0) = P(0)$: Para $n_0 = 0$, $r^0 = 1 = \frac{r^{0+1}-1}{r-1} = \frac{r-1}{r-1} = 1$ e a fórmula é verdadeira para $n_0 = 0$.
2. Passo indutivo: se a fórmula é verdadeira para $n = k$ então deve ser verdadeira para $n = k + 1$, i.e., $P(k) \rightarrow P(k + 1)$.

Princípio da indução matemática (fraca)
Exemplo 3

- $P(k) : \sum_{i=0}^{k} r^i = \frac{r^{k+1}-1}{r-1}$, para $k \geq 0$. [hipótese indutiva]
- Deve-se mostrar que $P(k + 1) : \sum_{i=0}^{k+1} r^i = \frac{r^{k+2}-1}{r-1}$

$$\begin{aligned}
\sum_{i=0}^{k+1} r^i &= \sum_{i=0}^{k} r^i + r^{k+1} \\
&= \frac{r^{k+1} - 1}{r - 1} + r^{k+1} \\
&= \frac{r^{k+1} - 1}{r - 1} + \frac{r^{k+1}(r - 1)}{r - 1} \\
&= \frac{r^{k+1} - 1 + r^{k+2} - r^{k+1}}{r - 1} \\
&= \frac{r^{k+2} - 1}{r - 1}
\end{aligned}$$

Princípio da indução matemática (fraca)
Exemplo 4

Prove que

$$P(n) : 2^{2n} - 1 \quad \text{é divisível por 3,}$$

para $n \geq 1$.

Prova (por indução matemática):

1. Passo base: $P(n_0) = P(1)$: Para $n_0 = 1$, $2^{2 \cdot 1} - 1 = 3$ que é divisível por 3.
2. Passo indutivo: se a fórmula é verdadeira para $n = k$ então deve ser verdadeira para $n = k + 1$, i.e., $P(k) \rightarrow P(k + 1)$.

Princípio da indução matemática (fraca)
Exemplo 4

– $P(k) : 2^{2k} - 1$ é divisível por 3. [hipótese indutiva]
– Deve-se mostrar que $P(k + 1) : 2^{2(k+1)} - 1$ é divisível por 3.

$$\begin{aligned} 2^{2(k+1)} - 1 &= 2^{2k+2} - 1 \\ &= 2^{2k} \cdot 2^2 - 1 \\ &= 2^{2k} \cdot 4 - 1 \\ &= 2^{2k} \cdot (3 + 1) - 1 \\ &= 2^{2k} \cdot 3 + (2^{2k} - 1) \end{aligned}$$

que é divisível por 3.

Princípio da indução matemática (fraca)
Exemplo 5

Prove que

$$P(n) : 2^0 + 2^1 + 2^2 + \ldots + 2^n = 2^{n+1} - 1,$$

para $n \geq 0$.

Prova (por indução matemática):

1. Passo base: $P(n_0) = P(0)$: Para $n_0 = 2^0 = 1$, $2^1 - 1 = 1$.
2. Passo indutivo: se a fórmula é verdadeira para $n = k$ então deve ser verdadeira para $n = k + 1$, i.e., $P(k) \rightarrow P(k + 1)$.

Princípio da indução matemática (fraca)
Exemplo 5

- $P(k) : 2^0 + 2^1 + 2^2 + \ldots + 2^k = 2^{k+1} - 1$, para $k \geq 0$. [hipótese indutiva]
- Deve-se mostrar que $P(k + 1) : 2^0 + 2^1 + 2^2 + \ldots + 2^{k+1} = 2^{k+2} - 1$

$$\begin{aligned} 2^0 + 2^1 + 2^2 + \ldots + 2^k + 2^{k+1} &= (2^{k+1} - 1) + 2^{k+1} \\ &= 2 \cdot 2^{k+1} - 1 \\ &= 2^{k+2} - 1 \end{aligned}$$

Princípio da indução matemática (fraca)
Exemplo 6

Prove que

$$P(n) : H_{2^n} \geq 1 + \frac{n}{2},$$

para $n \geq 0$.

H_j representa o número harmônico e é definido por:

$$H_j = 1 + \frac{1}{2} + \frac{1}{3} + \ldots + \frac{1}{j}.$$

Prova (por indução matemática):

1. Passo base: $P(n_0) = P(0)$:
   Para $n_0 = 0$, temos $H_{2^0} = H_1 = 1 \geq 1 + \frac{0}{2}$.
2. Passo indutivo: se a fórmula é verdadeira para $n = k$ então deve ser verdadeira para $n = k + 1$, i.e., $P(k) \rightarrow P(k + 1)$.

Princípio da indução matemática (fraca)
Exemplo 6

– $P(k) : H_{2^k} \geq 1 + \frac{k}{2}$, para $k \geq 0$. [hipótese indutiva]
– Deve-se mostrar que $P(k + 1) : H_{2^{k+1}} \geq 1 + \frac{k+1}{2}$

$$\begin{aligned}
H_{2^{k+1}} &= 1 + \frac{1}{2} + \frac{1}{3} + \ldots + \frac{1}{2^k} + \frac{1}{2^k + 1} + \frac{1}{2^k + 2} + \ldots + \frac{1}{2^{k+1}} \\
&\quad \text{[Definição de número harmônico.]} \\
&= H_{2^k} + \frac{1}{2^k + 1} + \frac{1}{2^k + 2} + \ldots + \frac{1}{2^{k+1}} \\
&\quad \text{[Definição de número harmônico.]} \\
&\geq \left(1 + \frac{k}{2}\right) + 2^k \cdot \frac{1}{2^{k+1}} \\
&\quad \text{[Hipótese indutiva e existem } 2^k \text{ termos, cada um pelo menos } 1/2^{k+1}.\text{]} \\
&\geq \left(1 + \frac{k}{2}\right) + \frac{1}{2} \\
&\geq 1 + \frac{k+1}{2}.
\end{aligned}$$

Princípio da indução matemática (fraca)
Exemplo 7

Seja a sequência $a_1, a_2, a_3, \ldots$ definida como

$$\begin{aligned} a_1 &= 2 \\ a_k &= 5a_{k-1}, k \geq 2 \end{aligned}$$

Prove que

$$P(n) : a_n = 2 \cdot 5^{n-1}$$

para $n \geq 1$.

Prova (por indução matemática):

1. Passo base: $P(n_0) = P(1)$: Para $n_0 = 1$, $2 \cdot 5^{1-1} = 2$ e $a_1 = 2$. Logo, a fórmula é válida para $n = 1$.
2. Passo indutivo: se a fórmula é verdadeira para $n = k$ então deve ser verdadeira para $n = k + 1$, i.e., $P(k) \rightarrow P(k + 1)$.

Princípio da indução matemática (fraca)
Exemplo 7

– $P(k) : a_k = 2 \cdot 5^{k-1}$. [hipótese indutiva]
– Deve-se mostrar que $P(k + 1) : a_{k+1} = 2 \cdot 5^{(k+1)-1} = 2 \cdot 5^k$.

$$\begin{aligned} a_{k+1} &= 5 \cdot a_{(k+1)-1} \\ &= 5 \cdot a_k \\ &= 5 \cdot (2 \cdot 5^{k-1}) \\ &= 2 \cdot (5 \cdot 5^{k-1}) \\ &= 2 \cdot 5^k \end{aligned}$$

Princípio da indução matemática (fraca)
Exemplo 8

Prove que

$$P(n)\colon\ 2n+1 < 2^n$$

para todos os inteiros $n \geq 3$.

Prova (por indução matemática):

1. Passo base: $P(n_0) = P(3)$. Para $n_0 = 3$,

   $$2 \cdot 3 + 1 < 2^3.$$

   Logo, a fórmula é válida para $n_0 = 3$.
2. Passo indutivo: se a fórmula é verdadeira para $n = k$ então deve ser verdadeira para $n = k+1$, i.e., $P(k) \rightarrow P(k+1)$.

Princípio da indução matemática (fraca)
Exemplo 8

– $P(k)\colon\ 2k+1 < 2^k$, para $k \geq 3$. [hipótese indutiva]
– Deve-se mostrar que $P(k+1)\colon\ 2(k+1)+1 < 2^{k+1}$.

$$\begin{array}{rcl} 2k+2+1 & = & \\ (2k+1)+2 & = & \\ (2k+1)+2 & < & 2^k+2 \\ 2(k+1)+1 & < & 2^k+2 \overset{?}{<} 2^{k+1} \end{array}$$

Se puder ser mostrado que $2^k + 2 < 2^{k+1}$ então o predicado $P(k+1)$ é verdadeiro.

$$\begin{array}{rcl} 2^k+2 & \overset{?}{<} & 2^{k+1} \\ 2 & \overset{?}{<} & 2^{k+1}-2^k \\ 2 & \overset{?}{<} & 2^k(2-1) \\ 2 & \overset{?}{<} & 2^k \\ 1 & < & 2^{k-1}, \text{ que é verdade para } k \geq 2. \end{array}$$

Em particular, a inequação $(1 < 2^{k-1})$ é válida para $k \geq 3$. Assim, $P(k+1)$ é V.

Princípio da indução matemática (fraca)
Exemplo 9

- Prove que para todos os inteiros $n \geq 1$

$$P(n)\text{: } n^3 - n \text{ é divisível por 3.}$$

Prova (por indução matemática):

1. Passo base: $P(n_0) = P(1)$. Para $n_0 = 1$,

$$1^3 - 1 = 0 \text{ é divisível por 3.}$$

   Logo, a fórmula é válida para $n_0 = 3$.
2. Passo indutivo: se a fórmula é verdadeira para $n = k$ então deve ser verdadeira para $n = k + 1$, i.e., $P(k) \rightarrow P(k + 1)$.

Princípio da indução matemática (fraca)
Exemplo 9

– $P(k)$: $k^3 - k$ é divisível por 3, para $k \geq 1$. [hipótese indutiva]
– Deve-se mostrar que $P(k + 1)$: $(k + 1)^3 - (k + 1)$ é divisível por 3, para $k \geq 1$.

$$\begin{aligned}&(k+1)^3 - (k+1) = \\ &(k^3 + 3k^2 + 3k + 1) - (k+1) = \\ &(k^3 - k) + 3(k^2 + k)\end{aligned}$$

O primeiro termo é divisível por 3 (hipótese indutiva) e o segundo também. Como a soma de dois números divisíveis por 3 é um número divisível por 3, então o predicado $P(k + 1)$ é V.

Princípio da indução matemática (fraca)
Exemplo 10

Seja um inteiro $n \geq 1$. Prove que

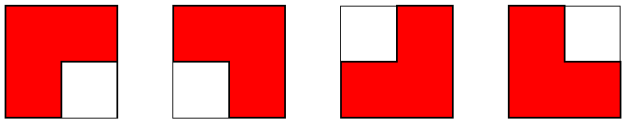
$P(n)$: qualquer região quadrada de tamanho $2^n \times 2^n$, com um quadrado removido, pode ser preenchida com peças no formato L, como mostrado abaixo.



Princípio da indução matemática (fraca)
Exemplo 10

Prova (por indução matemática):

1. Passo base: $P(n_0) = P(1)$. P(1) é V já que uma região quadrada de tamanho 2×2, com um quadrado removido, pode se preenchida com peças no formato L, como mostrado abaixo.

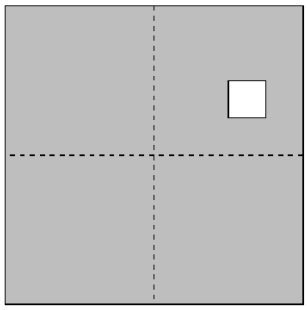


2. Passo indutivo: se a fórmula é verdadeira para $n = k$ então deve ser verdadeira para $n = k + 1$, i.e., $P(k) \rightarrow P(k + 1)$.

Princípio da indução matemática (fraca)
Exemplo 10

- $P(k)$: Qualquer região quadrada de tamanho $2^k \times 2^k$, com um quadrado removido, pode ser preenchida com peças no formato L. [hipótese indutiva]
- Deve-se mostrar $P(k+1)$: Qualquer região quadrada de tamanho $2^{k+1} \times 2^{k+1}$, com um quadrado removido, pode ser preenchida com peças no formato L.

Considere uma região quadrada de tamanho $2^{k+1} \times 2^{k+1}$, com um quadrado removido. Divida essa região em quatro regiões de tamanho $2^k \times 2^k$ como mostrado abaixo.



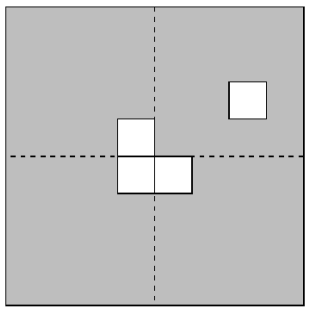
Temos três regiões $2^k \times 2^k$ com nenhum quadrado removido e uma região $2^k \times 2^k$ com um quadrado removido. Ou seja, a região $2^{k+1} \times 2^{k+1}$ possui apenas um quadrado removido.

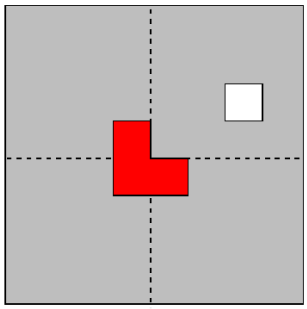
Pela hipótese indutiva, a região $2^k \times 2^k$, com um quadrado removido, pode ser preenchida com peças no formato L. O problema passa a ser como a mesma hipótese indutiva pode ser aplicada às outras três regiões.

Princípio da indução matemática (fraca)
Exemplo 10

Temporariamente remova um quadrado de cada região $2^k \times 2^k$ que está "completa" como mostrado na figura abaixo à esquerda.

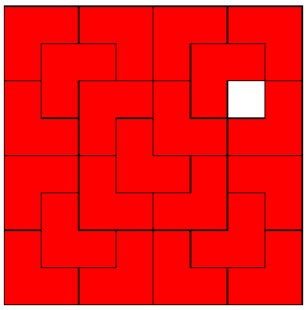
Pela hipótese indutiva cada uma dessas três regiões $2^k \times 2^k$ pode ser preenchida com peças no formato L. No entanto, para resolvermos o problema da peça removida em cada uma dessas três regiões basta colocarmos uma peça L exatamente sobre esses três "buracos" como mostrado na figura abaixo à direita.





Princípio da indução matemática (fraca)
Exemplo 10

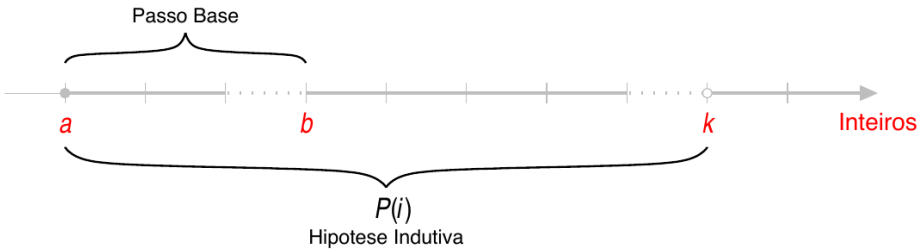
Assim, uma região quadrada de tamanho $2^{k+1} \times 2^{k+1}$, com um quadrado removido, pode ser preenchida com peças no formato L, como mostrado na figura abaixo.



Princípio da indução matemática (forte)

Seja $P(n)$ um predicado que é definido para inteiros n, e seja a e b inteiros fixos, sendo $a \leq b$. Suponha que as duas afirmações seguintes sejam verdadeiras:

1. $P(a), P(a+1), \ldots, P(b)$ são V. (Passo base)
2. Para qualquer inteiro $k \geq b$,
   se $P(i)$ é V para $a \leq i < k$ então $P(k)$ é V, i.e., $P(i) \rightarrow P(k)$.
   ➜ Logo, a afirmação "para todos inteiros $n \geq a$, $P(n)$" é V. (A suposição que $P(i)$ é V para $a \leq i < k$ é chamada de hipótese indutiva.)



Princípio da indução matemática (forte)
Exemplo 11

Prove que qualquer inteiro maior que 1 é divisível por um número primo.

Prova (por indução matemática):

1. Passo base: Para $n = 2$ a propriedade é válida já que $2|2$.
2. Passo indutivo: Vamos supor que para todos inteiros i, $2 \leq i < k$, i é divisível por um número primo. [hipótese indutiva]

   Se a propriedade é válida para $2 \leq i < k$, então é válida para k, ou seja, k é divisível por um número primo [o que deve ser mostrado].
   Seja k um inteiro, $k > 2$. Ou k é primo ou k não é primo. Se k é primo, então k é divisível por um primo (ele próprio). Se k não é primo então $k = u \cdot v$, onde u e v são inteiros tais que $2 \leq u < k$ e $2 \leq v < k$. Pela hipótese indutiva, u é divisível por um número primo p e pela transitividade da divisibilidade k também é divisível por p. Assim, independente se k é primo ou não, k é divisível por um primo.

Princípio da indução matemática (forte): Exemplo 12

Seja a sequência $a_1, a_2, a_3, \ldots$ definida como

$$\begin{aligned} a_1 &= 0 \\ a_2 &= 2 \\ a_k &= 3 \cdot a_{\lfloor k/2 \rfloor} + 2, \ k \geq 3 \end{aligned}$$

Prove que a_n é par, para $n \geq 1$.

Prova (por indução matemática):

1. Passo base: Para $n = 1$ e $n = 2$ a propriedade é válida já que $a_1 = 0$ e $a_2 = 2$.
2. Passo indutivo: Vamos supor que a_i é par para todos inteiros i, $1 \leq i < k$. [hipótese indutiva]

Princípio da indução matemática (forte): Exemplo 12

Se a propriedade é válida para $1 \leq i < k$, então é válida para k, ou seja, a_k é par [o que deve ser mostrado].

Pela definição de $a_1, a_2, a_3, \ldots$

$$a_k = 3 \cdot a_{\lfloor k/2 \rfloor} + 2, \ k \geq 3$$

O termo $a_{\lfloor k/2 \rfloor}$ é par pela hipótese indutiva já que $k \geq 3$ e $1 \leq \lfloor k/2 \rfloor < k$. Desta forma, $3 \cdot a_{\lfloor k/2 \rfloor}$ é par e $3 \cdot a_{\lfloor k/2 \rfloor} + 2$ também é par, o que mostra que a_k é par.

Sequências

- Exemplo: Número de ancestrais—Um limite superior

Geração	1	2	3	4	5	6	7...
# ancestrais	2	4	8	16	32	64	128...

- Mais definições:
  - Termo: cada elemento de uma sequência.
    Exemplo: $a_1, a_2, a_3, \ldots, a_n$
  - Índice ou subscrito: indica a posição do termo na sequência.
    Exemplo: O número 3 no termo a_3 indica o terceiro elemento da sequência.
  - Sequência finita: possui um conjunto finito de termos.
  - Sequência infinita: possui um conjunto infinito de termos.
    Exemplo: $a_1, a_2, a_3, \ldots$
  - Fórmula explícita ou fórmula geral: é a regra que mostra como os valores de a_k podem ser obtidos a partir de k.

Exemplo 1

Prove que para todos inteiros n≥1,

$$1 + 2 + \ldots + n = \frac{n(n+1)}{2}$$

Prova por indução matemática

Passo Base

P(1), para $n_0=1$, tem-se $1 = 1(1+1)/2 = 1$

A fórmula é verdadeira para $n_0=1$

Passo Indutivo

se a fórmula é verdadeira para $n = k$ então deve ser verdadeira para $n=k+1$, ou seja, $P(k) \rightarrow P(k+1)$.

Exemplo 1

Supondo que a fórmula é verdadeira para $n=k$

$$P(k) : 1 + 2 + \ldots + k = \frac{k(k+1)}{2}$$ para algum inteiro k ≥ 1.

Deve-se mostrar que

$$P(k+1) : 1 + 2 + \ldots + (k+1) = \frac{(k+1)(k+2)}{2}$$

Exemplo 1

$$
\begin{aligned}
1+2+\ldots+k+(k+1) &= \frac{k(k+1)}{2}+(k+1) \\
&= \frac{k(k+1)}{2}+\frac{2(k+1)}{2} \\
&= \frac{k^2+3k+2}{2} \\
&= \frac{(k+1)(k+2)}{2}
\end{aligned}
$$

Exemplo 2

Prove que para todos inteiros n≥0,

$$0+1+2+\ldots+n=\frac{n(n+2)}{2}$$ ERRADO

Passo Base

P(0), para $n_0=0$, tem-se $0 = 0(0+2)/2 = 0$

A fórmula é verdadeira para $n_0=0$

Passo Indutivo

se a fórmula é verdadeira para $n = k$ então deve ser verdadeira para $n=k+1$, ou seja, $P(k) \rightarrow P(k+1)$.

Exemplo 2

Supondo que a fórmula é verdadeira para $n=k$

$$P(k): 0+1+2+\ldots+k=\frac{k(k+2)}{2}=\frac{k^2+2k}{2}$$

Deve-se mostrar que

$$P(k+1): 0+1+2+\ldots+(k+1)=\frac{(k+1)(k+3)}{2}=\frac{k^2+4k+3}{2}$$

Exemplo 2

$$
\begin{aligned}
0+1+2+\ldots+k+(k+1) &= \frac{k^2+2k}{2}+(k+1) \\
&= \frac{k^2+2k+2(k+1)}{2} \\
&= \frac{k^2+4k+2}{2}
\end{aligned}
$$

Assim, não foi possível derivar a conclusão a partir da hipótese. Isto significa que o predicado original é falso.

Exemplo 3

Prove que $$P(n): \sum_{i=0}^{n} r^i = \frac{r^{n+1}-1}{r-1}$$

para todos inteiros n≥0 e para todos números reais r, r ≠ 1.

Passo Base

P(0), para $n_0=0$, tem-se $r^0=1=(r^{0+1}-1)/(r-1)=(r-1)/(r-1)=1$

A fórmula é verdadeira para $n_0=0$

Passo Indutivo

se a fórmula é verdadeira para $n = k$ então deve ser verdadeira para $n=k+1$, ou seja, $P(k) \rightarrow P(k+1)$.

Exemplo 3

Supondo que a fórmula é verdadeira para $n=k$

$$P(k): \sum_{i=0}^{k} r^i = \frac{r^{k+1}-1}{r-1} \quad \text{para } k \geq 0$$

Deve-se mostrar que

$$P(k+1): \sum_{i=0}^{k+1} r^i = \frac{r^{k+2}-1}{r-1}$$

Exemplo 3

$$
\begin{aligned}
\sum_{i=0}^{k+1} r^i &= \sum_{i=0}^{k} r^i + r^{k+1} \\
&= \frac{r^{k+1}-1}{r-1} + r^{k+1} \\
&= \frac{r^{k+1}-1}{r-1} + \frac{r^{k+1}(r-1)}{r-1} \\
&= \frac{r^{k+1}-1+r^{k+2}-r^{k+1}}{r-1} \\
&= \frac{r^{k+2}-1}{r-1}
\end{aligned}
$$

Exemplo 4

Prove que $P(n) : 2^{2n} - 1$ é divisível por 3, para n≥1.

Passo Base

P(1), para n_0=*1*, tem-se $2^{2\times1}$-1= 2^2-1= *4-1=3* que é divisível por 3

Passo Indutivo

se a fórmula é verdadeira para *n* = *k* então deve ser verdadeira para *n=k+1*, ou seja, *P(k)* $\rightarrow$ *P(k +1)*.

Exemplo 4

Supondo que a fórmula é verdadeira para *n=k*

$$P(k) : 2^{2k} - 1 \text{ é divisível por 3}$$

Deve-se mostrar que

$$P(k+1) : 2^{2(k+1)} - 1 \text{ é divisível por 3}$$

Exemplo 4

$$
\begin{aligned}
2^{2(k+1)} - 1 &= 2^{2k+2} - 1 \\
&= 2^{2k} \cdot 2^2 - 1 \\
&= 2^{2k} \cdot 4 - 1 \\
&= 2^{2k} \cdot (3+1) - 1 \\
&= \boxed{2^{2k} \cdot 3} + \boxed{(2^{2k} - 1)}
\end{aligned}
$$

que é divisível por 3.

Exemplo 5

Prove que

$$P(n) : 2^0 + 2^1 + 2^2 + \ldots + 2^n = 2^{n+1} - 1,$$

para n≥0.

Passo Base

P(0), para $n_0=0$, tem-se $2^0 = 1 = 2^{0+1}-1 = 2^1-1 = 2-1 = 1$

Passo Indutivo

se a fórmula é verdadeira para $n = k$ então deve ser verdadeira para $n=k+1$, ou seja, $P(k) \rightarrow P(k+1)$.

Exemplo 5

Supondo que a fórmula é verdadeira para $n=k$

$$P(k) : 2^0 + 2^1 + 2^2 + \ldots + 2^k = 2^{k+1} - 1$$

Deve-se mostrar que

$$P(k+1) : 2^0 + 2^1 + 2^2 + \ldots + 2^{k+1} = 2^{k+2} - 1$$

Exemplo 5

$$
\begin{aligned}
2^0 + 2^1 + 2^2 + \ldots + 2^k + 2^{k+1} &= \\
&= (2^{k+1} - 1) + 2^{k+1} \\
&= 2 \cdot 2^{k+1} - 1 \\
&= 2^{k+2} - 1
\end{aligned}
$$

Exemplo 6

Prove que $P(n) : H_{2^n} \geq 1 + \frac{n}{2},$ para n≥0.

H_j representa o número harmônico e é definido por:

$$H_j = 1 + \frac{1}{2} + \frac{1}{3} + \ldots + \frac{1}{j}.$$

Passo Base

P(0), para n_0=*0*, tem-se $H_2^0 = H_1 = 1 \geq 1+0/2 = 1$

Passo Indutivo

se a fórmula é verdadeira para *n* = *k* então deve ser verdadeira para *n=k+1*, ou seja, *P(k)* → *P(k +1)*.

Exemplo 6

Supondo que a fórmula é verdadeira para *n=k*

$$P(k) : H_{2^k} \geq 1 + \frac{k}{2}$$

Deve-se mostrar que

$$P(k+1) : H_{2^{k+1}} \geq 1 + \frac{k+1}{2}$$

Exemplo 6

$$H_{2^{k+1}} = 1 + \frac{1}{2} + \frac{1}{3} + \ldots + \frac{1}{2^k} + \frac{1}{2^k+1} + \frac{1}{2^k+2} + \ldots + \frac{1}{2^{k+1}}$$

[Definição de número harmônico.]

$$= H_{2^k} + \frac{1}{2^k+1} + \frac{1}{2^k+2} + \ldots + \frac{1}{2^{k+1}}$$

[Definição de número harmônico.]

$$\geq \left(1 + \frac{k}{2}\right) + 2^k \cdot \frac{1}{2^{k+1}}$$

[Hipótese indutiva e existem 2^k termos, cada um pelo menos $1/2^{k+1}$.]

$$\geq \left(1 + \frac{k}{2}\right) + \frac{1}{2} \qquad \geq 1 + \frac{k+1}{2}.$$

Exemplo 7

Dada a seqüência a_1, a_2, a_3, ... definida como

$$\begin{aligned} a_1 &= 2 \\ a_k &= 5a_{k-1}, k \geq 2 \end{aligned}$$

Prove que $a_n = 2 \times 5^{n-1}$ para n≥1

Passo Base

P(1), para $n_0=1$, tem-se $a_1 = 2 = 2 \times 5^{1-1} = 2 \times 1 = 2$

Passo Indutivo

se a fórmula é verdadeira para $n = k$ então deve ser verdadeira para $n=k+1$, ou seja, $P(k) \rightarrow P(k+1)$.

Exemplo 7

Supondo que a fórmula é verdadeira para $n=k$

$$P(k) : a_k = 2 \cdot 5^{k-1}$$

Deve-se mostrar que

$$P(k+1) : a_{k+1} = 2 \cdot 5^{(k+1)-1} = 2 \cdot 5^k$$

Exemplo 7

$$
\begin{aligned}
a_{k+1} &= 5 \cdot a_{(k+1)-1} \\
&= 5 \cdot a_k \\
&= 5 \cdot (2 \cdot 5^{k-1}) \\
&= 2 \cdot (5 \cdot 5^{k-1}) \\
&= 2 \cdot 5^k
\end{aligned}
$$

Exemplo 8

Prove que $P(n)$: $n^3 - n$ é divisível por 3 para n≥1.

Passo Base

P(1), para $n_0=1$, tem-se $1^3-1 = 1-1=0$ que é divisível por 3.

Passo Indutivo

se a fórmula é verdadeira para $n = k$ então deve ser verdadeira para $n=k+1$, ou seja, $P(k) \rightarrow P(k+1)$.

Exemplo 8

Supondo que a fórmula é verdadeira para $n=k$

$P(k)$: $k^3 - k$ é divisível por 3

Deve-se mostrar que

$P(k+1)$: $(k+1)^3 - (k+1)$ é divisível por 3

Exemplo 8

$$(k+1)^3 - (k+1) =$$
$$(k^3 + 3k^2 + 3k + 1) - (k+1) =$$
$$\boxed{(k^3 - k)} + \boxed{3(k^2 + k)}$$

que é divisível por 3.

Existem muitas identidades matemáticas envolvendo números naturais que podem ser verificadas com o uso do princípio de indução matemática. Abaixo, veremos alguns exemplos:

Exemplo 1 Demonstre que para qualquer $n \in \mathbb{N}$ é válida a igualdade: $2 + \cdots + 2n = n(n+1)$.

Solução:

Definimos a proposição $P(n): 2 + \cdots + 2n = n(n+1)$.. Notemos que a mesma vale para $n = 1$, pois: $P(1): 2 = 1(1+1)$. Suponhamos agora, que $P(k)$ seja verdadeira pra um certo $k > 1, k \in \mathbb{N}$.
Daí:
$2 + \cdots + 2k + 2(k+1) = k(k+1) + 2(k+1) = (k+1)(k+2)$, o que prova que $P(k+1)$ é verdade. Assim, $P(n)$ é verdade para qualquer $n \in \mathbb{N}$, pelo princípio da indução.

Observação 1 Uma consequência imediata do exemplo acima é que $1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$, o que pode facilmente ser provada pelo leitor.

Exemplo 2 Demonstre que para qualquer $n \in \mathbb{N}$ é válida a igualdade: $1 + 3 + 5 + \cdots + 2n - 1 = n^2$.

Solução:

Definimos a proposição $P(n): 1 + 3 + 5 + \cdots + 2n - 1 = n^2$. Notemos que a mesma vale para $n = 1$, pois: $P(1): 1^2 = 1 = 2.1 - 1$.
Suponhamos agora, que $P(k)$ seja verdadeira pra um certo $k > 1, k \in \mathbb{N}$.
Daí:
$1 + 3 + 5 + \cdots + 2k - 1 + 2k + 1 = k^2 + 2k + 1 = (k+1)^2$, o que prova que $P(k+1)$ é verdade. Assim, $P(n)$ é verdade para qualquer $n \in \mathbb{N}$, pelo princípio da indução.

Exemplo 3 Os números de Fermat são números definidos pela expressão $F_n = 2^{2^n} + 1$ para todo natural n. Fermat conjecturou na época que esta fórmula era geradora de números primos. De fato, os quatro primeiros números de Fermat são primos. No entanto, Leonard Euller, provou mais tarde que para $n = 5$ a mesma resulta em um número composto. Uma propriedade interessante dos números de Fermat é a seguinte: $F_0 F_1 \ldots F_{n-1} = F_n - 2$ para todo natural n. Use o princípio de indução matemática para provar a veracidade da mesma.

Solução:

Vamos definir a proposição $P(n)$: $F_n = 2^{2^n} + 1$. Notemos que a mesma vale para $n = 1$, pois: $P(1) = F_0 = 2^{2^0} + 1 = 3 = 2^{2^1} + 1 - 2 = F_1 - 2$ Suponhamos agora, que $P(k)$ seja verdadeira pra um certo $k > 1, k \in \mathbb{N}$. Daí:

$F_0 F_1 \ldots F_{k-1} F_k = (F_k - 2) F_k = (2^{2^k} + 1 - 2)(2^{2^k} + 1) = (2^{2^k} - 1)(2^{2^k} + 1) = 2^{2.2^k} - 1 = 2^{2^{k+1}} - 1 = 2^{2^{k+1}} + 1 - 2 = F_{k+1} - 2$, o que prova que $P(k+1)$ é verdade. Assim, $P(n)$ é verdade para qualquer $n \in \mathbb{N}$, pelo princípio da indução.

SILVA, R. L.; ALMEIDA, R. L. da S. O poderoso princípio da indução matemática. C.Q.D.– Revista Eletrônica Paulista de Matemática, Bauru, v. 22, n. 1, p. 63-72, jul. 2022. Edição Iniciação Científica.
DOI: 10.21167/cqdv22n1ic2022062072 Disponível em: http://www.fc.unesp.br/departamentos/matematica/revista-cqd/

3.2 Demonstração de desigualdades

Assim como as identidades, também há inúmeras desigualdades que podem ser demonstradas através da indução matemática. Nos exemplos a seguir ilustraremos algumas dessas desigualdades, destacando uma desigualdade famosa, conhecida como desigualdade de Bernoulli.

Exemplo 4 Demonstre que para qualquer $n \in \mathbb{N}$ é válida a desigualdade: $3^{n-1} < 2^{n^2}$.

Solução:

Definimos a proposição $P(n)$: $3^{n-1} < 2^{n^2}$. Notemos que a mesma vale para $n = 1$, pois: $P(1)$: $3^{1-1} = 1 < 2^{1^2} = 2$. Suponhamos agora, que $P(k)$ seja verdadeira pra um certo $k > 1, k \in \mathbb{N}$. Usando o fato de que $3 < 2^{2k+1}, k \in \mathbb{N}$, temos que:
$3^k = 3.3^{k-1} < 2^{k^2}.2^{2k+1} = 2^{(k+1)^2}$, o que prova que $P(k+1)$ é verdade. Assim, $P(n)$ é verdade para qualquer $n \in \mathbb{N}$, pelo princıpio da indução.

Exemplo 5 Prove que para qualquer $n > 3, n \in \mathbb{N}$ é válida a desigualdade: $2^n < n!$.

Solução:

Definimos a proposição $P(n)$: $2^n < n!$. Notemos que a mesma vale para n = 4, pois: $P(4)$: $24 = 16 < 4! = 24$. Suponhamos agora, que P(k) seja verdadeira pra um certo $k > 4, k \in \mathbb{N}$. Usando o fato de que $2 < (k+1), k \in \mathbb{N}, k > 4$, temos que:
$2^{k+1} = 2^k.2 < k!(k+1) = (k+1)!$, o que prova que $P(k+1)$ é verdade. Assim, $P(n)$ é verdade para qualquer $n \in \mathbb{N}, n > 3$, pelo princípio da indução.

Exemplo 6 Demonstre, que para todo $n \in \mathbb{N}$, $\sqrt{2+\sqrt{2+\sqrt{2+\cdots+\sqrt{2}}}} < 2$, com n radicais.

Solução:

A desigualdade é claramente verdade para $n = 1$, pois $\sqrt{2} < 2$. Suponhamos que para um certo n natural, vale a desigualdade com n radicais. Adicionando 2 em ambos os lados dessa desigualdade e extraindo a raiz quadrada de ambos os lados, temos que:
$\sqrt{\sqrt{2+\sqrt{2+\sqrt{2+\cdots+\sqrt{2}}}}} < \sqrt{2+2} = 2$, com $n+1$ radicais, o que prova a veracidade da proposição.

Exemplo 7 Uma desigualdade interessante da Matemática é a chamada Desigualdade de Bernoulli. Esta desigualdade afirma o seguinte: Sejam $n \in \mathbb{N}$ e $x > 1$. Então $(1+x)^n \geq 1+nx$. Use o princípio de indução matemática para demonstrar a veracidade dessa desigualdade.

Solução:

A desigualdade é claramente verdade para $n = 1$, pois $(1 + x)^1 \geq 1 + x$. Suponhamos que para um certo n natural, vale a desigualdade $(1 + x)^n \geq 1 + nx$. Iremos provar que $(1 + x)^{n+1} \geq 1 + (n + 1)x$.

Temos que:
$(1 + x)^{n+1} = (1 + x)^n(1 + x) \geq (1 + nx)(1 + x) = 1 + nx + x + nx^2 = 1 + (n + 1)x + nx^2 \geq 1 + (n + 1)x$, o que prova a veracidade da proposição.

3.3 Problemas de divisibilidade

O princípio da indução finita também é muito utilizado em problemas que envolvem a divisibilidade, como mostramos nos exemplos a seguir:

Exemplo 8 Mostre que para qualquer $n \in \mathbb{N}$, $n^3 + 2n$ é sempre divisível por 3.

Solução:

Para $n = 1$, temos que a proposição é válida, pois: $1^3 + 2.1 = 3$, que claramente é divisível por 3. Suponhamos agora, que para um certo $k > 1, k \in \mathbb{N}$, $k^3 + 2k$ é divisível por 3. Mostraremos que $(k + 1)^3 + 2(k + 1)$ é divisível por 3. Assim, temos que:
$(k + 1)^3 + 2(k + 1) = k^3 + 3k^2 + 3k + 1 + 2k + 2 = k^3 + 2k + 3(k^2 + k + 1)$. Como por hipótese $k^3 + 2k$ é divisível por 3 e como $3(k^2 + k + 1)$ também é claramente divisível por 3, segue-se que $(k + 1)^3 + 2(k + 1)$ é também divisível por 3, o que mostra a veracidade de $P(k + 1)$. Dessa forma, $P(n)$ é verdade para qualquer $n \in \mathbb{N}$.

Exemplo 9 Prove que a soma dos cubos de três números naturais consecutivos é divisível por 9.

Solução:

Seja $P(n)$ a propriedade: $P(n)$: $n^3 + (n + 1)^3 + (n + 2)^3$ é divisível por 9. Admitindo que $P(n)$ seja válida para um certo n natural, mostraremos que $P(n + 1)$ é verdade, ou seja, que $P(n + 1)$: $(n + 1)^3 + (n + 2)^3 + (n + 3)^3$ é divisível por 9.

Temos então que:
$(n + 1)^3 + (n + 2)^3 + (n + 3)^3 = (n + 1)^3 + (n + 2)^3 + (n^3 + 9n^2 + 27n + 27) = n^3 + (n + 1)^3 + (n + 2)^3 + 9(n^2 + 3n + 3)$, que é claramente divisível por 9, pois $n^3 + (n + 1)^3 + (n + 2)^3$ é divisível por 9 por hipótese e $9(n^2 + 3n + 3)$ também é divisível por 9. Assim, $P(n + 1)$ é verdade para todo n, o que prova a veracidade de $P(n)$.

3.4 Problemas de geometria

Outra aplicação interessante do princípio de indução matemática é no tocante a muitos problemas de Geometria, pois existem muitas fórmulas geométricas que envolvem números naturais. Abaixo apresentaremos alguns exemplos dessa aplicação:

Exemplo 10 Mostre que a soma dos ângulos internos de um polígono convexo de n-lados é $S_n = (n-2)\pi$ radianos.

Solução:

Para $n = 3$ temos que $S_3 = (3-2)\pi = \pi$, que é verdade, pois a soma dos ângulos internos de um triângulo é π radianos. Vamos agora assumir que $P(n)$ seja verdade e mostraremos que para um polígono de $n+1$ lados, $S_{n+1} = (n-1)\pi$. Vamos denotar os vértices do polígono de $n+1$ lados por $A_1, A_2, \dots, A_n, A_{n+1}$. Podemos decompor este polígono como a união do polígono de n lados $A_1, A_2, \dots, A_n$ e do triângulo A_1, A_n, A_{n+1}. Dessa forma, a soma dos ângulos internos do polígono $n+1$ é: a soma dos ângulos internos do polígono de vértices $A_1, A_2, \dots, A_n$ mais a soma dos ângulos internos do triângulo A_1, A_n, A_{n+1}, isto é:
$S_{n+1} = (n-2)\pi + \pi = (n-1)\pi$, como queríamos provar.

Exemplo 11 Mostre que o número de diagonais de um polígono convexo de n-lados é $d_n = \frac{n(n-3)}{2}$.

Solução:

Para $n = 3$ temos que $d_3 = \frac{3(3-3)}{2} = 0$, que é verdade, pois um triângulo não tem diagonais. Vamos agora assumir que $P(n)$ seja verdade e mostraremos que para um polígono de $n+1$ lados, $d_{n+1} = \frac{(n+1)(n-2)}{2}$. Vamos denotar os vértices do polígono de $n+1$ lados por $A_1, A_2, \dots, A_n, A_{n+1}$. Podemos decompor este polígono como a união do polígono de n lados $A_1, A_2, \dots, A_n$ e do triângulo A_1, A_n, A_{n+1}. Dessa forma, o número de diagonais do polígono de $n+1$ é: o número de diagonais do polígono de vértices $A_1, A_2, \dots, A_n$ mais $n-2$ diagonais que partem A_{n+1} mais a diagonal A_1A_n, isto é:
$d_{n+1} = \frac{n(n-3)}{2} + (n-2) + 1 = \frac{n^2-n-2}{2} = \frac{(n+1)(n-2)}{2}$, como queríamos provar.

4 O princípio da indução matemática e recorrências lineares

Estudaremos agora o princípio da indução matemática aplicado a recorrências lineares. Inicialmente, iremos definir o que é recorrência linear e propriedades, elucidando alguns exemplos do uso do princípio de indução matemática na prova da veracidade das mesmas. Também estudaremos uma recorrência famosa, conhecida como sequência de Fibonacci.

Definição 1 Uma recorrência linear de grau k é uma expressão da forma:
$x_{n+1} = r_{k-1}x_n + r_{k-2}x_{n-1} + \cdots + r_0x_{n-k+1}, x_1 = a_1, x_2 = a_2, \dots, x_k = a_k$, onde $r_0, r_1, \dots, r_{k-1}$ são números reais com $r_0 \neq 0$.

Exemplo 12 As recorrências $3x_n + 4x_{n-1} = 0$ e $-5x_n + \frac{2}{5}x_{n+1} = 3x_{n+2}$ são lineares, ao passo que as recorrências $(x_n)^3 - 2x_{n-1} = 2_n$ e $-4x_n + \frac{1}{3}x_{n+1} = 2x_{n+2} + 5$ não são lineares.

Exemplo 13 Seja a relação de recorrência $a_0 = 8, a_1 = 10$ e $a_n = 4a_{n-1} - 3a_{n-2}$. Prove que $a_n = 7 + 3^n, \forall\, n \in \mathbb{N}$.

Solução:

Temos que $P(0)$ é verdade, pois: $P(0) = 7 + 3^0 = 7 + 1 = 8$. Suponhamos agora que $P(n)$ seja verdade para algum $n > 2$. Mostraremos que $P(n + 1)$ também é verdade. Com efeito:
$a_{n+1} = 4a_n - 3a_{n-1} = 4(7 + 3^n) - 3(7 + 3^{n-1}) = 7 + 4.3^n - 3.3^{n-1} = 7 + 3^{n-1}(4.3 - 3) = 7 + 9.3^{n-1} = 7 + 3^2.3^{n-1} = 7 + 3^{n+1}$. Assim, $P(n + 1)$ é verdade, o que prova a veracidade de $P(n)$.

4.1 A sequência de Fibonacci

A sequência de Fibonacci, em homenagem ao matemático italiano Leonardo de Pisa (1170 - 1250), é um exemplo muito interessante de recorrência linear. Esta sequência foi estudada por muitos matemáticos, que descobriram inúmeras propriedades interessantes da mesma. Esta fantástica sequência está interligada com um enigmático número, conhecido como número de ouro. Em virtude disso, ela aparece em inúmeras situações da natureza: nas plantas, na galáxia, em animais, no corpo humano, em obras de artistas famosos, etc. Nesta seção iremos definir essa sequência e descrever algumas de suas principais propriedades, provando-as com o uso do princípio da indução matemática.

4.1.1 Definição da sequência

Definição: A sequência de inteiros positivos (F_n): $(1,1,2,3,5,8,13,21, \ldots)$, onde $F_1 = F_2 = 1$ e $F_n = F_{n-1} + F_{n-2}$, para todo $n > 2$, $n \in \mathbb{N}$, recebe o nome de sequência de Fibonacci. Os seus termos chamam-se números de Fibonacci.

4.1.2 Propriedades elementares

A sequência de Fibonacci é repleta de propriedades muito interessantes. Abaixo segue algumas de suas propriedades elementares e as demonstrações das mesmas.

Propriedade 1: A soma dos n primeiros números de Fibonacci é igual a $F_{n+2} - 1$.

Prova:

De fato, temos que:
$F_1 = F_3 - F_2 = F_3 - 1$
$F_2 = F_4 - F_3$
$F_3 = F_5 - F_4$
... ...
$F_{n-1} = F_{n+1} - F_n$
$F_n = F_{n+2} - F_{n+1}$
Somando membro a membro as igualdades e simplificando, temos que: $\sum_{i=1}^{n} F_i = F_{n+2} - 1$.

Propriedade 2: A soma dos n primeiros números de Fibonacci com índices impares é igual a F_{2n} .

E61.Funções Definidas Recursivamente — p. 17

Funções Definidas Recursivamente

Uma função f cujo domínio é o conjunto $\mathbb{Z}^+$ podem ser definidas recursivamente através de dois passos:

1. **Passo Básico**: Especificar o valor de $f(0)$
2. **Passo Recursivo**: Dar uma regra (**relação de recorrência**) para encontrar seu valor para um inteiro a partir de seus valores para inteiros menores.

Tal definição é chamada de **definição recursiva** ou **definição indutiva**

Exemplo Na função f definida recursivamente abaixo, encontre $f(1)$, $f(2)$, $f(3)$ e $f(4)$.

- $f(0) = 3$
- $f(n+1) = 2 \cdot f(n) + 3$

$f(1) = 2 \cdot f(0) + 3 = 2 \cdot 3 + 3 = 9$

$f(2) = 2 \cdot f(1) + 3 = 2 \cdot 9 + 3 = 21$

$f(3) = 2 \cdot f(2) + 3 = 2 \cdot 21 + 3 = 45$

$f(4) = 2 \cdot f(3) + 3 = 2 \cdot 45 + 3 = 93$

Exemplos

- **Exemplo**: dê uma definição recursiva para $f(n) = a^n$, onde $a \in \mathbb{R}^*$ e $n \in \mathbb{Z}^+$
- **Solução**:
  Passo Base: $f(0) = 1$
  Passo Recursivo: $f(n) = a \cdot f(n-1)$

- **Exemplo**: dê uma definição recursiva para o somatório

$$\sum_{k=0}^{n} a_k$$

- **Solução**:

  Passo Base: $\sum_{k=0}^{0} a_k = a_0$

  Passo Recursivo: $\sum_{k=0}^{n+1} a_k = \left(\sum_{k=0}^{n} a_k \right) + a_{n+1}$

Exemplos

- **Exemplo**: dê uma definição recursiva para a função fatorial $F(n) = n!$
- **Solução**
  Passo Base: $F(0) = 1$
  Passo Recursivo: $f(n+1) = (n+1) \cdot F(n)$

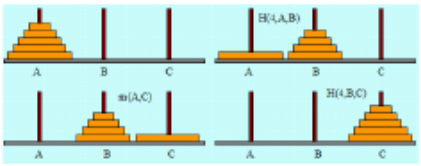
  Determine $F(6)$, usando a definição recursiva acima.

  $F(6) = 6 \cdot F(5) = 6 \cdot 5 \cdot F(4) = 6 \cdot 5 \cdot 4 \cdot F(3) = 6 \cdot 5 \cdot 4 \cdot 3 \cdot F(2)$
  $F(6) = 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot F(1) = 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 \cdot F(0)$
  $F(6) = 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 \cdot 1 = 720$

- **Exemplo**: Números de Fibonacci são definidos pela regra: $f_0 = 0$, $f_1 = 1$, e $f_n = f_{n-1} + f_{n-2}$, $n \geq 2$. Encontre os números de Fibonacci f_2, f_3, f_4, f_5 e f_6.
- **Solução**
  $f_2 = f_1 + f_0 = 1 + 0 = 1$
  $f_3 = f_2 + f_1 = 1 + 1 = 2$
  $f_4 = f_3 + f_2 = 2 + 1 = 3$
  $f_5 = f_4 + f_3 = 3 + 2 = 5$
  $f_6 = f_5 + f_4 = 5 + 3 = 8$
- Enumere a sequência de números de Fibonacci, de f_0 à f_{12}

Algoritmos Recursivos e Relações de Recorrência

Como o problema pode ser resolvido para n discos?



Algoritmo 7 `TORRE-HANOI`(n, *origem*, *destino*, *auxiliar*: inteiros)

```
1: if (n = 1) then
2:     Movimente n de origem para destino
3: else
4:     TORRE-HANOI(n − 1, origem, auxiliar, destino)
5:     Movimente n de origem para destino
6:     TORRE-HANOI(n − 1, auxiliar, destino, origem)
7: end if
```

Exercício: Esboce uma árvore com as chamadas recursivas do algoritmo acima para resolver o problema para $n = 4$ discos.

Exemplo: Solução em Forma Fechada

Encontre uma solução em forma fechada para a seguinte relação de recorrência:

- $T(1) = 1$
- $T(n) = T(n-1) + 3$, para $n \geq 2$

Passo 1: expandir

$$
\begin{aligned}
T(n) &= T(n-1) + 3 && \text{(apos 1 expansão)}\\
T(n) &= \big[T(n-2) + 3\big] + 3 = T(n-2) + 2 \cdot 3 && \text{(após 2 expansões)}\\
T(n) &= \big[T(n-3) + 3\big] + 2 \cdot 3 = T(n-3) + 3 \cdot 3 && \text{(após 3 expansões)}\\
&\vdots\\
T(n) &= T(n-k) + k \cdot 3 && \text{(após } k \text{ expansões)}\\
&\vdots\\
T(n) &= T\big(n-(n-1)\big) + (n-1) \cdot 3 && \text{(após } (n-1) \text{ expansões)}\\
T(n) &= T(1) + 3 \cdot (n-1) = 1 + 3n - 3
\end{aligned}
$$

Passo 2: conjecturar

$\boxed{T(n) = 3n - 2}$ (**solução em forma fechada**)

Exemplo: Solução em Forma Fechada

Passo 3: verificar

Teorema (Conjectura)

$T(n) = 3n - 2$ *para todo inteiro* $n \geq 1$

Prova por Indução.

- Passo Base: $T(1) = 1 = 3(1) - 2 = 1$
- Passo Indutivo: assumir $T(k) = 3k - 2$ é verdade

- Mostrar que $T(k) \to T(k+1)$, i.e., $\boxed{T(k+1) = 3(k+1) - 2}$

$$T(k+1) = T(k) + 3$$

$$T(k+1) = \underbrace{(3k-2)}_{} + 3 = 3k + 3 - 2$$

$$\boxed{T(k+1) = 3(k+1) - 2}$$

□

2.2.3 Série Harmônica

$$\sum_{k=1}^{n} \frac{1}{k} = \ln n + \gamma + O(1/n),$$

onde $\gamma = 0,577...$ é a *constante de Euler*.

2.2.4 Soma de Logaritmos

$$\sum_{k=1}^{n} \lfloor \log_2 k \rfloor = (n+1)\lfloor \log_2 n \rfloor - 2^{\lfloor \log_2 n \rfloor + 1} + 2 \in \Theta(n \log n).$$

2.2.5 Produtos

O produto finito $a_1 \cdot a_2 \cdot \ldots \cdot a_n$ é denotado por $\prod_{k=1}^{n} a_k$. Se $n = 0$, o valor do produto é definido como 1. Às vezes é conveniente converter uma fórmula contendo um produto em uma fórmula com um somatório usando a identidade: $\log\left(\prod_{k=1}^{n} a_k\right) = \sum_{k=1}^{n} \log a_k$.

2.2.6 Somatórios Interessantes

Exemplo 1

$$\sum_{k=1}^{n} k^2 = \frac{n(n+1)(2n+1)}{6}.$$

Exemplo 2

$$\sum_{k=1}^{n} k2^k = (n-1)2^{n+1} + 2.$$

Exemplo 3

$$\sum_{k=1}^{n} k2^{n-k} = 2^{n+1} - 2 - n.$$

3 Técnicas para a resolução de relações de recorrência

3.1 Expansão Telescópica

Como exemplo de um algoritmo cuja complexidade de tempo pode facilmente ser expressa através de uma relação de recorrência, considere o que realiza busca binária em um vetor ordenado. Esse algoritmo usa o seguinte procedimento recursivo: sejam V um vetor com n elementos ordenado crescentemente e x um valor que desejamos encontrar em V. Começamos por comparar x com o elemento $V(m)$ que se encontra na posição do meio ($m = (n+1)/2$) do vetor. Se x for igual a $V(m)$, então m é o índice procurado e a busca termina. Se x for menor que $V(m)$, o elemento procurado só pode se encontrar à esquerda

E66.Em decorrência da propriedade de linearidade, é possıvel co — p. 3

Em decorrência da propriedade de linearidade, é possível colocar em evidência termos multiplicativos que sejam independentes do somatório. Por exemplo:

$$\sum_{k=1}^{n}(f(x)a_k) = f(x)\sum_{k=1}^{n}(a_k),$$

onde $f(x)$ é uma função independente de k. Manipulação similar pode ser feita com somatórios que incorporam notação assintótica. Por exemplo:

$$\sum_{k=1}^{n}\Theta(f(k)) = \Theta(\sum_{k=1}^{n}f(k)).$$

Nesta equação, a notação Θ no lado esquerdo se aplica à variável k. No lado direito, porém, ela se aplica a n. Manipulações desta natureza também podem ser aplicadas a séries infinitas convergentes.

2.2 Exemplos de Somatórios

Saber encontrar a forma fechada de um somatório é muito importante para a resolução de relações de recorrência. Uma das técnicas que apresentamos a seguir faz uso do fato de que relações de recorrência podem ser reescritas como somatórios. Nesta seção, apresentamos alguns somatórios encontrados com freqüência na análise de algoritmos, junto com suas respectivas formas fechadas. O leitor interessado poderá encontrar uma rica discussão das técnicas em [2, 1].

2.2.1 Série Aritmética

Uma seqüência $a_1 + a_2 + a_3 + ... + a_n$ tal que $a_k - a_{k-1} = c$ para $1 < k \leq n$ e c constante é chamada de *progressão aritmética*. Sua soma $\sum_{k=1}^{n} a_k$ é chamada de *série aritmética* e sua forma fechada é dada por:

$$\sum_{k=1}^{n} a_k = \frac{n(a_n + a_1)}{2}.$$

2.2.2 Série Geométrica

Uma seqüência $a_1 + a_2 + a_3 + ... + a_n$ tal que $a_k/a_{k-1} = c$ para $1 < k \leq n$ e c constante é chamada de *progressão geométrica*. Sua soma $\sum_{k=1}^{n} a_k$ é chamada de *série geométrica* e sua forma fechada é dada por:

$$\sum_{k=1}^{n} a_k = a_1\frac{c^{n+1} - 1}{c - 1}.$$

Quando a série é infinita e $|c| < 1$, temos uma série geométrica infinita decrescente cuja soma é dada por:

$$\sum_{k=1}^{\infty} a_k = \frac{a_1}{1 - c}.$$

para $b \geq 1$.

Pode ser inicialmente contra-intuitivo considerar que se deve *subtrair* um termo (ainda que de menor ordem). Afinal de contas, se o algebrismo não funciona, não deveríamos tentar aumentar nosso palpite? A chave para entender este passo é lembrar que estamos usando indução matemática: podemos provar algo mais apertado para um dado valor através da suposição de algo mais apertado para valores menores.

É fácil errar quando usamos notação assintótica. Por exemplo, podemos tentar provar que a relação de recorrência (4) pertence a $O(n)$ usando o palpite de que $T(n) \leq cn$ e argumentando

$$\begin{aligned} T(n) &\leq 2(c\lfloor n/2 \rfloor) + n \\ &\leq cn + n \in O(n), \end{aligned}$$

uma vez que c é uma constante. O erro neste caso é que não provamos a forma exata da hipótese indutiva, isto é, não provamos que $T(n) \leq cn$. Começamos tentando provar uma coisa (que $T(n) \leq cn$) mas saltamos para uma conclusão alternativa que não corresponde ao que pretendíamos demonstrar. Note que $T(n) \leq cn$ implica que $T(n) \in O(n)$, mas, por outro lado, $T(n) \leq cn + n$ e $cn + n \in O(n)$ *não* implicam que $T(n) \in O(n)$.

Algumas vezes, um pouco de manipulação algébrica pode fazer uma recorrência aparentemente desconhecida tomar um aspecto mais familiar. Como exemplo, considere a seguinte relação de recorrência:

$$T(n) = 2T(\lfloor \sqrt{n} \rfloor) + \log_2 n.$$

Embora pareça complicada, essa recorrência pode ser simplificada através de uma troca de variáveis. Por conveniência, não vamos nos preocupar em arredondar valores. Renomeando $\log_2 n$ para m, obtemos

$$T(2^m) = 2T(2^{m/2}) + m.$$

Podemos então substituir $T(2^m)$ por $S(m)$ e produzir a nova recorrência

$$S(m) = 2S(m/2) + m,$$

que é muito parecida com a recorrência (4) e tem a mesma solução: $S(m) \in O(m \log m)$. Mudando de volta de $S(m)$ para $T(2^m) = T(n)$, obtemos que $T(n) \in O(\log n \log \log n)$.

3.3 O Teorema Master

Até aqui, muitas das relações de recorrência que vimos foram da forma $T(n) = aT(n/b) + f(n)$ e obtivemos soluções caso a caso. Seria interessante termos um teorema que nos permita obter, de modo simples, soluções para todas as relações de recorrência dessa forma. Isto é exatamente o que nos dá o seguinte Teorema Master:

de grau $< k$, precisamos percorrer apenas uma linha (correspondente ao vértice v) e uma coluna (correspondente aos vértices aos quais v é adjacente) da matriz. A complexidade para esta representação é descrita pela seguinte relação de recorrência:

$$T(n) = T(n-1) + n + n.$$

A solução para esta recorrência pertence a $O(n^2)$, ou seja, a representação da informação aumentou a velocidade do algoritmo.

É possível reduzir ainda mais o tempo necessário para se encontrar os vértices de grau menor que k através do uso de uma estrutura auxiliar como, por exemplo, um *heap* [2] com a chave de acesso sendo o grau de cada vértice. O tempo necessário para construir o *heap* inicial é $O(n \log n)$ e a remoção do i-ésimo elemento do *heap* toma tempo $O(\log(n - i + 1))$.

Exercício: Com o uso da estrutura auxiliar de *heap* como descrito acima, qual a complexidade final do algoritmo?

Uma importante lição que podemos tirar deste exemplo é que a eficiência de um algoritmo é freqüentemente melhorada se se escolhe uma estrutura de dados adequada para representação da informação de modo a facilitar os acessos requeridos pelo algoritmo.

4.3 O Problema da União e Busca de Conjuntos

4.3.1 A Função Logaritmo Iterado

Antes de entrarmos no Problema propriamente dito, vamos apresentar a *Função Logaritmo Iterado*.

Inicialmente, definimos a função[5] $\log^{(i)} n$ recursivamente para inteiros não-negativos por:

$$\log^{(i)} n = \begin{cases} n, & \text{se } i = 0 \\ \log(\log^{(i-1)} n), & \text{se } i > 0 \text{ e } \log^{(i-1)} n > 0 \\ \text{indefinida}, & \text{se } i > 0 \text{ e } \log^{(i-1)} n \leq 0 \text{ ou } \log^{(i-1)} \text{ é indefinida} \end{cases}$$

O leitor deve tomar o cuidado de não confundir as notações $\log^{(i)} n$ (i-ésima iterada da função logaritmo) e $\log^i n$ (i-ésima potência do logaritmo de n); esta última é o mesmo que $(\log n)^i$.

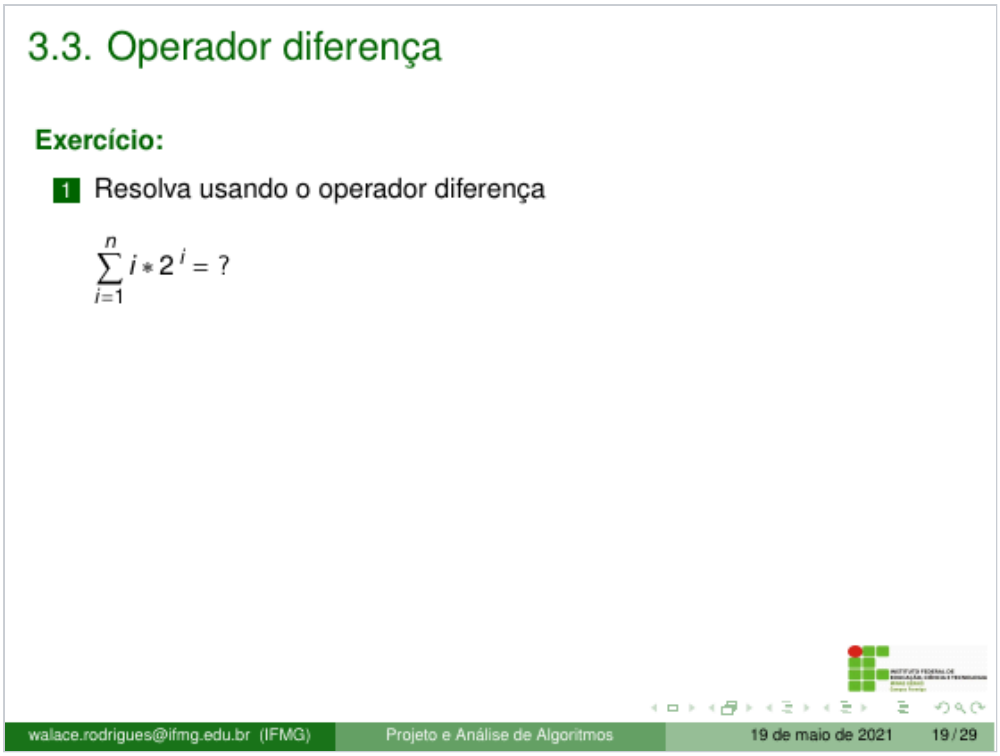
Definimos a *função logaritmo iterado*, denotada por $\log^* n$, lê-se "log estrela de n"como:

$$\log^* n = \min \left\{ i \geq 0 : \log^{(i)} n \leq 1 \right\}$$

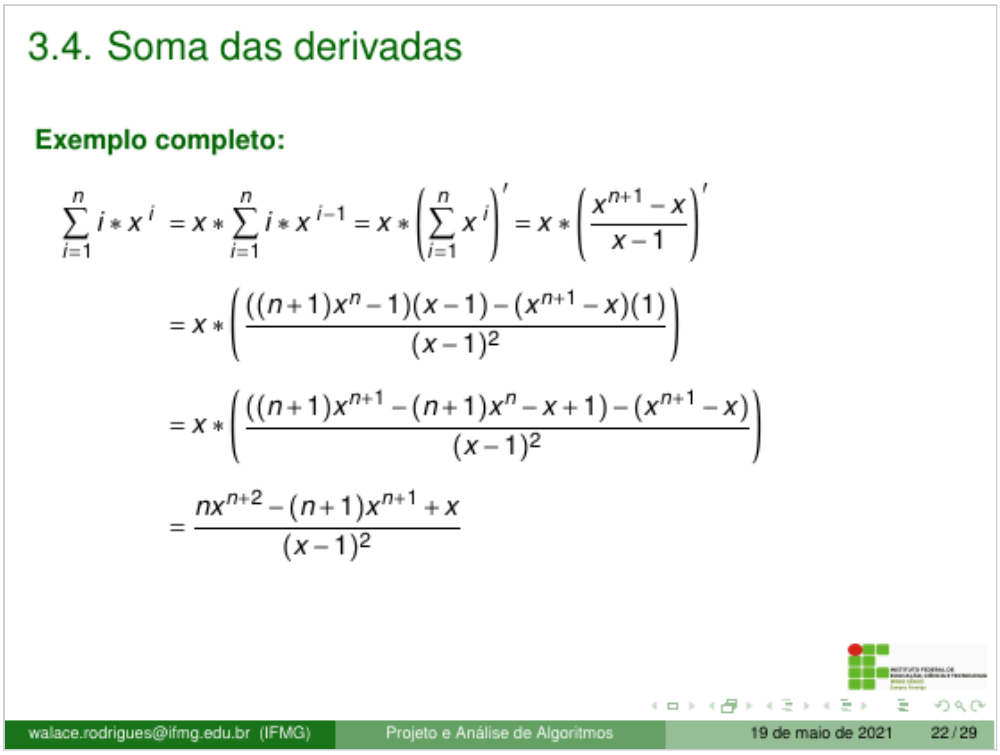
Vejamos como é lento o crescimento da função logaritmo iterado $\log_2^* n$:

[5]Omitimos aqui a base do logaritmo, mas, quando for conveniente, esta pode ser explicitada.

E69.3.3. Operador diferença — p. 19



E70.3.4. Soma das derivadas — p. 22



Prova

Seja S o conjunto dos inteiros m tais que $A(m)$ seja falsa. Suponhamos que a afirmação $A(n)$ seja falsa para algum inteiro. Então, o conjunto S é não-vazio.
Conforme o Principio da Boa Ordem (veja Aula 1) existe elemento minimal $m_0 = \min S$. Como $m_0 - 1 \notin S$ (pois m_0 é minimal) assim $A(m_0 - 1)$ é verdadeira. Conforme (ii), para $k = m_0 - 1$ teremos então que

$$A(k+1) = A(m_0 - 1 + 1) = A(m_0)$$

é verdadeira. Mas isso é uma contradição, pois m_0 é elemento em S ou seja $A(m_0)$ deve ser falso. Assim S é conjunto vazio e $A(n)$ é verdadeira para todo inteiro $n \geq a$.

2.2 *Exemplos*

Para aplicar o principio da indução finita nos exemplos precisamos fazer 3 coisas:

Base: Verificar a afirmação no menor caso possível.

Hipótese: Assumir que a afirmação seja verdadeira no caso $n = k$.

Passo: Usando hipótese mostrar que a afirmação é verdadeira no caso $n = k + 1$.

Exemplo 2.1

Provaremos agora que a formula

$$1 + 2 + \ldots + n = \frac{n(n+1)}{2}$$

é verdadeira para todo $n \geq 1$.
Base n=1. A fórmula acima dá

$$1 = \frac{1(1+1)}{2}, \text{ ou seja }, 1 = 1$$

Assim, a afirmação é verdadeira para $n = 1$. Deveremos mostrar agora que, se a afirmação é verdadeira para $n = k$, então também é verdadeira para $n = k + 1$.
Hipótese n=k. Suponha que

$$1 + 2 + \ldots + k = \frac{k(k+1)}{2}, \tag{2.1}$$

e mostremos que

$$1 + 2 + \ldots + k + (k+1) = \frac{(k+1)(k+2)}{2}.$$

Passo n=k+1. Somando $k + 1$ a ambos os lados da (2.1) temos

$$1 + 2 + \ldots + k + (k+1) = \frac{k(k+1)}{2} + (k+1) = \frac{k(k+1) + 2(k+1)}{2}$$

isto é

$$1 + 2 + \ldots + k + (k+1) = \frac{(k+1)(k+2)}{2}$$

que é exatamente o que queríamos demonstrar.

Ant ft). Prof. Kostiantyn

Exemplo 2.2

Provaremos agora que a formula

$$1+3+\ldots+(2n-1)=n^2$$

é verdadeira para todo $n \geq 1$.
Base n=1. Para $n=1$, a fórmula acima dá

$$1=1^2, \text{ ou seja }, 1=1$$

Assim, a afirmação é verdadeira para $n = 1$. Deveremos mostrar agora que, se a afirmação é verdadeira para $n=k$, então também é verdadeira para $n=k+1$.
Hipótese n=k. suponha que

$$1+3+\ldots+(2k-1)=k^2 \tag{2.2}$$

e mostremos que

$$1+3+\ldots+(2k-1)+(2k+1)=(k+1)^2$$

Passo n=k+1. Somando $2k+1$ a ambos os lados da (2.2) temos

$$1+3+\ldots+(2k-1)+(2k+1)=k^2+2k+1$$

isto é

$$1+3+\ldots+(2k-1)+(2k+1)=(k+1)^2$$

que é exatamente o que queríamos demonstrar.

Exemplo 2.3

Vamos mostrar que

$$n^3<2^n$$

é verdadeira para todo $n \geq 10$.
Base n=10. Para $n=10$, temos

$$10^3=1000, 2^{10}=1028, \text{ ou seja }, 10^3<2^{10}.$$

Assim, a afirmação é verdadeira para $n = 10$. Deveremos mostrar agora que, se a afirmação é verdadeira para $n=k$, então também é verdadeira para $n=k+1$.
Hipótese n=k. Assim, suponha que

$$k^3<2^k. \tag{2.3}$$

Passo n=k+1. Temos

$$(k+1)^3=k^3+3k^2+3k+1$$

Pelo hipótese (2.5) $k^3<2^k$, além disso obviamente $3k<3k^2$ e $1<k^2$. Assim

$$(k+1)^3=k^3+3k^2+3k+1<2^k+3k^2+3k^2+k^2=2^k+7k^2. \tag{2.4}$$

Como $k \geq 10$, assim $7k^2<k^3<2^k$, portanto (2.4) escreva-se como

$$(k+1)^3<2^k+7k^2<2^k+2^k=2^{k+1}$$

que é exatamente o que queríamos demonstrar.

Exemplo 2.4

Vamos mostrar que

$$\left(1-\frac{1}{4}\right)\left(1-\frac{1}{9}\right)\left(1-\frac{1}{16}\right)\cdots\left(1-\frac{1}{n^2}\right)=\frac{n+1}{2n}$$

é verdadeira para todo $n \geq 2$.

Base n=2. Para $n = 2$, temos

$$\left(1-\frac{1}{4}\right)=\frac{3}{4},\quad \frac{2+1}{2\cdot 2}=\frac{3}{4},\text{ ou seja },\frac{3}{4}=\frac{3}{4}.$$

Hipótese n=k. Suponha que

$$\left(1-\frac{1}{4}\right)\left(1-\frac{1}{9}\right)\left(1-\frac{1}{16}\right)\cdots\left(1-\frac{1}{k^2}\right)=\frac{k+1}{2k} \tag{2.5}$$

Passo n=k+1. Devemos mostrar que

$$\left(1-\frac{1}{4}\right)\left(1-\frac{1}{9}\right)\left(1-\frac{1}{16}\right)\cdots\left(1-\frac{1}{k^2}\right)\left(1-\frac{1}{(k+1)^2}\right)=\frac{k+2}{2(k+1)}$$

Pelo hipótese o lado esquerdo pode ser escrito como

$$\frac{k+1}{2k}\cdot\left(1-\frac{1}{(k+1)^2}\right)=\frac{k+1}{2k}\cdot\frac{(k+1)^2-1}{(k+1)^2}=\frac{k^2+2k+1-1}{2k(k+1)}=\frac{k+2}{2(k+1)},$$

que é exatamente o que queríamos demonstrar.

2.3 *Numeros de Fibonacci e Segunda forma da PIF*

Você pode ter ouvido falar dos números de Fibonacci. Eles ocorrem com frequência em matemática e ciências da vida. Pela primeira vez eles ocorreram no livro **Liber Abaci** (1202) de Fibonacci, onde foi usada para calcular o crescimento das populações de coelhos. Os números de Fibonacci formam uma sequência em que cada termo, exceto os dois primeiros, é a soma dos dois números anteriores. Matematicamente, se denotarmos o n-th número de Fibonacci F_n, então

$$F_n = F_{n-1} + F_{n-2}$$

Isso é chamado de relação de recorrência para F_n.



Livro **Liber Abaci** (1202)

A relação de recorrência implica que precisamos começar com dois valores iniciais. Frequentemente, começamos com $F_0 = 0$ e $F_1 = 1$. Combinamos a relação de recorrência para F_n e seus valores iniciais juntos em uma definição:

$$F_0 = 0,\quad F_1 = 1,\quad F_n = F_{n-1} + F_{n-2},\quad \text{para } n \geq 2$$

Temos que especificar que a relação de recorrência é válida apenas quando $n \geq 2$, porque este é o menor valor de n para o qual podemos usar a relação de recorrência.

A soma do zero e dos primeiros números de Fibonacci nos dá o

Anexo OBI (EO129 (D...

Sobre este documento. Contém as 43 questões dos materiais complementares e 73 blocos de exemplos dos slides. As listas 1–5 e a prova de 2025 **não** estão aqui — cada uma tem seu próprio PDF nesta pasta, e todas juntas estão em `banco-de-questoes.pdf`. As resoluções passaram por verificação adversarial em contexto isolado e conferência numérica (`RESOLUCOES/verificacao/`). IFMG Campus Formiga, 2026/2.